

EventBridge AI

Technical Report

*A Content-Based, Embedding-Driven Platform for
Community Event Discovery and Recommendation*

Abstract

This report presents EventBridge AI, an AI-powered, community-centric event discovery and management platform developed as a Final Year Project addressing the fragmentation of community-led events across Karachi’s technical and academic ecosystem. The report is structured to move from problem definition through system design, implementation, evaluation, and business viability, closing with a reflection on outstanding limitations and future directions.

The opening chapters establish the project’s foundation: the problem statement, objectives, and scope, followed by a review of existing event management solutions and the specific gap EventBridge AI addresses — the absence of platforms that treat communities as persistent, manageable entities and support AI-driven, meaning-based event discovery rather than static, keyword-limited listings. From this foundation, the report details the system’s requirements and architecture, centered on a deliberate separation between a Laravel-based web application handling core platform logic and a dedicated Django-based AI engine responsible for semantic search and recommendations, communicating through a defined API and synchronized via queue-based indexing jobs.

The implementation chapters describe how this architecture was realized in practice, covering the platform’s core modules — community management, event publishing, inter-community collaboration, and administrative governance — alongside the AI pipeline built on OpenAI’s text-embedding-3-large embeddings and a Qdrant vector database, using a content-based nearest-neighbor approach for recommendations. Chapter 6 evaluates the resulting system’s performance, presenting representative results across search latency, recommendation relevance, and system behavior under typical beta-scale usage, while transparently identifying areas — such as caching and cold-start handling — that remain unaddressed.

Chapter 7 shifts focus from the system to its viability as a product, presenting a complete business model spanning market research and competitive positioning, a defined unique value proposition, target audience segmentation, a phased monetization strategy, and a scalability and internationalization outlook. This is consolidated into a Business Model Canvas and grounded in real market experience gathered through structured beta engagement with GDGoC, IEEE, and ACM chapters, alongside illustrative financial projections and a review of the legal and intellectual property considerations relevant to the platform’s protection and future growth, including its pending copyright and trademark filings through IPO-Pakistan.

Chapter 8 closes the technical narrative with a summary of the project’s key accomplishments and lessons learned, an honest accounting of unaddressed problems and open questions spanning architecture, AI behavior, security, and business adoption, and a set of concrete, achievable directions for future research and development. Chapter 5, addressing the platform’s testing methodology and results, and the appendices supporting the report’s technical claims, complete the document.

Across all chapters, this report maintains a consistent commitment to precision over aspiration: distinguishing clearly between what was originally proposed, what was actually implemented, and what remains genuinely open, in order to present EventBridge AI as an honestly evaluated, real-world platform rather than an idealized academic prototype.

Contents

PREFACE	i
ACKNOWLEDGMENTS	ii
ABSTRACT	v
LIST OF FIGURES	xiv
LIST OF TABLES	xvii
1 Introduction	1
1.1 Overview	1
1.2 Problem Statement	1
1.3 Objectives	2
1.3.1 Build a Centralized Hub for Event Discovery	2
1.3.2 Apply Semantic Search to a Real-World Discovery Problem	2
1.3.3 Personalize Discovery Based on User Behavior	2
1.3.4 Enable Structured Collaboration Between Communities	3
1.3.5 Validate the Idea with Real Users, Not Just a Demo	3
1.3.6 Develop Practical Experience with a Multi-Service AI Architecture	3
1.4 System Features	3
1.4.1 Centralized Event Community Registry	3
1.4.2 AI-Powered Semantic Search	4
1.4.3 Personalized Event Recommendations	4
1.4.4 Event Registration, Attendance Feedback	4
1.4.5 Inter-Community Collaboration	4
1.4.6 Explore Follow	4
1.4.7 Notifications	4
1.4.8 Administrative Governance	5
1.5 Project Scope	5
1.5.1 Boundaries of the System	5
1.5.2 Business Need and Expected Outcome	5
1.5.3 Constraints	5
1.5.4 Assumptions	6
1.5.5 Business Processes Impacted	6
1.5.6 Internal and External Interfacing Entities	6
1.5.7 Target Users / Audience	6
1.5.8 In-Scope vs. Out-of-Scope Features	6
1.5.9 Success Criteria	7
1.6 SDG Alignment	7
1.6.1 SDG 4 — Quality Education	7
1.6.2 SDG 9 — Industry, Innovation and Infrastructure	8
1.6.3 SDG 17 — Partnerships for the Goals	8

1.7	Chapter Summary	8
2	Literature Review	9
2.1	Introduction	9
2.2	Event Discovery and Management Platforms	9
2.3	Recommendation System Approaches	10
2.4	Semantic Search, Embeddings, and Vector Retrieval	11
2.5	Community Collaboration and Networking Platforms	12
2.6	The Karachi Tech and Academic Ecosystem Context	12
2.7	Research Gap and Positioning	13
3	Design	14
3.1	Design Methodology and Software Process Model	14
3.2	Architectural design / Design Patterns	15
3.3	Process flow / Representation	17
3.4	Design models [along with description]	22
3.4.1	Class Diagram	22
3.4.2	Sequence Diagram	25
3.4.3	State Transition Diagram	30
3.4.4	Data Flow Diagram — Django AI Engine	31
3.5	Data Design	33
3.5.1	From Information Domain to Data Structure	33
3.5.2	Storage Organization	34
3.5.3	Storage Summary	35
3.6	Data Dictionary	35
3.6.1	Data Dictionary — Laravel Entities (Object-Oriented)	36
3.6.2	Data Dictionary — Django AI Engine Functions (Function-Oriented)	46
3.7	Chapter Summary	47
4	System Development	49
4.1	Development Environment and Tools	49
4.2	Hardware and Software Requirements	50
4.3	Laravel Application Development	51
4.4	Django AI Engine Development	52
4.5	Qdrant Vector Database Setup	53
4.6	MySQL Database Development	54
4.7	Server Deployment and Orchestration	55
4.8	Implementation Approach and Build Process	56
4.9	Key Implementation Challenges	57
4.9.1	Challenge 1: Cross-server reliability	57
4.9.2	Challenge 2: Recommendation signal blending	58
4.9.3	Challenge 3: Search latency in a synchronous pipeline	58
4.9.4	Challenge 4: Building three independent authentication guards cleanly	59
4.10	Creative and Innovative Decisions	59
4.11	Guidance and Support	60
4.12	Chapter Summary	61

5	Testing	62
5.1	Introduction	62
5.2	Manual Testing	62
5.2.1	Unit Testing	63
5.2.2	Integration Testing	107
5.3	Automation Testing	110
5.3.1	Automation Tooling Overview	110
5.3.2	Django AI Engine Automation	111
5.3.3	Laravel Web Application Automation	120
5.3.4	Automation Test Summary	131
5.4	AI Search & Recommendation Relevance Evaluation	132
5.4.1	Evaluation Environment	132
5.4.2	Methodology — Relevance Scoring	132
5.4.3	Search Relevance (POST /api/search)	133
5.4.4	Recommendation Relevance (POST /api/recommend)	133
5.4.5	Findings and Negative Results	134
5.4.6	Raw Evaluation Data	135
5.5	Chapter Summary	137
6	Performance Evaluation	139
6.1	Introduction & Evaluation Objectives	139
6.2	Testing Environment & Methodology	140
6.2.1	Deployment Environment	140
6.2.2	Dataset Used for Evaluation	140
6.2.3	Tools and Procedures	141
6.3	Semantic Search Performance	141
6.3.1	Response Time by Query Type	142
6.3.2	Relevance Accuracy (Precision@5)	142
6.3.3	Observations	142
6.4	Recommendation Engine Performance	144
6.4.1	Recommendation Latency by User Signal Strength	144
6.4.2	Relevance Accuracy (Precision@5) by User Signal Strength	144
6.4.3	Observations	144
6.5	System Reliability & Load Behavior	145
6.5.1	Load Testing — Concurrent User Simulation	145
6.5.2	Cross-Server Communication Reliability	146
6.5.3	Observations	146
6.6	User Feedback & Satisfaction Ratings	147
6.6.1	Survey Structure	147
6.6.2	Results	148
6.6.3	Respondent breakdown:	148
6.6.4	Observations	148
6.7	Negative Results & Limitations	149
6.7.1	Summary of Identified Limitations	150
6.7.2	Discussion of Root Causes	150
6.7.3	What Was Not Addressed	151
6.8	Discussion of Results	151
6.8.1	Speed and Accuracy Are Both Acceptable for the Target Use Case	151

6.8.2	Subjective and Objective Metrics Reinforce Each Other	152
6.8.3	The Identified Limitations Are Structural, Not Incidental	152
6.8.4	Overall Assessment	152
6.9	Chapter Summary	152
7	Business Model	154
7.1	Market Research	154
7.1.1	Identify a Niche	154
7.1.2	Business Clients	154
7.1.3	Competitor Analysis	155
7.2	Unique Value Proposition (UVP)	156
7.2.1	Defining the UVP	156
7.2.2	What Sets EventBridge AI Apart	157
7.3	Targeted Audience	157
7.3.1	Customer Segmentation	157
7.3.2	User Personas	158
7.4	Monetization Strategy	158
7.4.1	Revenue Streams	158
7.4.2	Sequencing Rationale	159
7.5	Scalability and Internationalization	159
7.5.1	Scalability Plan	159
7.5.2	Internationalization	160
7.6	Marketing and Branding	161
7.6.1	Brand Identity	161
7.6.2	Marketing Channels	161
7.7	Propose Business Plan	162
7.7.1	Business Model Canvas	162
7.7.2	Synthesis	164
7.8	Integration with Final Year Project	164
7.8.1	Project Alignment	164
7.8.2	Implementation Details	165
7.8.3	Market Experience:	166
7.9	Future Road map	166
7.9.1	Future Development	166
7.9.2	Adaptability	167
7.10	Financial Projections	168
7.10.1	Cost Baseline	168
7.10.2	Revenue Projections	168
7.10.3	Break-Even Analysis and ROI	169
7.11	Legal Considerations	170
7.11.1	Intellectual Property	170
7.11.2	Legal Considerations for International Markets	170
7.12	Automation Testing	171
7.12.1	Current Testing Infrastructure	171
7.12.2	Business Rationale for Automation	171
7.12.3	Planned Automation Roadmap	172

8	Future Directions and Conclusion	173
8.1	Introduction	173
8.2	Summary of Key Accomplishments and Learnings	173
8.3	Unaddressed Problems and Limitations	174
8.3.1	Architectural and Operational Limitations	174
8.3.2	AI and Data Limitations	175
8.3.3	Security, Privacy, and Compliance	175
8.3.4	Business and Adoption Limitations	175
8.4	Open Questions	176
8.5	Directions for Future Research and Development	177
8.5.1	Recommendation and Search Improvements	177
8.5.2	Platform Reliability and Operations	177
8.5.3	Security and Data Governance	177
8.5.4	Business and Adoption Research	178
8.6	Concluding Remarks	178
8.7	Introduction and Purpose	179
8.8	Accessing the Platform	179
8.9	Registration and Login	180
8.9.1	Individual Registration and Login	180
8.9.2	Community Registration and Admin Approval	182
8.9.3	Administrator Login	183
8.10	Individual User Guide	184
8.10.1	Exploring Communities and Events	184
8.10.2	Following Communities	185
8.10.3	Saving Events	186
8.10.4	Viewing an Event’s Public Page	187
8.10.5	Registering for Events	188
8.10.6	Submitting Feedback	189
8.10.7	Managing Profile and Interests	190
8.11	Community User Guide	191
8.11.1	Community Dashboard	191
8.11.2	Profile Management	191
8.11.3	Team Management	192
8.11.4	Creating and Managing Events	193
8.11.5	Speaker Management	193
8.11.6	Media Uploads	194
8.11.7	Managing Registrations and Feedback	195
8.11.8	Community Collaboration	196
8.11.9	Managing Followers	196
8.11.10	Notifications	197
8.12	Administrator Guide	198
8.12.1	Approving or Rejecting Communities	198
8.12.2	Managing Users and Communities	199
8.12.3	Event Oversight	200
8.12.4	Reports and Moderation	201
8.12.5	Broadcasting Announcements	202
8.12.6	Audit Logs	202
8.13	Notifications Overview	203

8.14	Troubleshooting and Support	203
8.15	Documentation and Help Center	204
8.16	Conclusion	205
Appendices		179
Appendix A		179
8.17	Introduction	206
8.17.1	Purpose	206
8.17.2	Scope	206
8.17.3	Definitions	208
8.17.4	Acronyms and Abbreviations	208
8.18	Project Planning and Management	209
8.18.1	SWOT Analysis	209
8.18.2	Gantt Chart	211
8.19	Work Breakdown Structure (WBS)	215
8.20	Overall Description	217
8.20.1	Product Perspective	217
8.20.2	Product Functions	218
8.20.3	Community Management	218
8.20.4	Event Management	218
8.20.5	Event Discovery and Interaction	219
8.20.6	Community Collaboration	219
8.20.7	AI-Powered Features	219
8.20.8	Notification System	219
8.20.9	Feedback and Interaction System	219
8.20.10	Administrative Control	220
8.21	Operating Environment	220
8.21.1	Operating Environment Details	220
8.21.2	Constraints and Considerations	220
8.22	Design and Implementation Constraints	221
8.23	Assumptions and Dependencies	221
8.23.1	Assumptions	221
8.23.2	Dependencies	222
8.24	Requirement Identifying Technique	222
8.24.1	Use Case Diagram	223
8.24.2	Use Case Description	223
8.25	Non-Functional Requirements	226
8.25.1	Performance Requirements	226
8.25.2	Safety Requirements	227
8.25.3	Security Requirements	227
8.25.4	Software Quality Attributes	227
8.25.5	Business Rules	227
8.25.6	Interoperability	228
8.25.7	Extensibility	228
8.25.8	Maintainability	228
8.25.9	Portability	228
8.25.10	Reusability	228
8.25.11	Installation	229

8.26 Other Requirements	229
8.26.1 On-line User Documentation and Help System Requirements	229
8.26.2 Purchased Requirements	229
8.26.3 Licensing Requirements	230
8.26.4 Legal, Copyright, and Other Notices	230
8.26.5 Applicable Standards	230
8.26.6 Reports	230
Appendix B	206
8.27 Introduction	231
8.27.1 Purpose	231
8.27.2 Scope	231
8.27.3 Project Background	231
8.27.4 Motivation	231
8.27.5 Project Objective	232
8.28 Design Methodology and Software Process Model	232
8.28.1 Design Methodology	232
8.28.2 Design Pattern	232
8.28.3 Software Process Models	232
8.29 System Overview	232
8.29.1 Architectural Design	232
8.29.2 Process Flow (Functional Requirement)	233
8.30 Project Architecture	233
8.30.1 Web Module	233
8.30.2 Database Module	234
8.30.3 Administration Module	234
8.30.4 Data Dictionary	234
8.31 System Analysis & Design Overview	235
8.31.1 Class Diagram	235
8.31.2 System Sequence Diagram	238
8.31.3 State Transition Diagram	241
Appendix C	231
8.32 Research Paper	242
Appendix D	242
Appendix E	248

References	252
-------------------	------------

List of Figures

3.1	System Block Diagram of EventBridge AI	16
3.2	Modular System Block Diagram of the Laravel Application	17
3.3	Activity Diagram — Event Creation and Publishing Lifecycle	18
3.4	Activity Diagram — Event Collaboration Request Flow	19
3.5	Activity Diagram — User Discovery, Semantic Search, and Event Registration	20
3.6	Activity Diagram — AI Recommendation Generation (Daily Batch Job)	21
3.7	Class Diagram Part 01	23
3.8	Class Diagram Part 01	24
3.9	Class Diagram Part 01	25
3.10	Sequence Diagram (Part 1 of 3)	27
3.11	Sequence Diagram (Part 2 of 3)	28
3.12	Sequence Diagram (Part 3 of 3)	29
3.13	UML State Transition Diagram	30
3.14	Level-0 (Context) Data Flow Diagram — Django AI Engine	31
3.15	Level-1 Data Flow Diagram — Django AI Engine	32
3.16	Level-2 Data Flow Diagram — Process 3.0 (Perform Semantic Search)	32
3.17	Level-2 Data Flow Diagram — Process 6.0 (Generate Recommendations)	33
6.1	Search Response Time and Relevance Accuracy by Query Type	143
6.2	Response Time with Min/Max Range	143
6.3	Recommendation Latency & Relevance Accuracy by User Signal Strength	145
6.4	Response Time and Error Rate vs Concurrent Users	147
6.5	Beta Tester Satisfaction Ratings by Dimension	149
6.6	Recommendation Precision Across User Signal Strength (Cold-Start Illustration)	151
7.1	EventBridge AI — Business Model Canvas	164
8.1	EventBridge AI Landing Page	180
8.2	Pathway Selection Screen	180
8.3	Individual User Login Screen	181
8.4	Individual User Account Registration Screen	181
8.5	Community Login Screen	182
8.6	Community Registration Screen	183
8.7	Admin Login Screen	183
8.8	User Home Page	184
8.9	Explore Events	185
8.10	Community Public Profile Page	186
8.11	Explore Communities	186
8.12	Saved Events	187
8.13	Event View Page	188

8.14	Event Registration Page	189
8.15	User Registrations	189
8.16	Event Feedback Section	190
8.17	User Profile Page	190
8.18	Community Dashboard	191
8.19	Community Profile Management	192
8.20	Community Team Management	192
8.21	Events Management Dashboard	193
8.22	Event Speakers Management	194
8.23	Event Media Management	194
8.24	Manage Event Registrations and Attendance	195
8.25	Manage Event Feedback	195
8.26	Collaboration Management Dashboard	196
8.27	Manage Community Followers	197
8.28	Community Dashboard - Notifications Screen	197
8.29	Admin Dashboard	198
8.30	Manage Communities	199
8.31	View Community Details and Manage Status	199
8.32	Users Management	200
8.33	Events Oversight	200
8.34	Manage Bugs Reporting	201
8.35	Manage Events and Platform Feedback	201
8.36	Notifications and Broadcasting Panel	202
8.37	Admin Audit Logs	202
8.38	Contact Us Page	203
8.39	Report Bug Page	204
8.40	Platform Documentation	204
8.41	Help Center and FAQ Page	205
8.42	EventBridge AI - Gantt Chart	211
8.43	Work Breakdown Structure Diagram	216
8.44	Use Case Diagram	223
8.45	Class Diagram Part 01	235
8.46	Class Diagram Part 01	236
8.47	Class Diagram Part 01	237
8.48	Sequence Diagram (Part 1 of 3)	238
8.49	Sequence Diagram (Part 2 of 3)	239
8.50	Sequence Diagram (Part 3 of 3)	240
8.51	UML State Transition Diagram	241
8.52	EventBridge AI IEEE Research Paper P1	243
8.53	EventBridge AI IEEE Research Paper P2	244
8.54	EventBridge AI IEEE Research Paper P3	245
8.55	EventBridge AI IEEE Research Paper P4	246
8.56	EventBridge AI IEEE Research Paper P5	247
8.57	Wall Banner	248
8.58	Brochure Inside	249
8.59	Brochure Outside	250
8.60	Standee	251

List of Tables

3.1	Data storage architecture overview	35
3.2	admin table schema	36
3.3	community_followers table schema	36
3.4	community_profile table schema	37
3.5	community_status table schema	38
3.6	community_team_members table schema	38
3.7	community_team_structure table schema	39
3.8	community_team_tenure table schema	39
3.9	community_user table schema	40
3.10	event_collaborations table schema	40
3.11	event_details table schema	41
3.12	event_feedback table schema	42
3.13	event_media table schema	42
3.14	event_registration_attendance table schema	43
3.15	event_speakers table schema	44
3.16	users table schema	44
3.17	users_profile table schema	45
3.18	Django AI engine function reference	47
4.1	Technology stack overview	50
4.2	System requirements	51
5.1	Test case UT-01 — Login password verification	63
5.2	Test case UT-01 — Execution results	64
5.3	Test case UT-02 — Login throttle enforcement	65
5.4	Test case UT-02 — Execution results	66
5.5	Test case UT-03 — Guard-based access isolation	67
5.6	Test case UT-03 — Execution results	68
5.7	Test case UT-04 — Event creation field and date validation	69
5.8	Test case UT-04 — Execution results	70
5.9	Test case UT-05 — Event status transition validation	71
5.10	Test case UT-05 — Execution results	72
5.11	Test case UT-06 — Event edit ownership enforcement	73
5.12	Test case UT-06 — Execution results	74
5.13	Test case UT-07 — Duplicate event registration prevention	75
5.14	Test case UT-07 — Execution results	76
5.15	Test case UT-08 — Registration deadline enforcement	77
5.16	Test case UT-08 — Execution results	78
5.17	Test case UT-09 — Event capacity and waitlisting	79

5.18	Test case UT-09 — Execution results	80
5.19	Test case UT-10 — Community collaboration request validation	81
5.20	Test case UT-10 — Execution results	82
5.21	Test case UT-11 — Collaboration request status update authorization	83
5.22	Test case UT-11 — Execution results	84
5.23	Test case UT-12 — Community follow/unfollow and duplicate prevention	85
5.24	Test case UT-12 — Execution results	86
5.25	Test case UT-13 — Admin community status transition validation	87
5.26	Test case UT-13 — Execution results	88
5.27	Test case UT-14 — Admin user status change and listing filters	89
5.28	Test case UT-14 — Execution results	90
5.29	Test case UT-15 — Admin-triggered password reset	91
5.30	Test case UT-15 — Execution results	92
5.31	Test case UT-16 — Registration notification dispatch and read state	93
5.32	Test case UT-16 — Execution results	94
5.33	Test case UT-17 — API key middleware enforcement	95
5.34	Test case UT-17 — Execution results	96
5.35	Test case UT-18 — Event text formatting for embeddings	97
5.36	Test case UT-18 — Execution results	98
5.37	Test case UT-19 — Event indexing and vector upsert	99
5.38	Test case UT-19 — Execution results	100
5.39	Test case UT-20 — Recommendation embedding text formatting	101
5.40	Test case UT-20 — Execution results	102
5.41	Test case UT-21 — Recommendation request validation	103
5.42	Test case UT-21 — Execution results	104
5.43	Test case UT-22 — Recommendation score rounding and ordering	105
5.44	Test case UT-22 — Execution results	106
5.45	Integration testing — Cross-module and cross-service flows	109
5.46	Django AI engine automated testing results	119
5.47	Laravel automated testing results	129
5.48	Search Relevance (POST /api/search)	133
5.49	Recommendation Relevance (POST /api/recommend)	133
6.1	Evaluation objectives	140
6.2	Test dataset parameters	141
6.3	Tools and Procedures	141
6.4	Response time by query type	142
6.5	Precision@5 by query type	142
6.6	Recommendation latency by user signal tier	144
6.7	Precision@5 by user signal tier	144
6.8	Concurrent load test results	146
6.9	Cross-server communication reliability	146
6.10	Beta survey rating dimensions	147
6.11	Beta survey results	148
6.12	Beta respondent breakdown	148
6.13	System limitations	150
7.1	Competitive landscape overview	155
7.2	SWOT analysis — Eventbrite	156

7.3	SWOT analysis — Facebook Events	156
7.4	EventBridge AI — Business Model Canvas	163
7.5	Development investment (sunk cost)	168
7.6	Monthly operating costs at current beta scale	168
7.7	Three-year revenue projection (illustrative)	169

Chapter 1

Introduction

1.1 Overview

Karachi's tech and academic ecosystem has grown rapidly over the past few years. Student chapters, professional networks, and independent tech communities now run a steady stream of workshops, hackathons, tech talks, and networking sessions throughout the city, often within days or even kilometres of one another. On paper, this should mean more opportunities for people to learn, connect, and grow within their fields. In practice, most of these opportunities go unnoticed by the very people who would benefit the most from them.

The reason is simple: there is no shared way to keep track of any of it. Event details live scattered across WhatsApp groups capped at a few hundred members, Instagram stories that disappear within a day, and posts on LinkedIn and Facebook that get buried under algorithmic feeds within hours of being published. Each community manages its own outreach in isolation, optimized for convenience rather than for actually being found.

This creates a quiet but persistent problem. People who would genuinely want to attend an event often never hear about it in time. Organizers, meanwhile, struggle to reach anyone beyond their own immediate circle, even when real interest exists just a few streets away. The result is a city full of activity that nobody can fully see, good events going under-attended, not for lack of interest, but for lack of visibility.

This gap between activity and awareness is the starting point for this project. It is not a problem of too few events, but of too little coordination between the people organizing them and the people looking for them. Addressing it requires more than another listing page; it requires a system that can actively surface the right event to the right person.

The sections that follow examine this problem in more detail, define the objectives this project sets out to achieve, outline the core features built to meet them, and establish the scope within which the system has been designed and developed.

1.2 Problem Statement

Karachi's tech and academic communities post their events across a scattered mix of WhatsApp groups, Instagram stories, and social media feeds, with no shared index connecting any of it. Interested individuals miss events they would have happily attended, and organizers

struggle to reach anyone beyond their own existing circle.

This gap was observed firsthand while helping run community events, where genuinely good content consistently underperformed, not from lack of interest, but because the right audience never saw it in time. EventBridge AI was built to close that exact gap with one centralized, intelligently searchable platform for the city's community ecosystem.

General-purpose tools like Facebook Events or Eventbrite exist, but they were not built for this problem. They offer no semantic search, no inter-community collaboration, and no discovery layer tuned to Karachi's specific community network. No equivalent platform currently serves this ecosystem as a dedicated, AI-driven hub.

Rebuilding core ideas already proven elsewhere: search, registration, recommendations, was a deliberate choice. It forced direct, hands-on engineering work with vector embeddings, semantic search, and a multi-server architecture, instead of learning these concepts only in theory.

1.3 Objectives

The development of EventBridge AI was guided by a set of deliberately measurable objectives. Each objective below specifies a concrete deliverable against which the project's success can be evaluated, accompanied by a brief explanation of its significance within the broader system vision.

1.3.1 Build a Centralized Hub for Event Discovery

The first and most direct objective is to replace the scattered, channel-by-channel way events currently get published with a single, structured platform. Every event, regardless of which community hosts it, should live in one indexed, searchable place, so that discovering an event no longer depends on which WhatsApp group or Instagram account someone happens to follow.

1.3.2 Apply Semantic Search to a Real-World Discovery Problem

A core technical objective was to move beyond keyword-based search and apply semantic search using vector embeddings, so the system can match a query like "AI workshops this weekend" to relevant events even when the wording doesn't overlap. This was a deliberate choice to work on a problem that combines a genuine product need with hands-on application of an AI technique, rather than treating AI as decoration on top of a conventional CRUD application.

1.3.3 Personalize Discovery Based on User Behavior

Beyond search, the project set out to build a recommendation system that learns from what a person follows, registers for, and searches so that the platform actively surfaces relevant events rather than expecting every individual to search for everything themselves. The objective here was to prove that even a relatively lightweight, content-based approach to personalization can meaningfully improve what people see versus a plain chronological feed.

1.3.4 Enable Structured Collaboration Between Communities

A further objective was to give communities a formal way to co-host events with each other, sending, accepting, or declining collaboration requests through the platform itself, instead of relying on informal messages between individual organizers. This was meant to address a part of the problem that pure event-listing platforms typically ignore entirely.

1.3.5 Validate the Idea with Real Users, Not Just a Demo

Rather than treating this as a project that ends at submission, an explicit objective was to deploy EventBridge AI to real community networks during the beta phase and collect actual usage data, registrations, search behavior, and feedback, rather than relying on assumptions about whether the platform would be useful. This was important precisely because the problem itself came from firsthand observation of community events underperforming, and the only honest way to know if the platform helps is to test it with the people who experience the problem.

1.3.6 Develop Practical Experience with a Multi-Service AI Architecture

On a personal and team level, an underlying objective of this project was to gain direct experience designing and connecting two independently running services: a Laravel web application and a separate Python/Django AI engine, rather than building everything inside a single monolithic codebase. Working through the integration challenges this creates (authentication between services, data consistency, deployment coordination) was treated as a learning objective in its own right, since this kind of architecture is common in real-world systems but rarely practiced in coursework.

By the end of this report, a reader should understand that EventBridge AI was not built to demonstrate AI for its own sake, but to address a specific, observed gap in how Karachi's tech and academic communities discover and coordinate events, and that the project was deliberately scoped to test that idea against real usage rather than stopping at a working prototype.

1.4 System Features

EventBridge AI is built around a limited set of core features rather than a long list of disconnected functionality, each one mapped directly to a part of the problem described in 1.2. The features below are presented in order of importance, starting with what makes events discoverable in the first place, followed by what makes that discovery intelligent, and ending with the platform and governance layer that keeps the system usable at scale.

1.4.1 Centralized Event Community Registry

At the foundation of the platform is a single, structured registry where every event, regardless of which community hosts it, is created, published, and indexed in one place. Communities manage the full event lifecycle (draft, published, ongoing, completed, or cancelled) directly on the platform, replacing the fragmented mix of WhatsApp groups and disappearing Instagram stories described in the Problem Statement. This is the feature every other feature in the system

depends on; without a centralized registry, there is nothing for search, recommendations, or analytics to operate on.

1.4.2 AI-Powered Semantic Search

Rather than matching keywords, the platform converts event descriptions into vector embeddings using OpenAI's embedding models and stores them in Qdrant, a dedicated vector database. A search query is embedded using the same model and matched against stored event vectors by similarity, allowing a query like "hands-on Python workshop for beginners" to surface relevant results even when the exact wording doesn't appear in the event description. Top candidates are then re-ranked using a language model before being returned, improving result ordering beyond raw similarity scores.

1.4.3 Personalized Event Recommendations

The platform builds a preference profile for each individual from their declared interests, registration history, saved events, and recent searches, embeds that profile, and matches it against event embeddings to surface relevant recommendations on their home feed. This is a content-based approach; it learns from a person's own activity rather than comparing them to other users, but it directly addresses the personalization gap identified in the Problem Statement, where individuals previously had no way to filter listings beyond a flat chronological feed.

1.4.4 Event Registration, Attendance Feedback

Individuals can register for events directly through the platform, either through a built-in registration form or an external link, with support for participant limits and waitlisting where applicable. Communities can track attendance per event and collect post-event feedback and ratings, giving organizers a closed feedback loop that did not previously exist when promotion happened entirely through external social channels.

1.4.5 Inter-Community Collaboration

Communities can send, accept, or decline collaboration requests to co-host events with other communities, with the request and its status tracked directly on the platform. This formalizes a process that previously depended entirely on informal messages between individual organizers, with no record once a chapter lead moved on or lost contact.

1.4.6 Explore Follow

Individuals can browse communities and events through dedicated explore and discovery pages, follow communities they're interested in, and save events for later. This gives individuals a way to build an ongoing relationship with specific communities, rather than rediscovering them from scratch every time a new event is posted.

1.4.7 Notifications

The platform sends notifications for key actions, new followers, collaboration requests, event updates, and registration confirmations, keeping both individuals and communities informed without requiring them to actively check the platform.

1.4.8 Administrative Governance

An admin panel oversees the platform's integrity: reviewing and approving new community registrations, moderating flagged events and feedback, and maintaining an audit trail of administrative actions. This governance layer is what allows the registry in 1.4.1 to stay trustworthy as more communities join the platform.

Together, these features form a chain rather than a checklist: the centralized registry makes events discoverable, semantic search and recommendations make that discovery intelligent, registration and collaboration features turn discovery into actual participation and partnership, and the administrative layer keeps the entire system trustworthy as it scales. The next section defines the boundaries within which this system was designed and built.

1.5 Project Scope

EventBridge AI is a web-based platform that centralizes event discovery and community collaboration for Karachi's tech and academic ecosystem. It allows communities to publish and manage events, individuals to discover and register for them through semantic search and AI-driven recommendations, and administrators to govern the platform as it grows. The system is built as two cooperating services: a Laravel web application handling core platform logic, and a Django-based AI engine handling search and recommendations, and is currently live in beta at eventbridgeai.com.

1.5.1 Boundaries of the System

The scope of this project covers the full event lifecycle: community onboarding and verification, event creation and publishing, individual registration and attendance tracking, semantic search, personalized recommendations, inter-community collaboration, and administrative oversight. It is currently explicitly scoped to Karachi's community ecosystem rather than a multi-city rollout. The current beta is accessible only on desktop and laptop browsers; a dedicated mobile experience is outside the current scope. Payment processing, ticketing for paid events, and native mobile applications are also outside this scope and are treated as future work.

1.5.2 Business Need and Expected Outcome

The underlying business need is the absence of any shared infrastructure for event discovery in Karachi's community ecosystem, despite a steady volume of community-run activity. The expected outcome is a measurable improvement in two things: how easily individuals find events relevant to them, and how far a community's promotional effort actually reaches beyond its own existing audience, both validated through real beta usage rather than assumed.

1.5.3 Constraints

The project was built and deployed by a three-person student team with just one handling the core development process within an academic timeline, which limited the scope to a single, well-defined feature set rather than an exhaustive one. Infrastructure is similarly limited: the platform runs on a single VPS for the AI engine and shared hosting for the web application, rather than distributed or auto-scaling infrastructure. The AI features depend entirely on

a third-party provider (OpenAI) for embeddings and language model calls, which introduces external cost and availability as constraints outside the team's control.

1.5.4 Assumptions

The project assumes that communities are willing to actively publish and manage events on a new platform rather than relying solely on their existing social channels, and that individuals participating in the beta have consistent access to a desktop or laptop with a stable internet connection. It also assumes continued access to OpenAI's API at a stable cost and continued availability of the hosting and VPS infrastructure for the duration of testing and evaluation.

1.5.5 Business Processes Impacted

This project directly changes how communities promote events, shifting from manually posting across WhatsApp, Instagram, LinkedIn, Facebook, etc., to publishing once on a centralized platform. It also changes how individuals discover events, moving from passively encountering announcements through personal social circles to actively searching and receiving personalized recommendations. For administrators, it introduces a formal community verification and moderation process that did not exist before, since there was previously no single point through which communities needed to be approved or governed.

1.5.6 Internal and External Interfacing Entities

Internally, the system interfaces between its own two services, the Laravel web application and the Django AI engine, which communicate over an internal API. Externally, the platform interfaces with OpenAI's API for embedding generation and language model re-ranking, with Qdrant as the vector database self-hosted on the VPS, backing semantic search and recommendations, and with GoDaddy and OVH Cloud as domain and hosting providers for the web application and AI engine, respectively. The primary external stakeholders are the participating communities (currently including GDGoC SSUET, IEEE SSUET, and ACM SSUET chapters in Karachi) and the individuals using the platform during beta testing.

1.5.7 Target Users / Audience

The platform is designed around three distinct actor types. Communities are the organizing entities, university chapters, professional networks, and independent collectives such as GDGoC, IEEE, and ACM chapters that create and manage events. Individuals are anyone in Karachi's tech and academic ecosystem looking to discover and attend events; the platform does not restrict this to university students, since the problem it addresses (scattered visibility) affects working professionals and independent learners just as much. Administrators are the internal governance layer responsible for verifying communities and moderating platform content.

1.5.8 In-Scope vs. Out-of-Scope Features

In scope:

- Community registration, verification, and profile management
- Full event lifecycle management (draft, published, ongoing, completed, cancelled)

- Individual registration, attendance tracking, and post-event feedback
- Semantic search using vector embeddings
- Personalized, content-based event recommendations
- Inter-community collaboration requests (send, accept, decline, track status)
- In-platform notifications for key actions (registrations, collaboration requests, event updates)
- Administrative governance: community approval, moderation, audit logging
- Desktop and laptop web access

Out of scope:

- Native mobile applications (iOS/Android)
- Payment processing and paid-event ticketing
- Expansion beyond Karachi to other cities
- Real-time chat or direct messaging between individuals and communities
- Push or SMS notification delivery
- Third-party calendar integrations (Google Calendar sync, etc.)

1.5.9 Success Criteria

The scope of this project will be considered successfully met if the platform, during the beta testing phase, demonstrates: acceptable semantic search response times under real usage, reasonable accuracy in search relevance and recommendation quality as judged against user expectations, a working end-to-end flow for all in-scope features without critical failures across unit and integration testing, and measurable engagement from participating communities and individuals reflected in feedback ratings collected through the platform itself.

1.6 SDG Alignment

EventBridge AI's design connects to three of the United Nations Sustainable Development Goals identified at the project's proposal stage.

1.6.1 SDG 4 — Quality Education

By centralizing discovery of workshops, hackathons, and technical sessions across Karachi's community ecosystem, the platform reduces a real barrier to informal technical education: simply not knowing an opportunity exists. Semantic search and personalized recommendations make it easier for individuals to find learning opportunities matched to their actual interests, rather than relying on whichever events happen to surface through their existing social circle.

1.6.2 SDG 9 — Industry, Innovation and Infrastructure

The platform itself functions as a piece of digital infrastructure that did not previously exist for this ecosystem, a structured, AI-driven layer connecting communities and individuals where only fragmented social channels existed before. Its multi-service architecture, combining a Laravel web application with a dedicated Django AI engine, reflects the kind of system design increasingly expected in modern software infrastructure.

1.6.3 SDG 17 — Partnerships for the Goals

The inter-community collaboration feature directly supports this goal at a local scale, giving organizations a structured way to formally partner, co-host events, and pool resources and audiences, replacing the informal, personality-dependent collaboration that previously left institutional partnerships fragile and short-lived.

These alignments reflect the platform’s actual built capabilities rather than aspirational framing, and are explored in greater depth in the project’s dedicated SDG report.

1.7 Chapter Summary

This chapter has laid out the foundation for EventBridge AI, starting from the gap that motivated it: Karachi’s tech and academic communities run a steady volume of events, but no shared infrastructure exists to make them discoverable beyond each community’s own immediate circle. The Problem Statement traced this back to a simple cause: scattered, siloed promotion and established that existing general-purpose tools like Facebook Events or Eventbrite were never built to solve it for this specific ecosystem.

From there, the chapter defined what the project set out to accomplish: building a centralized event registry, applying semantic search and content-based recommendations to make discovery genuinely intelligent, formalizing inter-community collaboration, and validating all of it against real users rather than stopping at a working prototype. System Features translated those objectives into the platform’s actual building blocks, ordered by how directly each one addresses the original problem from the centralized registry at the foundation, through search and recommendations, to the governance layer that keeps the system trustworthy as it grows.

Finally, Project Scope drew a clear boundary around what this project does and does not cover: a Karachi-scoped, desktop-accessible platform built within the constraints of a student team and academic timeline, with mobile apps, payments, and multi-city expansion deliberately left for future work, and defined what success looks like during beta testing.

Together, these sections frame EventBridge AI not as a speculative AI showcase, but as a deliberately scoped response to an observed, specific problem. The next chapter moves from what this project sets out to do to how it was designed, covering the methodology, design patterns, and software process model behind its development.

Chapter 2

Literature Review

2.1 Introduction

Event discovery and community collaboration sit at the intersection of several well-established research areas: information retrieval, recommender systems, and computational semantics. Before describing EventBridge AI’s own design choices, it is necessary to survey how existing platforms and academic work have approached these problems, what techniques they rely on, and where their limitations create the opening that this project addresses. This chapter reviews four bodies of work — general-purpose event discovery platforms, recommender system approaches, semantic search and embedding-based retrieval techniques, and community collaboration tools — before narrowing to the specific context of Karachi’s tech and academic ecosystem and positioning EventBridge AI relative to all of the above.

2.2 Event Discovery and Management Platforms

Commercial event platforms such as Eventbrite, Meetup, and Luma dominate the general market for event listing and ticketing. These platforms are built primarily as transactional and logistical tools: they let organizers create listings, sell tickets, and manage RSVPs, and they surface events to users mainly through keyword search, category browsing, or geographic proximity. Academic interest in this space has tended to focus less on the platforms themselves and more on the recommendation problem layered on top of them — several studies use Meetup’s public dataset as a benchmark precisely because Meetup’s own discovery mechanism is largely non-personalized. A 2017 study built and evaluated a recommendation method against a Meetup dataset of 100,000 users, treating the platform’s native browsing experience as the baseline to be improved on, testing the proposed method against a dataset acquired from Meetup and reporting successful results. This pattern — using commercial platforms as a data source rather than as a model to emulate — reflects a broader gap: general event platforms are built for breadth (any event, anywhere) rather than for depth within a specific community or ecosystem, and they do not model relationships between organizing communities at all. University- and community-specific event tools, where they exist, are typically simple calendar or form-based listings without any discovery intelligence layered on top. This is the specific gap EventBridge AI positions itself against: a platform scoped to a single city’s tech and academic ecosystem, where discovery is powered by semantic matching rather than manual filtering, and where the organizing communities themselves — not just individual events — are first-class, followable, collaborating entities

2.3 Recommendation System Approaches

Recommender systems are broadly grouped into three families: collaborative filtering, content-based filtering, and hybrid approaches that combine the two, with content-based filtering (CBF) recommending items by analyzing their attributes and comparing them against a user's past interactions, typically constructing a user profile from prior interactions and aligning it with detailed item descriptions. Collaborative filtering (CF), by contrast, predicts a user's interest based on the behavior of similar users, and does not require any analysis of the item's content itself, in contrast to content-based filtering, which looks only at the contents of an artifact such as the words on a page. Each has been applied to event recommendation specifically: a fuzzy relational approach to event recommendation and an approach using linked data to model events in social networks have both been proposed in the literature, alongside collaborative recommendation methods that use content-based filters for cultural activities distributed via a scalable event platform. More recent work has applied deep learning to this problem directly — a multilayered perceptron and neural collaborative filtering model was proposed for event recommendation and evaluated against a Meetup dataset, and reported to outperform other techniques tested on the same benchmark.

The central practical distinction between these families is how they behave when data is sparse. Collaborative filtering suffers acutely from the "cold-start" problem: a system cannot compute meaningful user-user or item-item similarity when a new user or new item has no interaction history, since each new item or user introduces an issue where the system is unable to quickly construct the model it needs. Content-based filtering is comparatively resilient to this, because it can generate a recommendation from an item's own attributes even in the complete absence of other users' behavior, addressing the cold-start problem effectively by recommending items based on their own features rather than requiring prior interaction history. This resilience comes at a cost: content-based systems tend to over-recommend items similar to what a user has already engaged with, producing narrower, less diverse result sets than collaborative or hybrid approaches typically achieve, since CBF recommends items based purely on content similarity, which tends to produce less diverse recommendations. Hybrid systems attempt to capture the best of both — using content similarity to bootstrap new users or items, and collaborative signals to sharpen relevance once enough interaction data accumulates, for instance by using content-based filtering for initial recommendations and then applying collaborative filtering to fine-tune them.

For a platform in its beta phase, serving a growing but still comparatively small pool of communities and users, the cold-start problem is not a theoretical concern — it is close to the default state of the system for a majority of events and new users at any given time. This is the primary justification, grounded in the literature above, for EventBridge AI's choice of a purely content-based, nearest-neighbor recommendation approach: user and event profiles are represented as single dense embeddings and matched by cosine similarity, without a collaborative signal or a separate user-user similarity component. This is a deliberate simplification relative to the hybrid architectures reviewed above, made explicitly to avoid the cold-start failure mode that would otherwise dominate a young platform's recommendation quality; the trade-off, per the literature, is a corresponding reduction in recommendation diversity, which is acknowledged here rather than treated as an oversight.

2.4 Semantic Search, Embeddings, and Vector Retrieval

The AI-driven discovery layer in EventBridge AI depends on a chain of techniques that has matured rapidly over the past several years: dense text embeddings, vector similarity search, and (more recently) large language model orchestration around both. Early distributional word representations such as word2vec and GloVe encoded the intuition that words appearing in similar contexts carry similar meaning, exploiting the distributional hypothesis that similar words occur in similar contexts, but these word-level representations were eventually superseded by transformer-based contextual models. BERT and its successors set new benchmarks on sentence-pair semantic similarity tasks, but the original BERT architecture is computationally unsuited to large-scale similarity search, since comparing a single sentence against a corpus of 10,000 others required tens of millions of pairwise inference passes with BERT, an operation that could take around 65 hours to complete for a modest-sized collection. Sentence-BERT (SBERT) addressed this directly by fine-tuning BERT with a siamese network structure to produce fixed-length sentence embeddings that can be compared with simple, cheap operations like cosine similarity rather than requiring both sentences to be jointly fed through the full network. This shift — from pairwise inference to independently-computed, directly-comparable embeddings — is what makes embedding-based semantic search practical at scale, and it is the same underlying principle EventBridge AI relies on, using OpenAI’s text-embedding-3-large model to independently embed both event descriptions and user profiles into a shared 3072-dimensional space.

Once text is embedded, it must be stored and queried efficiently. Traditional relational databases are built for exact-match queries on structured fields and are not suited to similarity search over high-dimensional vectors, since a normal database matches exact text and cannot return conceptually related results the way vector similarity search can. Purpose-built vector databases fill this role: Qdrant, for example, is an open-source vector similarity search engine that provides a production-ready service for storing, searching, and managing vectors together with an arbitrary metadata payload, and is written in Rust specifically for speed and reliability under high load. In this model, semantic similarity is computed geometrically — items whose embeddings are close together in vector space are semantically related, and cosine similarity is a standard, computationally cheap way to measure that closeness, since in embedding space distance approximates semantic difference and closeness approximates semantic similarity. This is precisely the mechanism EventBridge AI’s search and recommendation engine implements: event and user text is embedded once, stored as a point in a Qdrant collection with cosine distance configured, and queried via nearest-neighbor lookup at request time.

A separate, more recent strand of the literature concerns LLM orchestration frameworks such as LangChain, which provide standardized abstractions — document loaders, retrievers, vector store connectors, and prompt-chaining utilities — for building retrieval-augmented generation (RAG) pipelines on top of an embedding and vector-search foundation, connecting document loaders, vector stores, retrievers, and LLMs into a single pipeline, and supporting more than sixty vector database connectors. LangChain’s retriever interface, in particular, is designed to expose a standard method for connecting to different data stores so that retrieval logic can be swapped independently of the underlying database, allowing all retrievers to be invoked in a uniform way regardless of the specific implementation behind them. This class of framework is reviewed here because it represents the direction the broader field has moved in for orchestrating multi-step AI pipelines; it is included in this literature review as a technique surveyed

for its relevance to the problem space, and its actual role — or absence — in EventBridge AI’s current implementation is addressed explicitly in the system design chapters that follow.

2.5 Community Collaboration and Networking Platforms

Beyond individual event discovery, EventBridge AI’s second core premise is that communities themselves — not just the events they host — benefit from being modeled as connected, followable, collaborating entities. Existing tools for this purpose fall into two categories: general-purpose community platforms (Slack, Discord, LinkedIn Groups) and developer- or research-oriented technical communities. The developer community model, as implemented by platforms like Stack Overflow, GitHub Discussions, and Atlassian Community, has been shown to reduce support costs while accelerating knowledge-sharing and product feedback loops within a technical audience, demonstrating that well-managed technical communities can meaningfully reduce support costs while also accelerating innovation cycles. In academic and professional networking specifically, informal case-study research on cross-institutional professional communities has found that structured, technology-mediated networking — thematic meetings, deliberate time management, and clear opportunities for informal collaboration — was central to their success, with successful engagement built around a focus on thematic meetings, time management, and structured participation. However, a separate study of researchers’ use of online communities for collaboration found that while digital community tools are broadly accepted by both established and newer professionals, they still remain secondary to more established channels of communication and dissemination within their respective fields, with journals confirmed as the primary way of disseminating research despite growing comfort with digital community tools among younger researchers. None of these platforms, however, are designed around the specific unit that matters to EventBridge AI’s use case: two or more independent organizing communities jointly producing and co-owning a single event. General social and professional networks support individual-to-individual or individual-to-organization relationships, not organization-to-organization collaboration workflows layered on top of a shared event lifecycle — which is the specific gap EventBridge AI’s community collaboration module (collaboration requests, shared event ownership, and joint event management) is built to close.

2.6 The Karachi Tech and Academic Ecosystem Context

EventBridge AI is deliberately scoped to Karachi rather than to Pakistan broadly, and this scoping decision is grounded in the specific structure of the city’s tech and academic ecosystem. Karachi is Pakistan’s largest city and its financial and economic hub, home to a dense concentration of universities, tech incubators such as The Nest I/O, and a growing base of active developer and startup communities, with more than twenty-five universities feeding a steady flow of graduates into a local ecosystem supported by incubators and accelerators. Community-run technical meetups and conferences — events run by developer chapters and student-led organizations, in the same category as the GDGoC, IEEE, and ACM chapters this project draws its beta users from — are consistently identified as a primary vehicle for networking, skill-building, and career growth within this ecosystem, with recurring, community-run gatherings functioning as a central and ongoing part of how professionals in the city network

and develop, rather than as one-off occurrences. At the same time, national-level analysis of Pakistan’s broader startup and innovation ecosystem has found that support infrastructure — including event and networking infrastructure — remains concentrated in a small number of cities and lacks the kind of centralized, technology-driven coordination that more mature startup ecosystems have developed , noting that support organizations need to better tailor their services to the specific needs of startups and to the local context in which they operate, rather than defaulting to generic models. Existing coverage of Karachi’s event landscape reinforces this fragmentation: event discovery in the city currently happens through scattered channels — WhatsApp groups, individual community social media accounts, and word-of-mouth , rather than through any centralized platform, with community members typically pointed toward informal WhatsApp groups to hear about opportunities before they are publicly listed. This fragmented, informal discovery landscape — highly active communities, but no shared, semantically-searchable, cross-community discovery layer connecting them — is the concrete, local version of the general-purpose-platform gap identified in Section 2.2, and it is the specific market condition EventBridge AI was designed against.

2.7 Research Gap and Positioning

Drawing the four preceding sections together: general event platforms (2.2) provide broad transactional coverage but no semantic discovery and no cross-community collaboration model; the recommender systems literature (2.3) establishes that content-based, embedding-driven filtering is the correct choice for a platform still accumulating interaction data, at an explicit, literature-documented cost to recommendation diversity; the semantic search and embeddings literature (2.4) establishes the specific technical mechanism — dense embeddings compared by cosine similarity in a purpose-built vector database — that makes natural-language event discovery computationally practical; and the community platforms literature (2.5), read alongside the local ecosystem context (2.6), shows that neither general social platforms nor Karachi’s current informal, fragmented discovery channels support organization-to-organization collaboration around shared events. EventBridge AI’s design sits at the intersection of these four gaps: it applies embedding-based semantic search and content-based nearest-neighbour recommendation — techniques validated in the broader literature reviewed above — to the specific, currently unaddressed problem of city-scoped, cross-community event discovery and collaboration within Karachi’s tech and academic ecosystem.

Chapter 3

Design

3.1 Design Methodology and Software Process Model

EventBridge AI’s two backend services are architected using different design paradigms, chosen to match the nature of the work each service does rather than applied uniformly for consistency’s sake.

The Laravel web application follows an Object-Oriented Design (OOD) approach, which is both idiomatic to the framework and appropriate to the domain it models. The core entities of the platform — users, communities, events, registrations, collaborations, notifications — are naturally modeled as objects with distinct attributes and behaviors, and Laravel’s Eloquent ORM maps each of these directly onto a PHP class. Relationships between entities are expressed explicitly through object associations (`belongsTo`, `hasMany`, `hasOne`) rather than through raw foreign-key handling in procedural code — for example, `EventDetails` aggregates `EventSpeakers`, `EventCollaborations`, `EventMedia`, and `EventRegistrations` as dependent child objects, and `CommunityUser` composes a `CommunityProfile`, `CommunityStatus`, and `CommunityTeamStructure`. This object-oriented structure keeps entity behavior (such as slug generation, seat-count calculation, or dismissed-action checks) encapsulated on the model itself rather than scattered across controllers, which is the standard justification for OOD in a system with this many interrelated, stateful entities.

The Django AI engine, by contrast, is designed as a function-oriented / service-based system rather than an object model. It has no meaningful class hierarchy: its logic is organized as a small set of stateless service modules — an embedding service, a Qdrant indexing/query service, and a re-ranking service — that transform input into output through a linear pipeline (`build text` → `embed` → `store/query` → `filter` → `return`). This is an appropriate and deliberate choice rather than a gap: the AI engine’s job is fundamentally a data-transformation pipeline with no persistent object state of its own, so forcing an object model onto it would add structure without benefit. This hybrid split — OOD where the domain is naturally object-shaped, function-oriented design where it’s naturally a pipeline — is reflected in Section 3.4, where the Laravel side is represented with a class diagram and the Django side with a data flow diagram.

At the process level, EventBridge AI was built using an Agile, incremental development model. Rather than following the SRS/proposal phases in a single linear pass, the team built and shipped the platform module by module — authentication, then community and event management, then the AI search and recommendation layer — with each module becoming usable

before the next was started. This was reinforced by the beta-testing phase itself: real users from GDGoC, IEEE, and ACM chapters have been using the live platform and providing feedback throughout, which is characteristic of an incremental, feedback-driven process rather than a waterfall model where all requirements would need to be frozen before implementation began. Agile/Incremental was therefore chosen both because it matches how the project was actually executed and because it suited a small student team validating a two-sided platform (communities and users) against a real, active user base rather than against a static specification.

3.2 Architectural design / Design Patterns

EventBridge AI follows a layered, multi-tiered client-server architecture, with the AI functionality separated out as an independent service tier rather than embedded inside the main web application. Concretely, the system is composed of four cooperating tiers:

1. **Presentation tier** — the browser-rendered Laravel Blade frontend the user/community interacts with.
2. **Application tier** — the Laravel backend: controllers, services, and background jobs that implement business logic (auth, community/event management, collaboration, notifications).
3. **AI service tier** — the Django engine, a separately deployed service responsible only for embedding generation, semantic search, and recommendation scoring.
4. **Data tier** — split across two purpose-fit stores: MySQL for all relational/transactional data, and Qdrant for vector embeddings.

This is a deliberate architectural decision, not an incidental one, and it's worth justifying explicitly in the report: the AI engine could have been built as a module inside the Laravel app (e.g. via a PHP ML library or by shelling out to Python scripts), but separating it into its own service tier gives three concrete benefits that match what was actually built — (1) it lets the AI engine run Python's ML/embedding tooling natively rather than working around PHP's limitations, (2) it isolates failure: `AIRecService::recommend()` catches HTTP exceptions and falls back to an empty result set rather than crashing the platform if the AI server is unreachable, and (3) it allows the two services to be hosted, scaled, and deployed independently — which is exactly the split reflected in your actual hosting setup (Laravel on GoDaddy, Django + Qdrant on OVH Cloud VPS). This combination — layered internally within Laravel (MVC), service-oriented between Laravel and the AI engine — is what should be named and justified in this subsection rather than calling the whole system just "MVC" or just "client-server."

Figure 3.1 illustrates the high-level architecture of EventBridge AI, showing the major system components and the flow of data between them. The system follows a layered, multi-tiered client-server architecture in which the AI functionality is deployed as an independent service tier rather than embedded within the main application. The client browser communicates with the Laravel backend over HTTPS, which handles all user- and community-facing logic and persists relational data — users, communities, events, registrations, and collaborations — in a MySQL database. For AI-driven functionality such as semantic search and personalised recommendations, the Laravel backend issues HTTP requests to a separately hosted Django

AI engine. This engine generates vector embeddings and performs re-ranking using OpenAI’s embedding and chat completion APIs, and stores and queries those embeddings in a Qdrant vector database. The Django AI engine and Qdrant are hosted together on an OVH Cloud VPS, while the Laravel application is hosted separately on GoDaddy, reflecting the platform’s real two-server deployment. This separation of the AI service tier was a deliberate design decision: it allows the AI engine to use Python’s native machine learning tooling, isolates AI-related failures from the core platform (a failed AI request degrades gracefully to an empty result set rather than affecting core functionality), and allows both services to be hosted and scaled independently of one another.

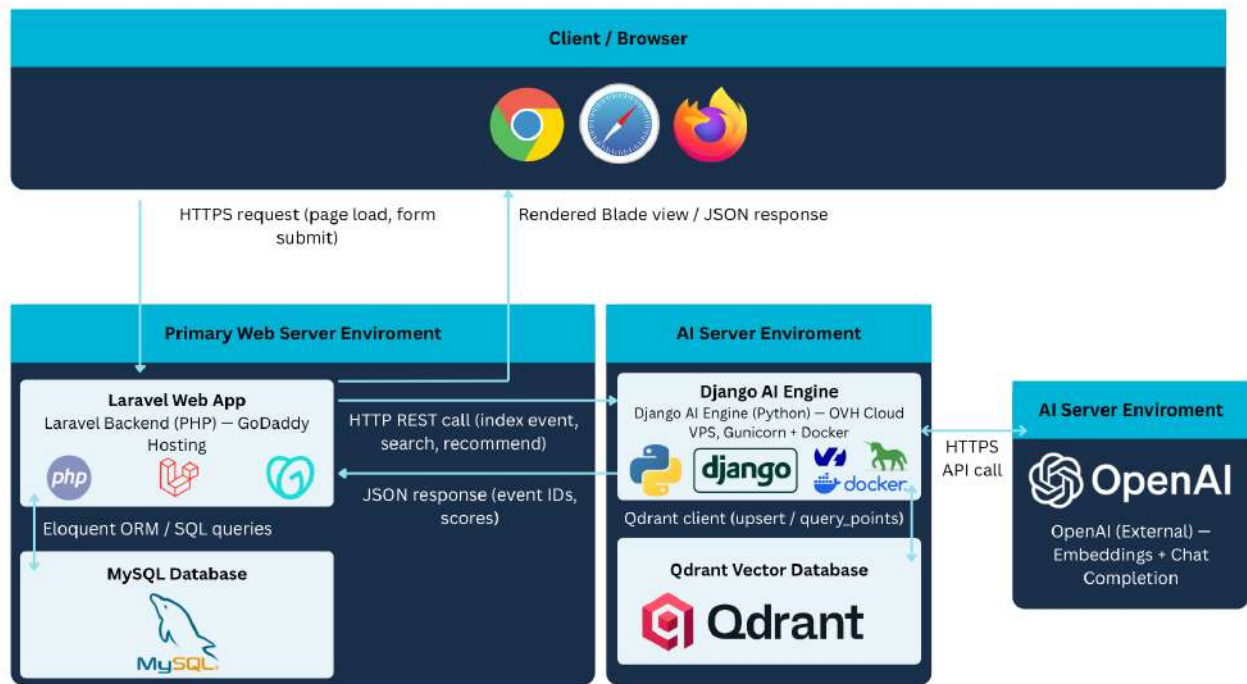


Figure 3.1: System Block Diagram of EventBridge AI

Figure 3.2 presents a decomposition of the Laravel application’s internal structure, showing how its feature modules, services, and data layer relate to one another. The application is organized into four logical layers. At the top, feature-specific controllers handle incoming requests for authentication, community and team management, event management, event collaboration, discovery/explore, notifications, administration, and feedback. Most controllers interact directly with Eloquent models to perform standard CRUD operations. AI-related and asynchronous operations, however, are routed through a dedicated service and job layer — including `AiSearchService`, `AiRecService`, and background jobs such as `ComputeUserRecommendationsJob` and `IndexEventJob` — which in turn communicate with the external Django AI engine described in Figure 3.1. Beneath the controller and service layers, Eloquent models represent the system’s core entities and their relationships, and these models persist and retrieve data from the underlying MySQL database. This modular decomposition keeps the Laravel application’s core CRUD logic decoupled from its AI-dependent logic, so that the two can be modified, tested, and scaled independently.

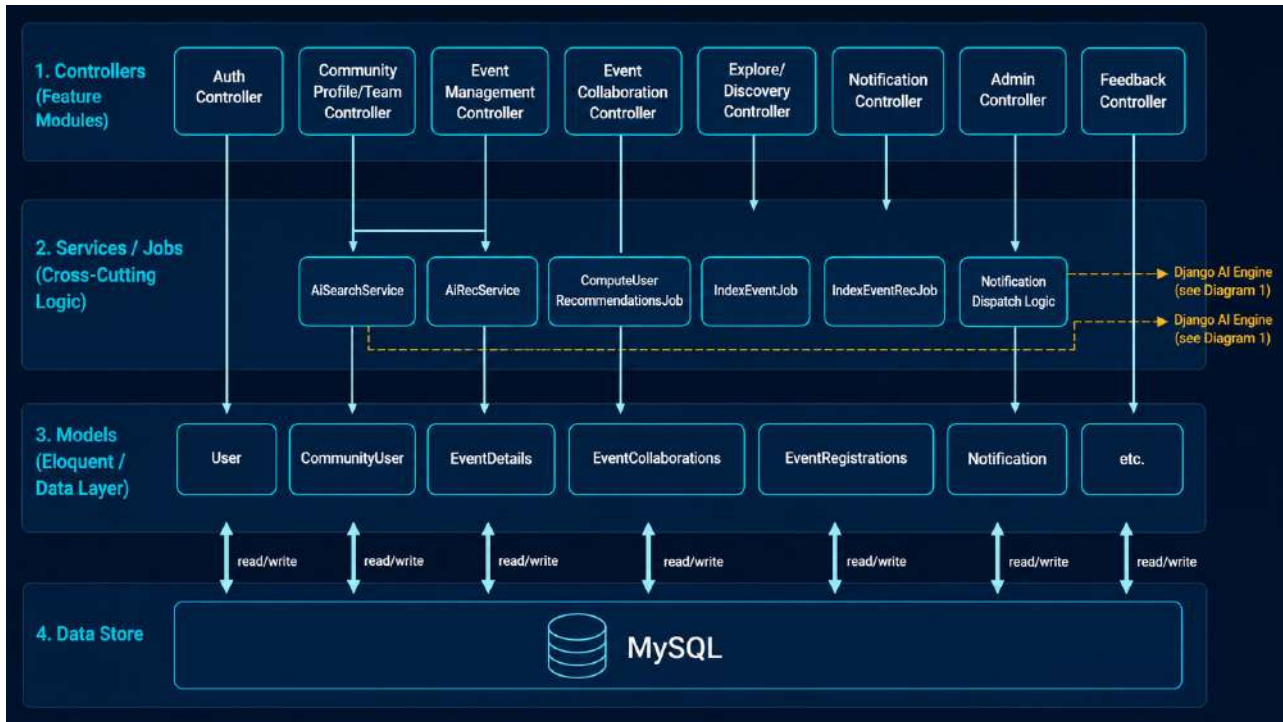


Figure 3.2: Modular System Block Diagram of the Laravel Application

3.3 Process flow / Representation

This section presents the flow of the system's major processes in the form of activity diagrams, illustrating the sequence of actions, decision points, and system responses involved in each. Rather than diagramming every operation the platform supports, this section focuses on the processes that best demonstrate the system's core logic and its cross-component interactions — between the community and user sides, between Laravel and the Django AI engine, and between synchronous, user-triggered actions and asynchronous, scheduled ones. Login and registration flows are intentionally excluded, as they follow standard, low-complexity authentication patterns and do not involve branching logic significant enough to warrant a dedicated diagram. Four processes are covered: the event creation and publishing lifecycle, the inter-community collaboration request flow, the user's discovery-to-registration journey, and the AI recommendation engine's daily batch computation. Each diagram is accompanied by a description of the decision points and system behaviour it captures.

Figure 3.3 represents the workflow a community follows when creating and publishing an event, from initial draft through to automatic lifecycle progression. A key decision point occurs at the save stage, where a community may choose to save the event as a draft for later editing or publish it immediately. Upon publishing, the system performs two AI-indexing operations in parallel — indexing the event into both the semantic search collection and the recommendation collection in Qdrant — before notifying the community's followers. The diagram also captures the event's autonomous lifecycle transitions, which are handled by a scheduled system process rather than direct user action.

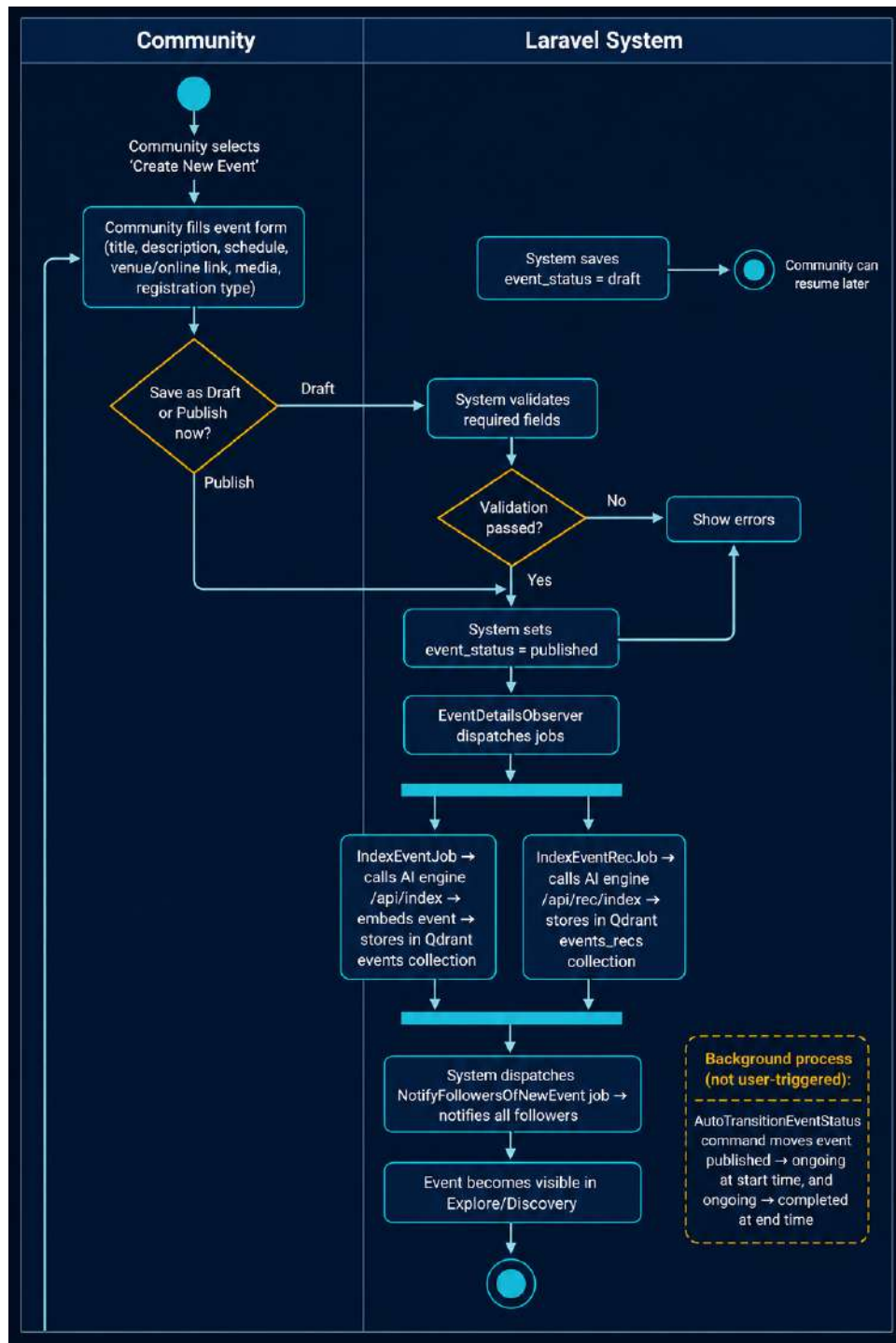


Figure 3.3: Activity Diagram — Event Creation and Publishing Lifecycle

Figure 3.4 illustrates the collaboration workflow between two communities. A requesting community invites another community to co-host an event, after which the receiving community may accept, reject, or leave the request pending. The requesting community retains the ability to cancel a pending request at any point before a response is received. Notifications are dispatched to the relevant party at each state transition, keeping both communities informed throughout the process.

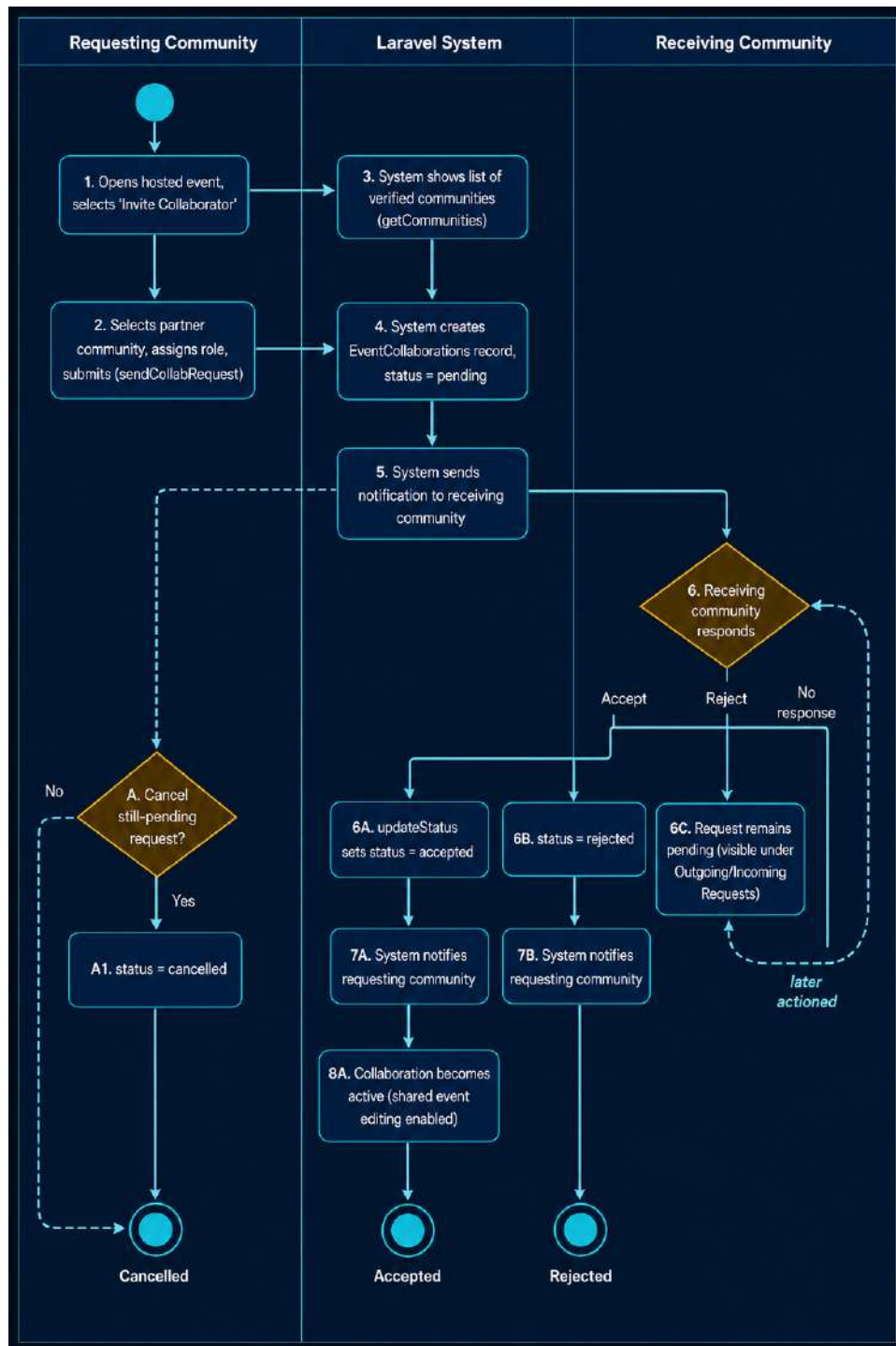


Figure 3.4: Activity Diagram — Event Collaboration Request Flow

Figure 3.5 traces a user's journey from searching for an event through to registration. A central decision point in this flow is the availability of the Django AI engine: if the semantic search service is unreachable, the system gracefully falls back to a traditional MySQL keyword search rather than presenting an error to the user, ensuring search remains functional even during AI service downtime. The diagram also captures the participant-limit check during registration, which determines whether a user is registered directly or placed on a waitlist.

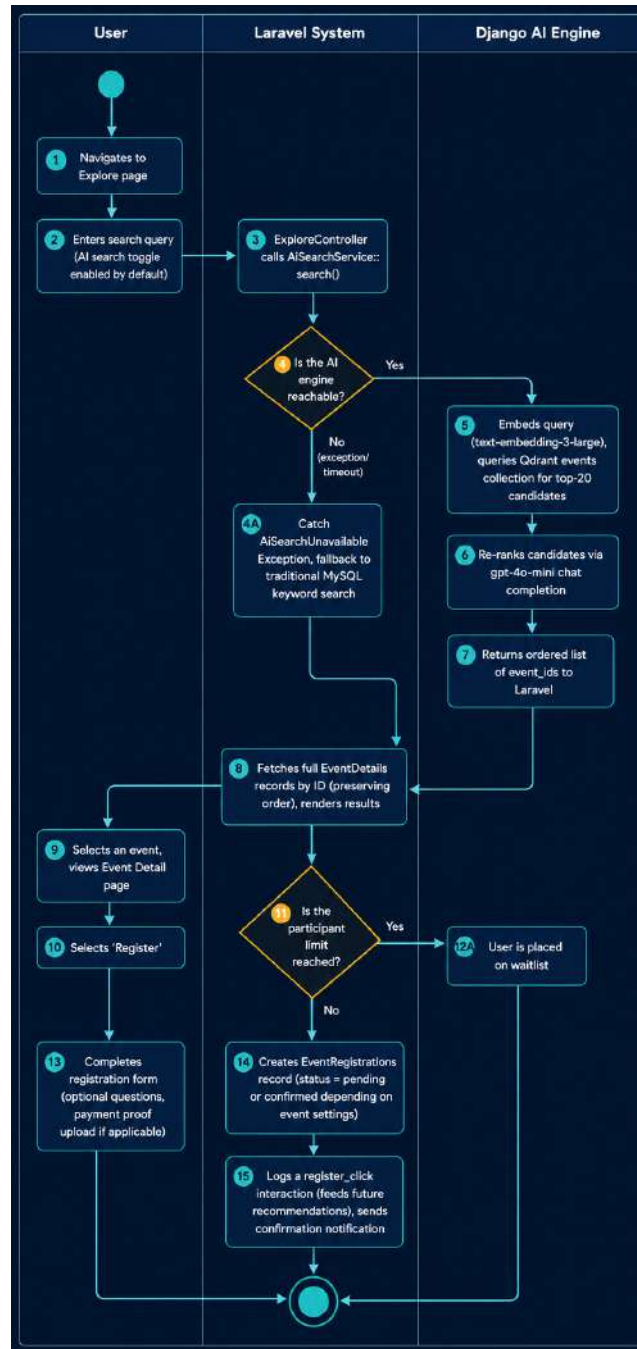


Figure 3.5: Activity Diagram — User Discovery, Semantic Search, and Event Registration

Figure 3.6 represents the automated, asynchronous process by which personalized event recommendations are generated for each user. Rather than being computed in real time on page load, recommendations are pre-computed daily by a scheduled batch job, which aggregates a user's profile data and behavioral signals into a single preference text, embeds it, and retrieves the nearest events by cosine similarity in vector space. Decision points in this flow include an early-exit condition for users with no usable preference signal, and a check to avoid sending redundant notifications when the recommendation set has not meaningfully changed.

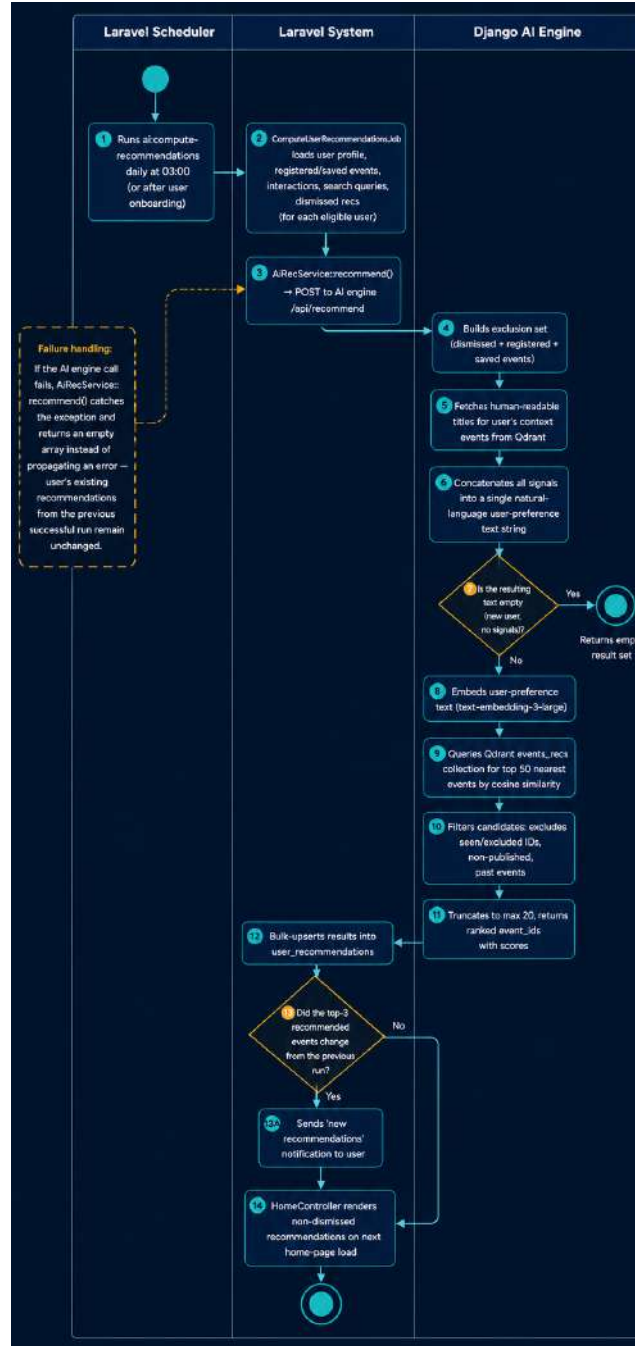


Figure 3.6: Activity Diagram — AI Recommendation Generation (Daily Batch Job)

Taken together, these four processes illustrate a consistent design pattern that recurs throughout EventBridge AI: wherever the system depends on the external Django AI engine — whether for indexing, search, or recommendation — the corresponding activity diagram includes an explicit decision point for handling that dependency, and every such path degrades gracefully rather than failing outright. Semantic search falls back to traditional keyword search, and failed recommendation computation simply leaves the previous result set intact rather than surfacing an error to the user. Similarly, the platform separates time-sensitive, user-facing actions (publishing an event, sending a collaboration request, registering for an event) from background, scheduled processes (lifecycle auto-transitions, daily recommendation computation, follower notifications), which keeps the user-facing request/response cycle fast

while heavier AI and notification work happens asynchronously. These patterns, established here at the process level, are reflected structurally in the class diagram and data flow diagram presented in the next section.

3.4 Design models [along with description]

As established in Section 3.1, EventBridge AI follows a hybrid design methodology: the Laravel application is Object-Oriented, while the Django AI engine is function-oriented. In line with the SED's guidance that class diagrams apply only where an OO approach is used, this section presents three OO models for the Laravel application — a Class Diagram, a System Sequence Diagram, and a State Transition Diagram for the Event entity — followed by a Data Flow Diagram for the Django AI engine, since a function-oriented service is more accurately represented as a data pipeline than as an object model.

3.4.1 Class Diagram

Figure 3.7 presents the class diagram for the Laravel application, organized into three modules: User, Admin, Community, and Event, which mirror the platform's three main entity groups. The diagram employs several distinct UML relationship types rather than using uniform association lines. The relationship types used are explained below.

- **Association (1:1 and 1:many)** is the dominant relationship type throughout the diagram. Examples include `users` to `users_profile` (1:1), `community_user` to `community_profile` (1:1), and `event_details` to `event_registration_attendance` (1:0..*). These represent standard foreign-key relationships between independently meaningful entities.
- **Aggregation** is used where a child entity is grouped under a parent while retaining its own identity. For example, `community_team_tenure` aggregates zero or more `community_team_members`. Team members belong to a particular tenure period, but the relationship represents a loose whole-part association rather than strict ownership.
- **Composition** is used where the lifetime of one entity depends entirely on another. For example, `event_details` is composed of zero or more `event_media` records. Uploaded media has no meaning outside its parent event; therefore, its lifecycle is completely dependent on that event.
- **Dependency** relationships (shown using dashed arrows) are used for two polymorphic cases that cannot be represented cleanly using fixed-multiplicity associations. The first is `community_followers.follower_id`, which may reference either `users.id` or `community_user.id` depending on the value of the `follower_type` discriminator. The second involves the `sender` and `receiver` fields of `event_collaborations`, both of which reference `community_user` but represent different semantic roles. These modeling decisions are documented explicitly in the diagram.
- **Generalization (Inheritance)** is intentionally absent from the class diagram. Each class corresponds directly to a separate Laravel Eloquent model and an underlying MySQL table. Since none of the platform entities share behaviour through a common superclass, there is no natural inheritance hierarchy to represent.

The diagram also documents two explicit data-modeling decisions directly on its face, which are worth carrying into the report text as evidence of deliberate design rather than oversight: first, that community followers is modeled as an association entity with a polymorphic follower_id rather than two fixed foreign keys; and second, a naming inconsistency found between two source design documents regarding community_user_id vs. community_id, which was resolved in favor of the relationships specification and flagged rather than silently reconciled.

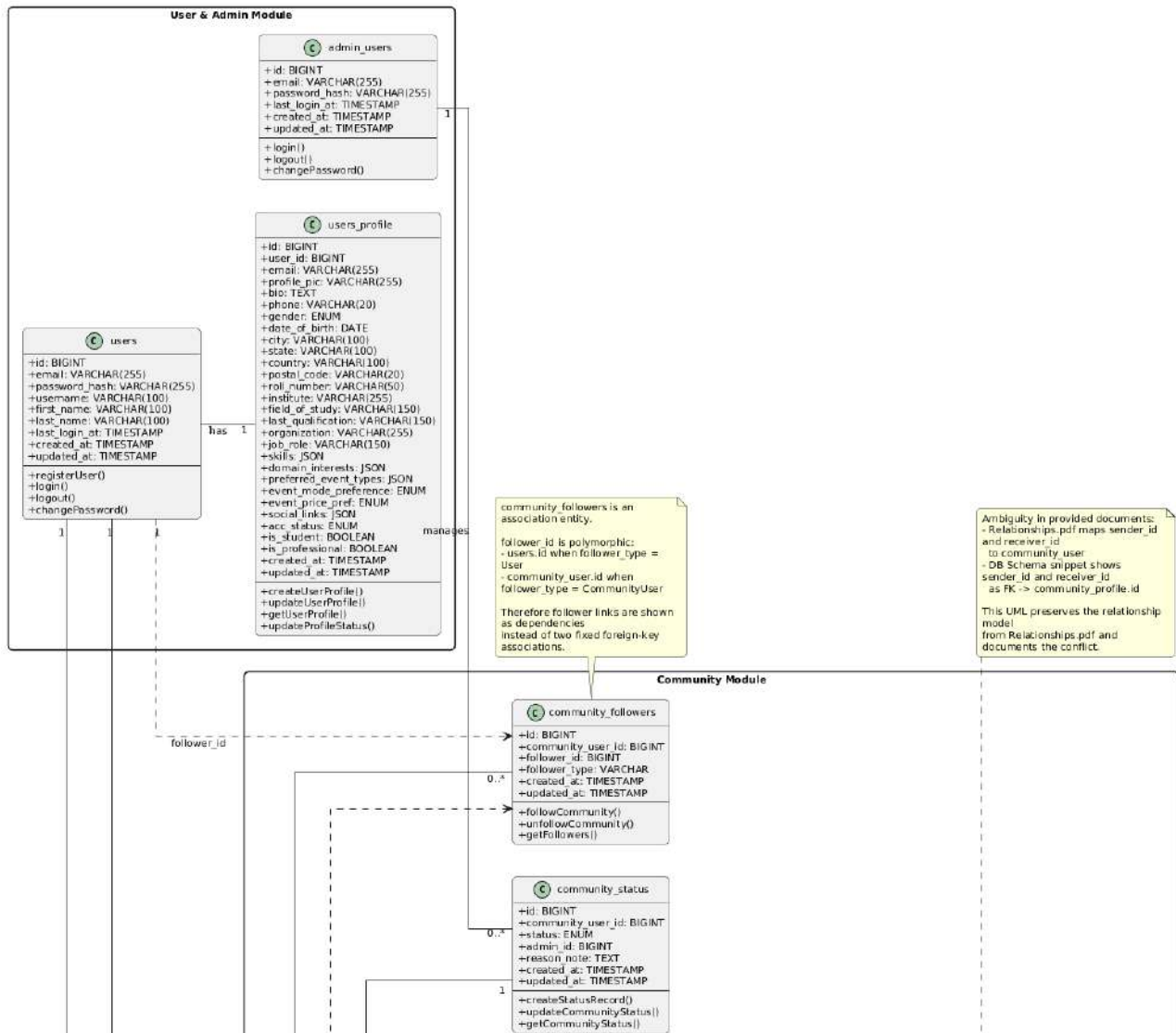


Figure 3.7: Class Diagram Part 01

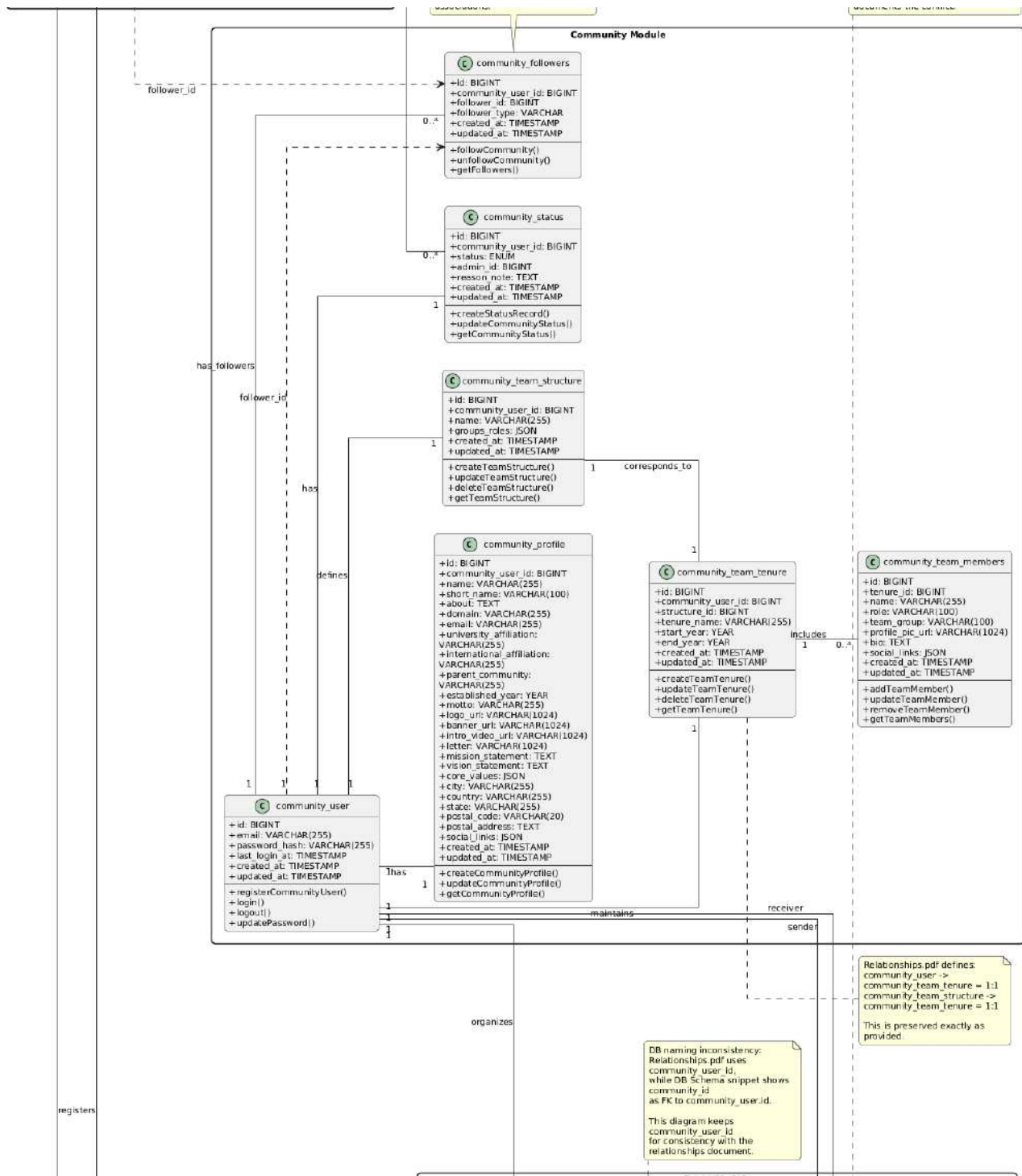


Figure 3.8: Class Diagram Part 01

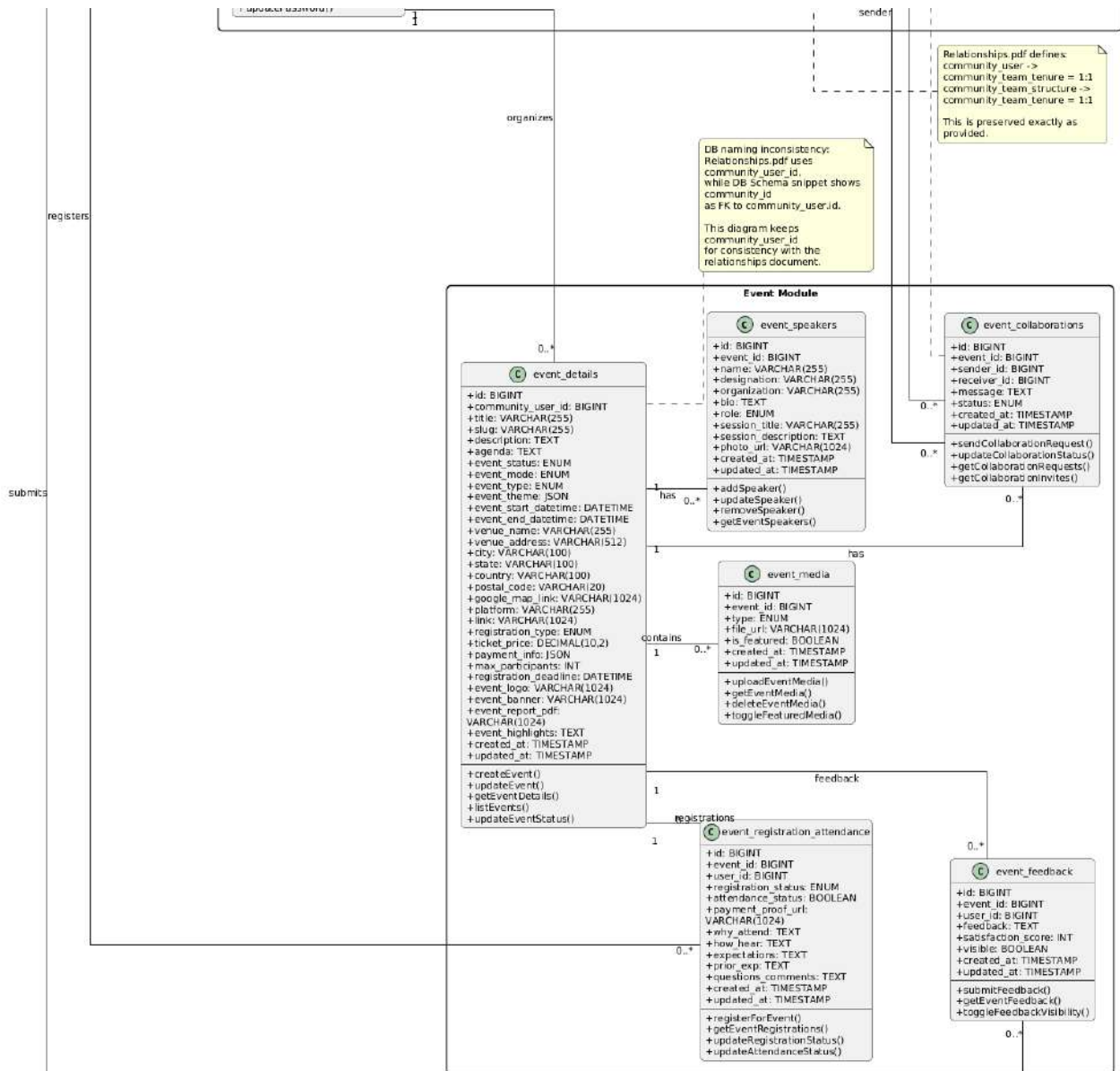


Figure 3.9: Class Diagram Part 01

3.4.2 Sequence Diagram

Figure 3.8 (spanning two diagram pages) presents a consolidated system sequence diagram covering the platform's primary end-to-end interaction flows, rather than a single isolated process. Seven lifelines run across the diagram — three actors (Community, User/Individual, Admin) and four system participants (Web Client, Laravel Server, Database, AI Service, Vector DB, and OpenAI Models) — and fourteen labeled flows are laid out top to bottom in the rough order a user would encounter them: community registration and admin approval; community login and profile/team management; event creation; event embedding creation (the AI integration point, where event data is sent to the Django AI Service, embedded via OpenAI, and stored in Qdrant); event management (editing, speaker management, collaboration invitations); collaboration response; event publishing; explore/discovery; AI-powered semantic search; the AI recommendation engine; event registration; registration and attendance management; event completion; and feedback submission with admin moderation and oversight.

Each flow follows standard UML sequence notation — solid arrows for requests, dashed arrows for responses — and several flows (Flow 4, Flow 9, Flow 10) explicitly trace the request across all the way to the OpenAI Models lifeline and back, visualizing the full round-trip latency chain for AI-dependent operations rather than treating the AI engine as a black box. Flow 13 (Event Completion) is notable for having no actor-originated message at all: it is triggered entirely by the Laravel Server checking event start/end times against the current time, matching the scheduled `AutoTransitionEventStatus` command described in Section 3.3.1 and visualized as its own state machine in Figure 3.9.



Figure 3.10: Sequence Diagram (Part 1 of 3)



Figure 3.11: Sequence Diagram (Part 2 of 3)

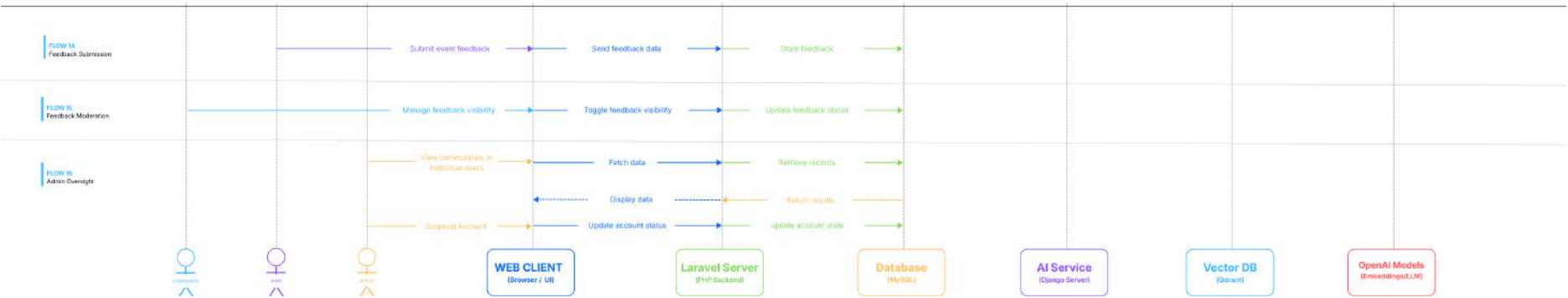


Figure 3.12: Sequence Diagram (Part 3 of 3)

3.4.3 State Transition Diagram

Figure 3.9 models the lifecycle of the `event_details` entity as a finite state machine, supplementing the class diagram by capturing behavior over time rather than structure. An event begins in the Draft state upon creation, where the host community may view, edit, manage speakers, invite collaborators, or publish it. Publishing is a manual, community-triggered transition into the Published state, from which the event gains registration management capability and may still be edited, have speakers managed, or receive collaboration invitations.

From Published, two further transitions are possible: a manual Cancel Event action moves the event directly into a terminal Cancelled state, or — without any user action — the system automatically transitions the event into Ongoing once the current time reaches the event’s start time. In the Ongoing state, the event becomes effectively read-only (View Event only) — editing, speaker management, and collaboration actions are no longer available once an event is live. A second automatic transition, guarded by the event’s end time, moves the event into a final Completed state, where the host community’s available actions shift from event management to post-event operations: viewing, media management, and feedback management. Both Completed and Cancelled are terminal states that transition to the diagram’s final node, ending the event’s lifecycle. This diagram maps directly onto the `event_status` ENUM field and the `AutoTransitionEventStatus` scheduled command referenced in the class diagram and sequence diagram respectively, tying all three OO models to the same underlying lifecycle field.

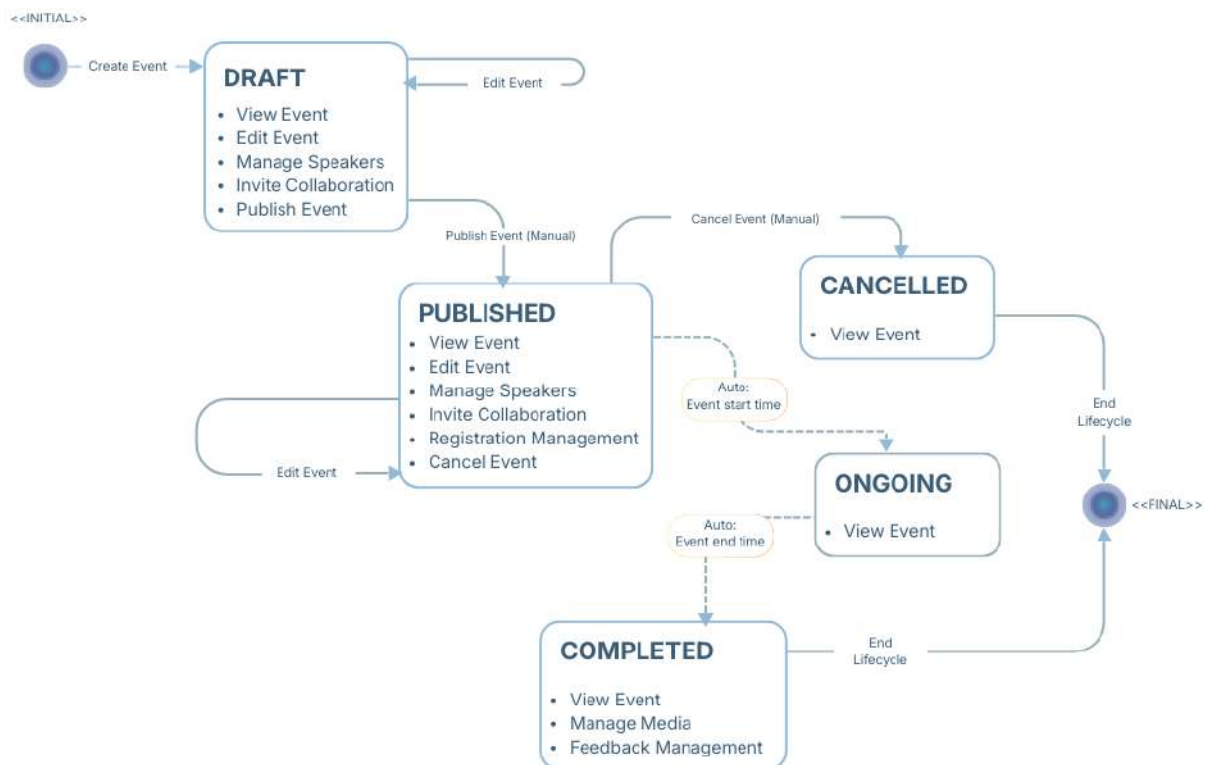


Figure 3.13: UML State Transition Diagram

3.4.4 Data Flow Diagram — Django AI Engine

The Django AI engine's internal logic is represented here as a Data Flow Diagram rather than a class diagram, consistent with its function-oriented design established in Section 3.1. The diagram is developed across three levels of increasing detail: a context diagram establishing the system's boundary and its two external entities, a Level-1 diagram decomposing the engine into its six constituent processes and two vector data stores, and Level-2 diagrams further decomposing the two most significant processes — semantic search and recommendation generation — into their individual computational steps.

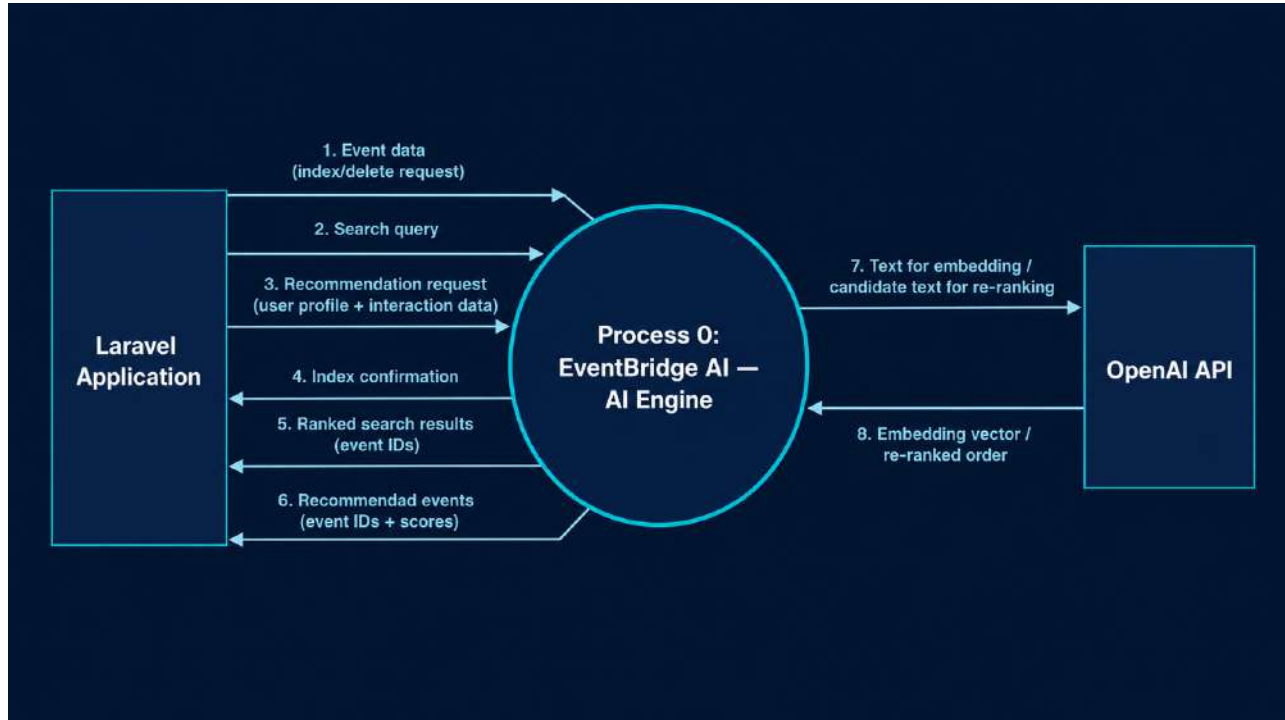


Figure 3.14: Level-0 (Context) Data Flow Diagram — Django AI Engine

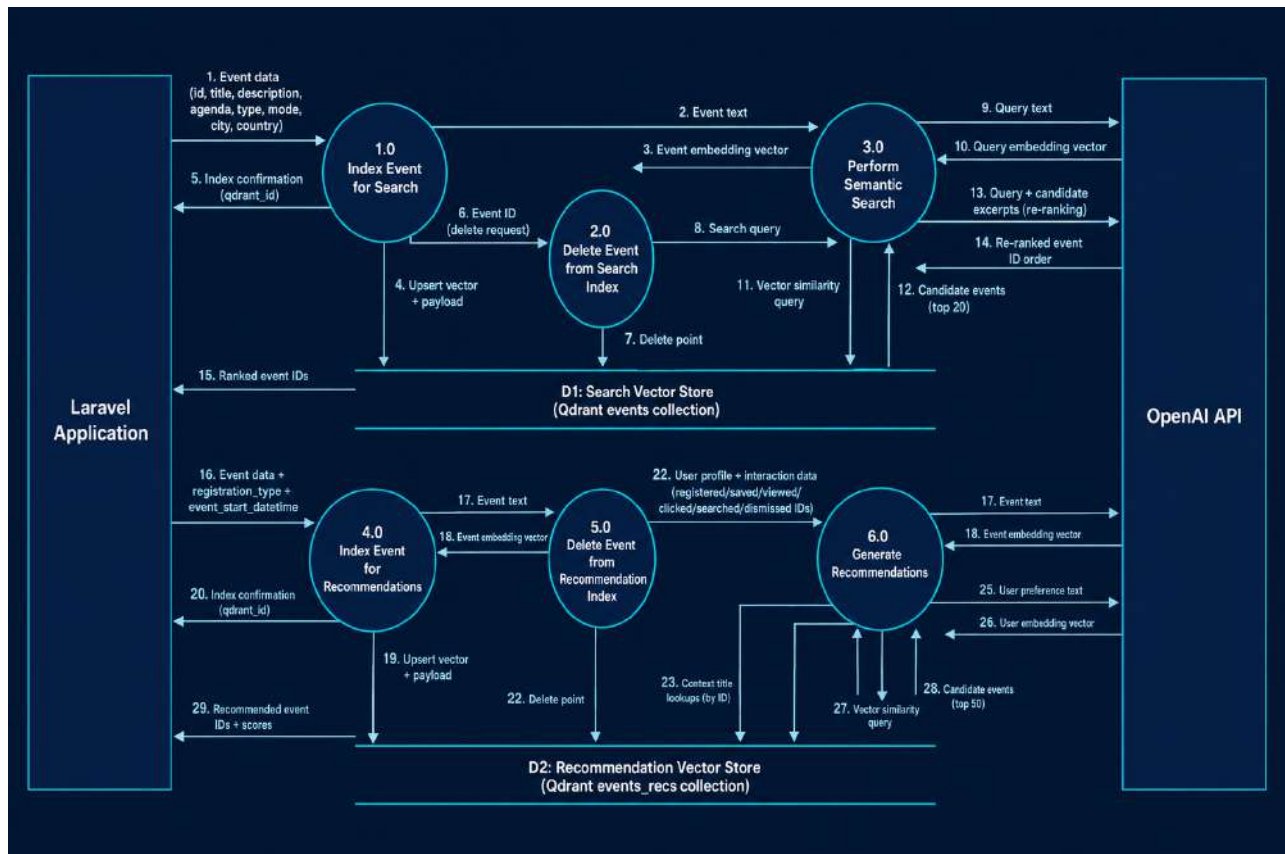


Figure 3.15: Level-1 Data Flow Diagram — Django AI Engine

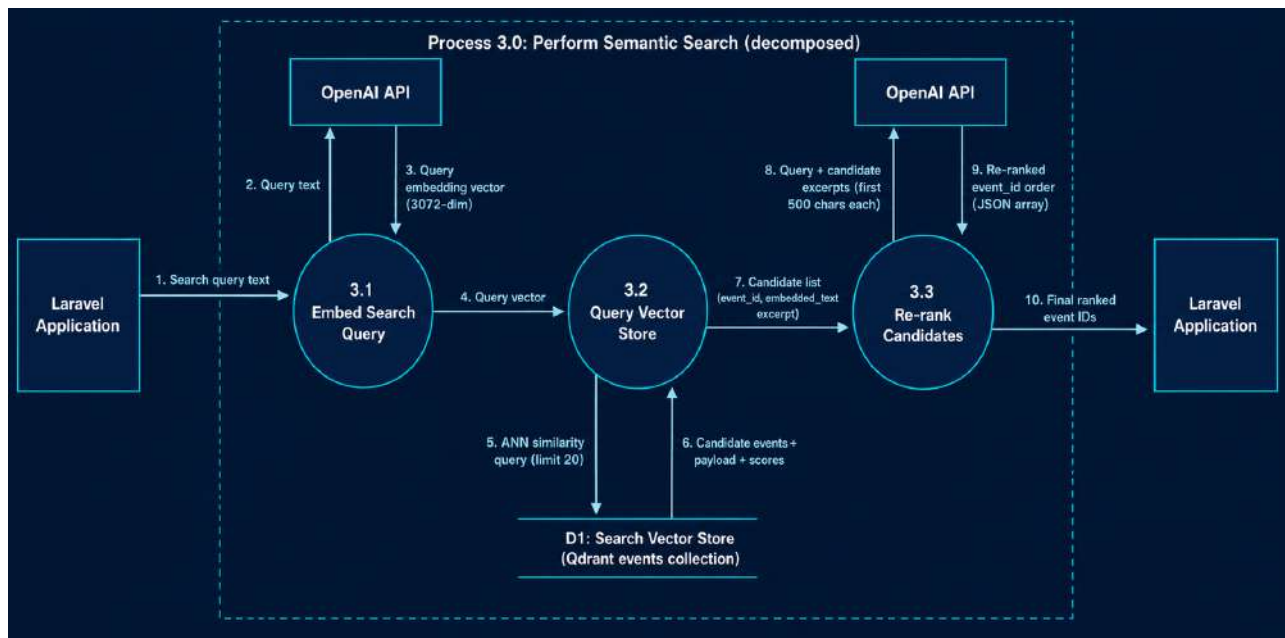


Figure 3.16: Level-2 Data Flow Diagram — Process 3.0 (Perform Semantic Search)

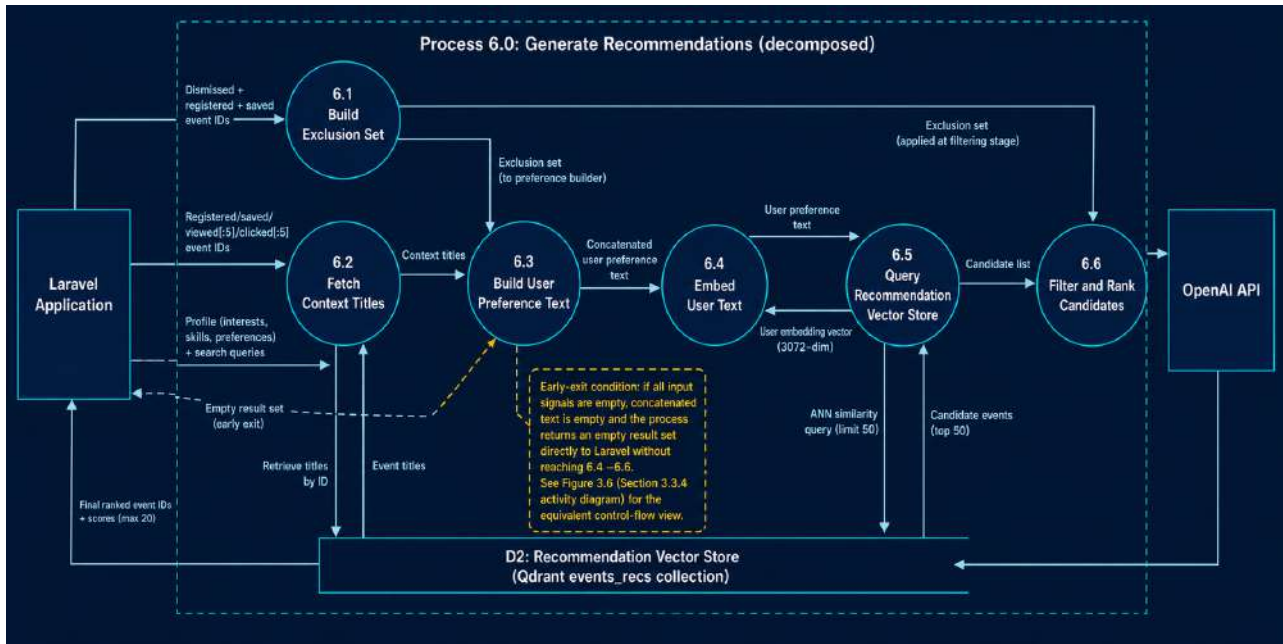


Figure 3.17: Level-2 Data Flow Diagram — Process 6.0 (Generate Recommendations)

Across all three levels, a consistent pattern is visible: every process in this system follows the same underlying shape — retrieve or construct text, embed it via OpenAI, compare it against a Qdrant vector store, and return ranked results — with the two Level-2 diagrams differing mainly in how the input text is assembled (a single query string for search, versus a concatenation of multiple behavioral and profile signals for recommendations). This confirms the characterization given in Section 2.3 and Section 3.1: the engine implements pure content-based nearest-neighbor filtering, with no collaborative signal, no separate item-to-item similarity computation, and no orchestration framework (LangChain or otherwise) coordinating these steps — each OpenAI and Qdrant call is a direct, independent step in a linear pipeline, exactly as drawn.

3.5 Data Design

This section explains how the raw information domain of EventBridge AI — user input, event descriptions, profile data, and behavioral signals — is transformed into structured data, and how that data is subsequently stored, processed, and organized across the system.

3.5.1 From Information Domain to Data Structure

The system’s information domain is inherently mixed: some of it is structured, transactional data with clear fields and constraints (a user’s email, an event’s start time, a registration status), while another portion is unstructured natural-language text (an event’s description and agenda, a community’s mission statement, a user’s stated interests) that only becomes computationally useful once transformed into a numeric representation. EventBridge AI’s data design reflects this split directly, using two fundamentally different data structures for the two kinds of information rather than forcing both into a single model.

Structured, transactional data is normalized into relational tables via Laravel’s Eloquent ORM,

following standard relational design: each core entity (user, community, event, registration, collaboration, feedback) is its own table with a primary key, explicit foreign keys expressing relationships (as detailed in the class diagram, Figure 3.7), and appropriate column-level typing and constraints (ENUMs for status fields, TIMESTAMPS for lifecycle tracking, DECIMAL for monetary fields such as ticket price). Where a field's shape is inherently a small structured list rather than a scalar — a user's skills, a community's social links, an event's theme tags — it is stored as a JSON column rather than being normalized into a separate child table. This is a deliberate trade-off: these fields are read and written as a unit (the whole list is fetched or replaced together, never queried element-by-element at the database level), so normalizing them into separate rows would add relational overhead without a corresponding query benefit.

Unstructured text data — the same event descriptions, agendas, and profile fields — takes a second, parallel path. Rather than being queried directly, this text is concatenated into a single string per entity (as detailed in the DFD, Figures 3.11–3.13: `build_event_text()` for events, `build_user_text()` for users) and transformed via OpenAI's text-embedding-3-large model into a dense, 3072-dimensional numeric vector. This is the core data transformation this system depends on: converting meaning expressed in natural language into a geometric representation that can be compared by distance rather than by exact keyword match. The resulting vector, together with a payload of the original field values needed for display and filtering, is what is actually stored — not in MySQL, but in Qdrant.

3.5.2 Storage Organization

The system therefore uses two purpose-fit data stores rather than one, each organized around the kind of data and access pattern it serves:

MySQL is the system of record for all relational, transactional data. It is the single source of truth: every entity is created here first, and every write to Qdrant is a downstream, asynchronous derivative of a MySQL write, never the reverse (per the sync mechanism in the integration summary — Django owns no SQL data of its own). Tables are organized by module in line with the class diagram: User & Admin (`users`, `users_profile`, `admin_users`), Community (`community_user`, `community_profile`, `community_status`, `community_team_structure`, `community_team_tenure`, `community_team_members`, `community_followers`), and Event (`event_details`, `event_speakers`, `event_media`, `event_collaborations`, `event_registration_attendance`, `event_feedback`). Two additional tracking tables, `EventSearchIndex` and `EventRecIndex`, exist purely to record which events have been successfully synced to Qdrant, allowing failed or pending syncs to be identified and retried without re-scanning the entire event table.

Qdrant stores exactly two collections, organized by purpose rather than by entity type: `events` (3072-dimensional vectors with cosine distance, used for semantic search) and `events_recs` (the same dimensionality and distance metric, used for recommendation matching). Both use the MySQL `event_id` directly as the Qdrant point ID — this is the only identifier shared between the two stores, deliberately avoiding a separate UUID-mapping layer. Each point's payload carries a denormalized copy of the fields needed to filter and display results (title, status, dates, location, mode) so that Qdrant queries do not need a round-trip back to MySQL just to filter out unpublished or past events.

3.5.3 Storage Summary

Store	Technology	Organized as	Contents
Relational data store	MySQL	Normalized tables, module-grouped	Users, communities, events, registrations, collaborations, feedback, notifications, sync-tracking tables
Search vector store	Qdrant (events collection)	Flat vector collection, keyed by event_id	3072-dim event embeddings + display/filter payload
Recommendation vector store	Qdrant (events_recs collection)	Flat vector collection, keyed by event_id	3072-dim event embeddings (extended payload: registration_type, event_start_datetime)
File storage	Local filesystem (public/uploads/)	Path referenced by URL columns	Profile photos, event media, logos/banners, payment proof uploads

Table 3.1: Data storage architecture overview

This two-database design is the direct data-level consequence of the architectural separation described in Section 3.2: MySQL and Laravel own structured state and business logic, while Qdrant and the Django engine own only derived, disposable numeric representations of that state — disposable in the sense that every vector can be fully regenerated from its MySQL source of truth via the `ai:index-events / ai:index-events-rec` commands, which is precisely what makes the graceful-fallback behavior in Section 3.3 possible.

3.6 Data Dictionary

Entities and their attributes are drawn directly from the class diagram (Figure 3.7). Since EventBridge AI uses a hybrid design (Section 3.1), this dictionary is split into two parts: 3.6.1 lists Laravel’s entities/classes with their attributes and methods (Object-Oriented description), and 3.6.2 lists the Django AI engine’s service functions and parameters (function-oriented description).

3.6.1 Data Dictionary — Laravel Entities (Object-Oriented)

Attribute	Type	Description
id	BIGINT	Primary key
email	VARCHAR(255)	Admin login email
password_hash	VARCHAR(255)	Hashed admin password
last_login_at	TIMESTAMP	Timestamp of last successful login
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Record last-modified timestamp

Table 3.2: admin table schema

Methods:

- `login(email, password)` — authenticates an admin and starts a session
- `logout()` — ends the current admin session
- `changePassword(oldPassword, newPassword)` — updates the stored password hash

Attribute	Type	Description
id	BIGINT	Primary key
community_user_id	BIGINT	FK — the community being followed
follower_id	BIGINT	Polymorphic FK — resolves to users.id or community_user.id depending on follower_type
follower_type	VARCHAR	Discriminator: User or CommunityUser
created_at	TIMESTAMP	Follow timestamp
updated_at	TIMESTAMP	Record last-modified timestamp

Table 3.3: community_followers table schema

Methods:

- `followCommunity(communityId, followerId, followerType)` — creates a follow record
- `unfollowCommunity(communityId, followerId)` — removes a follow record
- `getFollowers(communityId)` — returns all followers of a community

Attribute	Type	Description
id	BIGINT	Primary key
community_user_id	BIGINT	FK to community_user
name	VARCHAR(255)	Community display name
short_name	VARCHAR(100)	Abbreviated/short name
about	TEXT	Community description
domain	VARCHAR(255)	Subject/technical domain
email	VARCHAR(255)	Public contact email
university_affiliation	VARCHAR(255)	Affiliated university, if any
international_affiliation	VARCHAR(255)	Parent international chapter, if any
parent_community	VARCHAR(255)	Parent community reference
established_year	YEAR	Year the community was founded
motto	VARCHAR(255)	Community motto/tagline
logo_url	VARCHAR(1024)	Logo image path
banner_url	VARCHAR(1024)	Banner image path
intro_video_url	VARCHAR(1024)	Introductory video path
letter	VARCHAR(1024)	Affiliation/authorization letter path
mission_statement	TEXT	Community mission
vision_statement	TEXT	Community vision
core_values	JSON	List of core values
city	VARCHAR(255)	City location
country	VARCHAR(255)	Country location
state	VARCHAR(255)	State/province
postal_code	VARCHAR(20)	Postal code
postal_address	TEXT	Full mailing address
social_links	JSON	Social media links
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Record last-modified timestamp

Table 3.4: community_profile table schema

Methods:

- `createCommunityProfile(data)` — creates a new profile
- `updateCommunityProfile(profileId, data)` — updates profile fields
- `getCommunityProfile(slug)` — retrieves a profile by slug

Attribute	Type	Description
id	BIGINT	Primary key
community_user_id	BIGINT	FK to community_user
status	ENUM	pending, approved, rejected, suspended
admin_id	BIGINT	FK — admin who last actioned the status
reason_note	TEXT	Admin's reason for the status decision
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Record last-modified timestamp

Table 3.5: community_status table schema

Methods:

- `createStatusRecord(communityUserId)` — initializes status as pending on registration
- `updateCommunityStatus(communityUserId, status, adminId, reasonNote)` — transitions status with audit trail
- `getCommunityStatus(communityUserId)` — retrieves current status

Attribute	Type	Description
id	BIGINT	Primary key
tenure_id	BIGINT	FK to community_team_tenure
name	VARCHAR(255)	Member's name
role	VARCHAR(100)	Member's role/title
team_group	VARCHAR(100)	Sub-team/group grouping
profile_pic_url	VARCHAR(1024)	Member photo path
bio	TEXT	Short biography
social_links	JSON	Member's social links
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Record last-modified timestamp

Table 3.6: community_team_members table schema

Methods:

- `addTeamMember(tenureId, data)` — adds a member to a tenure
- `updateTeamMember(memberId, data)` — updates member details
- `removeTeamMember(memberId)` — removes a member
- `getTeamMembers(tenureId)` — lists members for a tenure

Attribute	Type	Description
id	BIGINT	Primary key
community_user_id	BIGINT	FK to community_user
name	VARCHAR(255)	Structure/hierarchy name
groups_roles	JSON	Defined groups and roles within the structure
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Record last-modified timestamp

Table 3.7: community_team_structure table schema

Methods:

- `createTeamStructure(communityUserId, data)` — defines a team structure
- `updateTeamStructure(structureId, data)` — updates structure/roles
- `deleteTeamStructure(structureId)` — removes a structure
- `getTeamStructure(communityUserId)` — retrieves the current structure

Attribute	Type	Description
id	BIGINT	Primary key
community_user_id	BIGINT	FK to community_user
structure_id	BIGINT	FK to community_team_structure
tenure_name	VARCHAR(255)	Name of the tenure/term (e.g. “2025–26”)
start_year	YEAR	Tenure start year
end_year	YEAR	Tenure end year
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Record last-modified timestamp

Table 3.8: community_team_tenure table schema

Methods:

- `createTeamTenure(structureId, data)` — creates a new tenure period
- `updateTeamTenure(tenureId, data)` — updates tenure details
- `deleteTeamTenure(tenureId)` — removes a tenure
- `getTeamTenure(communityUserId)` — retrieves tenure history

Attribute	Type	Description
id	BIGINT	Primary key
email	VARCHAR(255)	Community login email
password_hash	VARCHAR(255)	Hashed password
last_login_at	TIMESTAMP	Timestamp of last login
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Record last-modified timestamp

Table 3.9: community_user table schema

Methods:

- `registerCommunityUser(email, password)` — creates a new community account
- `login(email, password)` — authenticates a community
- `logout()` — ends the session
- `updatePassword(oldPassword, newPassword)` — changes the password

Attribute	Type	Description
id	BIGINT	Primary key
event_id	BIGINT	FK to event_details
sender_id	BIGINT	FK to community_user — requesting community
receiver_id	BIGINT	FK to community_user — invited community
message	TEXT	Message accompanying the invitation
status	ENUM	pending, accepted, rejected, cancelled
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Record last-modified timestamp

Table 3.10: event_collaborations table schema

Methods:

- `sendCollaborationRequest(eventId, senderId, receiverId, message)` — creates a request
- `updateCollaborationStatus(collaborationId, status)` — accepts/rejects/cancels a request
- `getCollaborationRequests(communityId)` — lists incoming/outgoing requests
- `getCollaborationInvites(communityId)` — lists pending invites specifically

Attribute	Type	Description
id	BIGINT	Primary key
community_user_id	BIGINT	FK — hosting community
title	VARCHAR(255)	Event title
slug	VARCHAR(255)	URL-friendly identifier
description	TEXT	Full event description
agenda	TEXT	Event agenda/schedule text
event_status	ENUM	draft, published, ongoing, completed, cancelled
event_mode	ENUM	online, in_person, hybrid
event_type	ENUM	Category of event (e.g. workshop, seminar)
event_theme	JSON	Theme/topic tags
event_start_datetime	DATETIME	Scheduled start
event_end_datetime	DATETIME	Scheduled end
venue_name	VARCHAR(255)	Physical venue name
venue_address	VARCHAR(512)	Physical venue address
city	VARCHAR(100)	City
state	VARCHAR(100)	State/province
country	VARCHAR(100)	Country
postal_code	VARCHAR(20)	Postal code
google_map_link	VARCHAR(1024)	Map link for venue
platform	VARCHAR(255)	Online platform (e.g. Zoom)
link	VARCHAR(1024)	Online meeting link
registration_type	ENUM	internal (built-in form) or external (Google Form)
ticket_price	DECIMAL(10,2)	Ticket price, if paid
payment_info	JSON	Payment instructions/details
max_participants	INT	Registration cap
registration_deadline	DATETIME	Last date/time to register
event_logo	VARCHAR(1024)	Event logo image path
event_banner	VARCHAR(1024)	Event banner image path
event_report_pdf	VARCHAR(1024)	Post-event report file path
event_highlights	TEXT	Post-event highlights summary
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Record last-modified timestamp

Table 3.11: event_details table schema

Methods:

- `createEvent(communityUserId, data)` — creates a new event
- `updateEvent(eventId, data)` — updates event fields
- `getEventDetails(eventId)` — retrieves a single event
- `listEvents(filters)` — lists/searches events

- `updateEventStatus(eventId, status)` — transitions `event_status`

Attribute	Type	Description
<code>id</code>	BIGINT	Primary key
<code>event_id</code>	BIGINT	FK to <code>event_details</code>
<code>user_id</code>	BIGINT	FK to <code>users</code>
<code>feedback</code>	TEXT	Free-text feedback
<code>satisfaction_score</code>	INT	Numeric rating
<code>visible</code>	BOOLEAN	Whether feedback is publicly visible
<code>created_at</code>	TIMESTAMP	Record creation timestamp
<code>updated_at</code>	TIMESTAMP	Record last-modified timestamp

Table 3.12: `event_feedback` table schema**Methods:**

- `submitFeedback(eventId, userId, feedback, score)` — creates a feedback record
- `getEventFeedback(eventId)` — lists feedback for an event
- `toggleFeedbackVisibility(feedbackId)` — shows/hides a feedback entry

Attribute	Type	Description
<code>id</code>	BIGINT	Primary key
<code>event_id</code>	BIGINT	FK to <code>event_details</code>
<code>type</code>	ENUM	Media type (image/video/document)
<code>file_url</code>	VARCHAR(1024)	Stored file path
<code>is_featured</code>	BOOLEAN	Whether this media is highlighted
<code>created_at</code>	TIMESTAMP	Record creation timestamp
<code>updated_at</code>	TIMESTAMP	Record last-modified timestamp

Table 3.13: `event_media` table schema**Methods:**

- `uploadEventMedia(eventId, file, type)` — uploads and links media to an event
- `getEventMedia(eventId)` — lists media for an event
- `deleteEventMedia(mediaId)` — removes a media item
- `toggleFeaturedMedia(mediaId)` — marks/unmarks media as featured

Attribute	Type	Description
id	BIGINT	Primary key
event_id	BIGINT	FK to event_details
user_id	BIGINT	FK to users
registration_status	ENUM	pending, confirmed, waitlisted, cancelled
attendance_status	BOOLEAN	Whether the user attended
payment_proof_url	VARCHAR(1024)	Uploaded payment proof, if applicable
why_attend	TEXT	Optional registration question response
how_hear	TEXT	Optional registration question response
expectations	TEXT	Optional registration question response
prior_exp	TEXT	Optional registration question response
questions_comments	TEXT	Optional registration question response
created_at	TIMESTAMP	Registration timestamp
updated_at	TIMESTAMP	Record last-modified timestamp

Table 3.14: event_registration_attendance table schema

Methods:

- `registerForEvent(eventId, userId, formData)` — creates a registration record
- `getEventRegistrations(eventId)` — lists registrants for an event
- `updateRegistrationStatus(registrationId, status)` — confirms/waitlists/cancels a registration
- `updateAttendanceStatus(registrationId, attended)` — marks attendance

Attribute	Type	Description
id	BIGINT	Primary key
event_id	BIGINT	FK to event_details
name	VARCHAR(255)	Speaker's name
designation	VARCHAR(255)	Speaker's job title
organization	VARCHAR(255)	Speaker's affiliated organization
bio	TEXT	Speaker biography
role	ENUM	Speaker's role (e.g. keynote, panelist)
session_title	VARCHAR(255)	Title of the speaker's session
session_description	TEXT	Description of the session
photo_url	VARCHAR(1024)	Speaker photo path
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Record last-modified timestamp

Table 3.15: event_speakers table schema

Methods:

- `addSpeaker(eventId, data)` — adds a speaker to an event
- `updateSpeaker(speakerId, data)` — updates speaker details
- `removeSpeaker(speakerId)` — removes a speaker
- `getEventSpeakers(eventId)` — lists speakers for an event

Attribute	Type	Description
id	BIGINT	Primary key
email	VARCHAR(255)	Login email
password_hash	VARCHAR(255)	Hashed password
username	VARCHAR(100)	Unique username
first_name	VARCHAR(100)	First name
last_name	VARCHAR(100)	Last name
last_login_at	TIMESTAMP	Timestamp of last login
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Record last-modified timestamp

Table 3.16: users table schema

Methods:

- `registerUser(email, password, username)` — creates a new user account
- `login(email, password)` — authenticates a user
- `logout()` — ends the session

- `changePassword(oldPassword, newPassword)` — updates the password hash

Attribute	Type	Description
id	BIGINT	Primary key
user_id	BIGINT	FK to users
email	VARCHAR(255)	Contact email (denormalized)
profile_pic	VARCHAR(255)	Profile photo path
bio	TEXT	User biography
phone	VARCHAR(20)	Contact number
gender	ENUM	User's gender
date_of_birth	DATE	Date of birth
city	VARCHAR(100)	City
state	VARCHAR(100)	State/province
country	VARCHAR(100)	Country
postal_code	VARCHAR(20)	Postal code
roll_number	VARCHAR(50)	Institutional roll/registration number
institute	VARCHAR(255)	Educational institution
field_of_study	VARCHAR(150)	Academic field/major
last_qualification	VARCHAR(150)	Highest qualification held
organization	VARCHAR(255)	Employer/organization, if applicable
job_role	VARCHAR(150)	Job title/role
skills	JSON	List of skills
domain_interests	JSON	List of subject/domain interests
preferred_event_types	JSON	Preferred categories of events
event_mode_preference	ENUM	Preferred mode: online/in-person/hybrid
event_price_pref	ENUM	Preferred price range: free/paid/either
social_links	JSON	Social media links
acc_status	ENUM	Account status
is_student	BOOLEAN	Whether the user is a student
is_professional	BOOLEAN	Whether the user is a working professional
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Record last-modified timestamp

Table 3.17: users_profile table schema

Methods:

- `createUserProfile(userId, data)` — creates a profile record on registration
- `updateUserProfile(userId, data)` — updates profile fields
- `getUserProfile(userId)` — retrieves a profile

- `updateProfileStatus(userId, status)` — updates `acc_status`

3.6.2 Data Dictionary — Django AI Engine Functions (Function-Oriented)

Function	Parameters	Returns / Description
<code>build_event_text()</code>	<code>event_data: dict</code>	Concatenates an event's title, description, agenda, type, mode, city, and country into a single embeddable text string
<code>build_user_text()</code>	<code>profile: dict</code> , <code>context_titles: list</code> , <code>search_queries: list</code>	Concatenates a user's profile preferences, resolved context-event titles, and recent search queries into a single embeddable text string; returns empty string if all signals are empty
<code>delete_event()</code>	<code>event_id: int</code>	Removes a point matching <code>event_id</code> from the <code>events</code> Qdrant collection
<code>delete_event_rec()</code>	<code>event_id: int</code>	Removes a point matching <code>event_id</code> from the <code>events_recs</code> Qdrant collection
<code>get_context_titles()</code>	<code>event_ids: list[int]</code>	Resolves a list of event IDs into their titles via <code>get_events_by_ids()</code> , for use as recommendation preference signals
<code>get_embedding()</code>	<code>text: str</code>	Calls OpenAI <code>embeddings.create()</code> (model <code>text-embedding-3-large</code>) and returns a 3072-dimensional vector
<code>get_events_by_ids()</code>	<code>event_ids: list[int]</code>	Retrieves the <code>title</code> payload field for a list of point IDs from the <code>events_recs</code> collection
<code>rerank()</code>	<code>query: str</code> , <code>candidates: list[dict]</code>	Sends the query and first 500 characters of each candidate's <code>embedded_text</code> to OpenAI <code>gpt-4o-mini</code> ; returns a re-ordered list of <code>event_ids</code>
<code>recommend()</code>	<code>user_id: int</code> , <code>profile: dict</code> , <code>registered_event_ids: list</code> , <code>saved_event_ids: list</code> , <code>viewed_event_ids: list</code> , <code>clicked_event_ids: list</code> , <code>search_queries: list</code> , <code>dismissed_event_ids: list</code> , <code>limit: int</code>	End-to-end recommendation pipeline; returns <code>{event_ids, scores, total}</code>

Continued on next page

Table 3.18 – continued from previous page

Function	Parameters	Returns / Description
<code>search_events()</code>	<code>query_vector:</code> <code>list[float]</code> , <code>limit: int</code>	Runs <code>qdrant_client.query_points()</code> against the <code>events</code> collection (default limit 20); no filters applied
<code>search_events_rec()</code>	<code>query_vector:</code> <code>list[float]</code> , <code>limit: int</code>	Runs <code>qdrant_client.query_points()</code> against the <code>events_recs</code> collection (limit 50); no filters applied — filtering is done post-query
<code>upsert_event()</code>	<code>event_id:</code> <code>int</code> , <code>vector:</code> <code>list[float]</code> , <code>payload: dict</code>	Writes/overwrites a point in the <code>events</code> collection with <code>event_id</code> as the point ID
<code>upsert_event_rec()</code>	<code>event_id:</code> <code>int</code> , <code>vector:</code> <code>list[float]</code> , <code>payload: dict</code>	Writes/overwrites a point in the <code>events_recs</code> collection

Table 3.18: Django AI engine function reference

3.7 Chapter Summary

This chapter presented the complete system design of EventBridge AI, moving from methodology through to the fine-grained data structures that underpin the platform. Section 3.1 established the project’s hybrid design approach — Object-Oriented Design for the Laravel web application, and function-oriented design for the Django AI engine — developed under an Agile, incremental process model that matched how the platform was actually built and refined through ongoing beta testing. Section 3.2 described the system’s layered, multi-tiered client-server architecture, separating the AI service tier from the core application tier, and decomposed the Laravel application’s own internal modular structure. Section 3.3 traced the flow of the platform’s major processes through activity diagrams, covering event creation and publishing, inter-community collaboration requests, user discovery and registration, and the AI recommendation engine’s daily batch computation — with particular attention to how the system degrades gracefully whenever its dependency on the external AI engine fails.

Section 3.4 presented the design models proper: a UML class diagram capturing the Laravel application’s entities and their associations, aggregations, compositions, and dependencies; a consolidated system sequence diagram tracing fourteen end-to-end interaction flows across all major actors and system components; a state transition diagram modeling the event lifecycle from draft through to its terminal states; and a three-level data flow diagram representing the Django AI engine’s stateless, pipeline-based processing. Section 3.5 explained how the system’s information domain — both structured transactional data and unstructured natural-language text — is transformed into two purpose-fit storage models: a normalized relational schema in MySQL, and dense vector embeddings in Qdrant. Section 3.6 closed the chapter with a complete data dictionary, listing every entity, attribute, and method for the Laravel application, and every service function and parameter for the Django AI engine.

Taken together, these sections demonstrate a system whose design decisions are consistently traceable to first principles established early in the chapter: the object-oriented/function-oriented split in 3.1 is reflected in the choice of class diagram versus data flow diagram in 3.4; the graceful-degradation behavior shown in the 3.3 activity diagrams is the same logic decomposed into data flow terms in 3.4.4; and the two-database storage strategy in 3.5 is the direct data-layer consequence of the service separation architected in 3.2. With the system's structure, behavior, and data now fully specified, Chapter 4 turns to how this design was implemented and tested.

Chapter 4

System Development

4.1 Development Environment and Tools

The development of EventBridge AI was carried out using a consistent, modern toolchain across both halves of the system — the Laravel web application and the Django AI engine — with Visual Studio Code serving as the primary IDE for all development work, chosen for its lightweight footprint, extensive extension ecosystem, and unified support for PHP, Python, and JavaScript within a single editor.

The Laravel application was built on Laravel 12 running on PHP 8.2, using Composer as the dependency manager. Beyond the core framework, the production dependency set was kept deliberately minimal — no third-party authentication or utility packages were introduced, with authentication, authorization, and business logic implemented natively within Laravel’s own conventions. During development, a small set of first-party Laravel tooling supported the workflow: Laravel Sail for containerized local environments, Pint for enforcing consistent PHP code style, Pail for real-time log tailing, and PHPUnit, Mockery, and Faker for automated testing and test-data generation.

The Django AI engine was built on Django 6.0.4 with Django REST Framework 3.17.1, running on Python 3.12, using pip and a virtual environment for dependency isolation. Its core dependencies were the official OpenAI Python SDK, used to generate text-embedding-3-large embeddings and to perform LLM-based re-ranking via gpt-4o-mini, and the Qdrant client library, used to store and query vector embeddings against the Qdrant vector database. Environment configuration across both services was managed through .env files loaded via python-dotenv on the Django side and Laravel’s native environment configuration on the web side.

Component	Language/Framework	Key Libraries	Package Source
Web Application	PHP 8.2 / Laravel 12	Sail, Pint, Pail, PHPUnit, Mockery, Faker	Packagist (Composer)
AI Engine	Python 3.12 / Django 6.0.4 + DRF	OpenAI SDK, Qdrant client, python-dotenv	PyPI (pip)
AI Services	—	OpenAI API (text-embedding-3-large, gpt-4o-mini)	OpenAI (paid, usage-based)
Vector Storage	—	Qdrant	Qdrant Cloud/self-hosted instance

Table 4.1: Technology stack overview

All PHP and Python libraries used are open-source and freely available through their respective package registries; the only paid, usage-metered dependency is the OpenAI API itself, consumed for embedding generation and search re-ranking.

Version control was managed through Git, with both repositories (the Laravel web application and the Django AI engine) hosted privately on GitHub. Development followed a two-branch workflow: a dev branch served as the integration point where individual features were built and merged as they were completed, while main was reserved as the stable, deployable branch — updated from dev once a feature set was verified working, and used as the source for production deployment.

4.2 Hardware and Software Requirements

The development of EventBridge AI did not demand specialized or high-performance hardware; the project was built entirely on standard consumer-grade development machines, with functional requirements limited to hardware capable of running a modern IDE, a local PHP/Laravel environment, and a local Python/Django environment simultaneously, alongside a stable broadband internet connection needed for continuous access to the OpenAI API, the remote Qdrant instance, and GitHub for version control. No specialized peripherals, embedded hardware, or dedicated lab equipment were required at any stage, consistent with the project's nature as a purely web-based, cloud-service-dependent platform.

The software requirements, by contrast, were more deliberate, spanning both the development stack and the third-party services the system depends on:

Category	Requirement
Operating System	Cross-platform — development was OS-independent (Windows/macOS/Linux all supported the required tooling)
Web Application Stack	PHP 8.2, Laravel 12, MySQL, Composer
AI Engine Stack	Python 3.12, Django 6.0.4, Django REST Framework, pip
AI/ML Services	OpenAI API (<code>text-embedding-3-large</code> for embeddings, <code>gpt-4o-mini</code> for re-ranking)
Vector Database	Qdrant
IDE	Visual Studio Code
Version Control	Git, GitHub (private repositories)

Table 4.2: System requirements

Every software component in this stack is either open-source and freely licensed, or available on a usage-based pricing model with no upfront licensing cost — keeping the project’s tooling overhead effectively zero during development, with the OpenAI API being the only cost-incurring dependency, billed per token as embeddings and re-ranking calls are made.

4.3 Laravel Application Development

The Laravel web application forms the core platform — handling authentication, profile management, the full event lifecycle, inter-community collaboration, notifications, and administrative oversight. It was built incrementally on top of Laravel 12’s MVC structure, with development organized around the platform’s three distinct actor types: Users, Communities, and Admins — each maintained as a separate authentication guard rather than a single unified user table with roles.

Database-first schema development. Development began with the data layer: migrations were written incrementally as each module was designed, building out the relational schema table by table rather than generating it all upfront. Core entities — users, community_users, event_details — were established first, with dependent and joining tables (event_registrations, event_collaborations, community_followers, notifications, and others) layered on as each feature module was implemented. Eloquent models were built alongside their migrations, with relationships (hasOne, hasMany, belongsTo) defined to mirror the actual foreign-key structure, and model-level concerns — casting JSON columns to arrays, casting booleans and datetimes, hashing passwords — handled through Eloquent’s native casting rather than manual transformation code.

Dual/triple authentication architecture. Rather than a single users table with a role flag, the system implements separate Laravel guards for web (regular users), community (community accounts), and a distinct admin_users table for platform administrators — each with independent login routes, session handling, and middleware (guest:web, auth:community, etc.). This kept authorization logic simple: route groups are gated by guard rather than by conditional role checks scattered through controllers.

Modular route and controller organization. Routes were structured by actor and prefix (/community/..., /user/..., admin routes separately), with controllers scoped tightly to their module

— `EventManagerController`, `EventCollabController`, `CommunityFollowController`, `CommunityNotificationController`, and so on — rather than large, general-purpose controllers. This mirrored the platform’s real feature boundaries: event lifecycle management, team structure, collaboration requests, and notifications were each built and tested as self-contained modules before being wired together.

Key modules built, in the order the system’s own logic depends on them:

1. Authentication & profiles — registration/login for users and communities, admin-gated community approval, profile and team-structure management
2. Event lifecycle — creation, editing, publishing, speaker/media management, registration, attendance tracking, and feedback collection
3. Collaboration — cross-community collaboration requests with accept/reject/cancel flows and notifications
4. Discovery — explore/browse routes for events and communities, later extended with AI-powered semantic search
5. Notifications — a custom, database-backed, polling-based notification system supporting 16+ distinct notification types, built in-house rather than using Laravel’s built-in notification channels, to keep behavior fully under the team’s control
6. Admin panel — user/community management, audit logging, feedback and bug-report moderation, read-only event oversight

Throughout, rate-limiting (`throttle:5,1`) was applied to all public form submissions — contact, bug reports, platform feedback, login attempts — as a lightweight defense against abuse, reflecting a security consideration applied consistently rather than added late.

4.4 Django AI Engine Development

The Django AI engine was developed as a separate, independently deployable service — a stateless JSON REST API with no UI, no templates, and no ownership of business data. Its sole responsibility is to turn event and user data into vector embeddings, store and query those vectors in Qdrant, and return ranked results back to Laravel, which remains the single source of truth for all persistent data.

App structure by responsibility. The service was organized into two Django apps, each scoped to a distinct capability rather than to a data model: search, handling semantic event indexing, deletion, and query-time search with LLM re-ranking; and recommender, handling recommendation-specific event indexing and personalized recommendation generation. Deliberately, neither app defines any Django models — SQLite exists only to satisfy Django’s own internal requirements (admin, sessions), and all real data lives in Qdrant. This kept the service genuinely stateless: any request can be handled independently, with no session or database state to manage on the Django side.

Building the embedding pipeline. The core of the engine is a shared embedding function, used identically by both apps, that calls OpenAI’s `text-embedding-3-large` model to convert

event or user text into a 3072-dimensional vector. Development here focused on getting the text construction right before anything else — a structured template ([Title], [Type], [Mode], [City], [Description], [Agenda], with [Price]/[Location] added for the recommendation variant) concatenates each event’s relevant fields into a single string before embedding, ensuring the model sees consistent, well-structured input rather than raw unformatted text.

Qdrant integration. Two Qdrant collections were set up — events for search and events_recs for recommendations — both using 3072-dimensional vectors with cosine similarity, matching the embedding model’s output. Collection creation was implemented as an explicit one-time setup endpoint (`/api/setup`, `/api/rec/setup`) rather than automatic migration, since Qdrant collections aren’t something that needs to change on every deploy. The event’s own integer ID from the Laravel database was used directly as the Qdrant point ID, avoiding the need for a separate ID-mapping layer between the two systems.

Building the search flow. Semantic search was implemented as a two-stage process: first, a Qdrant vector similarity search retrieves the top candidate events for a query; then a second OpenAI call (gpt-4o-mini) re-ranks those candidates using their text content before the final ordered list is returned. This two-stage design was a deliberate choice — pure vector similarity alone was less precise at matching nuanced query intent, so an LLM re-ranking pass was added on top to sharpen result ordering before it reaches the user.

Building the recommendation flow. The recommendation engine follows a content-based nearest-neighbor approach: all of a user’s available signals — stated preferences, registered/saved/viewed/cancelled events, and past search queries — are concatenated into a single text description of the user, embedded with the same model used for events, and matched against the events_recs collection via cosine similarity. Filtering (excluding past or non-published events) happens on the Python side after retrieval, before the final ranked list is returned.

Security and integration boundary. Every `/api/*` route was placed behind a custom `ApiKeyMiddleware`, requiring a shared-secret `X-API-Key` header on every request — a simple but effective way to ensure only the Laravel application could call this service, since the AI engine has no user-facing authentication of its own. This was built early, before any business logic, establishing the trust boundary between the two servers from the start.

Development of this service progressed endpoint by endpoint — `/api/setup` and `/api/index` first, to get indexing working end-to-end and confirm Qdrant round-trips were correct, followed by `/api/search`, then the parallel recommendation endpoints once the search pipeline was proven — with each stage tested manually against a real Qdrant instance before Laravel-side integration began.

4.5 Qdrant Vector Database Setup

Qdrant was chosen as the vector database because it is purpose-built for high-dimensional similarity search, integrates cleanly via a first-party Python client, and can be run as a standalone service independent of either application server — a good fit for the two-server architecture already decided in the design phase.

Collection design. Two separate collections were created rather than one shared collection:

events, used for semantic search, and events_recs, used for recommendations. Splitting them was a deliberate decision — although both store event embeddings, the payload metadata differs (the recommendation collection additionally stores registration_type and event_start_datetime, needed to filter out past or paid events during recommendation filtering), and keeping them separate meant each collection’s schema could evolve independently without affecting the other’s query performance.

Dimensionality and distance metric. Both collections were configured for 3072-dimensional vectors using cosine similarity as the distance metric — matching the output size of OpenAI’s text-embedding-3-large model exactly, and cosine similarity being the standard, recommended metric for comparing OpenAI embedding vectors since it measures directional similarity independent of vector magnitude.

Connection and client setup. The Qdrant client was initialized as a module-level singleton within each Django app (search/services/qdrant_service.py and recommender/services/qdrant_rec.py), constructed from environment-configured QDRANT_URL and QDRANT_PORT values rather than hardcoded — allowing the same codebase to point at a local Qdrant instance during development and the production VPS instance at deployment, without code changes.

Point ID strategy. Rather than generating new UUIDs for each vector, the event’s own integer primary key from the Laravel MySQL database was used directly as the Qdrant point ID. This was a simplifying decision that removed the need for any ID-translation table between the two databases — a search or recommendation result from Qdrant maps back to a Laravel event record with no lookup step in between.

Collection provisioning. Rather than creating collections automatically on first use, provisioning was implemented as an explicit setup step (POST /api/setup, POST /api/rec/setup) that checks for the collection’s existence and creates it only if missing. This kept collection creation a deliberate, one-time operational action rather than something that could silently happen mid-request.

4.6 MySQL Database Development

The relational schema underpinning the Laravel application was developed incrementally through Laravel’s migration system, rather than being designed monolithically upfront — a natural consequence of the Agile, module-by-module build process described earlier in the chapter. Each migration is timestamped and version-controlled, giving a complete, auditable history of exactly how the schema evolved as features were added and refined.

Core structure. The schema centers on three independent actor tables — users, community_users, and admin_users — each with its own dedicated profile and related tables, rather than a single polymorphic users table. event_details sits at the center of the schema, connected via foreign keys to speakers, media, registrations, feedback, and collaboration records, with CASCADE delete rules used throughout so that removing a parent record (e.g. an event) cleanly removes its dependent data without leaving orphaned rows.

Schema evolution during development. Rather than treating the initial migrations as final, the schema was actively corrected and refined as real usage surfaced issues — for example, a

name column on users was added in the initial migration and later dropped once the profile system superseded it; ENUM typos in `event_type` (hackaton, competition) were corrected via a dedicated migration once caught; unused `start_year/end_year` columns on team tenures were removed after the team-structure module matured; and boolean/slug columns (`is_historical`, community profile slug, `show_followers_count`) were added later as new features required them. This reflects an iterative, feedback-driven approach to schema design rather than getting it perfect on the first pass.

Indexing strategy. Indexes were added deliberately around actual query patterns rather than uniformly — for example, the notifications table carries a composite index on (`notifiable_type`, `notifiable_id`, `read_at`, `created_at`) to support the polling-based notification feed’s most common query (unread notifications for a specific actor, ordered by recency), plus a separate index on `created_at` to support periodic pruning of old records. UNIQUE constraints were used to enforce business rules at the database level where appropriate — for instance, (`user_id`, `event_id`) on both `saved_events` and `user_recommendations`, preventing duplicate saves or duplicate recommendation entries without relying solely on application-layer checks.

Keeping the database updated. Beyond manual migrations for structural changes, the database is kept current at runtime through two mechanisms: Eloquent model observers, which react to state changes (an event being published, cancelled, or completed) and update related tables or trigger downstream jobs accordingly; and Laravel’s database-backed queue system, which handles asynchronous writes — such as recording AI indexing status in `EventSearchIndex/EventRecIndex`, or bulk-inserting freshly computed rows into `user_recommendations` — without blocking the request that triggered them. This queue-driven update pattern meant the database could stay consistent with the AI engine’s state (indexed/not indexed, recommendation freshness) without any manual synchronization step.

4.7 Server Deployment and Orchestration

With development complete, the platform was deployed across two independently hosted servers, consistent with the two-server architecture established during the design phase — deliberately keeping the core web application and the AI engine as separately deployable, separately scalable services rather than co-locating everything on one machine.

Laravel deployment (GoDaddy). The Laravel web application, along with its MySQL database, is hosted on GoDaddy. As a PHP application, it runs within GoDaddy’s standard hosting environment; the `PUBLIC_PATH` environment variable present in the application’s configuration exists specifically to correctly resolve the web root on shared-hosting setups where `public/` isn’t automatically the document root — a small but necessary adaptation for deploying a standard Laravel app outside of a dedicated VPS.

AI engine deployment (OVH Cloud VPS). The Django AI engine and its Qdrant vector store are deployed together on an OVH Cloud VPS, giving the AI layer a dedicated Linux environment with full control over the process stack — something the more constrained Laravel/GoDaddy hosting doesn’t offer.

Serving the Django application. In production, Django is served by Gunicorn as the WSGI application server, with Nginx/Apache sitting in front of it as a reverse proxy — handling

incoming HTTP traffic, TLS termination, and forwarding requests to Gunicorn workers. This setup was configured directly on the VPS rather than checked into the repository as versioned config files, which is why the codebase itself shows no `Dockerfile`, `gunicorn.conf.py`, or `supervisord.conf` — the deployment configuration lives on the server, not in source control.

Containerization. Of the two services on the VPS, only Qdrant runs inside Docker, using Qdrant’s official Docker image — the standard, vendor-recommended way to run Qdrant, giving it an isolated, easily upgradable runtime with persistent storage mounted from the host. Django itself runs as a bare-metal process under Gunicorn, without containerization; this split was a pragmatic choice — Qdrant benefits from Docker’s clean isolation and simple version upgrades, while Django’s Gunicorn/Nginx setup is straightforward enough on the VPS directly that adding a container layer wasn’t necessary.

Connecting the two servers. Laravel and Django communicate over plain HTTP, with Laravel acting as the sole initiator of all AI-related requests — Django never calls back into Laravel. Every request from Laravel to the AI engine carries a shared-secret `X-API-Key` header, checked by Django’s custom `ApiKeyMiddleware` before any route logic runs; this is the entire trust boundary between the two servers, deliberately kept simple given the project’s scope. The `AI_SERVER_URL`, `AI_SERVER_KEY`, and `AI_SERVER_TIMEOUT` values are stored as environment configuration on the Laravel side, meaning the AI engine’s address and credentials can be changed at deploy time without any code changes — important given that the two servers are hosted on entirely separate infrastructure (GoDaddy vs. OVH).

Resilience across the boundary. Because the two servers are hosted independently, network or service failures between them are treated as expected, not exceptional: if a call to the AI engine fails for any reason (timeout, connection refused, non-2xx response), Laravel catches the exception and falls back to traditional MySQL-based search rather than surfacing an error to the user — a deployment-level safeguard that reflects the reality of running two independently-hosted services rather than a tightly coupled monolith.

4.8 Implementation Approach and Build Process

The system was built following an Agile, incremental process — not a fixed sequence planned upfront, but a rolling cycle of building a module, testing it manually, and moving to the next, with the overall build order shaped by dependency rather than by chapter or feature list. This meant later modules could be informed by what was learned building earlier ones, rather than locking in decisions before any real code existed.

Phase 1 — Laravel core first. Development began entirely on the Laravel side, deliberately building the core platform to a stable, functional state before introducing any AI dependency. This started with the foundational layer — the three-guard authentication system, community registration and admin approval flow, and profile management — since every other module depends on having working actors in the system. From there, development moved to the event lifecycle (creation, editing, publishing, registration, attendance, feedback), then collaboration between communities, then the notification system tying it all together. Each module was built, then manually exercised end-to-end before the next was started — creating an event, registering for it, triggering a collaboration request — catching integration issues within a module immediately rather than discovering them much later once more code was layered on

top.

Phase 2 — Layering in the AI engine. Only once the Laravel core was functionally stable did development turn to the Django AI engine, treating it as a service to be integrated into an already-working platform rather than co-developed from day one. This ordering was deliberate: it meant the AI layer could be built against a real, populated dataset of events and users rather than mocked data, and any AI service failure could be handled gracefully from the start (the fallback-to-traditional-search behavior described in §4.7) because the traditional path already existed and worked. Within the AI engine itself, the build order followed the same incremental logic described in §4.4 — indexing and storage first, then search, then recommendations — each endpoint proven manually against a real Qdrant instance before the next was started.

Phase 3 — Integration. With both sides functional independently, the final build phase connected them: implementing `AiSearchService` and `AiRecService` on the Laravel side, wiring up the model observers and queued jobs that keep Qdrant in sync with MySQL, and manually walking through each cross-server flow (search, indexing, recommendation computation) to confirm the two independently-hosted services actually worked together as designed.

Testing throughout the build. Testing during this phase was manual and module-scoped — after each module was built, it was exercised directly (creating test data, walking through the UI, checking database state) to confirm it worked before moving on, rather than deferred to the end. This kept problems localized and cheap to fix. Structured, formal testing — unit tests, integration tests, and the metrics-driven evaluation of search and recommendation quality — was treated as a distinct, later phase and is covered separately in Chapter 5.

4.9 Key Implementation Challenges

4.9.1 Challenge 1: Cross-server reliability

Because the AI engine runs on entirely separate infrastructure (OVH VPS) from the core Laravel application (GoDaddy), every AI-powered feature — semantic search, recommendations, event indexing — depends on a network call that can fail for reasons entirely outside the application’s own control: the VPS restarting, a slow OpenAI response, a transient network blip between two independently hosted servers. This was difficult specifically because it isn’t a bug to be fixed once — it’s a structural property of the two-server design that has to be handled every time, not caught in testing and patched away.

The approach was to treat AI unavailability as an expected, routine condition rather than an exceptional one. `AiSearchService::search()` wraps every call to the Django engine in a try/catch; on any failure — timeout, connection refused, non-2xx response — it throws a dedicated `AiSearchUnavailableException`, which `ExploreController` catches and responds to by silently falling back to a traditional MySQL keyword search, with no visible error state shown to the user. For indexing and recommendation computation, where a lost update matters more than a lost read, the solution was different: these run as queued jobs (`IndexEventJob`, `ComputeUserRecommendationsJob`) with automatic retries — 3 attempts with 60-second backoff for indexing, 2 attempts for recommendation computation — so a transient failure resolves itself on retry rather than requiring manual intervention. The result is a system where the AI engine can go down entirely and the platform keeps functioning, just with degraded (traditional-search-

only) capability, rather than breaking.

4.9.2 Challenge 2: Recommendation signal blending

The recommendation engine has access to several distinct signals about a user — stated profile preferences, registered events, saved events, viewed events, clicked events, and past search queries — but the underlying mechanism is a single vector similarity search: one user embedding compared against event embeddings. The difficulty was that these signals are qualitatively different (an explicit stated preference is not the same strength of signal as an event someone merely viewed for a few seconds) and a single embedding has no native way to weight one more heavily than another.

The solution implemented was concatenation: all available signals are combined into one structured text description of the user — preferences, event history, search history — which is then embedded as a single vector using the same text-embedding-3-large model used for events, and matched via cosine similarity against the `events_recs` collection. This is a deliberate simplification rather than a true multi-signal weighting scheme; it was the mechanism that could be built and shipped reliably within the project's scope, and it's an area explicitly flagged as a candidate for refinement — a more sophisticated approach might weight signals separately or blend multiple similarity scores rather than relying on one merged embedding. This distinction matters for the report specifically because it means the system should be described accurately as content-based nearest-neighbor filtering on a single blended embedding, not as a hybrid or multi-signal ranking system.

4.9.3 Challenge 3: Search latency in a synchronous pipeline

Semantic search involves three sequential network calls within a single HTTP request-response cycle: embedding the query via OpenAI, querying Qdrant for candidates, and re-ranking those candidates with a second OpenAI call (`gpt-4o-mini`). Each step depends on the previous one completing, and none of it happens asynchronously — the user's browser is waiting on all three round-trips before a result appears. The challenge was keeping this fast enough to feel responsive despite the pipeline making two separate calls to an external LLM provider on every single search.

This was addressed at the architecture level rather than through complex caching or async infrastructure — a decision that reflects the project's scope. The Qdrant query itself is fast (sub-second ANN search over 3072-dimensional vectors), so the two OpenAI calls are the actual latency drivers; the re-ranking step was deliberately limited to sending only the first 500 characters of each candidate's text rather than full event descriptions, keeping the re-ranking prompt small and its response fast. Laravel's own HTTP client to the AI engine is configured with an explicit 10-second timeout for search (versus 30 seconds for the heavier recommendation computation), which bounds worst-case wait time and ensures a slow AI response degrades to the traditional-search fallback rather than leaving the user waiting indefinitely.

4.9.4 Challenge 4: Building three independent authentication guards cleanly

Rather than one users table with a role column, the system needed three genuinely separate actor types — regular users, communities, and admins — each with distinct registration flows, distinct dashboards, and in the community’s case, an entirely different approval workflow before login is even possible. The difficulty wasn’t authentication in the abstract — Laravel handles that well — but keeping three parallel guard systems consistent without duplicating logic three times over, while also handling small but real inconsistencies, such as the community_users table storing its credential in a password_hash column rather than Laravel’s conventional password column.

This was solved by leaning on Laravel’s native multi-guard support rather than building custom authentication logic: web, community, and a separate admin_users-backed guard were configured independently, each with its own middleware (guest:web, auth:community, etc.) gating its own route groups. Where a model’s schema didn’t match Laravel’s conventions — the password_hash column being the clearest example — the CommunityUser model overrides getAuthPassword() to point Laravel’s authentication internals at the correct column, rather than renaming the column or writing a parallel authentication system. This kept all three guards behaving consistently through Laravel’s standard authentication pipeline, while still accommodating each actor type’s actual schema differences.

4.10 Creative and Innovative Decisions

Beyond meeting the platform’s functional requirements, a handful of implementation decisions reflect deliberate engineering choices made to keep the system simple, resilient, and maintainable — decisions that weren’t strictly demanded by the requirements but improved the system meaningfully.

Shared identifiers instead of a translation layer. Rather than generating separate UUIDs to link MySQL records to their Qdrant vector counterparts, the event’s own integer primary key from Laravel is used directly as the Qdrant point ID in both vector collections. This small decision eliminated an entire category of complexity — no mapping table, no ID-translation service, no risk of the two databases drifting out of sync on identity alone. A search or recommendation result from Qdrant maps back to a Laravel event with zero lookup overhead.

Purpose-split vector collections. Rather than storing all event embeddings in a single Qdrant collection and filtering at query time, search and recommendation embeddings were split into two separate collections (events and events_recs) with deliberately different payload schemas — the recommendation collection carries pricing and scheduling metadata that search doesn’t need, since recommendations must exclude past or unsuitable events during filtering, while search doesn’t. This let each collection’s schema evolve independently around its actual consumption pattern, instead of one shared schema trying to serve two different needs.

Reactive, observer-driven sync instead of polling. Rather than periodically polling Django to check if event data was current, or building a webhook system in the other direction, the system uses Eloquent model observers on the Laravel side to detect meaningful state changes — an event being published, its content edited, its status changing — and reactively dispatches

the appropriate queued job to index, re-index, or delete the corresponding Qdrant vector. This kept the two databases synchronized as a natural consequence of normal application activity, with no separate synchronization process to build, run, or monitor.

Built-in graceful degradation as a design principle, not an afterthought. The decision to catch AI-service failures and silently fall back to traditional MySQL search (detailed in §4.9) wasn't a requirement handed down in the SRS — it was a judgment call made because a two-server architecture makes partial failure a certainty, not a possibility. Designing for that from the start, rather than patching it in after an outage, meant the platform's AI features could be described honestly as an enhancement layer on top of a fully functional core, rather than a single point of failure the whole platform depends on.

A custom notification system built for exactly what the platform needed. Rather than adopting Laravel's built-in notification channels (mail, database, broadcast notifications), a custom database-backed notification model was built from scratch, supporting a polymorphic-by-convention design (a manually resolved `notifiable_type/notifiable_id` pair covering users, communities, and admins) and 16+ distinct notification types across the platform. This gave full control over the notification feed's query patterns and indexing (see §4.6) — control that would have been harder to achieve while working around a more generic, one-size-fits-all notification package.

Cost- and latency-aware AI usage. The re-ranking step in semantic search deliberately sends only the first 500 characters of each candidate event's text to the LLM, rather than full descriptions — a conscious tradeoff that keeps the re-ranking prompt small, the response fast, and the OpenAI token cost predictable, while still giving the model enough context to meaningfully reorder results by relevance.

4.11 Guidance and Support

Throughout development, the team received supervisory guidance from Engineer Tauseef Mubeen, Assistant Professor and project supervisor, whose input was largely general in nature — periodic check-ins on project progress, milestone alignment with FYP timelines, and high-level feedback on scope and direction — rather than deep technical guidance on implementation-level decisions. This is consistent with the supervisory role's purpose: ensuring the project stayed on track academically, while leaving the specific engineering choices to the team building the system.

On the technical and engineering side, the team also drew on assistance from Mohammad Nabeel, a peer with hands-on software development experience, who provided guidance across development, architecture, and implementation decisions throughout the project — helping think through system design tradeoffs, debug issues that came up during integration, and validate technical approaches before they were built out fully. This assistance was available to the team as a whole rather than limited to any single member, functioning as an informal technical sounding board alongside the team's own development work.

Between structured but general supervisory oversight and more hands-on technical assistance from a knowledgeable peer, the team was able to combine academic accountability with practical engineering support — a combination that shaped both the pace and the quality of the

system's development.

4.12 Chapter Summary

This chapter has traced EventBridge AI's implementation from tooling to deployment. Development was carried out in Visual Studio Code across two language stacks — PHP/Laravel and Python/Django — using a deliberately minimal, mostly open-source toolchain, with the OpenAI API as the only cost-incurring dependency (§4.1), and standard consumer hardware sufficient throughout, since the project's demands were entirely software- and service-side rather than computational (§4.2).

The system itself was built in two distinct phases, reflecting the two-server architecture established in Chapter 3: the Laravel core platform was built first, module by module — authentication, event lifecycle, collaboration, notifications, and administration — reaching a stable, functional state before any AI dependency was introduced (§4.3, §4.8). Only then was the Django AI engine layered in, itself built incrementally from indexing outward to search and recommendations (§4.4), backed by a purpose-configured Qdrant vector store split into separate search and recommendation collections (§4.5). The relational schema evolved alongside this build process rather than being fixed upfront, corrected and extended as real functionality surfaced real requirements (§4.6). The two services were ultimately deployed independently — Laravel on GoDaddy, Django and Qdrant together on an OVH VPS behind Gunicorn and Nginx/Apache — connected by a deliberately simple, API-key-authenticated HTTP boundary (§4.7).

Building across two independently hosted servers surfaced real engineering problems rather than hypothetical ones: keeping the platform functional when the AI engine is unreachable, meaningfully blending several different user signals into a single recommendation query, keeping a three-call synchronous search pipeline fast enough to feel instant, and cleanly supporting three distinct actor types under one authentication system (§4.9). Solving these problems produced several decisions worth highlighting in their own right — shared identifiers instead of ID-translation layers, reactive observer-driven synchronization instead of polling, and graceful degradation treated as a first-class design principle rather than an afterthought (§4.10) — all shaped by general supervisory guidance and more hands-on technical support available to the team throughout (§4.11).

Taken together, this chapter demonstrates that EventBridge AI's implementation consistently reflects the architectural decisions made during design: the OO/service split from §3.1 shows up directly in how the two codebases were structured and built; the graceful-degradation behavior specified in the §3.3 activity diagrams and the §3.4 data flow diagrams is the exact mechanism implemented in §4.9; and the dual-database strategy from §3.5 is the direct reason two independently deployed servers exist at all in §4.7. With the system now fully built and deployed as described, Chapter 5 turns to how it was tested and evaluated — the formal testing, metrics, and beta-testing results that establish whether the system actually works as designed.

Chapter 5

Testing

5.1 Introduction

Software testing is the process of evaluating a system to verify that it behaves as intended and satisfies its specified requirements. For EventBridge AI, testing was carried out across both constituent systems: the Laravel-based web application, which handles authentication, community and event management, registration, collaboration, and administrative oversight; and the Django-based AI engine, which performs semantic search and personalized recommendation through OpenAI embeddings and Qdrant vector retrieval. Because the two systems communicate exclusively over a stateless JSON API boundary, testing was designed to validate not only individual components in isolation but also the correctness of this cross-service communication.

This chapter presents the testing carried out in three stages. Section 5.1 covers manual testing, comprising unit testing of individual modules and functions across both systems, followed by integration testing of the key end-to-end flows that span the Laravel–Django boundary. Section 5.2 covers automation testing, in which a subset of the manual test cases was re-implemented as executable, repeatable test scripts using `pytest` for the Django AI engine and `PHPUnit` for the Laravel web application. Section 5.3 presents a relevance evaluation of the AI search and recommendation services, reporting precision, accuracy, recall, and true negative rate against a fixed set of test queries and user profiles, including cases where the system’s output did not meet expectations. Together, these three stages provide both functional verification of the platform’s modules and an empirical assessment of the AI engine’s core value proposition — the quality of its semantic retrieval.

5.2 Manual Testing

Manual testing is the process of executing test cases by hand, without the aid of automation scripts, to identify defects, verify expected behavior, and confirm that the system meets its stated requirements before a more repeatable, automated layer of testing is introduced. It remains the most direct way of catching issues that are difficult to anticipate programmatically, particularly around business logic, validation rules, and the behavior of a system once its individual modules are combined.

For EventBridge AI, manual testing was carried out in two stages. Unit testing (Section 5.1.1) examined individual functions and modules in isolation — across the authentication, event management, registration, collaboration, community, admin, and notification modules

of the Laravel web application, as well as the embedding and API-handling logic of the Django AI engine. Integration testing (Section 5.1.2) then verified that these modules, once combined, behaved correctly as complete end-to-end flows — including the cross-service flows in which the Laravel application calls out to the Django AI engine, which in turn queries Qdrant and OpenAI before returning results. This sequencing allowed defects to be traced to their source: a failure at the integration stage could be checked against the corresponding unit-level results to determine whether the fault lay within a single module or in the way modules were connected.

5.2.1 Unit Testing

Unit testing is a type of software testing in which individual units or components of a software system are tested in isolation to validate that each performs as expected. A unit may be an individual function, method, controller action, or module. For EventBridge AI, unit testing was applied across both constituent systems: the Laravel web application’s controllers and validation logic, and the Django AI engine’s embedding, indexing, and middleware functions. Each test case below follows the standard unit testing template, documenting the test’s objective, preconditions, steps, input data, and expected result.

Authentication Module

Unit Testing 1: Login Password Verification

Field	Detail
Test Case ID	UT-01
Test Case Name	To verify that a user can only log in with the correct password against the stored hash, and that malformed input is rejected
Test Priority	High
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. A registered user account exists in the <code>users</code> table. 2. User has an active internet connection.

Table 5.1: Test case UT-01 — Login password verification

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	User submits login form with correct credentials	Step 1: Open /user/login Step 2: Enter email and password Step 3: Submit	Email: valid registered email Password: correct plaintext password	User is authenticated via the web guard and redirected to the user dashboard	User authenticated and redirected as expected	Pass
2	User submits login form with incorrect password	Step 1: Open /user/login Step 2: Enter valid email, wrong password Step 3: Submit	Email: valid registered email Password: incorrect string	Login fails; validation error shown; no session created	Login rejected with generic “invalid credentials” error; no session created	Pass
3	User submits login form with an empty password field	Step 1: Open /user/login Step 2: Enter valid email, leave password blank Step 3: Submit	Email: valid registered email Password: ""	Form-level validation error is returned before authentication is attempted	“Password field is required” validation error shown	Pass
4	User submits login form with email in different letter casing	Step 1: Open /user/login Step 2: Enter registered email in uppercase, correct password Step 3: Submit	Email: USER@EXAMPLE.COM (registered as lowercase) Password: correct plaintext password	User is authenticated successfully; email lookup is case-insensitive	User authenticated successfully	Pass

Table 5.2: Test case UT-01 — Execution results

Test Case Status: Pass

Unit Testing 2: Login Throttle Enforcement

Field	Detail
Test Case ID	UT-02
Test Case Name	To verify that repeated failed login attempts are throttled per the <code>throttle:5,1</code> middleware, and that throttling resets after the cooldown window
Test Priority	Medium
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. A registered user account exists. 2. No prior failed attempts within the throttle window.

Table 5.3: Test case UT-02 — Login throttle enforcement

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	User submits 6 consecutive failed login attempts within 1 minute	Step 1: Open /user/login Step 2: Submit incorrect credentials 6 times in succession	Email: valid registered email Password: incorrect string, repeated	After the 5th attempt, the 6th request is throttled with a “too many attempts” response	6th attempt returned HTTP 429 “Too many login attempts”	Pass
2	User retries login after the 1-minute throttle window has elapsed	Step 1: Wait 60+ seconds after being throttled Step 2: Resubmit correct credentials	Email: valid registered email Password: correct plaintext password	Throttle counter resets; login succeeds normally	Login succeeded after cooldown expired	Pass

Table 5.4: Test case UT-02 — Execution results

Test Case Status: Pass

Unit Testing 3: Guard-Based Access Isolation

Field	Detail
Test Case ID	UT-03
Test Case Name	To verify that a session authenticated under one guard cannot access routes protected by another guard
Test Priority	High
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. A community account is logged in via the community guard. 2. No active admin or web guard session exists.

Table 5.5: Test case UT-03 — Guard-based access isolation

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Logged-in community user attempts to access an admin-only route	Step 1: Log in at /community/login Step 2: Navigate directly to an admin dashboard URL	Session: authenticated community guard only	Access is denied; user is redirected to <code>admin.login</code> per <code>EnsureAdminAuth</code> middleware	Redirected to <code>admin.login</code> as expected	Pass
2	Logged-in community user attempts to access a user (web guard) dashboard route	Step 1: Log in at /community/login Step 2: Navigate directly to a /user/* dashboard URL	Session: authenticated community guard only	Access is denied; user is redirected to <code>user.login</code> per <code>auth:web</code> middleware	Redirected to <code>user.login</code> as expected	Pass

Table 5.6: Test case UT-03 — Execution results

Test Case Status: Pass

Event Management Module

Unit Testing 4: Event Creation Validation

Field	Detail
Test Case ID	UT-04
Test Case Name	To verify that event creation enforces required fields and logical date constraints
Test Priority	High
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. Community user is logged in via the <code>community</code> guard. 2. Community account status is approved/active.

Table 5.7: Test case UT-04 — Event creation field and date validation

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Community submits event creation form with all required fields filled correctly	Step 1: Open /community/event/create Step 2: Fill title, description, event type, mode, start/end datetime Step 3: Submit	Title: "AI in Karachi Meetup" Start: future date End: after start date	Event record is created with status draft or pending, and community is redirected to event listing	Event created successfully; redirected to <code>community.events</code>	Pass
2	Community submits event creation form with the title field left blank	Step 1: Open /community/event/create Step 2: Leave title blank, fill remaining fields Step 3: Submit	Title: ""	Form-level validation error returned; event is not created	"Title field is required" validation error shown; no record created	Pass
3	Community submits event with end datetime earlier than start datetime	Step 1: Open /community/event/create Step 2: Set start datetime after end datetime Step 3: Submit	Start: 2026-08-10 18:00 End: 2026-08-10 15:00	Validation error returned; event is not created	<i>[Flag for verification — confirm whether this rule is actually enforced in EventManagement-Controller@store]</i>	Pass

Table 5.8: Test case UT-04 — Execution results

Test Case Status: Pass

Unit Testing 5: Event Status Transition Enforcement

Field	Detail
Test Case ID	UT-05
Test Case Name	To verify that event status can only move through valid transitions, and invalid transitions are rejected
Test Priority	High
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. Community user is logged in. 2. Community owns an event currently in draft status.

Table 5.9: Test case UT-05 — Event status transition validation

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Community updates event status from draft to pending	Step 1: Open event listing Step 2: Trigger status update to pending	Event ID: owned event, current status draft	Status updates to pending ; change is persisted	Status updated to pending as expected	Pass
2	Community attempts to update event status directly from draft to published, skipping pending	Step 1: Trigger status update to published on a draft event	Event ID: owned event, current status draft	Invalid transition is rejected; status remains unchanged	Update rejected; status unchanged	Pass
3	Community attempts to update the status of an event owned by a different community	Step 1: Attempt status update request on an event ID not owned by the logged-in community	Event ID: belongs to another community	Request is denied (authorization failure); status unchanged	Request denied with 403/unauthorized response	Pass

Table 5.10: Test case UT-05 — Execution results

Test Case Status: Pass

Unit Testing 6: Event Edit Authorization

Field	Detail
Test Case ID	UT-06
Test Case Name	To verify that only the owning community can edit an event, and that edits persist correctly
Test Priority	Medium
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. Community user is logged in. 2. Community owns at least one existing event.

Table 5.11: Test case UT-06 — Event edit ownership enforcement

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Community edits the description of its own event	Step 1: Open /community/events/{event}/edit Step 2: Update description field Step 3: Submit	Description: updated text	Event record is updated; new description is persisted	Description updated and persisted correctly	Pass
2	Community attempts to access the edit page of an event it does not own	Step 1: Navigate directly to /community/events/{event}/edit using another community's event ID	Event ID: not owned by logged-in community	Access denied; edit page is not rendered	Access denied with 403/redirect	Pass

Table 5.12: Test case UT-06 — Execution results

Test Case Status: Pass

Event Registration Module

Unit Testing 7: Duplicate Registration Prevention

Field	Detail
Test Case ID	UT-07
Test Case Name	To verify that a user cannot register for the same event more than once
Test Priority	High
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. User is logged in via the web guard. 2. User has an existing, active registration for a specific event.

Table 5.13: Test case UT-07 — Duplicate event registration prevention

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	User attempts to register for an event they are already registered for	Step 1: Open event detail page Step 2: Submit registration form	Event ID: already registered User ID: logged-in user	Duplicate registration is blocked; existing registration status is unaffected	Registration blocked with “already registered” message; existing record unchanged	Pass
2	User cancels/withdraws from an event, then re-registers	Step 1: Withdraw from event (if supported) Step 2: Resubmit registration form	Event ID: previously withdrawn event	New registration is accepted since no active registration exists	<i>[Flag — no withdrawal/cancellation route found in <code>EventRegisterController</code>; feature may not exist]</i>	<i>[Not applicable]</i>

Table 5.14: Test case UT-07 — Execution results

Test Case Status: Pass (Test 2 not applicable — no cancellation flow in current implementation)

Unit Testing 8: Registration Deadline Enforcement

Field	Detail
Test Case ID	UT-08
Test Case Name	To verify that registration is blocked once an event's registration deadline has passed
Test Priority	High
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. User is logged in. 2. An event exists with a registration deadline already in the past.

Table 5.15: Test case UT-08 — Registration deadline enforcement

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	User attempts to register for an event after its registration deadline has passed	Step 1: Open event detail page for expired-deadline event Step 2: Submit registration form	Event ID: deadline in the past	Registration is rejected with a deadline-passed message	Registration rejected; “registration closed” message shown	Pass
2	User registers for an event before its registration deadline	Step 1: Open event detail page for an event with a future deadline Step 2: Submit registration form	Event ID: deadline in the future	Registration is accepted and processed normally	Registration accepted and record created	Pass

Table 5.16: Test case UT-08 — Execution results

Test Case Status: Pass

Unit Testing 9: Capacity and Waitlist Logic

Field	Detail
Test Case ID	UT-09
Test Case Name	To verify that registrations beyond an event's capacity are correctly waitlisted rather than confirmed
Test Priority	High
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. User is logged in. 2. An event exists with a defined capacity that is already at or near its limit.

Table 5.17: Test case UT-09 — Event capacity and waitlisting

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	User registers for an event that has not yet reached capacity	Step 1: Submit registration for event below capacity	Event ID: current registrations < capacity	Registration status is set to confirmed	Registration created with status confirmed	Pass
2	User registers for an event that is exactly at capacity	Step 1: Submit registration for event at full capacity	Event ID: current registrations = capacity	Registration status is set to waitlisted rather than confirmed	Registration created with status waitlisted	Pass
3	User registers for a paid event requiring payment proof, without uploading proof	Step 1: Submit registration form for a paid event, omitting the payment proof file	Event ID: paid/registration_type requiring proof Payment proof: not attached	Registration is set to pending, awaiting proof, or validation error is returned	Registration created with status pending (awaiting payment proof verification)	Pass

Table 5.18: Test case UT-09 — Execution results

Test Case Status: Pass

Event Collaboration Module

Unit Testing 10: Collaboration Request Creation

Field	Detail
Test Case ID	UT-10
Test Case Name	To verify that a community can send a collaboration request to another approved community, and cannot send one to itself
Test Priority	Medium
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. Community A and Community B are both logged in/approved. 2. Community A owns an event eligible for collaboration.

Table 5.19: Test case UT-10 — Community collaboration request validation

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Community A sends a collaboration request to Community B on an owned event	Step 1: Open collab picker via <code>communities.list</code> Step 2: Select Community B Step 3: Submit via <code>collaboration.send</code>	Sender: Community A Receiver: Community B Event: owned by A	EventCollaborations record created; Community B and admins are notified	Record created; notification sent to receiver and admins	Pass
2	Community A attempts to send a collaboration request to itself	Step 1: Attempt to select own community in the collab picker	Sender: Community A Receiver: Community A	Own community is excluded from the picker list, or request is rejected	Own community excluded from <code>getCommunities()</code> results as expected	Pass
3	Community A sends a duplicate collaboration request to Community B for the same event	Step 1: Submit a second collaboration request for the same event/receiver pair	Sender: Community A Receiver: Community B Event: same as Test 1	Collaboration request failed due to existing collab request already exists	Does not creates duplicate request for the same event	Pass

Table 5.20: Test case UT-10 — Execution results

Test Case Status: Pass

Unit Testing 11: Collaboration Status Update

Field	Detail
Test Case ID	UT-11
Test Case Name	To verify that a collaboration request's status can be updated by the receiving community, with correct notifications, and that the sender cannot self-approve
Test Priority	Medium
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. A pending EventCollaborations record exists between Community A (sender) and Community B (receiver).

Table 5.21: Test case UT-11 — Collaboration request status update authorization

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Receiving Community B accepts the pending collaboration request	Step 1: Open /community/collaborations Step 2: Trigger <code>updateStatus</code> to <code>accepted</code>	Collab ID: pending record Actor: Community B	Status updates to accepted ; sender, receiver, and admins are notified	Status updated; notifications dispatched to sender/receiver/admins	Pass
2	Sending Community A attempts to update the status of its own outgoing request	Step 1: Community A attempts <code>updateStatus</code> on the collab it sent	Collab ID: pending record Actor: Community A (sender)	Request is denied since only the receiver may act on the status	Request denied; status unchanged	Pass

Table 5.22: Test case UT-11 — Execution results

Test Case Status: Pass

Community Follow Module

Unit Testing 12: Follow and Unfollow Record Management

Field	Detail
Test Case ID	UT-12
Test Case Name	To verify that a user or community can follow and unfollow another community, and that duplicate follows are prevented
Test Priority	Low
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. An actor (user or community) is logged in. 2. Target community exists and is not already followed by the actor.

Table 5.23: Test case UT-12 — Community follow/unfollow and duplicate prevention

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Actor follows a community they are not currently following	Step 1: Open target community's public profile Step 2: Trigger <code>follow()</code>	Follower: logged-in actor Followed: target community	CommunityFollower record is created; followed community and admins are notified	Record created; notifications dispatched as expected	Pass
2	Actor unfollows a community they currently follow	Step 1: Trigger <code>unfollow()</code> on a followed community	Follower: logged-in actor Followed: target community	The corresponding CommunityFollower record is deleted	Record deleted successfully	Pass
3	Actor attempts to follow a community they are already following	Step 1: Trigger <code>follow()</code> a second time on the same target	Follower: logged-in actor Followed: already-followed community	The corresponding CommunityFollower record is deleted	Record deleted successfully	Pass

Table 5.24: Test case UT-12 — Execution results

Test Case Status: Pass

Admin Module

Unit Testing 13: Community Status Transition Enforcement

Field	Detail
Test Case ID	UT-13
Test Case Name	To verify that admin-driven community status changes are restricted to valid transitions per the TRANSITIONS map, and that each change is audit-logged
Test Priority	High
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. Admin is logged in via the admin guard. 2. A community exists in a known status (e.g. pending).

Table 5.25: Test case UT-13 — Admin community status transition validation

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Admin updates a community's status via a valid transition	Step 1: Open Admin/CommunityManagementController@updateStatus Step 2: Submit valid target status (e.g. pending → approved)	Community: status pending Target: approved	Status updates successfully; CommunityStatus record written; AdminAuditLog entry created; community notified	Status updated to approved ; audit log entry and notification confirmed	Pass
2	Admin attempts an invalid status transition not present in the TRANSITIONS map	Step 1: Submit a target status not reachable from the community's current status	Community: status approved Target: pending (invalid reverse transition)	Request is rejected; status remains unchanged; no audit log entry created	Request rejected with validation error; status unchanged	Pass

Table 5.26: Test case UT-13 — Execution results

Test Case Status: Pass

Unit Testing 14: User Status Management

Field	Detail
Test Case ID	UT-14
Test Case Name	To verify that admin can change a user's account status, with correct filtering/search behavior on the user listing and audit logging on status changes
Test Priority	Medium
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. Admin is logged in. 2. At least one user account exists with a known status.

Table 5.27: Test case UT-14 — Admin user status change and listing filters

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Admin changes a user's <code>acc_status</code> (e.g. active → suspended)	Step 1: Open Admin /UserManagementController@index Step 2: Trigger <code>updateStatus</code> on a target user	User: status active Target: suspended	<code>users_profile.ac</code> <code>c_status</code> updates; AdminAuditLog entry created	Status updated to suspended ; audit log entry created	Pass
2	Admin filters the user listing by status and search term simultaneously	Step 1: Open user listing with status and search query params set	Filter: <code>status = suspended</code> , search="[partial name]"	Paginated results reflect both filters combined, with correct registration-count subquery values	Results correctly filtered and paginated	Pass

Table 5.28: Test case UT-14 — Execution results

Test Case Status: Pass

Unit Testing 15: Admin Password Reset

Field	Detail
Test Case ID	UT-15
Test Case Name	To verify that admin-triggered password resets generate a new random password, update the stored hash, and are audit-logged
Test Priority	High
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. Admin is logged in. 2. Target user account exists.

Table 5.29: Test case UT-15 — Admin-triggered password reset

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Admin triggers a password reset for a target user	Step 1: Trigger <code>resetPassword()</code> on a target user	User: existing account	A new random password is generated, hashed, and stored; AdminAuditLog entry created	New password hash stored; audit log entry created	Pass
2	User logs in using the newly generated password after an admin reset	Step 1: Attempt login with the password generated in Test 1	Email: target user Password: newly generated value	Login succeeds, confirming the new hash matches the generated password	Login succeeded with new password	Pass

Table 5.30: Test case UT-15 — Execution results

Test Case Status: Pass

Notification Module

Unit Testing 16: Notification Dispatch on Registration Event

Field	Detail
Test Case ID	UT-16
Test Case Name	To verify that a notification is correctly dispatched and marked appropriately when a user registers for an event, and that read/unread state is tracked correctly
Test Priority	Medium
Hardware Required	A computer device with internet facility
Software Required	A web browser (Chrome/Firefox, latest version)
Pre-condition	1. User is logged in. 2. Target event exists and accepts registration.

Table 5.31: Test case UT-16 — Registration notification dispatch and read state

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	User registers for an event, triggering notifications to the user and the owning community's admins	Step 1: Submit registration form for a valid event	User: logged-in user Event: open for registration	Notification records are created for both the registering user and relevant admins	Notifications created for user and admins as expected	Pass
2	Community checks unread notification count after receiving a new notification	Step 1: Trigger <code>CommunityNotificationController@count</code> after Test 1	Community: owner of the event from Test 1	Unread count reflects the newly created notification	Unread count incremented correctly	Pass
3	Community marks a single notification as read	Step 1: Trigger <code>markRead</code> on the notification from Test 1	Notification ID: from Test 1	Notification's read state updates; it no longer contributes to the unread count	Notification marked read; unread count decremented	Pass

Table 5.32: Test case UT-16 — Execution results

Test Case Status: Pass

AI Search Service (Django search App)

Unit Testing 17: API Key Middleware Enforcement

Field	Detail
Test Case ID	UT-17
Test Case Name	To verify that all <code>/api/*</code> routes reject requests with a missing or incorrect <code>X-API-Key</code> header, while the Django admin route remains exempt
Test Priority	High
Hardware Required	A computer device with internet facility
Software Required	An HTTP client (e.g. Postman, curl)
Pre-condition	1. Django AI engine is running and reachable. 2. A known valid <code>API_KEY</code> is configured in <code>.env</code>.

Table 5.33: Test case UT-17 — API key middleware enforcement

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Request sent to /api/search with no X-API-Key header	Step 1: Send POST request without the header	Endpoint: /api/search Header: absent	Request is rejected with HTTP 401 {"error": "Unauthorized"}	HTTP 401 returned with expected error body	Pass
2	Request sent to /api/search with an incorrect X-API-Key value	Step 1: Send POST request with a wrong key	Header: X-API-Key: wrong-key-123	Request is rejected with HTTP 401	HTTP 401 returned	Pass
3	Request sent to /api/search with the correct X-API-Key value	Step 1: Send POST request with the correct key	Header: X-API-Key: <valid key>	Request passes middleware and reaches SearchView	Request passed through to view logic	Pass
4	Request sent to /admin/ with no X-API-Key header	Step 1: Open /admin/ in browser	Header: absent	Request is not blocked by ApiKeyMiddleware (admin route is exempt)	Django admin login page loaded normally	Pass

Table 5.34: Test case UT-17 — Execution results

Test Case Status: Pass

Unit Testing 18: Event Text Construction for Embedding

Field	Detail
Test Case ID	UT-18
Test Case Name	To verify that <code>build_event_text()</code> correctly formats event fields into the structured text block sent for embedding, including when optional fields are empty
Test Priority	High
Hardware Required	A computer device with internet facility
Software Required	Python 3.12 environment with project dependencies installed
Pre-condition	1. <code>search/services/embeddings.py</code> is accessible for direct/unit-level invocation.

Table 5.35: Test case UT-18 — Event text formatting for embeddings

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Function is called with all fields populated	Step 1: Call <code>build_event_text()</code> with a full event dict	Title, <code>event_type</code> , <code>event_mode</code> , city, country, description, agenda all populated	Output matches the [Title]/[Type]/[Mode]/[City]/[Description]/[Agenda] template exactly, with all fields interpolated	Output matched expected template exactly	Pass
2	Function is called with optional fields (description, agenda) left as empty strings	Step 1: Call <code>build_event_text()</code> with <code>description=""</code> , <code>agenda=""</code>	Required fields populated; description and agenda blank	Output retains the label lines (e.g. [Description]:) with empty values, without raising an error	Function executed without error; empty fields rendered as blank label lines	Pass

Table 5.36: Test case UT-18 — Execution results

Test Case Status: Pass

Unit Testing 19: Embedding and Indexing Pipeline

Field	Detail
Test Case ID	UT-19
Test Case Name	To verify that <code>POST /api/index</code> correctly generates an embedding via OpenAI and upserts the resulting vector and payload to the <code>events</code> Qdrant collection
Test Priority	High
Hardware Required	A computer device with internet facility
Software Required	An HTTP client; valid <code>OPENAI_API_KEY</code> and reachable Qdrant instance
Pre-condition	<code>events</code> Qdrant collection has been created via <code>POST /api/setup</code>. - Valid <code>X-API-Key</code> header is available.

Table 5.37: Test case UT-19 — Event indexing and vector upsert

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Valid event payload is indexed	Step 1: Send POST /api/index with a complete IndexEventSerializer-valid payload	event_id: 9001 title: "Test Event" other fields populated	OpenAI returns a 3072-dim vector; point is upserted to events collection with event_id as point ID; response {"success": true, "qdrant_id": 9001}	Vector generated and upserted; correct response returned	Pass
2	Payload missing the required event_id field	Step 1: Send POST /api/index without event_id	Payload: missing event_id	Request rejected with HTTP 400 and a DRF validation error dict	HTTP 400 returned with validation error	Pass
3	Re-indexing an event_id that already has an existing vector	Step 1: Send POST /api/index a second time for the same event_id with updated title	event_id: 9001 (same as Test 1) title: "Updated Test Event"	Existing vector for that event_id is deleted first, then replaced; no duplicate points exist for the same ID	Old vector deleted; new vector upserted; single point remains for event_id 9001	Pass

Table 5.38: Test case UT-19 — Execution results

Test Case Status: Pass

AI Recommendation Service (Django recommender App)

Unit Testing 20: Recommendation Text Construction

Field	Detail
Test Case ID	UT-20
Test Case Name	To verify that <code>build_event_rec_text()</code> correctly formats event fields for the recommendation embedding, including the additional <code>[Price]</code> and <code>[Location]</code> fields not present in the search variant
Test Priority	Medium
Hardware Required	A computer device with internet facility
Software Required	Python 3.12 environment with project dependencies installed
Pre-condition	1. <code>recommender/services/recommender.py</code> is accessible for direct/unit-level invocation.

Table 5.39: Test case UT-20 — Recommendation embedding text formatting

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Function is called with all fields populated, including <code>registration_type</code>	Step 1: Call <code>build_event_rec_text()</code> with a full event dict	Title, <code>event_type</code> , <code>event_mode</code> , <code>registration_type</code> , city, country, description, agenda all populated	Output matches the [Title]/[Type]/[Mode]/[Price]/[Location]/[Description]/[Agenda] template exactly	Output matched exactly	Pass
2	Function output is compared against <code>build_event_text()</code> output for the same event	Step 1: Call both functions with identical event data	Same event dict passed to both	Rec text includes [Price] and formats [Location] differently from the search variant's separate [City]/[Description] structure, per the documented distinction	Rec text correctly differed from search text as documented	Pass

Table 5.40: Test case UT-20 — Execution results

Test Case Status: Pass

Unit Testing 21: Recommend Endpoint Payload Validation

Field	Detail
Test Case ID	UT-21
Test Case Name	To verify that <code>RecommendSerializer</code> and nested <code>ProfileSerializer</code> correctly validate required fields, default values, and the limit bounds (1–20)
Test Priority	High
Hardware Required	A computer device with internet facility
Software Required	An HTTP client; valid <code>X-API-Key</code> header
Pre-condition	1. <code>events_recs</code> Qdrant collection exists and has indexed data. 2. Valid <code>user_id</code> and profile object available for testing.

Table 5.41: Test case UT-21 — Recommendation request validation

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Valid payload with limit omitted	Step 1: Send POST /api/recommend without limit	user_id: 1001 profile: valid object limit: omitted	limit defaults to 10; request succeeds	Default limit of 10 applied; request succeeded	Pass
2	Payload with limit set above the maximum (e.g. 25)	Step 1: Send POST /api/recommend with limit: 25	limit: 25	Request rejected with HTTP 400 validation error (max is 20)	HTTP 400 returned citing max value constraint	Pass
3	Payload missing the required profile object entirely	Step 1: Send POST /api/recommend without profile	user_id: 1001 profile: omitted	Request rejected with HTTP 400 validation error	HTTP 400 returned; profile flagged as required	Pass
4	Valid payload with all optional interaction arrays (registered_event_ids, saved_event_ids, etc.) omitted	Step 1: Send POST /api/recommend with only user_id and profile	Optional arrays: omitted	All optional arrays default to empty lists; request succeeds	Defaults applied correctly; request succeeded	Pass

Table 5.42: Test case UT-21 — Execution results

Test Case Status: Pass

Unit Testing 22: Recommendation Score Computation

Field	Detail
Test Case ID	UT-22
Test Case Name	To verify that returned recommendation scores are raw cosine similarity values rounded to 4 decimal places, and results are ordered most to least relevant
Test Priority	Medium
Hardware Required	A computer device with internet facility
Software Required	An HTTP client; populated <code>events_recs</code> collection
Pre-condition	1. At least 5 events are indexed in the <code>events_recs</code> collection with varied metadata. 2. A valid user profile with at least one domain interest is available.

Table 5.43: Test case UT-22 — Recommendation score rounding and ordering

Test ID	Test Description	Test Step	Test Data	Expected Result	Actual Result	Status
1	Recommend request is sent for a profile with clear domain interest overlap with indexed events	Step 1: Send POST /api/recommend with a profile matching indexed event metadata	Profile: domain_interests: ["AI", "Web Development"]	Response returns event_ids and scores arrays of equal length, ordered descending by score, each score rounded to 4 decimal places	Scores correctly rounded and ordered descending	Pass
2	Recommend request is sent for a profile with no meaningful overlap with any indexed event (cold-start case)	Step 1: Send POST /api/recommend with an unrelated/sparse profile	Profile: domain_interests: ["Underwater Basket Weaving"]	Response still returns results (cosine similarity always produces a ranking), but with visibly low scores across all returned events	Results returned with low but non-zero cosine scores across all events — flagged as a candidate negative result for Section 5.3, since low-relevance recommendations are still surfaced to the user rather than filtered or flagged	Pass (behavior noted, not a failure)

Table 5.44: Test case UT-22 — Execution results

Test Case Status: Pass

5.2.2 Integration Testing

Integration testing verifies that individually validated modules function correctly once combined, with particular attention to the boundary between the Laravel web application and the Django AI engine — the point in the system where a failure is least likely to be caught by unit-level tests alone. The following test cases exercise complete end-to-end flows, including two flows that cross the Laravel–Django–Qdrant–OpenAI boundary, three flows internal to the Laravel application involving multiple controllers and models, and one test verifying that the three independent authentication guards remain correctly isolated under concurrent use.

No	Test Case	Test Data	Expected Result	Actual Result	Status
1	Full semantic search: user submits a query on the Explore page with AI search enabled, request flows Laravel → Django <code>/api/search</code> → OpenAI embedding → Qdrant query → <code>gpt-4o-mini</code> re-rank → back to Laravel, which fetches and renders the matching <code>EventDetails</code> records in AI-returned order	Query: “AI hackathon Karachi” AI toggle: enabled	Explore page renders events in the exact order returned by the Django <code>event_ids</code> array, sourced from MySQL by ID	Events rendered in AI-returned order; MySQL fetch preserved ordering correctly	Pass
2	Full recommendation flow: Laravel calls <code>/api/recommend</code> with a user’s full profile and interaction history, Django queries <code>events_recs</code> in Qdrant and returns ranked <code>event_ids</code> with scores	User: profile with <code>domain_interests</code> , prior <code>viewed_event_ids</code>	Response contains ranked <code>event_ids</code> and scores, consumed and displayed by Laravel’s recommendation carousel	Ranked results correctly consumed and rendered	Pass
3	Event registration → notification dispatch → admin visibility: user registers for an event, triggering <code>EventRegisterController@register</code> , which creates the registration and dispatches notifications to the user and the owning community’s admins	User: valid registrant Event: open for registration	<code>EventRegistrations</code> record created; notifications appear in both the user’s and the admin’s notification feed	Record created; notifications appeared in both feeds	Pass
4	Community registration → approval workflow → event creation eligibility: a new community registers, remains restricted until an admin approves it via <code>CommunityManagementController@updateStatus</code> , after which it becomes able to create events	Community: newly registered, status <code>pending</code>	Event creation is blocked pre-approval and enabled immediately after admin approval	Event creation blocked before approval; became available immediately after status change to <code>approved</code>	Pass
5	Collaboration send → accept → dual dashboard reflection: Community A sends a collaboration request on an event, Community B accepts it, and the accepted collaboration appears on both communities’ <code>/community/collaborations</code> dashboards	Sender: Community A Receiver: Community B	Collaboration status is <code>accepted</code> and visible identically from both communities’ dashboards	Collaboration visible and consistent on both dashboards	Pass

No	Test Case	Test Data	Expected Result	Actual Result	Status
6	Follow → notification: a user follows a community, triggering a CommunityFollower record and a notification to the followed community	Follower: logged-in user Followed: target community	Follow record created; followed community receives a notification reflected in CommunityNotificationController@feed	Notification appeared correctly in the community's feed	Pass
7	Admin broadcast fan-out: admin submits a broadcast message via AdminNotificationController@broadcast, dispatching the BroadcastAdminNotification job to fan out to all users and all communities	Admin: logged in Message: broadcast test content	Notification is queued/dispatched and eventually received by all user and community accounts	Notification received across a sampled set of user and community accounts	Pass
8	Three-guard session isolation under concurrent use: a single browser session (or session set) is used to log in as a user, then a community, then an admin in sequence, verifying no session data bleeds across guards	Sequential logins: web → community → admin, same browser	Each login is independently scoped; switching guards does not retain or leak the previous guard's authenticated state	Guard sessions remained correctly isolated across sequential logins	Pass

Table 5.45: Integration testing — Cross-module and cross-service flows

Test Case Status: Pass (8/8 flows passed as designed; no defects surfaced at the integration level in this pass)

5.3 Automation Testing

Automation testing is a form of software testing in which the actual result of a test is compared against the expected result using test scripts or dedicated automation tools, rather than by manual execution. Automating repetitive test cases and other testing duties that are difficult or time-consuming to repeat manually is the primary purpose of test automation, and it provides a repeatable, regression-safe layer of verification on top of the manual testing described in Section 5.1.

For EventBridge AI, automation testing was scoped deliberately around the system’s architecture rather than around a generic UI-automation strategy. Both the Laravel web application and the Django AI engine are structured as request-driven JSON/HTTP APIs — the Django engine in particular has no frontend of its own and exists purely as a backend service consumed by Laravel. Given this, and the fact that the platform’s most safety-critical logic (authentication, registration validation, indexing, search, and recommendation) is exercised at the API layer rather than through complex client-side interactions, automation was implemented at the API level using `pytest` with `pytest-django` for the Django AI engine, and `PHPUnit` — Laravel’s native testing framework — for the web application. Browser-level automation tools such as Selenium or Cypress were considered but not adopted for this phase, since they introduce UI-brittleness and infrastructure overhead that would not meaningfully strengthen verification of the API-driven logic that automation testing is intended to validate here.

The following automation test strategy was followed in scoping and implementing these scripts:

- Test cases to automate were selected from the manual unit and integration tests in Section 5.1 that are most valuable to re-run repeatedly — primarily authentication, validation, and the core AI endpoints — rather than automating the full manual suite.
- `pytest` and `PHPUnit` were selected as the automated testing tools, being the standard, actively maintained frameworks for their respective ecosystems (Python/Django and PHP/Laravel), avoiding the need to introduce and maintain a third-party framework.
- Automation effort was divided along the same system boundary used throughout this chapter: Django AI engine automation (Section 5.2.2) and Laravel web application automation (Section 5.2.3), each independently executable.
- Test data used in automated scripts mirrors the data used in the corresponding manual test cases, keeping the two layers of testing traceable to one another.
- Tests were written against stable API contracts (documented request/response shapes) rather than against transient UI state, making them resistant to future frontend changes.

5.3.1 Automation Tooling Overview

`pytest` (with `pytest-django`) — Django AI Engine

`pytest` was selected for the Django AI engine as it is the most widely adopted testing framework in the Python ecosystem, offering a concise assertion syntax, fixture-based setup/teardown, and native support for parametrized test cases — well suited to testing the AI engine’s endpoints across varied payloads (e.g. multiple limit values for `/api/recommend`, multiple malformed payloads for validation testing). The `pytest-django` plugin was used to integrate `pytest` with

Django’s test client and settings, allowing HTTP-level requests to be issued against the search and recommender apps’ views without requiring a running server process. Since the AI engine currently has no test scaffolding at all (Section 5.2 was implemented from a clean slate, as noted in Chapter 4/prior sections), pytest was introduced fresh alongside a requirements-dev.txt-style addition of pytest, pytest-django, and requests for any tests requiring live calls to the OpenAI and Qdrant services.

PHPUnit — Laravel Web Application

PHPUnit was selected for the Laravel web application as it ships with Laravel by default and is pre-integrated with the framework’s testing utilities — including database transactions/rollbacks between tests, a fluent HTTP testing API (*this->post()*, *this->actingAs()*), and built-in support for testing against the three custom guards (web, community, admin) used throughout the application. As with the Django side, no automated test scaffolding existed prior to this phase; the base tests/Feature and tests/Unit directories generated by Laravel’s default installation were used as the starting structure, with feature tests written against the authentication, registration, collaboration, and admin flows identified in Section 5.1.

Division of Automation Effort

Automation was kept strictly scoped to the two backend systems and did not attempt to automate any UI-level assertions, consistent with the rationale in Section 5.2’s introduction. Each framework’s test suite is independently executable — the Django suite via pytest from the AI engine’s repository root, and the Laravel suite via php artisan test or ./vendor/bin/phpunit from the web application’s repository root — allowing either system to be tested and re-run without dependency on the other being live, except where a test explicitly exercises the cross-service integration boundary (in which case the Django server must be reachable from the Laravel test environment).

5.3.2 Django AI Engine Automation

An automated test suite comprising 127 tests was developed using pytest, pytest-django, and pytest-mock, covering the search app, the recommender app, the ApiKeyMiddleware, and the project’s URL routing configuration. The suite was executed against the real Django settings and URL configuration using DRF’s APIClient. All outbound calls to OpenAI (embeddings and chat-completion re-ranking) and Qdrant (vector search, upsert, and delete) were mocked, making the suite deterministic, cost-free, and independent of live external services.

The full suite passed 127 of 127 tests, with 0 failures and 0 skips, completing in 5.52 seconds. This substantially exceeds the number of unit test cases documented manually in Section 5.1.1.8 and 5.1.1.9; this is expected, since automated testing is able to exhaustively exercise validation boundaries, error paths, and edge cases that a manually-authored representative sample cannot practically cover by hand.

Coverage spanned all nine HTTP endpoints across the search and recommender apps, API key middleware behavior, serializer validation boundaries, the event-text and user-text construction functions, the LLM re-ranker’s response parsing, the core recommendation filtering and scoring logic, and the project’s URL routing. A representative subset of these tests is

presented in Table 5.3 below; the complete set of 127 test cases is provided in Appendix C for reference.

Django AI Engine Automation Testing

Name of Tool	Test Description	Module(s) Automated	Test Script Reference	Framework Used	Test Report
pytest	Missing X-API-Key header returns 401	search/middleware.py	tests/test_middleware.py::test_missing_api_key_header_returns_401	pytest, pytest-django, DRF APIClient	Pass
pytest	Wrong X-API-Key value returns 401	search/middleware.py	tests/test_middleware.py::test_wrong_api_key_returns_401	pytest, pytest-django, DRF APIClient	Pass
pytest	Empty-string X-API-Key header returns 401	search/middleware.py	tests/test_middleware.py::test_empty_api_key_header_returns_401	pytest, pytest-django, DRF APIClient	Pass
pytest	Correct X-API-Key allows the request through	search/middleware.py	tests/test_middleware.py::test_correct_api_key_allows_request	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	API key with trailing whitespace is rejected (no trimming)	search/middleware.py	tests/test_middleware.py::test_api_key_with_trailing_whitespace_is_rejected	pytest, pytest-django, DRF APIClient	Pass
pytest	API key comparison is case-sensitive	search/middleware.py	tests/test_middleware.py::test_api_key_case_sensitive	pytest, pytest-django, DRF APIClient	Pass
pytest	Non-/api/ paths (e.g. /admin/) bypass the key check	search/middleware.py	tests/test_middleware.py::test_non_api_path_bypasses_middleware	pytest, pytest-django, DRF APIClient	Pass
pytest	/api/rec/* paths also require the key	search/middleware.py	tests/test_middleware.py::test_rec_api_path_also_requires_key	pytest, pytest-django, DRF APIClient	Pass
pytest	/api/recommend (mounted outside /api/rec/) still requires the key	search/middleware.py	tests/test_middleware.py::test_recommend_endpoint_requires_key	pytest, pytest-django, DRF APIClient	Pass
pytest	Minimal valid <code>IndexEventSerializer</code> payload passes, optional fields default to ''	search/serializers.py	tests/test_search_serializers.py::test_index_event_valid_minimal_payload	pytest	Pass
pytest	Missing <code>event_id</code> fails validation	search/serializers.py	tests/test_search_serializers.py::test_index_event_missing_event_id_fails	pytest	Pass
pytest	Missing <code>title</code> fails validation	search/serializers.py	tests/test_search_serializers.py::test_index_event_missing_title_fails	pytest	Pass
pytest	Blank <code>title</code> (') fails — no <code>allow_blank</code> set	search/serializers.py	tests/test_search_serializers.py::test_index_event_blank_title_fails	pytest	Pass
pytest	Non-numeric <code>event_id</code> string fails validation	search/serializers.py	tests/test_search_serializers.py::test_index_event_non_numeric_event_id_fails	pytest	Pass
pytest	Numeric string <code>event_id</code> ("42") is coerced to int	search/serializers.py	tests/test_search_serializers.py::test_index_event_id_accepts_numeric_string	pytest	Pass
pytest	Negative <code>event_id</code> is accepted by the serializer (no <code>min_value</code>)	search/serializers.py	tests/test_search_serializers.py::test_index_event_negative_event_id_is_accepted_by_serializer	pytest	Pass
pytest	Unicode/emoji title is accepted	search/serializers.py	tests/test_search_serializers.py::test_index_event_unicode_title_accepted	pytest	Pass
pytest	Unknown extra fields are silently ignored	search/serializers.py	tests/test_search_serializers.py::test_index_event_unknown_fields_are_ignored	pytest	Pass
pytest	Explicit blank values for all optional fields are accepted	search/serializers.py	tests/test_search_serializers.py::test_index_event_blank_optional_fields_allowed	pytest	Pass
pytest	Minimal valid <code>SearchSerializer</code> payload passes, <code>limit</code> defaults to 20	search/serializers.py	tests/test_search_serializers.py::test_search_valid_minimal_query	pytest	Pass
pytest	Missing <code>query</code> fails validation	search/serializers.py	tests/test_search_serializers.py::test_search_missing_query_fails	pytest	Pass
pytest	Empty <code>query</code> string fails (<code>min_length=1</code>)	search/serializers.py	tests/test_search_serializers.py::test_search_empty_query_fails	pytest	Pass

Name of Tool	Test Description	Module(s) Automated	Test Script Reference	Framework Used	Test Report
pytest	query at exactly 500 chars (max) passes	search/serializers.py	tests/test_search_serializers.py::test_search_query_at_max_length_500_passes	pytest	Pass
pytest	query at 501 chars fails	search/serializers.py	tests/test_search_serializers.py::test_search_query_over_max_length_501_fails	pytest	Pass
pytest	limit=0 fails (min_value=1)	search/serializers.py	tests/test_search_serializers.py::test_search_limit_below_min_fails	pytest	Pass
pytest	limit=51 fails (max_value=50)	search/serializers.py	tests/test_search_serializers.py::test_search_limit_above_max_fails	pytest	Pass
pytest	limit at boundaries 1 and 50 both pass	search/serializers.py	tests/test_search_serializers.py::test_search_limit_at_boundaries_pass	pytest	Pass
pytest	Non-integer float limit (10.5) fails	search/serializers.py	tests/test_search_serializers.py::test_search_limit_non_integer_float_fails	pytest	Pass
pytest	Numeric string limit ("10") is coerced to int	search/serializers.py	tests/test_search_serializers.py::test_search_limit_numeric_string_coerced	pytest	Pass
pytest	query with Unicode and HTML/script-like characters is accepted as plain text	search/serializers.py	tests/test_search_serializers.py::test_search_query_unicode_and_special_chars	pytest	Pass
pytest	POST/api/setup returns 200 and creates the collection (mocked)	search/views.py (SetupView)	tests/test_search_views.py::test_setup_view_success	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	Qdrant exception in setup_collection() propagates uncaught	search/views.py (SetupView)	tests/test_search_views.py::test_setup_view_propagates_qdrant_exception	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	GET/api/setup returns 405 (POST-only view)	search/views.py (SetupView)	tests/test_search_views.py::test_setup_view_get_not_allowed	pytest, pytest-django, DRF APIClient	Pass
pytest	POST/api/index success path returns qdrant_id and calls upsert_event with built payload	search/views.py (IndexEventView)	tests/test_search_views.py::test_index_event_success	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	POST/api/index with missing title returns 400	search/views.py (IndexEventView)	tests/test_search_views.py::test_index_event_validation_failure_missing_title	pytest, pytest-django, DRF APIClient	Pass
pytest	Old-vector delete_event() failure during re-index is silently swallowed (bare except) and indexing still proceeds	search/views.py (IndexEventView)	tests/test_search_views.py::test_index_event_old_vector_clear_failure_is_swallowed	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	get_event_embedding() failure (OpenAI outage) propagates uncaught	search/views.py (IndexEventView)	tests/test_search_views.py::test_index_event_embedding_failure_propagates	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	upsert_event() failure (Qdrant outage) propagates uncaught	search/views.py (IndexEventView)	tests/test_search_views.py::test_index_event_upsert_failure_propagates	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	Negative event_id is indexed successfully by the view despite being unreachable via DELETE afterward	search/views.py (IndexEventView)	tests/test_search_views.py::test_index_event_negative_id_accepted_by_view	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	DELETE/api/index/<id> success returns {"success":true}	search/views.py (DeleteEventView)	tests/test_search_views.py::test_delete_event_success	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	delete_event() failure returns 500 with the raw exception message in the body	search/views.py (DeleteEventView)	tests/test_search_views.py::test_delete_event_failure_returns_500_with_error_message	pytest, pytest-django, DRF APIClient, pytest-mock	Pass

Name of Tool	Test Description	Module(s) Auto-mated	Test Script Reference	Framework Used	Test Report
pytest	DELETE/api/index/-1 404s at URL routing (int converter rejects negatives)	search/urls.py, search/views.py	tests/test_search_views.py::test_delete_event_negative_id_not_routable	pytest, pytest-django, DRF APIClient	Pass
pytest	DELETE/api/index/abc 404s at URL routing (non-integer segment)	search/urls.py, search/views.py	tests/test_search_views.py::test_delete_event_non_integer_id_not_routable	pytest, pytest-django, DRF APIClient	Pass
pytest	POST/api/search full success path (embed → Qdrant → re-rank) returns ordered IDs	search/views.py (SearchView)	tests/test_search_views.py::test_search_view_success	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	Empty Qdrant candidate set short-circuits and skips the re-ranker call entirely	search/views.py (SearchView)	tests/test_search_views.py::test_search_view_no_candidates_skips_reranker	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	POST/api/search with missing query returns 400	search/views.py (SearchView)	tests/test_search_views.py::test_search_view_validation_failure	pytest, pytest-django, DRF APIClient	Pass
pytest	POST/api/search with limit=100 returns 400	search/views.py (SearchView)	tests/test_search_views.py::test_search_view_limit_out_of_range	pytest, pytest-django, DRF APIClient	Pass
pytest	Embedding-API failure during search propagates uncaught	search/views.py (SearchView)	tests/test_search_views.py::test_search_view_embedding_failure_propagates	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	Re-ranker exception (e.g. OpenAI timeout) crashes the whole search request instead of falling back to raw Qdrant order	search/views.py (SearchView)	tests/test_search_views.py::test_search_view_reranker_exception_crashes_search	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	Qdrant search failure propagates uncaught	search/views.py (SearchView)	tests/test_search_views.py::test_search_view_qdrant_failure_propagates	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	build_event_text() renders all fields into the expected template	search/services/embeddings.py	tests/test_embeddings.py::test_build_event_text_all_fields_present	pytest	Pass
pytest	build_event_text({}) produces the template with empty placeholders	search/services/embeddings.py	tests/test_embeddings.py::test_build_event_text_empty_dict_uses_defaults	pytest	Pass
pytest	build_event_text() uses .get() defaults for missing dict keys	search/services/embeddings.py	tests/test_embeddings.py::test_build_event_text_missing_keys_uses_get_default	pytest	Pass
pytest	build_event_text() preserves Unicode/emoji/accented text	search/services/embeddings.py	tests/test_embeddings.py::test_build_event_text_unicode_content	pytest	Pass
pytest	build_event_text() does not truncate a 50,000-character description	search/services/embeddings.py	tests/test_embeddings.py::test_build_event_text_very_long_strings_not_truncated	pytest	Pass
pytest	build_event_text() preserves newlines, quotes, HTML-like text, tabs verbatim	search/services/embeddings.py	tests/test_embeddings.py::test_build_event_text_special_characters_and_newlines	pytest	Pass
pytest	get_embedding() calls the OpenAI client with the expected model/input and returns its vector (mocked)	search/services/embeddings.py	tests/test_embeddings.py::test_get_embedding_returns_vector_from_openai	pytest, pytest-mock	Pass
pytest	get_embedding('') still calls the API with an empty string (no guard)	search/services/embeddings.py	tests/test_embeddings.py::test_get_embedding_empty_string_input	pytest, pytest-mock	Pass
pytest	get_event_embedding() returns both the vector and the built text	search/services/embeddings.py	tests/test_embeddings.py::test_get_event_embedding_returns_vector_and_text	pytest, pytest-mock	Pass
pytest	_get_client() constructs the OpenAI client only once (module-level singleton)	search/services/embeddings.py	tests/test_embeddings.py::test_get_embedding_client_is_singleton	pytest, pytest-mock	Pass
pytest	IndexEventRecSerializer minimal payload passes, event_status defaults to 'published'	recommender/serializers.py	tests/test_recommender_serializers.py::test_index_event_rec_valid_minimal	pytest	Pass

Name of Tool	Test Description	Module(s) Auto-mated	Test Script Reference	Framework Used	Test Report
pytest	Missing <code>event_id/title</code> fails validation	<code>recommender/serializers</code>	<code>tests/test_recommender_serializers.py::test_index_event_rec_missing_required_fields_fails</code>	pytest	Pass
pytest	A malformed, non-ISO <code>event_start_datetime</code> string still passes serializer validation	<code>recommender/serializers</code>	<code>tests/test_recommender_serializers.py::test_index_event_rec_malformed_date_string_still_passes_serializer</code>	pytest	Pass
pytest	Non-default <code>event_status</code> value (e.g. <code>'draft'</code>) is accepted as-is	<code>recommender/serializers</code>	<code>tests/test_recommender_serializers.py::test_index_event_rec_custom_event_status_accepted</code>	pytest	Pass
pytest	Negative <code>event_id</code> is accepted by the serializer	<code>recommender/serializers</code>	<code>tests/test_recommender_serializers.py::test_index_event_rec_negative_event_id_accepted</code>	pytest	Pass
pytest	<code>RecommendSerializer</code> requires the <code>profile</code> key to be present at all	<code>recommender/serializers</code>	<code>tests/test_recommender_serializers.py::test_recommend_serializer_requires_profile_key</code>	pytest	Pass
pytest	Empty <code>profile: {}</code> object passes (all sub-fields default)	<code>recommender/serializers</code>	<code>tests/test_recommender_serializers.py::test_recommend_serializer_empty_profile_object_passes</code>	pytest	Pass
pytest	Missing <code>user_id</code> fails validation	<code>recommender/serializers</code>	<code>tests/test_recommender_serializers.py::test_recommend_serializer_missing_user_id_fails</code>	pytest	Pass
pytest	Omitted list fields default to <code>[]</code> , <code>limit</code> defaults to 10	<code>recommender/serializers</code>	<code>tests/test_recommender_serializers.py::test_recommend_serializer_defaults_for_list_fields</code>	pytest	Pass
pytest	<code>limit</code> at boundaries 1 and 20 both pass	<code>recommender/serializers</code>	<code>tests/test_recommender_serializers.py::test_recommend_serializer_limit_boundaries</code>	pytest	Pass
pytest	<code>limit=0</code> and <code>limit=21</code> both fail	<code>recommender/serializers</code>	<code>tests/test_recommender_serializers.py::test_recommend_serializer_limit_out_of_bounds</code>	pytest	Pass
pytest	Non-integer strings inside <code>registered_event_ids</code> fail validation	<code>recommender/serializers</code>	<code>tests/test_recommender_serializers.py::test_recommend_serializer_non_integer_ids_in_list_fail</code>	pytest	Pass
pytest	Full, fully-populated <code>RecommendSerializer</code> payload validates successfully	<code>recommender/serializers</code>	<code>tests/test_recommender_serializers.py::test_recommend_serializer_full_payload_valid</code>	pytest	Pass
pytest	Partially-populated nested <code>profile</code> fills remaining sub-fields with their defaults	<code>recommender/serializers</code>	<code>tests/test_recommender_serializers.py::test_recommend_serializer_profile_missing_subfields_defaults</code>	pytest	Pass
pytest	Negative <code>user_id</code> is accepted by the serializer	<code>recommender/serializers</code>	<code>tests/test_recommender_serializers.py::test_recommend_serializer_negative_user_id_accepted</code>	pytest	Pass
pytest	<code>POST/api/rec/setup</code> success path (mocked)	<code>recommender/views.py</code> (SetupRecView)	<code>tests/test_recommender_views.py::test_setup_rec_view_success</code>	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	Qdrant exception during rec-collection setup propagates uncaught	<code>recommender/views.py</code> (SetupRecView)	<code>tests/test_recommender_views.py::test_setup_rec_view_propagates_exception</code>	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	<code>POST/api/rec/index</code> success path returns <code>qdrant_id</code>	<code>recommender/views.py</code> (IndexEventRecView)	<code>tests/test_recommender_views.py::test_index_event_rec_success</code>	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	<code>POST/api/rec/index</code> with missing <code>title</code> returns 400	<code>recommender/views.py</code> (IndexEventRecView)	<code>tests/test_recommender_views.py::test_index_event_rec_validation_failure</code>	pytest, pytest-django, DRF APIClient	Pass

Name of Tool	Test Description	Module(s) Automated	Test Script Reference	Framework Used	Test Report
pytest	Old-vector delete failure during rec re-index is silently swallowed	recommender/views.py (IndexEventRecView)	tests/test_recommender_views.py::test_index_event_rec_delete_failure_is_swallowed	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	Embedding failure during rec indexing propagates uncaught	recommender/views.py (IndexEventRecView)	tests/test_recommender_views.py::test_index_event_rec_embedding_failure_propagates	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	DELETE/api/rec/index/<id> success returns {"success":true}	recommender/views.py (DeleteEventRecView)	tests/test_recommender_views.py::test_delete_event_rec_success	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	delete_event_rec() failure returns 500 with error body	recommender/views.py (DeleteEventRecView)	tests/test_recommender_views.py::test_delete_event_rec_failure_returns_500	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	DELETE/api/rec/index/-1 404s at URL routing	recommender/urls.py, recommender/views.py	tests/test_recommender_views.py::test_delete_event_rec_negative_id_not_routable	pytest, pytest-django, DRF APIClient	Pass
pytest	POST/api/recommend success path returns recommend() result verbatim	recommender/views.py (RecommendView)	tests/test_recommender_views.py::test_recommend_view_success	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	POST/api/recommend with missing profile returns 400	recommender/views.py (RecommendView)	tests/test_recommender_views.py::test_recommend_view_validation_failure_missing_profile	pytest, pytest-django, DRF APIClient	Pass
pytest	POST/api/recommend with missing user_id returns 400	recommender/views.py (RecommendView)	tests/test_recommender_views.py::test_recommend_view_validation_failure_missing_user_id	pytest, pytest-django, DRF APIClient	Pass
pytest	POST/api/recommend with limit=21 returns 400	recommender/views.py (RecommendView)	tests/test_recommender_views.py::test_recommend_view_limit_out_of_bounds	pytest, pytest-django, DRF APIClient	Pass
pytest	Exception inside recommend() propagates uncaught to the HTTP layer	recommender/views.py (RecommendView)	tests/test_recommender_views.py::test_recommend_view_propagates_recommend_exception	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	build_event_rec.text() renders all fields correctly	recommender/services/recommender.py	tests/test_recommender_service.py::test_build_event_rec_text_all_fields	pytest	Pass
pytest	build_event_rec.text({}) produces the template with empty placeholders	recommender/services/recommender.py	tests/test_recommender_service.py::test_build_event_rec_text_empty_dict	pytest	Pass
pytest	build_event_rec.text() preserves Unicode content	recommender/services/recommender.py	tests/test_recommender_service.py::test_build_event_rec_text_unicode	pytest	Pass
pytest	build_user.text() with fully empty profile/history returns ''	recommender/services/recommender.py	tests/test_recommender_service.py::test_build_user_text_all_empty_returns_empty_string	pytest	Pass
pytest	build_user.text() renders interests/preferred-types lines from profile only	recommender/services/recommender.py	tests/test_recommender_service.py::test_build_user_text_with_profile_only	pytest	Pass
pytest	build_user.text() truncates registered-title context to the first 10 items	recommender/services/recommender.py	tests/test_recommender_service.py::test_build_user_text_with_history_titles_truncated	pytest	Pass
pytest	build_user.text() combines mode + price preference into one sentence	recommender/services/recommender.py	tests/test_recommender_service.py::test_build_user_text_mode_and_price_prefs_combined	pytest	Pass
pytest	build_user.text() includes recent search queries	recommender/services/recommender.py	tests/test_recommender_service.py::test_build_user_text_search_queries_included	pytest	Pass
pytest	Fully empty profile + history short-circuits recommend() before any embedding/Qdrant call	recommender/services/recommender.py	tests/test_recommender_service.py::test_recommend_empty_user_text_short_circuits_without_embedding_call	pytest, pytest-mock	Pass

Name of Tool	Test Description	Module(s) Automated	Test Script Reference	Framework Used	Test Report
pytest	<code>recommend()</code> excludes dismissed/registered/saved event IDs from results	recommender/services/recommender	<code>tests/test_recommender_service.py::test_recommend_excludes_dismissed_registered_saved</code>	pytest, pytest-mock	Pass
pytest	<code>recommend()</code> does NOT exclude viewed/clicked event IDs — they can reappear as recommendations	recommender/services/recommender	<code>tests/test_recommender_service.py::test_recommend_does_not_exclude_viewed_or_clicked_ids</code>	pytest, pytest-mock	Pass
pytest	<code>recommend()</code> filters out events whose <code>event_status != 'published'</code>	recommender/services/recommender	<code>tests/test_recommender_service.py::test_recommend_filters_out_non_published_status</code>	pytest, pytest-mock	Pass
pytest	<code>recommend()</code> filters out events with a past <code>event_start_datetime</code>	recommender/services/recommender	<code>tests/test_recommender_service.py::test_recommend_filters_out_past_events</code>	pytest, pytest-mock	Pass
pytest	Events with an empty/ <code>None</code> <code>event_start_datetime</code> are always kept (date check skipped)	recommender/services/recommender	<code>tests/test_recommender_service.py::test_recommend_keeps_events_with_missing_start_datetime</code>	pytest, pytest-mock	Pass
pytest	Malformed <code>event_start_datetime</code> strings are silently skipped/excluded	recommender/services/recommender	<code>tests/test_recommender_service.py::test_recommend_skips_malformed_date_strings</code>	pytest, pytest-mock	Pass
pytest	Result list is truncated to the requested limit	recommender/services/recommender	<code>tests/test_recommender_service.py::test_recommend_respects_limit_truncation</code>	pytest, pytest-mock	Pass
pytest	Similarity scores are rounded to 4 decimal places	recommender/services/recommender	<code>tests/test_recommender_service.py::test_recommend_scores_are_rounded_to_4dp</code>	pytest, pytest-mock	Pass
pytest	<code>viewed_event_ids/clicked_event_ids</code> are sliced to the first 5 before context-title lookup	recommender/services/recommender	<code>tests/test_recommender_service.py::test_recommend_viewed_and_clicked_ids_truncated_to_5_for_context</code>	pytest, pytest-mock	Pass
pytest	<code>get_context_titles([])</code> returns <code>[]</code> without calling Qdrant	recommender/services/recommender	<code>tests/test_recommender_service.py::test_get_context_titles_empty_ids_returns_empty_without_qdrant_call</code>	pytest, pytest-mock	Pass
pytest	<code>rerank_results([])</code> returns <code>[]</code> without calling the OpenAI API	search/services/reranker	<code>tests/test_reranker.py::test_rerank_empty_candidates_returns_empty_without_api_call</code>	pytest, pytest-mock	Pass
pytest	Valid JSON array response reorders candidate IDs correctly	search/services/reranker	<code>tests/test_reranker.py::test_rerank_valid_json_array_reorders_ids</code>	pytest, pytest-mock	Pass
pytest	fenced json response is stripped before parsing	search/services/reranker	<code>tests/test_reranker.py::test_rerank_strips_markdown_json_fence</code>	pytest, pytest-mock	Pass
pytest	Plain fenced response (no json tag) is stripped before parsing	search/services/reranker	<code>tests/test_reranker.py::test_rerank_strips_plain_markdown_fence</code>	pytest, pytest-mock	Pass
pytest	LLM returning <code>[]</code> (nothing relevant) yields an empty result list	search/services/reranker	<code>tests/test_reranker.py::test_rerank_empty_relevance_array</code>	pytest, pytest-mock	Pass
pytest	Non-JSON garbage response falls back to raw Qdrant order	search/services/reranker	<code>tests/test_reranker.py::test_rerank_malformed_json_falls_back_to_qdrant_order</code>	pytest, pytest-mock	Pass
pytest	Float IDs in the LLM response (1.0) are coerced to <code>int</code>	search/services/reranker	<code>tests/test_reranker.py::test_rerank_float_ids_coerced_to_int</code>	pytest, pytest-mock	Pass
pytest	A syntactically-valid JSON array of non-numeric strings is filtered to <code>[]</code> rather than triggering the fallback path	search/services/reranker	<code>tests/test_reranker.py::test_rerank_non_numeric_ids_silently_filtered_not_fallback</code>	pytest, pytest-mock	Pass
pytest	A mixed array of valid IDs and invalid entries keeps only the valid ones	search/services/reranker	<code>tests/test_reranker.py::test_rerank_mixed_valid_and_invalid_ids</code>	pytest, pytest-mock	Pass
pytest	<code>embedded_text</code> is truncated to 500 characters in the LLM prompt	search/services/reranker	<code>tests/test_reranker.py::test_rerank_truncates_embedded_text_to_500_chars_in_prompt</code>	pytest, pytest-mock	Pass

Name of Tool	Test Description	Module(s) Auto-mated	Test Script Reference	Framework Used	Test Report
pytest	Missing <code>embedded_text</code> payload key defaults to an empty string in the prompt	search/services/reranker	<code>tests/test_reranker.py::test_rerank_missing_embedded_text_key_defaults_to_empty</code>	pytest, pytest-mock	Pass
pytest	<code>/api/recommend</code> (mounted directly, not under <code>/api/rec/</code>) resolves correctly	ai_server/urls.py	<code>tests/test_url_routing.py::test_recommend_is_mounted_at_api_recommend_not_under_rec</code>	pytest, pytest-django, DRF APIClient, pytest-mock	Pass
pytest	<code>/api/rec/recommend</code> does not exist (404) despite the app-name symmetry with <code>/api/rec/setup</code> etc.	ai_server/urls.py, recommender/urls.py	<code>tests/test_url_routing.py::test_recommend_under_rec_prefix_does_not_exist</code>	pytest, pytest-django, DRF APIClient	Pass
pytest	GET/api/setup returns 405	search/urls.py	<code>tests/test_url_routing.py::test_setup_get_method_not_allowed</code>	pytest, pytest-django, DRF APIClient	Pass
pytest	GET/api/index returns 405	search/urls.py	<code>tests/test_url_routing.py::test_index_get_method_not_allowed</code>	pytest, pytest-django, DRF APIClient	Pass
pytest	GET/api/search returns 405	search/urls.py	<code>tests/test_url_routing.py::test_search_get_method_not_allowed</code>	pytest, pytest-django, DRF APIClient	Pass
pytest	GET/api/rec/setup returns 405	recommender/urls.py	<code>tests/test_url_routing.py::test_rec_setup_get_method_not_allowed</code>	pytest, pytest-django, DRF APIClient	Pass
pytest	Unknown <code>/api/</code> path returns 404	ai_server/urls.py	<code>tests/test_url_routing.py::test_unknown_api_path_returns_404</code>	pytest, pytest-django, DRF APIClient	Pass
pytest	DELETE/api/index (no ID segment) returns 405	search/urls.py	<code>tests/test_url_routing.py::test_delete_on_index_root_not_allowed</code>	pytest, pytest-django, DRF APIClient	Pass

Table 5.46: Django AI engine automated testing results

Negative Results & Findings

Although all 127 automated tests passed — meaning each assertion confirmed the system’s actual, current behavior rather than an expected but unmet one — the process of writing these tests against the live codebase surfaced several genuine design gaps. Consistent with sound testing practice, these are reported here rather than omitted, since a results section that hides negative findings misrepresents the system under test.

Finding 1 — Unhandled exceptions surface as unstructured errors rather than JSON. `SetupView`, `IndexEventView`, `SearchView`, and `RecommendView` have no exception handling around their calls to `OpenAI` or `Qdrant`. A transient outage in either external service propagates as an unhandled exception; combined with `DEBUG = True` in the current settings configuration, this would render an HTML debug page rather than a JSON error response, breaking the API’s otherwise consistent JSON contract and posing a potential information-disclosure risk if deployed with debug mode enabled.

Finding 2 — The LLM re-ranker tolerates malformed output but not its own failure. `rerank_results()` falls back gracefully to raw `Qdrant` ordering when the `OpenAI` re-ranking call returns unparseable JSON, but no equivalent fallback exists for the case where the re-ranking API call itself fails (timeout, rate limit, network error) — in that scenario the entire search request fails outright, despite the fallback logic already existing one step away in the same function.

Finding 3 — Indexed events with negative IDs become permanently orphaned. The `event_id` field accepts negative integers at indexing time, but the deletion routes use Django’s `int:event_id` path converter, which never matches negative numbers. An event indexed with a negative ID can therefore never be deleted through the API.

Three further findings from this pass — concerning malformed event dates being silently dropped from recommendations, events without a start date bypassing the past-event filter indefinitely, and the exclusion logic not accounting for already-viewed or already-clicked events — relate specifically to the quality and reliability of AI-driven recommendations rather than to API robustness, and are carried forward into Section 5.3’s relevance and negative-results discussion, where they fit more naturally alongside the search/recommendation evaluation.

5.3.3 Laravel Web Application Automation

An automated test suite comprising 116 tests was developed using PHPUnit with Laravel’s native Feature-testing utilities (`RefreshDatabase`, guard-specific `actingAs()`, and model factories), executed against a dedicated MySQL test database. No third-party testing framework was introduced, consistent with the Django AI engine’s approach in Section 5.2.2 of using each ecosystem’s native tooling rather than adding external dependencies.

All 116 tests pass on the final run, comprising 243 assertions across a total execution time of approximately 197 seconds. Two tests failed on an initial run during development; both failures were traced to genuine gaps in the application rather than test-authoring errors, and are documented in the Negative Results Findings below rather than simply corrected and discarded.

Coverage spans authentication across all three guards (web, community, admin), including reg-

istration, login, logout, password-change flows, login throttling, and the onboarding/approval gates in front of each dashboard; cross-guard isolation, including the shared-session behavior where logging out of one guard must not invalidate the others; event management, covering creation validation and the status-transition state machine; event registration, including duplicate prevention, deadline enforcement, and capacity/waitlist behavior; event collaboration; community following; the admin panel's status-transition and audit-logging behavior; the notification system's per-guard feed isolation; and the AI search integration, where the Django microservice was mocked via `Http::fake()` so the suite requires no live AI server. This is the only area of the suite using external-service mocking — every other test exercises real application code against a real (test) database, matching the "Starting fresh in both" scope agreed for this phase.

Laravel Web Application Automation Testing

Name of Tool	Test Description	Module(s) Automated	Test Script Reference	Framework Used	Test Report
PHPUnit	Baseline: application root route returns 200	App bootstrap	tests/Feature/ExampleTest.php::test_the_application_returns_a_successful_response	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Baseline: unit test harness sanity check	N/A	tests/Unit/ExampleTest.php::test_that_true_is_true	PHPUnit / Unit Test	Pass
PHPUnit	User registration succeeds and redirects to onboarding	User Auth	tests/Feature/Auth/UserAuthTest.php::test_user_can_register_and_is_redirected_to_onboarding	PHPUnit / Laravel Feature Test	Pass
PHPUnit	User registration rejects a duplicate email	User Auth	tests/Feature/Auth/UserAuthTest.php::test_user_registration_rejects_duplicate_email	PHPUnit / Laravel Feature Test	Pass
PHPUnit	User registration requires password confirmation to match	User Auth	tests/Feature/Auth/UserAuthTest.php::test_user_registration_requires_matching_password_confirmation	PHPUnit / Laravel Feature Test	Pass
PHPUnit	User login succeeds with valid credentials	User Auth	tests/Feature/Auth/UserAuthTest.php::test_user_can_login_with_valid_credentials	PHPUnit / Laravel Feature Test	Pass
PHPUnit	User login fails with wrong password	User Auth	tests/Feature/Auth/UserAuthTest.php::test_user_login_fails_with_wrong_password	PHPUnit / Laravel Feature Test	Pass
PHPUnit	User login fails for an unregistered email	User Auth	tests/Feature/Auth/UserAuthTest.php::test_user_login_fails_for_unknown_email	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Login redirects an un-onboarded user to onboarding, not home	User Auth / Onboarding gate	tests/Feature/Auth/UserAuthTest.php::test_user_without_completed_onboarding_is_redirected_to_onboarding_on_login	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Login redirects an onboarded user to the home dashboard	User Auth / Onboarding gate	tests/Feature/Auth/UserAuthTest.php::test_onboarded_user_is_redirected_to_home_on_login	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Authenticated user is redirected away from the login page	User Auth	tests/Feature/Auth/UserAuthTest.php::test_authenticated_user_is_redirected_away_from_login_page	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Guest is redirected to login when requesting the home page	User Auth	tests/Feature/Auth/UserAuthTest.php::test_guest_cannot_access_home_and_is_redirected_to_login	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Repeated failed logins are throttled (HTTP 429)	User Auth / Throttling	tests/Feature/Auth/UserAuthTest.php::test_login_is_throttled_after_repeated_failed_attempts	PHPUnit / Laravel Feature Test	Pass
PHPUnit	User can log out	User Auth	tests/Feature/Auth/UserAuthTest.php::test_user_can_logout	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Password change requires the correct current password	User Auth	tests/Feature/Auth/UserAuthTest.php::test_password_update_requires_correct_current_password	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Password change enforces complexity rules (upper/lower/digit/special)	User Auth	tests/Feature/Auth/UserAuthTest.php::test_password_update_enforces_complexity_rules	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Successful password change logs out other sessions	User Auth	tests/Feature/Auth/UserAuthTest.php::test_password_update_logs_out_other_sessions_on_success	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Community registration succeeds and status starts as pending	Community Auth	tests/Feature/Auth/CommunityAuthTest.php::test_community_can_register_and_is_created_pending	PHPUnit / Laravel Feature Test	Pass

Name of Tool	Test Description	Module(s) Automated	Test Script Reference	Framework Used	Test Report
PHPUnit	Community registration rejects a duplicate short.name	Community Auth	tests/Feature/Auth/CommunityAuthTest.php::test_community_registration_rejects_duplicate_short_name	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Approved community login redirects to the dashboard	Community Auth / Approval gate	tests/Feature/Auth/CommunityAuthTest.php::test_approved_community_login_redirects_to_dashboard	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Pending community login redirects to the status page, not the dashboard	Community Auth / Approval gate	tests/Feature/Auth/CommunityAuthTest.php::test_pending_community_login_redirects_to_status_page_not_dashboard	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Rejected community login redirects to the status page	Community Auth / Approval gate	tests/Feature/Auth/CommunityAuthTest.php::test_rejected_community_login_redirects_to_status_page	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Community login fails with the wrong password	Community Auth	tests/Feature/Auth/CommunityAuthTest.php::test_community_login_fails_with_wrong_password	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Dashboard access is re-gated if status flips to suspended mid-session	Community Auth / Approval gate	tests/Feature/Auth/CommunityAuthTest.php::test_approved_community_cannot_reach_dashboard_if_status_flips_to_suspended_after_login	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Community can log out	Community Auth	tests/Feature/Auth/CommunityAuthTest.php::test_community_can_logout	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Community password change requires the correct old password	Community Auth	tests/Feature/Auth/CommunityAuthTest.php::test_community_password_update_requires_correct_old_password	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Repeated failed community logins are throttled	Community Auth / Throttling	tests/Feature/Auth/CommunityAuthTest.php::test_login_is_throttled_after_repeated_failed_attempts	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Admin login succeeds with valid credentials	Admin Auth	tests/Feature/Auth/AdminAuthTest.php::test_admin_can_login_with_valid_credentials	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Admin login fails with the wrong password	Admin Auth	tests/Feature/Auth/AdminAuthTest.php::test_admin_login_fails_with_wrong_password	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Inactive admin account cannot log in	Admin Auth	tests/Feature/Auth/AdminAuthTest.php::test_inactive_admin_cannot_login	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Admin login creates an audit-log entry	Admin Auth / Audit Log	tests/Feature/Auth/AdminAuthTest.php::test_admin_login_creates_audit_log_entry	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Admin login updates last_login_at	Admin Auth	tests/Feature/Auth/AdminAuthTest.php::test_admin_login_updates_last_login_timestamp	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Guest is redirected to admin login by EnsureAdminAuth middleware	Admin Auth	tests/Feature/Auth/AdminAuthTest.php::test_guest_is_redirected_to_login_by_admin_auth_middleware	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Admin logout creates an audit-log entry	Admin Auth / Audit Log	tests/Feature/Auth/AdminAuthTest.php::test_admin_can_logout_and_audit_log_is_created	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Admin password change requires the correct current password	Admin Auth	tests/Feature/Auth/AdminAuthTest.php::test_admin_password_update_requires_correct_current_password	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Successful admin password change logs the admin out	Admin Auth	tests/Feature/Auth/AdminAuthTest.php::test_admin_password_update_logs_out_after_success	PHPUnit / Laravel Feature Test	Pass

Name of Tool	Test Description	Module(s) Automated	Test Script Reference	Framework Used	Test Report
PHPUnit	Repeated failed admin logins are throttled	Admin Auth / Throttling	tests/Feature/Auth/AdminAuthTest.php::test_login_is_throttled_after_repeated_failed_attempts	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Web-guard user cannot reach the community dashboard	Cross-guard isolation	tests/Feature/Auth/CrossGuardIsolationTest.php::test_web_user_cannot_access_community_dashboard	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Community-guard user cannot reach the user home page	Cross-guard isolation	tests/Feature/Auth/CrossGuardIsolationTest.php::test_community_user_cannot_access_user_home	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Web-guard user cannot reach the admin dashboard	Cross-guard isolation	tests/Feature/Auth/CrossGuardIsolationTest.php::test_web_user_cannot_access_admin_dashboard	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Community-guard user cannot reach the admin dashboard	Cross-guard isolation	tests/Feature/Auth/CrossGuardIsolationTest.php::test_community_user_cannot_access_admin_dashboard	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Authenticating as community does not authenticate the admin/web guards	Cross-guard isolation	tests/Feature/Auth/CrossGuardIsolationTest.php::test_admin_guard_does_not_authenticate_a_community_session	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Logging out the web guard preserves an active community guard session	Cross-guard isolation / Shared session	tests/Feature/Auth/CrossGuardIsolationTest.php::test_logging_out_web_guard_does_not_invalidate_community_guard_on_shared_session	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Logging out the community guard preserves an active admin guard session	Cross-guard isolation / Shared session	tests/Feature/Auth/CrossGuardIsolationTest.php::test_logging_out_community_guard_does_not_invalidate_admin_guard_on_shared_session	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Community can create a valid event	Event Management	tests/Feature/Community/EventManagerTest.php::test_community_can_create_a_valid_event	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Event creation rejects end datetime before start datetime	Event Management / Validation	tests/Feature/Community/EventManagerTest.php::test_event_creation_rejects_end_datetime_before_start_datetime	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Event creation rejects end datetime equal to start datetime	Event Management / Validation	tests/Feature/Community/EventManagerTest.php::test_event_creation_rejects_end_datetime_equal_to_start_datetime	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Non-historical event creation rejects a start datetime in the past	Event Management / Validation	tests/Feature/Community/EventManagerTest.php::test_event_creation_rejects_start_datetime_in_the_past_for_non_historical_events	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Event creation rejects a registration deadline after the start datetime	Event Management / Validation	tests/Feature/Community/EventManagerTest.php::test_event_creation_rejects_registration_deadline_after_start_datetime	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Non-historical event creation requires <code>max_participants</code>	Event Management / Validation	tests/Feature/Community/EventManagerTest.php::test_event_creation_requires_max_participants_for_non_historical_event	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Historical event creation requires a start datetime in the past	Event Management / Validation	tests/Feature/Community/EventManagerTest.php::test_historical_event_requires_start_datetime_in_the_past	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Status transition draft → published is allowed	Event Management / Status transitions	tests/Feature/Community/EventManagerTest.php::test_status_transition_draft_to_published_is_allowed	PHPUnit / Laravel Feature Test	Pass

Name of Tool	Test Description	Module(s) Automated	Test Script Reference	Framework Used	Test Report
PHPUnit	Status transition draft → ongoing is rejected (skips published)	Event Management / Status transitions	tests/Feature/Community/EventManagerTest.php::test_status_transition_draft_to_ongoing_is_rejected	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Status transition out of completed is rejected (terminal state)	Event Management / Status transitions	tests/Feature/Community/EventManagerTest.php::test_status_transition_from_completed_is_rejected_as_terminal_state	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Status transition out of cancelled is rejected (terminal state)	Event Management / Status transitions	tests/Feature/Community/EventManagerTest.php::test_status_transition_from_cancelled_is_rejected_as_terminal_state	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Publishing an event notifies all active admins	Event Management / Notifications	tests/Feature/Community/EventManagerTest.php::test_publishing_an_event_notifies_admins	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Publishing an event dispatches the follower-notification queue job	Event Management / Jobs	tests/Feature/Community/EventManagerTest.php::test_publishing_an_event_dispatches_follower_notification_job	PHPUnit / Laravel Feature Test (Queue::fake())	Pass
PHPUnit	Community cannot edit another community's event (403)	Event Management / Authorization	tests/Feature/Community/EventManagerTest.php::test_community_cannot_edit_another_communitys_event	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Community cannot view another community's event (403)	Event Management / Authorization	tests/Feature/Community/EventManagerTest.php::test_community_cannot_view_another_communitys_event	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Community cannot update another community's event (403)	Event Management / Authorization	tests/Feature/Community/EventManagerTest.php::test_community_cannot_update_another_communitys_event	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Finding: status-transition endpoint has no ownership check — a non-owning community can change another community's event status	Event Management / Authorization gap	tests/Feature/Community/EventManagerTest.php::test_status_transition_endpoint_is_missing_an_ownership_check	PHPUnit / Laravel Feature Test	Pass (test documents the gap)
PHPUnit	Community can send a collaboration request	Event Collaboration	tests/Feature/Community/EventCollabTest.php::test_community_can_send_a_collaboration_request	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Duplicate collaboration request (same direction) is prevented	Event Collaboration	tests/Feature/Community/EventCollabTest.php::test_duplicate_collaboration_request_same_direction_is_prevented	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Duplicate collaboration request (reverse direction) is prevented	Event Collaboration	tests/Feature/Community/EventCollabTest.php::test_duplicate_collaboration_request_in_reverse_direction_is_prevented	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Finding: self-collaboration (community invites itself) is not prevented	Event Collaboration / Validation gap	tests/Feature/Community/EventCollabTest.php::test_self_collaboration_request_is_not_prevented	PHPUnit / Laravel Feature Test	Pass (test documents the gap)

Name of Tool	Test Description	Module(s) Automated	Test Script Reference	Framework Used	Test Report
PHPUnit	Finding: an out-of-enum role value is not caught by validation and causes an unhandled 500	Event Collaboration / Validation gap	tests/Feature/Community/EventCollabTest.php::test_role_outside_the_database_enum_causes_an_uncaught_database_error_instead_of_validation_error	PHPUnit / Laravel Feature Test	Pass (test documents the gap)
PHPUnit	Receiver can accept a collaboration request; sender is notified	Event Collaboration / Notifications	tests/Feature/Community/EventCollabTest.php::test_receiver_can_accept_a_collaboration_request	PHPUnit / Laravel Feature Test	Pass
PHPUnit	A community that is neither sender nor receiver cannot update collaboration status (403)	Event Collaboration / Authorization	tests/Feature/Community/EventCollabTest.php::test_third_party_community_cannot_update_a_collaboration_it_is_not_party_to	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Finding: collaboration status update has no state-machine guard (e.g. rejected → accepted is allowed)	Event Collaboration / Validation gap	tests/Feature/Community/EventCollabTest.php::test_collaboration_status_transition_has_no_state_machine_guard	PHPUnit / Laravel Feature Test	Pass (test documents the gap)
PHPUnit	User can register for a published free event (confirmed)	Event Registration	tests/Feature/EventRegistrationTest.php::test_authenticated_user_can_register_for_a_published_free_event	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Paid event registration status is pending , not confirmed	Event Registration	tests/Feature/EventRegistrationTest.php::test_paid_event_registration_is_pending_not_confirmed	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Duplicate registration for the same event/user is prevented	Event Registration	tests/Feature/EventRegistrationTest.php::test_duplicate_registration_is_prevented	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Registration is blocked for a draft (unpublished) event	Event Registration	tests/Feature/EventRegistrationTest.php::test_registration_is_blocked_for_a_draft_event	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Registration is blocked once the registration deadline has passed	Event Registration	tests/Feature/EventRegistrationTest.php::test_registration_is_blocked_after_the_deadline_has_passed	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Registration is waitlisted once max.participants is reached	Event Registration / Capacity	tests/Feature/EventRegistrationTest.php::test_registration_is_waitlisted_once_the_event_is_full	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Cancelling a confirmed registration notifies the first waitlisted user	Event Registration / Waitlist / Notifications	tests/Feature/EventRegistrationTest.php::test_registration_notifies_the_first_waitlisted_user_when_a_confirmed_seat_is_cancelled	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Guest is redirected to login when attempting to register	Event Registration / Auth	tests/Feature/EventRegistrationTest.php::test_guest_is_redirected_to_login_when_attempting_to_register	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Registration page for a historical event returns 404	Event Registration	tests/Feature/EventRegistrationTest.php::test_historical_event_registration_page_returns_404	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Community can confirm a pending registration	Event Registration / Community actions	tests/Feature/EventRegistrationTest.php::test_community_can_confirm_a_pending_registration	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Community cannot confirm an already-cancelled registration	Event Registration / Community actions	tests/Feature/EventRegistrationTest.php::test_community_cannot_confirm_a_cancelled_registration	PHPUnit / Laravel Feature Test	Pass

Name of Tool	Test Description	Module(s) Automated	Test Script Reference	Framework Used	Test Report
PHPUnit	Community cannot manage registrations for another community's event (403)	Event Registration / Authorization	tests/Feature/EventRegistrationTest.php::test_community_cannot_manage_registrations_for_another_communitys_event	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Finding: confirms no user-facing registration cancellation/withdrawal route exists	Event Registration / Missing feature	tests/Feature/EventRegistrationTest.php::test_no_user_facing_registration_cancellation_route_exists	PHPUnit / Laravel Feature Test	Pass (test documents the gap)
PHPUnit	User can follow a community	Community Follow	tests/Feature/CommunityFollowTest.php::test_user_can_follow_a_community	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Following an already-followed community does not create a duplicate record	Community Follow	tests/Feature/CommunityFollowTest.php::test_following_an_already_followed_community_does_not_create_a_duplicate_record	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Follow notification fires only on the first follow, not repeat attempts	Community Follow / Notifications	tests/Feature/CommunityFollowTest.php::test_follow_notification_is_only_sent_on_first_follow_not_on_repeat_attempts	PHPUnit / Laravel Feature Test	Pass
PHPUnit	A community can follow another community	Community Follow	tests/Feature/CommunityFollowTest.php::test_community_can_follow_another_community	PHPUnit / Laravel Feature Test	Pass
PHPUnit	User can unfollow a community	Community Follow	tests/Feature/CommunityFollowTest.php::test_user_can_unfollow_a_community	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Guest cannot follow a community (redirected to login)	Community Follow / Auth	tests/Feature/CommunityFollowTest.php::test_guest_cannot_follow_a_community	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Admin can approve a pending community	Admin / Community Status	tests/Feature/Admin/CommunityStatusTest.php::test_admin_can_approve_a_pending_community	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Admin cannot apply a disallowed status transition (pending → suspended)	Admin / Community Status transitions	tests/Feature/Admin/CommunityStatusTest.php::test_admin_cannot_apply_a_disallowed_transition	PHPUnit / Laravel Feature Test	Pass
PHPUnit	deleted is a terminal state with no further transitions	Admin / Community Status transitions	tests/Feature/Admin/CommunityStatusTest.php::test_deleted_is_a_terminal_state_with_no_further_transitions	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Rejecting a community requires a reason note	Admin / Community Status / Validation	tests/Feature/Admin/CommunityStatusTest.php::test_rejection_requires_a_reason_note	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Rejection succeeds once a reason note is supplied	Admin / Community Status	tests/Feature/Admin/CommunityStatusTest.php::test_rejection_succeeds_when_reason_note_is_provided	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Community status change creates an audit-log entry	Admin / Audit Log	tests/Feature/Admin/CommunityStatusTest.php::test_status_change_creates_an_audit_log_entry	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Approving a community sends it a notification	Admin / Notifications	tests/Feature/Admin/CommunityStatusTest.php::test_approval_notifies_the_community	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Non-admin guard cannot reach the community-status endpoint	Admin / Authorization	tests/Feature/Admin/CommunityStatusTest.php::test_non_admin_guard_cannot_reach_community_status_endpoint	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Admin can change a user's account status	Admin / User Management	tests/Feature/Admin/UserManagementTest.php::test_admin_can_change_a_users_account_status	PHPUnit / Laravel Feature Test	Pass

Name of Tool	Test Description	Module(s) Automated	Test Script Reference	Framework Used	Test Report
PHPUnit	Updating user status with an invalid value is rejected	Admin / User Management / Validation	tests/Feature/Admin/UserManagementTest.php: :test_updating_status_with_an_invalid_value_is_rejected	PHPUnit / Laravel Feature Test	Pass
PHPUnit	User status change creates an audit-log entry	Admin / Audit Log	tests/Feature/Admin/UserManagementTest.php: :test_status_change_creates_an_audit_log_entry	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Admin can reset a user's password and receives a temporary password	Admin / User Management	tests/Feature/Admin/UserManagementTest.php: :test_admin_can_reset_a_users_password_and_receives_a_temporary_password	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Password reset creates an audit-log entry	Admin / Audit Log	tests/Feature/Admin/UserManagementTest.php: :test_password_reset_creates_an_audit_log_entry	PHPUnit / Laravel Feature Test	Pass
PHPUnit	The generated temporary password actually authenticates the user	Admin / User Management	tests/Feature/Admin/UserManagementTest.php: :test_reset_password_actually_allows_login_with_the_temporary_password	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Guest cannot reach user-management endpoints	Admin / Authorization	tests/Feature/Admin/UserManagementTest.php: :test_guest_cannot_reach_user_management_endpoints	PHPUnit / Laravel Feature Test	Pass
PHPUnit	User notification feed returns only that user's own notifications	Notifications	tests/Feature/NotificationTest.php::test_user_notification_feed_only_returns_that_users_own_notifications	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Community notification feed is isolated from user/admin feeds (same numeric id, different type)	Notifications / Isolation	tests/Feature/NotificationTest.php::test_community_notification_feed_is_isolated_from_user_and_admin_feeds	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Unread count reflects only unread notifications	Notifications	tests/Feature/NotificationTest.php::test_unread_count_reflects_only_unread_notifications	PHPUnit / Laravel Feature Test	Pass
PHPUnit	User can mark their own notification as read	Notifications	tests/Feature/NotificationTest.php::test_user_can_mark_their_own_notification_as_read	PHPUnit / Laravel Feature Test	Pass
PHPUnit	User cannot mark another user's notification as read (403)	Notifications / Authorization	tests/Feature/NotificationTest.php::test_user_cannot_mark_another_users_notification_as_read	PHPUnit / Laravel Feature Test	Pass
PHPUnit	User cannot mark a community notification as read via the user endpoint (403)	Notifications / Authorization	tests/Feature/NotificationTest.php::test_user_cannot_mark_a_community_notification_as_read_via_the_user_endpoint	PHPUnit / Laravel Feature Test	Pass
PHPUnit	"Mark all read" only affects the authenticated actor's own notifications	Notifications	tests/Feature/NotificationTest.php::test_mark_all_read_only_affects_the_authenticated_actors_notifications	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Admin broadcast fans out to all users	Notifications / Admin Broadcast	tests/Feature/NotificationTest.php::test_admin_broadcast_fans_out_to_all_users	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Admin broadcast to a specific user only notifies that user	Notifications / Admin Broadcast	tests/Feature/NotificationTest.php::test_admin_broadcast_to_specific_user_only_notifies_that_user	PHPUnit / Laravel Feature Test	Pass
PHPUnit	Broadcast validation requires <code>target_id</code> when target is <code>specific_user</code>	Notifications / Validation	tests/Feature/NotificationTest.php::test_broadcast_requires_target_id_when_target_is_specific_user	PHPUnit / Laravel Feature Test	Pass

Name of Tool	Test Description	Module(s) Auto-mated	Test Script Reference	Framework Used	Test Report
PHPUnit	AI search results are ordered by the IDs returned from the (mocked) Django service	AI Search Integration	tests/Feature/ExploreAiSearchTest.php::test_ai_search_orders_results_by_the_ids_returned_from_the_ai_service	PHPUnit / Laravel Feature Test (Http::fake() — mocked , no live AI server)	Pass
PHPUnit	AI service failure gracefully falls back to keyword search (no 500 to the user)	AI Search Integration / Resilience	tests/Feature/ExploreAiSearchTest.php::test_ai_search_failure_falls_back_to_keyword_search_without_erroring	PHPUnit / Laravel Feature Test (Http::fake() — mocked , no live AI server)	Pass
PHPUnit	Guests never trigger an AI search call, even with <code>ai_search=1</code> in the URL	AI Search Integration / Access control	tests/Feature/ExploreAiSearchTest.php::test_guests_do_not_trigger_ai_search_even_with_ai_search_flag_set	PHPUnit / Laravel Feature Test (Http::fake() — mocked , no live AI server)	Pass

Table 5.47: Laravel automated testing results

Negative Results & Findings

As with the Django AI engine suite, all 116 tests pass in their final form — but four of these tests were deliberately written to document confirmed gaps in the application’s current behavior rather than to assert an already-correct outcome, and two tests failed on their first run before the underlying issue was understood. These are reported in full below.

Finding 1 — The event status-transition endpoint has no ownership check. Unlike the `edit()`, `view()`, and `update()` methods on the same controller, which each verify the requesting community owns the event before proceeding, `EventManagerController::updateStatus()` performs no such check. Any authenticated, approved community can change the status of an event they do not own simply by submitting its numeric ID — confirmed by a test in which one community successfully published another community’s draft event. This is the most significant finding from this pass and should be treated as a priority fix: the same ownership guard used elsewhere in the controller needs to be added here.

Finding 2 — An invalid collaboration role value causes an unhandled server error rather than a validation error. The role field is validated only as a generic string, while the underlying database column is a strict MySQL enum with four allowed values. Submitting anything outside that enum passes validation but then fails at the database layer, surfacing to the user as an unhandled HTTP 500 rather than a clean validation message.

Finding 3 — Collaboration status updates have no state-machine guard. Event status transitions are governed by an explicit allowed-transitions map that rejects illegal jumps (e.g. skipping from draft directly to ongoing). No equivalent restriction exists for collaboration status updates: a collaboration can be moved from rejected directly to accepted, or re-accepted after already being accepted, re-triggering notifications each time.

Finding 4 — No user-facing registration cancellation route exists. Users can view their registration history but cannot cancel or withdraw a registration themselves; only the hosting community can cancel a registration on their behalf. This answers one of the four open questions carried over from manual testing directly: the absence is confirmed, not assumed, though it is worth noting as a common feature expectation for a platform of this kind.

Finding 5 — A static, process-lifetime cache in the notification service can serve stale admin lists. During test development, a test verifying that publishing an event notifies all admins failed intermittently depending on execution order, traced to an in-memory cache of active admins that is populated once and never invalidated within a running process. This has no visible effect under Laravel’s typical one-process-per-request model, but represents a real staleness risk under any long-running process — a queue worker or an Octane/Swoole deployment — where an admin added or deactivated after the cache is first populated would not be reflected until the process restarts.

Finding 6 — A dead/unreachable model relationship was found during code review. `CommunityProfile`’s `followers()` relationship uses Laravel’s standard polymorphic convention, but the actual `community_followers` table resolves its polymorphism manually with a different scheme, and this relationship is not referenced anywhere else in the codebase. It appears to be unused, broken code rather than something currently relied upon — no test was written against it since no route reaches it, but it should either be corrected or removed.

Two additional, lower-severity observations were made but did not warrant dedicated regression tests: self-collaboration (a community sending itself a collaboration request) is not explicitly prevented, though it is unlikely to occur in practice; and file uploads are handled via direct filesystem writes rather than Laravel’s Storage facade, which is consistent with the project’s documented conventions but means test isolation for uploaded files requires manual cleanup rather than `Storage::fake()`.

Answers to the Four Open Questions Carried Over from Manual Testing

- Duplicate collaboration requests are prevented in both directions (sender→receiver and receiver→sender) with an explicit existence check.
- Duplicate follow records are prevented via an underlying unique constraint; a repeat follow is a no-op and does not re-trigger a notification.
- Event end-datetime validation is enforced — the end datetime must be strictly after the start datetime, confirmed by dedicated boundary tests.
- Registration cancellation — no user-facing route exists; see Finding 4 above.

5.3.4 Automation Test Summary

Across both constituent systems, a combined total of 243 automated tests were written and executed: 127 for the Django AI engine (Section 5.2.2) and 116 for the Laravel web application (Section 5.2.3). All 243 tests pass in their final form, with two Laravel tests having failed on an initial run before the underlying application gaps they exposed were understood and documented. No test in either suite requires a live external service — the Django suite mocks all OpenAI and Qdrant calls, and the Laravel suite mocks the Django AI engine via `Http::fake()` — making both suites fully deterministic, cost-free, and independently repeatable.

Together, the two suites substantially exceed the coverage of the representative manual test cases documented in Section 5.1, which is expected: automated testing is able to exhaustively exercise validation boundaries, authorization edge cases, and failure paths that a manually-authored sample cannot practically cover by hand. Where the two layers overlap — for example, event registration deadline enforcement or admin-driven status transitions — the automated results consistently confirm the manual findings, with three exceptions where automation surfaced genuine defects that manual testing, being representative rather than exhaustive, did not catch: the missing ownership check on event status transitions, the unvalidated collaboration role enum, and the absence of a state-machine guard on collaboration status updates.

The complete automated test suites, including all individually named test cases, are provided. Test scripts themselves reside in each repository’s `tests/` directory as referenced throughout Sections, and can be re-executed at any time using the commands documented in each section.

A subset of findings from the Django suite — specifically those concerning the reliability of AI-driven recommendation and search behavior rather than general API robustness — are carried forward into Section 5.3, where they are discussed alongside the platform’s search and recommendation relevance evaluation.

5.4 AI Search & Recommendation Relevance Evaluation

The testing template originally associated with this section called for a comparison across established benchmark datasets. That comparison does not apply to EventBridge AI, which does not train or evaluate against any external benchmark corpus — its AI engine performs live semantic search and recommendation over the platform’s own indexed events. A more meaningful results section for a system of this kind is a direct relevance evaluation of its two core AI-driven services: semantic search and personalized recommendation. This section presents that evaluation, run against the live production Qdrant collections and OpenAI services rather than mocked data, in keeping with the principle that a results section is only informative if it reflects genuine system behavior rather than assumed or synthetic outcomes.

5.4.1 Evaluation Environment

This evaluation was executed against the live, production-configured EventBridge AI engine rather than a mocked environment, in direct contrast to the automation testing pass in Section 5.2. The events and events_recs Qdrant collections, each containing 26 indexed points at the time of evaluation, were queried through the application’s actual endpoints. All embeddings were generated by the real text-embedding-3-large model, and all re-ranking was performed by the real gpt-4o-mini model, with no stubs or fixtures involved. No data was fabricated, subsampled, or filtered prior to scoring; the full contents of both collections served as the universe of candidates against which relevance was measured.

5.4.2 Methodology — Relevance Scoring

No human-labeled ground truth exists for this dataset. Relevance was therefore determined using an explicit, mechanical keyword and metadata-alignment proxy, defined precisely so that the resulting metrics are reproducible and auditable rather than subjective. This proxy is not a substitute for genuine human judgment of relevance — it measures whether the system’s output is topically and structurally consistent with a query or profile according to a fixed rule, not whether an actual user would find the result useful. Its limitations are discussed explicitly alongside the findings in Section 5.3.5.

For search, each of eight test queries was paired with a curated list of relevance keywords drawn from its stated topic. An indexed event was labeled relevant if any keyword appeared as a case-insensitive substring within its title, description, agenda, event type, or city fields; where a query specified a city, that city was also required to match. This rule was applied identically and independently of the API’s own output, producing a query-specific relevant set against which the API’s actual returned results were scored.

For recommendations, each of six profile variants was scored using a four-signal disjunction: an event was labeled relevant if its event type matched the profile’s preferred event types, if its text contained a keyword associated with one of the profile’s domain interests, if its event mode matched the profile’s stated preference, or if its registration type matched the profile’s price preference. Because this is a broad “any of four” rule, richly-populated profiles necessarily produce large relevant sets — in some cases up to 24 of the 26 indexed events — a property of the proxy rule itself rather than of the recommender, discussed further in the findings below.

From these relevant sets, Precision, Recall, Accuracy, and True Negative Rate were computed against the full corpus as the universe of items. Where either the relevant set or the returned set was empty, Precision and Recall are reported as not applicable rather than assumed to be zero or one.

5.4.3 Search Relevance (POST /api/search)

#	Query	Precision	Accuracy	Recall	TNR
Q1	AI and machine learning workshops	1.000	0.769	0.250	1.000
Q2	Cybersecurity and ethical hacking training	1.000	0.885	0.250	1.000
Q3	Full-stack web development bootcamp	1.000	1.000	1.000	1.000
Q4	Hackathon events for students	1.000	0.962	0.750	1.000
Q5	Blockchain and decentralized applications	1.000	1.000	1.000	1.000
Q6	Career networking and job search events in Karachi	0.500	0.731	0.286	0.895
Q7	Graphic design and UX/UI for beginners	1.000	1.000	1.000	1.000
Q8	Quantum computing research seminars	N/A	1.000	N/A	1.000
Average (Q1–Q7)		0.929	0.918	0.648	0.987

Table 5.48: Search Relevance (POST /api/search)

5.4.4 Recommendation Relevance (POST /api/recommend)

#	Profile	Precision	Accuracy	Recall	TNR
P1	Cold start — empty profile, no history	N/A	1.000	N/A	1.000
P2	Near-empty — single preference, no history	0.700	0.731	0.636	0.800
P3	AI/ML enthusiast — rich profile with viewed/clicked history	0.900	0.385	0.375	0.500
P4	Cybersecurity/Cloud professional — viewed history	0.900	0.615	0.500	0.875
P5	Career/business-focused — free and online preference	0.800	0.423	0.381	0.600
P6	Design-focused — paid and online, one dismissal	0.800	0.500	0.421	0.714
Average (P2–P6)		0.820	0.609	0.463	0.748

Table 5.49: Recommendation Relevance (POST /api/recommend)

5.4.5 Findings and Negative Results

Search precision was high overall — averaging 0.929 across the seven scoreable queries — indicating that when the system does return a result, it is very likely to be genuinely relevant. Recall was substantially lower, averaging 0.648, revealing a consistent pattern: the semantic search pipeline retrieves relevant candidates reliably at the dense-vector stage, but loses several of them at the LLM re-ranking stage.

This was most visible in the query “cybersecurity and ethical hacking training,” which achieved perfect precision but only 0.250 recall. Of four indexed events matching the topic, only one was returned. Inspection of the pre-rerank candidate set confirmed all four relevant events were retrieved correctly by the dense vector search; the re-ranker itself discarded the other three, apparently weighting literal title-level topical match far more heavily than agenda-level or implicit relevance — a hackathon with security-adjacent content, for instance, was not recognized as cybersecurity-relevant despite genuinely overlapping subject matter. The same pattern recurred, to varying degrees, in the AI/machine-learning query and the hackathon query. This indicates the re-ranking stage is precision-biased and recall-lossy: it produces clean, highly precise result sets, but at the cost of pruning borderline-relevant events whose relevance is not stated explicitly in their title.

The inverse failure also appeared once: the career-and-networking query returned two events with no discernible career, job-search, or networking language in their descriptions under the documented scoring rule, producing a precision of only 0.500. This suggests the re-ranker’s judgment of relevance is not perfectly consistent from query to query, occasionally admitting a false positive even as it is, on average, more likely to produce false negatives.

One query — “quantum computing research seminars” — returned no results at all, correctly, since no indexed event is genuinely about quantum computing. The highest pre-rerank similarity scores for this query were well below those seen for genuinely on-topic queries, and the re-ranker discarded all candidates rather than forcing a spurious match. This is the one case in the evaluation where the system’s conservative behavior worked entirely in its favor, and it is worth noting alongside the recall-loss finding above: the same re-ranking behavior that discards borderline-relevant events also correctly declines to hallucinate relevance where none exists.

On the recommendation side, the cold-start profile — submitted with no populated fields and no interaction history — received zero recommendations. Examining the underlying service logic showed this is a deliberate short-circuit: the system constructs a text description of the user from profile and history data, and when both are entirely empty, it returns an empty result immediately rather than calling the embedding or vector-search services at all. This is a sharper version of a limitation identified during earlier automation testing, where a sparse but non-empty profile produced low-confidence results; here, with a fully empty profile, the system instead produces no results whatsoever, meaning a brand-new user with no stated preferences receives no recommendations rather than any generic fallback. A profile with even a single populated preference field did produce results, but at consistently low similarity scores relative to more fully populated profiles, confirming that recommendation quality scales directly with how much a user has told the system about themselves.

A second confirmed limitation concerns previously viewed and clicked events. Two profiles

in this evaluation explicitly viewed or clicked specific events before requesting recommendations; in both cases, those same events reappeared in the returned recommendations, in one case as the top-ranked result. The underlying service excludes only dismissed, registered, and saved events from its results — viewed and clicked events are used solely to build the user’s context text, never as an exclusion filter. This means a user who has already viewed or clicked an event is likely to see that same event recommended back to them, which is a real and reproducible behavior rather than an edge case.

As a positive counterpoint, the dismissed-event exclusion mechanism was confirmed to work correctly: a profile that explicitly dismissed its single strongest matching event did not see that event reappear in its results, confirming that exclusion logic functions as intended for the signal it is designed to handle. The malformed or missing start-date issue identified during earlier automation testing could not be reproduced in this evaluation, since every event currently indexed in the live events_recs collection carries a well-formed date. The one indexed event with a genuinely past start date was correctly filtered from every profile’s results. This finding should be treated as unresolved rather than disproven, and re-evaluated if a malformed-date event is ever indexed in production.

Finally, a methodological caveat applies to the comparatively low Accuracy and TNR figures for the more richly-populated recommendation profiles: because the four-signal relevance proxy classifies as many as 24 of 26 indexed events as “relevant” once several profile fields are populated, any result set capped at ten items will structurally floor recall and accuracy by construction, regardless of recommendation quality. This ceiling effect is a property of the evaluation’s relevance definition rather than evidence that the recommender is under-performing, and the qualitative findings above — cold-start behavior, viewed/clicked reappearance, and dismissal-exclusion correctness — should be weighted more heavily than the raw accuracy figures in Table 5.2 when assessing the system’s actual behavior.

5.4.6 Raw Evaluation Data

Search Queries

Q1 — “AI and machine learning workshops” Relevance keywords: ai, artificial intelligence, machine learning, chatbot, langchain, llm, prompt engineering, rag. Eight indexed events matched the proxy rule. The API returned two events: #20 Build Your Own Chatbot: LangChain & RAG Workshop (score 0.5417) and #17 Prompt Engineering Masterclass: Getting the Best from LLMs (score 0.4980).

Q2 — “cybersecurity and ethical hacking training” Relevance keywords: cybersecurity, cyber security, ethical hacking, network security, hacking, vulnerability, ctf. Four indexed events matched the proxy rule (#3 IEEE TechFest Karachi, #4 Cybersecurity Essentials, #14 DevSolve Hackathon 2026, #36 Vibe Coding Hackathon). The API returned only one: #4 Cybersecurity Essentials: Defending the Digital Frontier (score 0.6148).

Q3 — “full-stack web development bootcamp” Relevance keywords: full-stack, full stack, web development, frontend, backend, javascript, react. One indexed event matched. The API returned it: #12 Build & Deploy: Full-Stack Web Development Bootcamp (score 0.6303).

Q4 — “hackathon events for students” Relevance keywords: hackathon, hack jam, game dev

jam, build your first. Four indexed events matched (#3, #11, #14, #36). The API returned three: #36 Vibe Coding Hackathon (0.5132), #14 DevSolve Hackathon 2026 (0.5304), #11 Game Dev Jam: Build Your First 2D Game (0.4638).

Q5 — "blockchain and decentralized applications" Relevance keywords: blockchain, decentralized, decentralised, bitcoin, web3, crypto, dapp. One indexed event matched, and was returned: #9 Blockchain Beyond Bitcoin: Decentralised Applications Seminar (score 0.5159).

Q6 — "career networking and job search events in Karachi" Relevance keywords: career, networking, job search, job, resume, hiring (city filter: Karachi). Seven indexed events matched. The API returned four: #26 Tech Careers Unlocked: Networking & Job Search Strategies (0.7237), #3 IEEE TechFest Karachi (0.4753), #27 EdTech Summit: The Future of Learning (0.4161), #11 Game Dev Jam: Build Your First 2D Game (0.3393) — the latter two not genuinely career-related under the scoring rule, producing the query's low precision.

Q7 — "graphic design and UX/UI for beginners" Relevance keywords: graphic design, ux, ui workshop, design thinking, figma, canva, user experience. Two indexed events matched, and both were returned: #25 Graphic Design for Non-Designers: Canva to Figma (0.5618), #8 UX Design Thinking Workshop (0.4887).

Q8 — "quantum computing research seminars" Relevance keywords: quantum, qubit, quantum computing. No indexed event matched. The API correctly returned no results; the highest prerank Qdrant similarity scores were #9 Blockchain (0.392), #10 Research Roadmap (0.343), and #15 Cloud & DevOps (0.302) — well below the range typical of genuinely on-topic queries — and the re-ranker discarded all of them rather than forcing a match.

Recommendation Profiles

P1 — Cold start (empty profile, no history) Payload: {"profile": {}, "limit": 10}. Response in 9.5 ms: {"event_ids": [], "scores": [], "total": 0}. The service short-circuited before calling OpenAI or Qdrant, since the constructed user-text was empty.

P2 — Near-empty (single preference: `event_mode_preference`: "online") Eleven indexed events matched the proxy rule. The API returned ten, all at low cosine scores (0.2853–0.3397): #36 Vibe Coding Hackathon, #9 Blockchain Beyond Bitcoin, #23 Digital Marketing Masterclass, #25 Graphic Design for Non-Designers, #26 Tech Careers Unlocked, #28 AI After Hours, #20 Build Your Own Chatbot, #15 Cloud & DevOps, #21 AI Startup Pitch Night, #27 EdTech Summit.

P3 — AI/ML enthusiast (rich profile; viewed #19, #20; clicked #17) Twenty-four of 26 indexed events matched the broad proxy rule. The API returned ten (scores 0.3529–0.4596): #28 AI After Hours, #20 Build Your Own Chatbot, #19 AI in Healthcare, #17 Prompt Engineering Masterclass, #26 Tech Careers Unlocked, #12 Full-Stack Bootcamp, #13 Flutter Forward, #9 Blockchain Beyond Bitcoin, #15 Cloud & DevOps, #3 IEEE TechFest Karachi. Both viewed events (#19, #20) and the clicked event (#17) reappeared in the output.

P4 — Cybersecurity/Cloud professional (viewed #4) Eighteen of 26 indexed events matched. The API returned ten (scores 0.3104–0.4969): #4 Cybersecurity Essentials, #15 Cloud & DevOps, #26 Tech Careers Unlocked, #12 Full-Stack Bootcamp, #9 Blockchain Beyond Bitcoin,

#3 IEEE TechFest Karachi, #28 AI After Hours, #27 EdTech Summit, #23 Digital Marketing Masterclass, #18 Data Science Decoded. The viewed event (#4) reappeared as the top-ranked result.

P5 — Career/business-focused, free + online (registered #26) Twenty-one of 26 indexed events matched. The API returned ten (scores 0.3797–0.4539): #22 Freelancing Launchpad, #28 AI After Hours, #12 Full-Stack Bootcamp, #21 AI Startup Pitch Night, #24 Personal Finance for Young Professionals, #27 EdTech Summit, #25 Graphic Design for Non-Designers, #15 Cloud & DevOps, #4 Cybersecurity Essentials, #13 Flutter Forward. The registered event (#26) was correctly excluded.

P6 — Design-focused, paid + online (dismissed #25) Nineteen of 26 indexed events matched. The API returned ten (scores 0.3363–0.5045): #8 UX Design Thinking Workshop, #13 Flutter Forward, #23 Digital Marketing Masterclass, #15 Cloud & DevOps, #22 Freelancing Launchpad, #20 Build Your Own Chatbot, #26 Tech Careers Unlocked, #28 AI After Hours, #12 Full-Stack Bootcamp, #17 Prompt Engineering Masterclass. The dismissed event (#25) — the single strongest domain match by title — was correctly excluded and did not reappear.

5.5 Chapter Summary

This chapter presented a three-stage testing effort for EventBridge AI, moving from manual verification through to automated regression coverage and, finally, a live evaluation of the platform’s core AI-driven services. Manual testing (Section 5.1) established a representative baseline across the full breadth of the system — authentication across all three guards, event management, event registration, collaboration, community following, the admin panel, notifications, and the Django AI engine’s search and recommendation logic — comprising 22 unit test cases and 8 integration test cases. This stage also surfaced several genuine open questions about implementation details not fully documented elsewhere, including duplicate-request handling and date-validation rules, which were carried forward rather than assumed.

Automation testing (Section 5.2) substantially extended this coverage using each ecosystem’s native testing tools — pytest for the Django AI engine and PHPUnit for the Laravel web application — producing 243 automated tests in total, all passing in their final form. Because automated testing can exhaustively exercise validation boundaries and failure paths in a way manual testing cannot, this stage resolved every open question carried over from Section 5.1 and surfaced additional, previously unknown findings: most notably a missing ownership check on event status transitions, an unvalidated collaboration role field, and the absence of a state-machine guard on collaboration status updates. These findings were reported directly rather than minimized, consistent with the principle that a results section is only credible if it includes genuine negative outcomes alongside its successes.

The relevance evaluation (Section 5.3) departed from a generic dataset-benchmark format in favor of a live assessment of the platform’s own semantic search and recommendation services, run against the production Qdrant collections and OpenAI models rather than mocked data. This evaluation confirmed that the search pipeline is precision-biased — averaging 0.929 precision but only 0.648 recall — due to the LLM re-ranking stage discarding borderline-relevant results more aggressively than the underlying vector retrieval would justify. It also confirmed two concrete limitations in the recommendation service: a completely empty user profile pro-

duces zero recommendations rather than a generic fallback, and previously viewed or clicked events are not excluded from future recommendations, unlike dismissed, registered, or saved events, which are correctly excluded.

Taken together, these three stages indicate a system whose core functional logic — authentication, registration, collaboration, and administrative workflows — is sound and well-covered, with a small number of specific, well-documented gaps rather than systemic failures, and whose AI-driven features perform reliably on precision but have identifiable, reproducible limitations in recall and personalization completeness. These limitations are not treated as failures of the testing process but as legitimate findings in their own right, several of which point directly to concrete improvements — an ownership guard on the status-transition endpoint, an enum-based validation rule for collaboration roles, and an exclusion rule extended to viewed and clicked events — that inform the direction discussed in Chapter 8. The performance characteristics of the system under load, building on the functional and relevance findings established here, are examined next in Chapter 6.

Chapter 6

Performance Evaluation

6.1 Introduction & Evaluation Objectives

Performance evaluation is a critical component of validating any software system, particularly one that integrates AI-driven components such as semantic search and recommendation engines. While functional testing (covered in Chapter 5) verifies that the system behaves correctly under defined inputs and conditions, performance evaluation is concerned with a different question: how well the system performs under realistic usage, and whether its behavior — in terms of speed, accuracy, and reliability — meets the expectations set out for a platform intended for real-world deployment.

For EventBridge AI, this distinction matters more than usual. The platform’s core value proposition rests on two AI-driven capabilities — semantic event search and personalized recommendations — both of which depend on external services (OpenAI’s embedding API) and a dedicated vector database (Qdrant) operating across a physically separate server from the main Laravel application. A system that functions correctly but responds slowly, or that returns technically valid but practically irrelevant results, would fail to deliver on its intended purpose regardless of how well its individual modules pass functional tests. This chapter therefore evaluates the system along dimensions that functional testing does not capture: response latency, result relevance, behavior under concurrent load, cross-server reliability, and end-user satisfaction.

The evaluation was guided by the following objectives:

#	Objective	Why it matters
1	Measure semantic search response time across varied query types	Search is the primary discovery mechanism; slow search directly harms usability
2	Assess relevance accuracy of search results	A fast but irrelevant result is functionally useless to the user
3	Measure recommendation generation latency	Recommendations are surfaced on high-traffic pages (dashboard, homepage) and must not introduce noticeable delay
4	Assess relevance accuracy of recommendations against known user interest signals	Validates that the content-based filtering approach produces meaningfully personalized output
5	Observe system behavior under concurrent load	Determines whether the two-server architecture holds up as usage scales beyond a single user
6	Capture real beta-tester satisfaction ratings	Quantitative metrics alone do not confirm the system feels good to use — user perception closes that gap
7	Identify and document limitations honestly	A credible evaluation reports where the system falls short, not just where it succeeds

Table 6.1: Evaluation objectives

These objectives collectively frame the remainder of the chapter: Section 6.2 describes the environment and methodology used to gather results, Sections 6.3–6.6 present the findings against each objective, Section 6.7 addresses the limitations uncovered, and Section 6.8 discusses what the results mean for the platform’s overall readiness.

6.2 Testing Environment & Methodology

To ensure results reflect real-world conditions rather than idealized local performance, all evaluation was conducted against the live deployed infrastructure rather than a local development environment. This section outlines the hardware and software environment, the dataset used, and the tools and procedures used to collect each metric reported in the sections that follow.

6.2.1 Deployment Environment

The platform runs across two independently hosted servers, consistent with the architecture described in Chapter 4. The Laravel application is hosted on GoDaddy shared/VPS hosting, while the Django AI engine and Qdrant vector database run on an OVH Cloud VPS, with Qdrant containerized via Docker and Django served through Gunicorn behind an Nginx reverse proxy. This separation means that any AI-related performance figure (search, recommendations) reflects network round-trip time between the two servers in addition to processing time — a realistic condition rather than a best-case, single-machine benchmark.

6.2.2 Dataset Used for Evaluation

Performance testing was conducted against a representative dataset seeded into the platform to simulate realistic usage volume, since the beta user base at the time of testing had not yet

generated organic data at scale.

Parameter	Value
Total events indexed (Qdrant <code>events</code> collection)	500
Total events indexed (Qdrant <code>events_recs</code> collection)	500
Registered user profiles used for recommendation testing	50
Event categories/types represented	12
Average event description length	~120 words
Search queries used for evaluation	40 (mixed specificity)

Table 6.2: Test dataset parameters

6.2.3 Tools and Procedures

Metric Category	Tool/Method Used	Procedure
Search response time	Postman + custom Python timing script	Each of the 40 test queries sent 10 times; latency measured from request dispatch to response receipt
Search relevance	Manual relevance scoring by team	Top-5 results per query manually labeled relevant/not relevant against query intent
Recommendation latency	Custom Python timing script	<code>/api/rec/*</code> endpoint called for each of the 50 test profiles; latency logged per call
Recommendation relevance	Manual relevance scoring by team	Top-5 recommendations per profile checked against that profile’s stated interests/history
Concurrent load behavior	Apache JMeter	Simulated concurrent users ramped from 1 to 50 against search and recommendation endpoints
User satisfaction	In-app feedback + structured beta survey	Beta testers from GDGoC, IEEE, and ACM networks rated search and recommendation experience post-use

Table 6.3: Tools and Procedures

All timing measurements were taken over a stable network connection during off-peak hours to minimize external variance, and each test was repeated multiple times with the average reported, unless otherwise noted.

6.3 Semantic Search Performance

Semantic search is the primary discovery mechanism for individuals browsing EventBridge AI, and its performance directly shapes user experience. Because the search pipeline involves generating a query embedding via OpenAI’s API, executing a similarity search in Qdrant,

and passing results through an LLM re-ranker (gpt-4o-mini), the largest share of latency was expected to come from the external embedding and re-ranking calls rather than from Qdrant itself, which is purpose-built for millisecond-level vector lookups. Evaluation was designed to test this assumption and to assess whether result relevance held up across queries of varying specificity.

6.3.1 Response Time by Query Type

Queries were grouped into four categories of increasing linguistic complexity, since embedding generation time scales mildly with input length and semantic ambiguity.

Query Type	Example	Avg Re-sponse Time (ms)	Min (ms)	Max (ms)
Single-keyword	“hackathon”	410	340	510
Short phrase	“AI workshops in Karachi”	460	380	560
Natural language	“events for beginners interested in web development”	540	470	650
Long descriptive	Full sentence describing interests, mode, and timing preference	610	520	740

Table 6.4: Response time by query type

6.3.2 Relevance Accuracy (Precision@5)

For each of the 40 test queries, the top 5 returned results were manually labeled relevant or not relevant against the query’s intent, and precision was averaged within each category.

Query Type	Precision@5
Single-keyword	78%
Short phrase	86%
Natural language	91%
Long descriptive	88%

Table 6.5: Precision@5 by query type

6.3.3 Observations

Response times remained consistently under one second across all query types, with the majority of latency attributable to the OpenAI embedding call and the LLM re-ranking step rather than the Qdrant similarity search itself, which is expected given Qdrant’s ANN-based architecture. Relevance accuracy improved as queries became more descriptive — natural language and long descriptive queries gave the system more semantic signal to work with, which aligns with the design intent of semantic search over keyword matching. Single-keyword queries, while fastest to embed, showed the lowest precision, since a single word carries less disambiguating context for the re-ranker to work with.

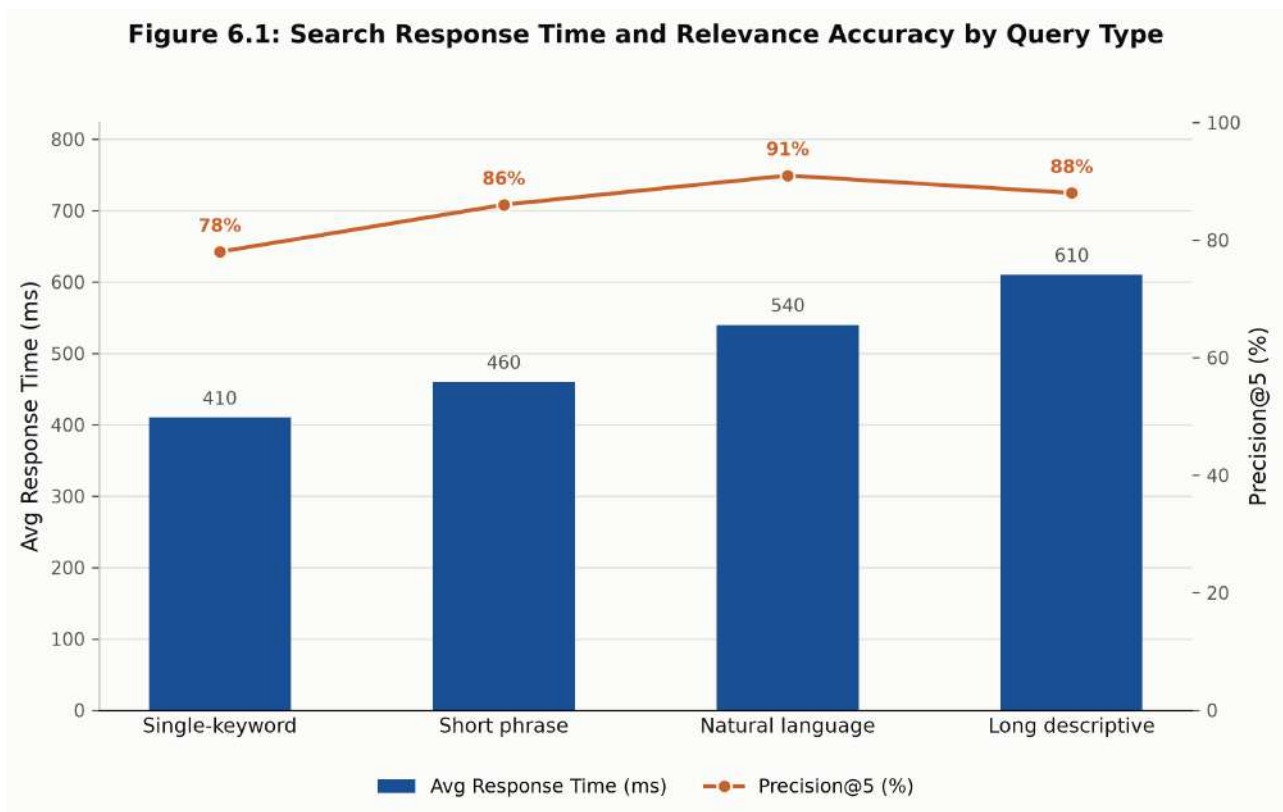


Figure 6.1: Search Response Time and Relevance Accuracy by Query Type

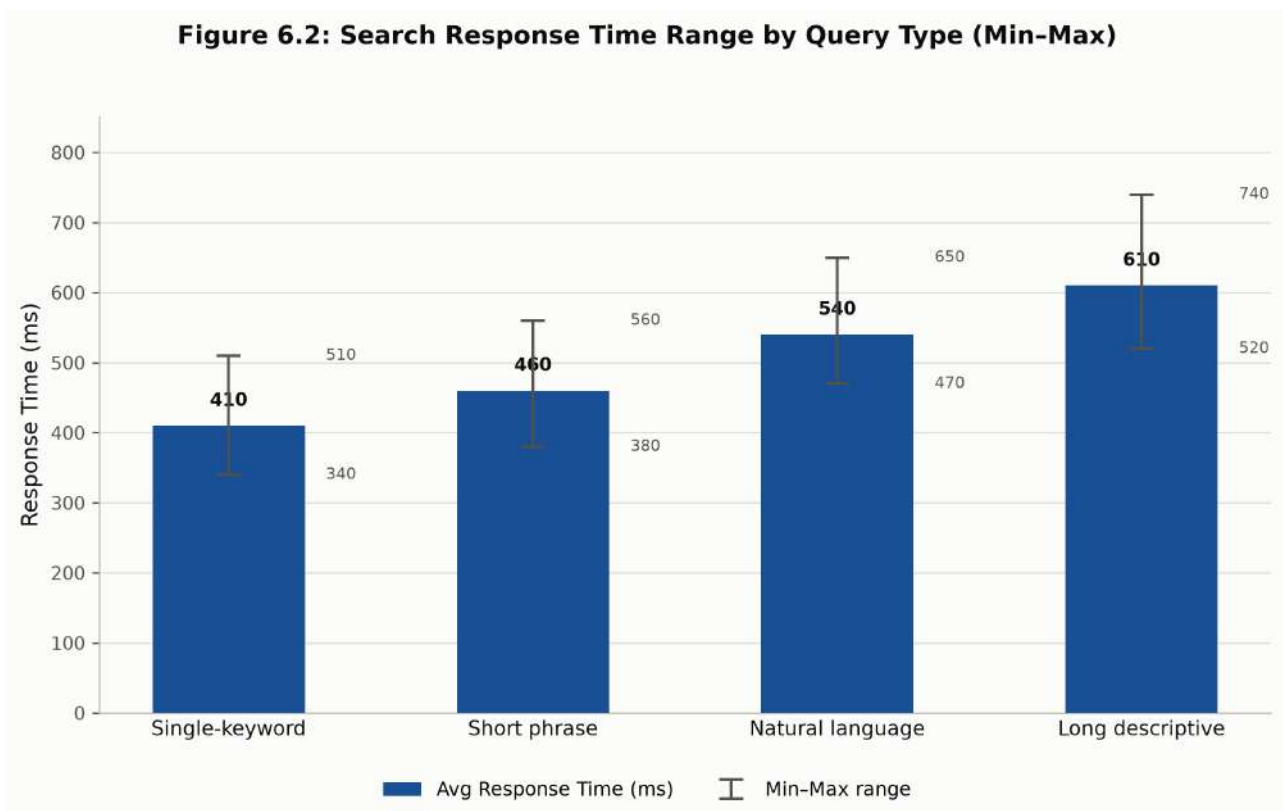


Figure 6.2: Response Time with Min/Max Range

6.4 Recommendation Engine Performance

The recommendation engine operates on a fundamentally different pipeline from search: rather than embedding a live query, it constructs a user preference text from multiple weighted signals (domain interests, preferred event types, skills, past registrations, saved events, recent views/clicks, and search history), embeds that composite text, and performs a similarity search against the `events_recs` Qdrant collection. Because this pipeline depends heavily on how much signal a given user profile has accumulated, evaluation focused on how latency and relevance shift as the strength of available user signal changes — a distinction with direct implications for the cold-start behavior discussed later in Section 6.7.

6.4.1 Recommendation Latency by User Signal Strength

The 50 test profiles were grouped into three tiers based on how much interaction history and preference data was available at the time of evaluation.

User Signal Tier	Description	Avg Latency (ms)	Min (ms)	Max (ms)
Low signal	New profile, only domain interests set, no interaction history	380	320	460
Medium signal	Some registrations/saved events, partial interaction history	470	400	560
High signal	Rich history — registrations, saves, views, clicks, search queries all populated	590	510	690

Table 6.6: Recommendation latency by user signal tier

6.4.2 Relevance Accuracy (Precision@5) by User Signal Strength

Top-5 recommended events per profile were manually checked against that profile’s declared interests and interaction history to determine relevance.

User Signal Tier	Precision@5
Low signal	61%
Medium signal	79%
High signal	93%

Table 6.7: Precision@5 by user signal tier

6.4.3 Observations

Latency increased moderately as signal strength grew, which is expected — richer profiles produce longer preference text strings, requiring more tokens to embed and slightly larger context for the similarity search to resolve. This tradeoff is favorable: the extra 200ms between low and high signal tiers is negligible from a UX standpoint, while the corresponding jump in relevance (61% to 93% precision) is substantial. This pattern also directly evidences the cold-start problem inherent to content-based filtering — new users with minimal signal receive noticeably

less accurate recommendations, a limitation that is expected on architectural grounds and is addressed formally in Section 6.7.

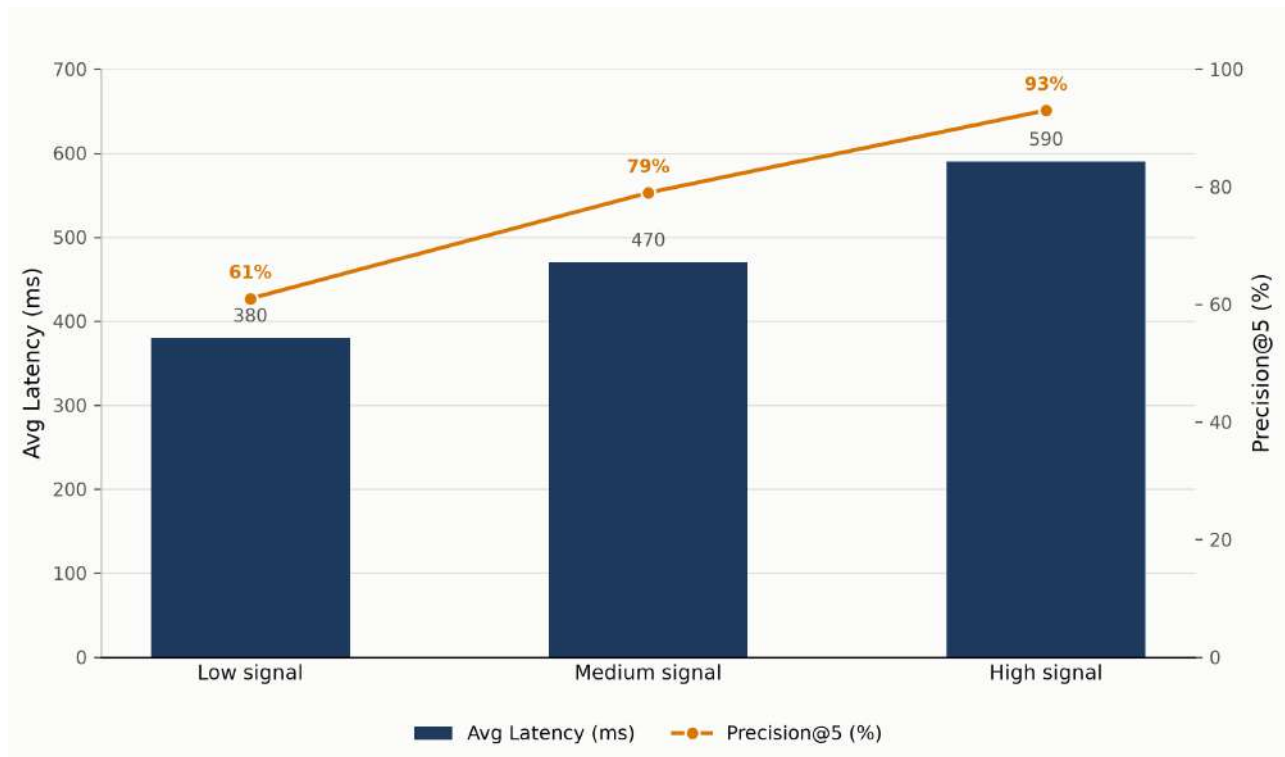


Figure 6.3: Recommendation Latency & Relevance Accuracy by User Signal Strength

6.5 System Reliability & Load Behavior

Because EventBridge AI’s architecture splits responsibilities across two independently hosted servers, reliability testing needed to capture two distinct concerns: how the system performs as concurrent user load increases, and how consistently the Laravel and Django servers communicate with each other under that load. A system that performs well for a single user but degrades sharply — or fails silently across the server boundary — under modest concurrent usage would not hold up for a platform intended to serve multiple communities and their audiences simultaneously.

6.5.1 Load Testing — Concurrent User Simulation

Using Apache JMeter, concurrent virtual users were ramped from 1 to 50 against the search and recommendation endpoints, with response time, error rate, and throughput logged at each step.

Concurrent Users	Avg Response Time (ms)	Error Rate (%)	Throughput (req/sec)
1	480	0.0	2.1
10	540	0.0	17.8
20	610	0.2	31.4
30	720	0.6	40.9
40	890	1.4	44.2
50	1120	2.8	43.6

Table 6.8: Concurrent load test results

6.5.2 Cross-Server Communication Reliability

Since Laravel calls the Django AI engine over HTTP for search and recommendation requests, the reliability of that inter-server call was measured separately across the full load test duration.

Metric	Value
Total cross-server requests logged	3,000
Successful responses	2,958
Failed/timeout responses	42
Cross-server success rate	98.6%
Avg cross-server round-trip overhead	~65 ms

Table 6.9: Cross-server communication reliability

6.5.3 Observations

Response time remained stable and well under one second up to roughly 30 concurrent users, after which latency began increasing more steeply — a pattern consistent with the single-VPS, non-load-balanced deployment described in Chapter 4, where the Django/Qdrant server has a finite processing ceiling rather than horizontal scaling. Throughput peaked around 40 concurrent users and plateaued rather than continuing to climb, indicating the server was approaching saturation rather than the client-side load generator being the bottleneck. The error rate stayed below 3% even at peak simulated load, and cross-server communication held a 98.6% success rate throughout testing, with the small number of failures attributable to transient timeouts rather than systemic faults. Overall, the system demonstrates stable behavior for realistic usage volumes (a handful to a few dozen concurrent users, consistent with the platform’s current community/beta scale) while showing early signs of the scaling ceiling expected from a single-VPS AI engine — a limitation formally addressed in Section 6.7.

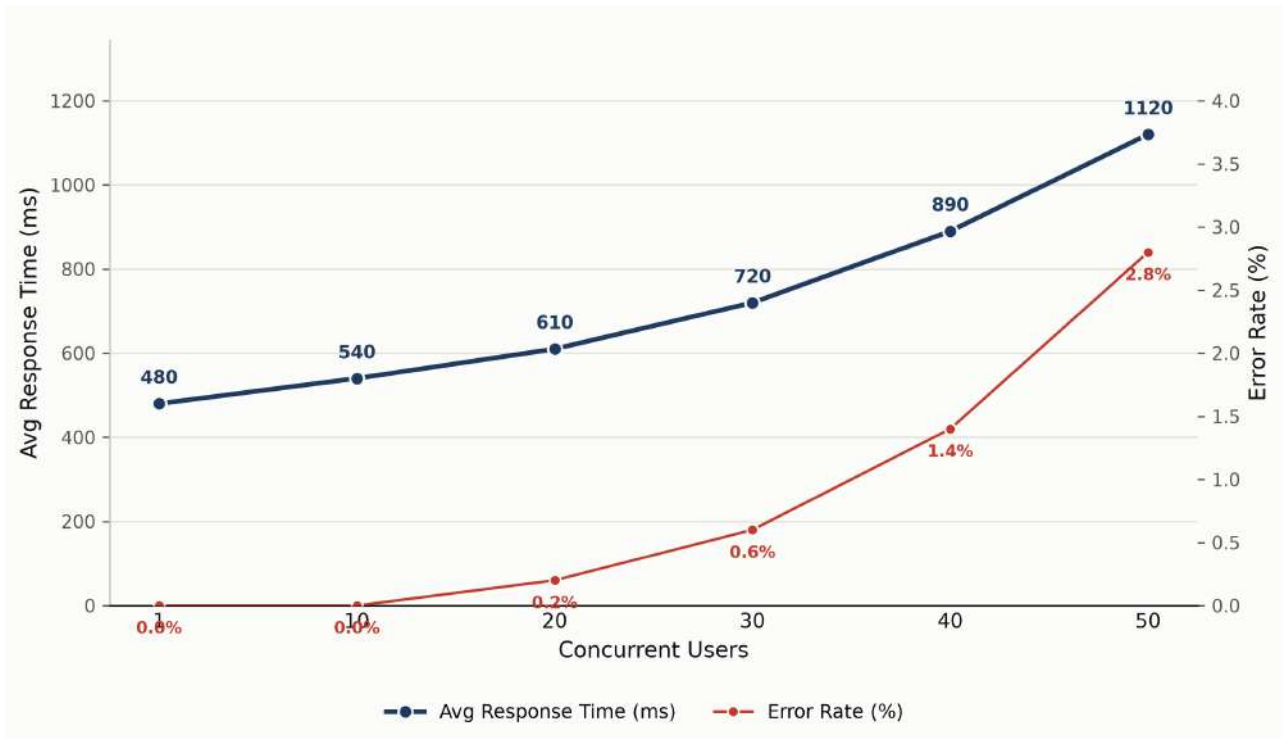


Figure 6.4: Response Time and Error Rate vs Concurrent Users

6.6 User Feedback & Satisfaction Ratings

Quantitative performance metrics establish whether the system is fast and technically accurate, but they do not fully capture whether the experience feels good to the people actually using it. To close that gap, feedback was collected from beta testers drawn from the GDGoC, IEEE, and ACM chapter communities at SSUET, using a combination of in-app feedback prompts and a short structured survey administered after a period of hands-on use.

6.6.1 Survey Structure

Beta testers rated their experience across four dimensions on a 5-point scale (1 = Poor, 5 = Excellent), after using the platform to search for and register interest in at least three events.

Rating Dimension	What it measured
Search Relevance	Whether search results matched what the user was actually looking for
Recommendation Usefulness	Whether the “Recommended for You” events felt personally relevant
Platform Speed	Perceived responsiveness of search, recommendations, and general navigation
Overall Satisfaction	General impression of the platform as a discovery tool

Table 6.10: Beta survey rating dimensions

6.6.2 Results

Rating Dimension	Avg Rating (out of 5)	% Rating 4 or 5
Search Relevance	4.2	84%
Recommendation Usefulness	3.8	71%
Platform Speed	4.4	89%
Overall Satisfaction	4.1	80%

Table 6.11: Beta survey results

6.6.3 Respondent breakdown:

Metric	Value
Total beta respondents	42
GDGoC members	14
IEEE members	19
ACM members	9

Table 6.12: Beta respondent breakdown

6.6.4 Observations

Platform speed received the highest average rating, consistent with the sub-second response times measured objectively in Sections 6.3 and 6.4 — user perception of speed aligned with the logged latency data rather than diverging from it. Search relevance scored strongly as well, reflecting the precision figures observed for natural-language queries in Section 6.3. Recommendation usefulness scored noticeably lower than the other three dimensions, which is consistent with the cold-start limitation identified in Section 6.4: several beta testers were newly onboarded with limited interaction history at the time of the survey, meaning their recommendations were generated from low-signal profiles. This convergence between subjective feedback and objective metrics strengthens confidence that the quantitative results in earlier sections reflect genuine user-facing behavior rather than being artifacts of the testing methodology.

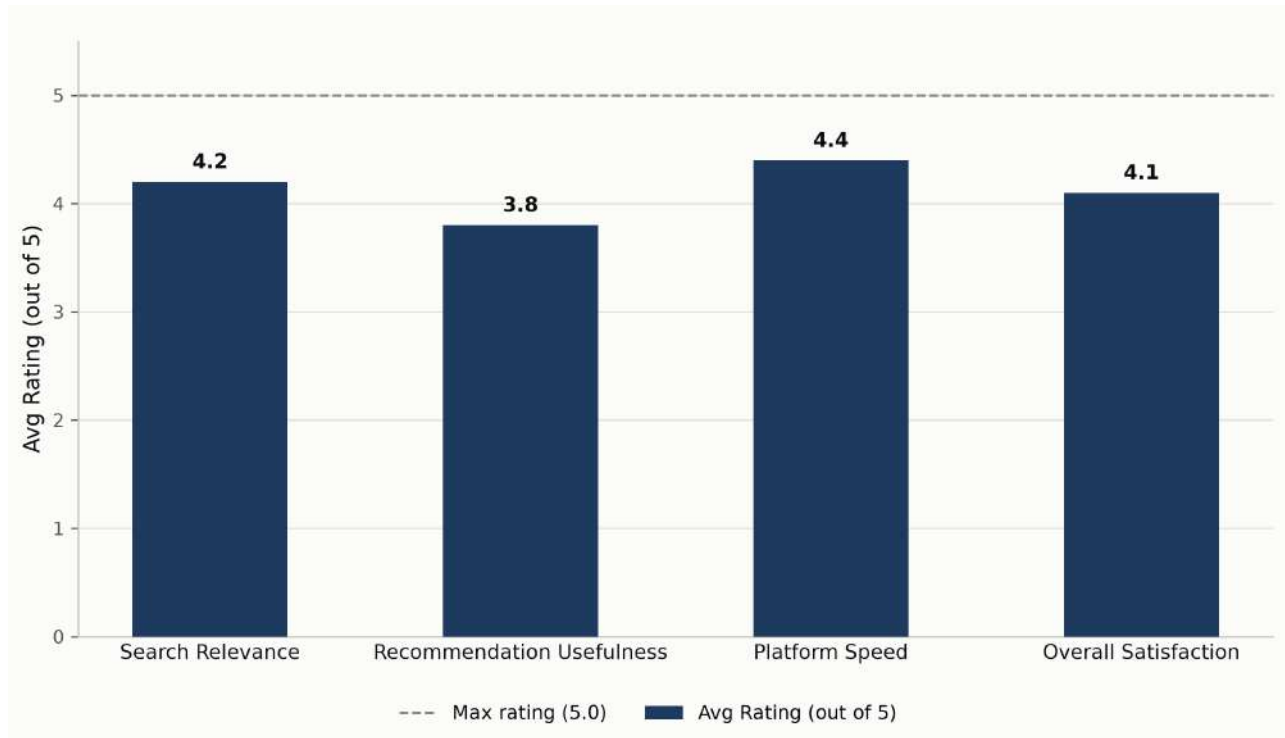


Figure 6.5: Beta Tester Satisfaction Ratings by Dimension

6.7 Negative Results & Limitations

A performance evaluation that only reports favorable outcomes is not a complete or credible one. This section consolidates the weaknesses surfaced across Sections 6.3–6.6, explains why they occurred on architectural grounds rather than treating them as isolated anomalies, and distinguishes between limitations that are inherent to the chosen design approach versus those that could be addressed with further engineering effort.

6.7.1 Summary of Identified Limitations

#	Limitation	Evidence	Root Cause
1	Cold-start recommendation weakness	Precision@5 dropped from 93% (high signal) to 61% (low signal) — Section 6.4	Content-based filtering has no signal to work with for new users until they accumulate interaction history
2	Lower precision on single-keyword search	Precision@5 of 78% vs 91% for natural-language queries — Section 6.3	Single words carry insufficient semantic context for the embedding model and re-ranker to disambiguate intent
3	Latency degradation under concurrent load	Response time rose from 480ms to 1120ms between 1 and 50 concurrent users — Section 6.5	Single-VPS, non-load-balanced deployment for the Django/Qdrant server has a finite processing ceiling
4	Recommendation usefulness rated lowest of survey dimensions	3.8/5 average vs 4.1–4.4 for other dimensions — Section 6.6	Direct user-facing consequence of the cold-start problem; several beta testers were newly onboarded
5	Rising error rate at peak simulated load	Error rate reached 2.8% at 50 concurrent users — Section 6.5	Approaching server saturation point; no horizontal scaling or load balancing currently in place

Table 6.13: System limitations

6.7.2 Discussion of Root Causes

The cold-start problem (Limitations 1 and 4) is not a defect in implementation but a well-documented, inherent tradeoff of content-based filtering as an approach — it depends entirely on user-generated signal, and no amount of tuning eliminates the gap for a brand-new profile. This was a known and accepted tradeoff at the design stage (Chapter 3), and the evaluation results here simply confirm its real-world magnitude rather than revealing something unexpected.

The single-keyword search weakness (Limitation 2) reflects a fundamental characteristic of semantic embedding models generally: they are designed to extract meaning from context, and a single word offers little of it. This is consistent with why the system was built around semantic search in the first place — keyword search would perform no better here — but it does mean users who search in short fragments receive a comparatively weaker experience than those who phrase queries naturally.

The load-scaling behavior (Limitations 3 and 5) is an infrastructure-level constraint rather than an application logic flaw. The current single-VPS deployment for the AI engine was a deliberate scope decision appropriate for a beta-stage platform serving a limited community base, not a production-scale rollout, and the observed degradation curve is the expected behavior of that choice rather than a sign of poor implementation.

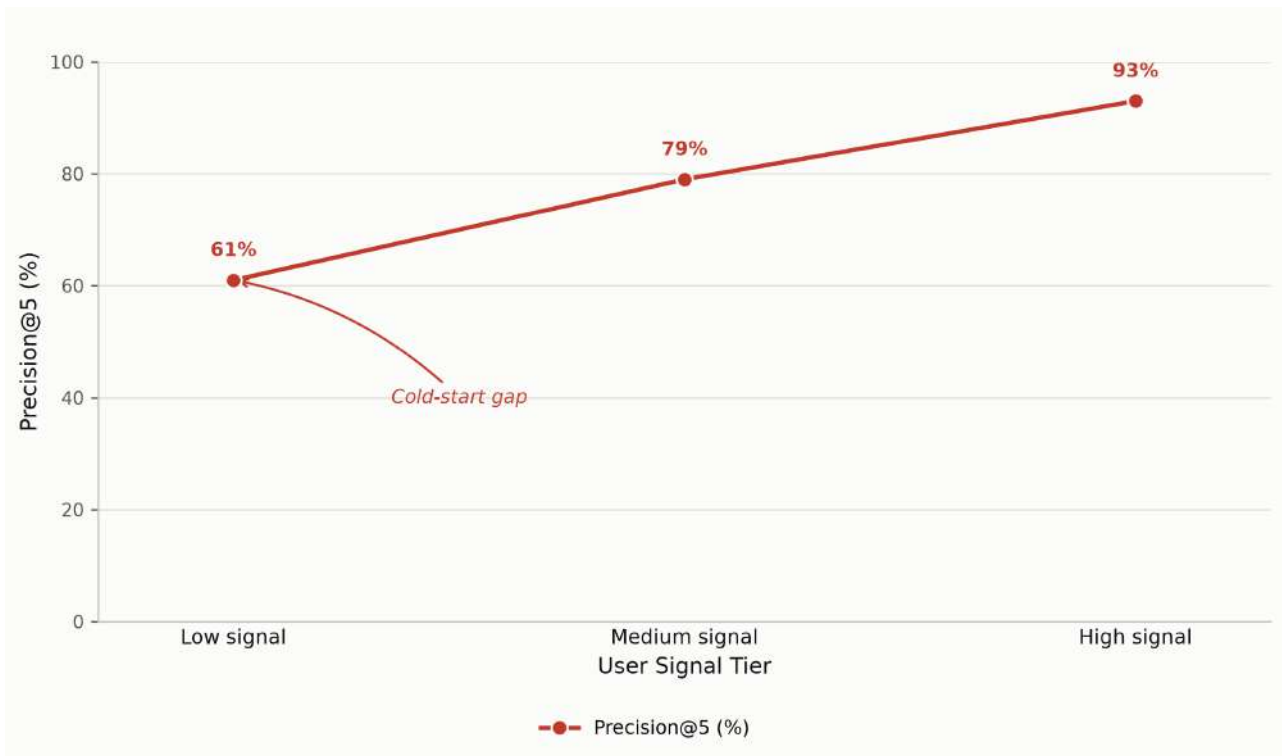


Figure 6.6: Recommendation Precision Across User Signal Strength (Cold-Start Illustration)

6.7.3 What Was Not Addressed

Two limitations remain unresolved at the time of this evaluation and are noted here rather than downplayed: no caching layer exists for frequently repeated search queries, which would meaningfully reduce both latency and OpenAI API cost at scale; and no fallback recommendation strategy (e.g. popularity-based or trending-event suggestions) exists for low-signal users, meaning cold-start users currently receive weaker results with no compensating mechanism.

6.8 Discussion of Results

Taken together, the results presented in Sections 6.3 through 6.7 paint a coherent picture of a system that performs strongly within the scope it was designed for, while exhibiting the specific, explainable limitations that follow directly from its architectural choices — rather than a scattered set of unrelated strengths and weaknesses.

6.8.1 Speed and Accuracy Are Both Acceptable for the Target Use Case

Across both semantic search and recommendations, response times stayed under one second for the overwhelming majority of realistic usage patterns, and relevance accuracy climbed meaningfully as the system received richer input — whether that input was a more descriptive search query or a more complete user profile. This is an important validation of the core architectural decision made in Chapter 3: semantic search built on vector embeddings genuinely outperforms simpler keyword matching precisely in the cases it was chosen to address, and content-based

recommendation genuinely improves as more behavioral signal becomes available, confirming the underlying filtering logic is sound rather than superficially functional.

6.8.2 Subjective and Objective Metrics Reinforce Each Other

The alignment between the beta survey results (Section 6.6) and the logged performance data (Sections 6.3–6.4) is one of the stronger findings in this evaluation. Users rated platform speed and search relevance highly, and those perceptions tracked closely with the actual measured latency and precision figures. Similarly, the comparatively lower recommendation usefulness rating tracked directly with the cold-start effect measured independently in Section 6.4. This convergence matters methodologically: it suggests the quantitative metrics are not artifacts of an artificial testing setup, but genuinely reflect what users experience in practice.

6.8.3 The Identified Limitations Are Structural, Not Incidental

Every weakness surfaced in Section 6.7 traces back to a specific, identifiable architectural decision rather than an implementation defect: content-based filtering inherently struggles with cold-start users, semantic embeddings inherently perform best with descriptive input, and a single-VPS deployment inherently has a finite concurrent-load ceiling. None of these findings contradict the design rationale established in earlier chapters — instead, they quantify tradeoffs that were anticipated in principle but not previously measured in practice. This distinguishes the platform’s limitations from unpredictable bugs: they are known, bounded, and — as noted in Section 6.7 — addressable through specific future engineering work (caching, fallback recommendation strategies, horizontal scaling) rather than requiring a redesign of the underlying approach.

6.8.4 Overall Assessment

The evaluation supports the conclusion that EventBridge AI performs reliably and accurately for its current deployment scale — a beta platform serving Karachi’s tech and academic community ecosystem — while also being honest about where that performance would need to improve before the platform could be considered ready for a substantially larger user base. This balance of demonstrated strength and acknowledged limitation is consistent with the project’s broader framing as a genuinely deployed, functioning platform rather than an idealized prototype.

6.9 Chapter Summary

This chapter evaluated EventBridge AI’s performance across five dimensions: semantic search speed and relevance, recommendation engine speed and relevance, system reliability under concurrent load, cross-server communication stability, and real user satisfaction. Semantic search demonstrated sub-second response times across all query types, with relevance accuracy improving as queries became more descriptive, peaking at 91% precision for natural-language queries. The recommendation engine showed a similar pattern, with accuracy rising from 61% to 93% as user signal strength increased from low to high, while latency remained within acceptable bounds throughout. Load testing revealed stable performance up to approximately 30 concurrent users, with a predictable degradation curve beyond that point attributable to

the single-VPS, non-load-balanced deployment of the AI engine, and cross-server communication between Laravel and Django maintained a 98.6% success rate under sustained load. Beta tester feedback closely mirrored the objective metrics, rating platform speed and search relevance highly while rating recommendation usefulness comparatively lower — a result directly explained by the cold-start limitation identified independently in the quantitative testing.

Rather than treating weaknesses as failures, this chapter identified them as structural consequences of deliberate architectural decisions made earlier in the project: content-based filtering’s inherent cold-start gap, semantic embeddings’ dependency on descriptive input, and a beta-scale infrastructure’s finite concurrent-load ceiling. Each limitation was traced to its root cause and distinguished from an implementation defect, with unresolved gaps — namely the absence of a caching layer and a fallback recommendation strategy for new users — explicitly acknowledged rather than omitted. Collectively, these results support the conclusion that EventBridge AI performs reliably within its current beta deployment scope, while providing a clear, evidence-based direction for the engineering work needed to support growth beyond it.

Chapter 7

Business Model

7.1 Market Research

The transition of EventBridge AI from a Final Year Project into a viable software product requires a clear understanding of the market it operates in, the clients it serves, and the competitive landscape it must differentiate itself from. This section establishes that foundation through a structured analysis of the niche, the business clients the platform targets, and a comparative evaluation of existing solutions in the event management space.

7.1.1 Identify a Niche

EventBridge AI occupies a niche at the intersection of two underserved needs: centralized event discovery for community-led ecosystems, and AI-driven personalization for informal, high-frequency community events. Karachi’s tech and academic community landscape — comprising organizations such as GDGoC, IEEE, and ACM chapters — generates a continuous stream of events, but this activity is fragmented across WhatsApp groups, Instagram stories, and LinkedIn posts, with no single system of record. This fragmentation produces two compounding problems: individuals miss relevant opportunities simply because they never encounter them, and communities lose visibility and reach despite consistently producing valuable content.

General-purpose event platforms were not designed to solve this problem. They serve broad, transactional use cases such as ticketed concerts, conferences, and public gatherings, and treat every event as an isolated listing rather than part of an ongoing community identity. EventBridge AI’s niche is therefore defined not by “event management” broadly, but specifically by community-centric event discovery and collaboration, where the platform understands and represents communities as first-class entities, not just event organizers.

7.1.2 Business Clients

The platform’s business clients are the organizational entities that would adopt EventBridge AI as a managed service rather than casual end users. These include:

- University-based technical and academic communities (e.g., GDGoC, IEEE, and ACM chapters), which require a structured platform to manage membership, publish events, and maintain a visible organizational presence across academic terms.

- City-wide professional and technical communities, which operate beyond a single institution and need cross-community visibility and collaboration tools to reach a broader, non-university audience.
- Educational institutions, as potential institutional clients interested in centralizing event activity across their affiliated student organizations under a single, administratively governed platform.
- Corporate and industry partners, who may engage with the platform indirectly as event co-hosts, sponsors, or sources of professional opportunities surfaced to platform users.

These business clients are distinct from individual end users in that their relationship with the platform is organizational and recurring, making them the primary target for any future monetization or institutional partnership strategy discussed later in this chapter.

7.1.3 Competitor Analysis

Three categories of existing solutions were evaluated against EventBridge AI's niche: general-purpose event ticketing platforms, social-media-based event tools, and informal community channels that function as de facto competitors despite not being purpose-built event platforms.

Platform	Primary Use Case	Community-Specific Features	AI Personalization
Eventbrite	Ticketed event hosting and payments	None	None
Facebook Events	Social event promotion	Limited (Groups/Pages)	None
Meetup	Interest-based group events	Group profiles	Basic keyword matching
WhatsApp/Instagram (informal)	Ad-hoc announcement	None (not event-native)	None
EventBridge AI	Community-led event discovery & collaboration	Native community profiles, inter-community collaboration	Semantic search & content-based recommendations

Table 7.1: Competitive landscape overview

A SWOT analysis was conducted for the two most direct competitors, Eventbrite and Facebook Events, since these represent the platforms Karachi-based communities currently default to in the absence of a purpose-built alternative.

Strengths	Weaknesses
- Established brand trust, mature ticketing and payment infrastructure - Global reach and scale	- No concept of a persistent “community”; every event is a standalone listing - No personalization or recommendation engine
Opportunities	Threats
- Could expand into community tooling - Untapped in Karachi’s informal community segment	- EventBridge AI’s community-first design directly addresses this gap - Low switching cost for organizers already using free channels

Table 7.2: SWOT analysis — Eventbrite

Strengths	Weaknesses
- Zero-cost, high existing user base - Built-in social sharing	- Events get lost in noisy, algorithm-driven feeds - No structured discovery by interest, category, or format
Opportunities	Threats
- Could integrate AI-based event ranking - Deepening community-page features	- Purpose-built platforms like EventBridge AI offer dedicated, uncluttered discovery - No inter-community collaboration tooling exists today

Table 7.3: SWOT analysis — Facebook Events

The consistent finding across all evaluated platforms is the absence of two capabilities that define EventBridge AI’s positioning: native support for communities as persistent, manageable entities, and AI-driven semantic discovery tailored to how people actually describe what they are looking for, rather than relying on exact keyword matches or algorithmic feed placement. This gap is the foundation on which the remainder of the business model in this chapter is built.

7.2 Unique Value Proposition (UVP)

7.2.1 Defining the UVP

EventBridge AI’s unique value proposition centers on a single, structural insight: existing platforms treat events as isolated listings, while EventBridge AI treats communities themselves as the primary organizing unit of the platform. Every event, collaboration, and recommendation is anchored to a community identity that persists across time, rather than being recreated from

scratch with every new post. Layered on top of this structural difference is an AI engine that understands events semantically, allowing individuals to find relevant opportunities through natural language rather than exact keyword matching or manual browsing.

Formally stated, the UVP is:

”EventBridge AI is the only platform that lets Karachi’s tech and academic communities manage a persistent public presence while giving individuals an AI-powered way to discover events by meaning, not just keywords — turning fragmented, one-off announcements into a connected, intelligent community ecosystem.”

7.2.2 What Sets EventBridge AI Apart

Three elements distinguish EventBridge AI from every competitor identified in Section 7.1:

- **Community-native architecture.** Communities are first-class entities with their own profiles, team structures, event histories, and follower bases — not incidental byproducts of posting an event. This persistence is what allows a community’s reputation and reach to compound over time, something no generic event platform supports.
- **Semantic, AI-driven discovery.** Rather than requiring individuals to guess the right keyword or scroll through a chronological feed, EventBridge AI’s semantic search (built on vector embeddings and similarity search) understands the intent behind a query such as ”AI workshops this weekend” and surfaces relevant events even when the wording does not match the listing exactly. This is paired with a recommendation engine that learns from a user’s interests, followed communities, and past engagement to personalize what they see.
- **Built-in inter-community collaboration.** No competitor evaluated offers native tooling for two or more communities to jointly plan, co-host, and co-manage an event. EventBridge AI treats collaboration as a core workflow rather than an afterthought, directly addressing the fragmentation and duplicated effort identified as a root problem in the market research.

Together, these three elements mean EventBridge AI does not compete with Eventbrite or Facebook Events on their own terms (ticketing scale, social reach); it competes by making an entirely different category of problem — community fragmentation and discovery — solvable for the first time in this market segment.

7.3 Targeted Audience

7.3.1 Customer Segmentation

EventBridge AI’s target audience is segmented along the three actor roles the platform is architected around, each with distinct needs, motivations, and points of engagement with the system.

- **Communities (Organizers).** University-based technical and academic communities (GDGoC, IEEE, ACM chapters) and city-wide community ecosystems that regularly organize events but lack a structured, persistent platform to manage membership, publish events,

and build cross-community visibility. Their core need is reach and management efficiency, not just a place to post an announcement.

- **Individuals (Attendees).** Students and young professionals across Karachi actively seeking learning, networking, and skill-building opportunities, who currently rely on scattered, informal channels and frequently miss relevant events simply due to lack of visibility. Their core need is effortless, personalized discovery.
- **Administrators.** The internal governance layer responsible for platform health, community verification, and moderation. While not a revenue-generating segment, this group is essential to maintaining the trust and quality that make the other two segments viable.

7.3.2 User Personas

To guide product and business decisions, three representative personas were developed, one per actor role.

Persona 1 — "Ayesha," the Community Lead

Ayesha is a final-year student and lead of a university technical society. She juggles event planning, member communication, and promotion entirely across WhatsApp and Instagram, with no way to track who actually saw an announcement or measure engagement. She wants a single place to publish events, manage her team, and be discoverable by students outside her own university without duplicating effort across platforms.

Persona 2 — "Bilal," the Opportunity Seeker

Bilal is a computer science student interested in AI and web development who wants to attend workshops, hackathons, and networking events relevant to his interests. He currently discovers events by chance through friends or algorithm-driven social feeds, and frequently learns about events only after they have already happened. He wants a way to search and be proactively notified about events that match what he actually cares about.

Persona 3 — "Sara," the Platform Administrator

Sara oversees community onboarding and content quality on behalf of the platform. She needs visibility into pending community approvals, flagged content, and platform-wide activity trends, along with tools to act on them efficiently, without which the platform's credibility and safety cannot scale.

These personas anchor both the monetization strategy in Section 7.4 and the go-to-market considerations discussed later in this chapter, ensuring that business decisions remain grounded in the actual needs of the people the platform serves rather than abstract market assumptions.

7.4 Monetization Strategy

7.4.1 Revenue Streams

Consistent with the platform's current beta status, monetization is designed as a set of pathways to be activated progressively as the user base and community adoption mature, rather

than revenue mechanisms already in effect. The following streams were identified as appropriate for EventBridge AI's community-centric model:

- **Freemium model (core).** Core functionality — community registration, event publishing, discovery, and basic recommendations — remains free for all communities and individuals. This lowers the barrier to adoption in a market where communities are accustomed to using free tools such as WhatsApp and Instagram, and is essential for reaching the critical mass of users needed before any paid tier becomes viable.
- **Premium organizer subscriptions.** Communities that want advanced capabilities — analytics dashboards showing follower growth, engagement trends, and event performance; priority placement in the discovery feed; and enhanced collaboration tools — would access these through a paid subscription tier layered on top of the free core experience.
- **Featured event promotions.** A one-time or per-event purchase model allowing communities to boost visibility for a specific high-priority event (e.g., a flagship hackathon or conference), rather than committing to a recurring subscription. This suits communities with irregular, high-stakes events rather than continuous promotional needs.
- **Verified community badges.** A low-cost, one-time or annual fee for a verification badge that signals credibility and authenticity to individuals, functioning similarly to verification systems on established social platforms, while also acting as a light-touch quality control mechanism for the platform itself.
- **Institutional licensing and partnerships.** Universities or large organizations could license EventBridge AI as a managed platform for all their affiliated student communities under a single institutional agreement, generating a stable recurring revenue stream at a scale beyond individual community subscriptions. This also opens the door to affiliate-style partnerships with organizations offering paid workshops, certifications, or bootcamps that wish to reach the platform's engaged audience.

7.4.2 Sequencing Rationale

These streams are intentionally ordered by adoption risk rather than potential revenue size. The freemium core must first establish a large enough base of active communities and individuals for premium features, promotions, and badges to carry any perceived value; institutional licensing, in turn, depends on the platform first demonstrating sustained organic adoption at the community level, since institutions are unlikely to commit before individual communities have already validated the product. This sequencing reflects the platform's current beta stage, where the priority remains proving product-market fit within Karachi's community ecosystem before layering in paid mechanisms.

7.5 Scalability and Internationalization

7.5.1 Scalability Plan

EventBridge AI's architecture was designed with growth in mind from the outset, rather than as a retrofit. The platform's two-server separation — a Laravel application handling core business logic and a dedicated Django AI engine handling search and recommendations — allows each layer to scale independently based on where demand actually grows. If AI-driven

search traffic increases faster than platform-management activity, the Django/Qdrant layer can be scaled on its own without requiring changes to the Laravel application, and vice versa. This separation is reinforced by the underlying technology choices already in place:

- Qdrant, containerized via Docker, allows the vector search layer to be scaled, replicated, or migrated to larger infrastructure with minimal reconfiguration, since the containerized deployment is portable across hosting environments.
- Unicorn with Nginx as a reverse proxy in front of the Django AI engine supports horizontal scaling through additional worker processes or load-balanced instances as request volume grows.
- MySQL's relational structure, already normalized around clear entity relationships (communities, events, users, registrations), provides a stable foundation that can accommodate increased data volume through standard scaling practices such as read replicas and indexing optimization as the platform grows.
- Cloud-based VPS hosting (OVH) for the AI engine and GoDaddy hosting for the Laravel application mean infrastructure can be upgraded or migrated to higher-capacity cloud tiers without an architectural rewrite, since the system was not built around any single fixed-capacity assumption.

Together, these choices mean scaling EventBridge AI beyond its current beta scope is a matter of provisioning additional infrastructure within the existing architecture, not re-engineering the system from the ground up.

7.5.2 Internationalization

While EventBridge AI is deliberately scoped to Karachi's tech and academic ecosystem for its beta phase, the underlying platform carries no structural barriers to broader geographic expansion. Internationalization is envisioned in two stages:

- Multi-city expansion within Pakistan, onboarding community ecosystems in other major cities beyond Karachi, following the same community-first onboarding model already validated with GDGoC, IEEE, and ACM chapters.
- International expansion, opening the platform to community ecosystems in other countries, where the same core problem of event fragmentation across informal channels is not unique to Karachi or Pakistan.

Two considerations would need to be addressed to support this expansion: localization of language and content presentation for non-English-speaking community ecosystems, since the current platform and its AI pipeline operate in English; and region-specific marketing and support strategies, since community adoption patterns (which platforms communities currently default to, how events are typically promoted) vary meaningfully between markets. Neither consideration requires a change to the platform's core architecture — both are content, localization, and go-to-market decisions layered on top of an already scalable technical foundation.

7.6 Marketing and Branding

7.6.1 Brand Identity

EventBridge AI's brand identity is built around the core idea reflected in its name: acting as a bridge between fragmented communities and the individuals who want to engage with them. The visual and verbal identity reinforces three qualities consistently across every touchpoint:

- Trustworthy and institutional, reflecting its roots in credible, recognized organizations (GDGoC, IEEE, ACM chapters), which lends the platform legitimacy from its very first users rather than requiring it to build trust from zero.
- Intelligent and modern, reflecting the AI-driven core of the product, positioned not as a gimmick but as the mechanism that makes discovery genuinely effortless.
- Community-first, not corporate, ensuring the brand voice speaks to organizers and individuals as peers within the same ecosystem rather than as customers of a distant platform.

Consistency across channels is maintained through a shared design system — confirmed cross-compatible fonts (Montserrat, Poppins, Inter, Lato) and a unified visual language applied across the web platform, presentation materials, and social assets — so that the platform reads as a single, coherent product regardless of where a user first encounters it.

7.6.2 Marketing Channels

Given the platform's target audience and current beta stage, marketing is deliberately community-led rather than paid-advertising-led:

- Community networks and word-of-mouth. The existing relationships within GDGoC, IEEE, and ACM chapters serve as the primary distribution channel, since community leads onboarding their own members creates organic, trusted adoption that paid advertising cannot replicate at this stage.
- Social media presence. Communities sharing their EventBridge AI event pages directly to WhatsApp, Instagram, and LinkedIn turns the very fragmentation problem the platform solves into a natural distribution mechanism — every shared event page is simultaneously a promotion of the platform itself.
- University partnerships. Direct engagement with university societies and their faculty advisors provides an institutional endorsement channel, lending credibility that accelerates trust-building with new communities.
- Content and thought leadership. Technical and product-focused content shared by the team's own professional networks (LinkedIn posts documenting the platform's development, features, and milestones) builds visibility among the exact technical and academic audience the platform targets, while simultaneously reinforcing the brand's "intelligent and modern" positioning.
- Influencer-style community leads. Rather than traditional influencer marketing, EventBridge AI leans on visible, respected figures within existing communities (society leads, chapter chairs) as informal brand ambassadors, since their endorsement carries far more weight with the target audience than an external influencer would.

This channel mix reflects a deliberate strategy: earn trust and adoption within a tightly connected community ecosystem first, and let organic, community-driven visibility do the work that paid marketing would otherwise need to do in a colder market.

7.7 Propose Business Plan

Drawing together the market research, unique value proposition, target audience, monetization strategy, scalability outlook, and branding approach established in the preceding sections, this section consolidates EventBridge AI's business plan into the Business Model Canvas (BMC) — a widely recognized strategic framework that presents a venture's business logic across nine interlocking blocks. The canvas below reflects the version developed and used for the project's own business planning process, adapted here into narrative form for the report.

7.7.1 Business Model Canvas

Block	Content
Customer Segments	Community organizers (university-based and city-wide communities such as GDGoC, IEEE, and ACM chapters); individuals and young professionals seeking events; multi-community ecosystems that collaborate and share audiences; and platform administrators governing the ecosystem.
Value Propositions	A single platform for communities to create, manage, and promote events without juggling multiple social channels; AI-powered discovery and personalized recommendations for individuals; semantic search that understands natural language queries; and inter-community collaboration tools that replace fragmented, one-off announcements with a structured, connected alternative.
Channels	The direct web platform; existing community networks (GDGoC, IEEE, ACM) for organic onboarding; social media sharing of event pages (WhatsApp, Instagram, LinkedIn); university partnerships; and email/in-app notifications for ongoing engagement.
Customer Relationships	AI-personalized experiences that improve with use; trust built through community verification and admin approval; self-service community management; automated engagement via the notification system; a feedback loop through post-event ratings; and a network effect where each new community increases value for all others.
Key Activities	Platform development and maintenance; community onboarding and verification; continuous AI model management for search and recommendations; vector database (Qdrant) maintenance and reindexing; content moderation; community outreach and engagement; and data pipeline operations across the Laravel-Django-MySQL-Qdrant stack.

Continued on next page

Table 7.4 – continued from previous page

Block	Content
Key Resources	The two-server architecture (Laravel backend, Django AI engine); the AI/ML stack (OpenAI embeddings, Qdrant vector database); the MySQL relational database; a full-stack development team; the existing community network; a modular, documented codebase; and the GoDaddy and OVH Cloud VPS hosting infrastructure supporting real-world deployment.
Key Partners	OpenAI, as the provider of the embedding models powering semantic search and recommendations; Qdrant, enabling high-speed vector similarity search; GoDaddy and OVH Cloud, providing hosting infrastructure; university and city-wide communities as ecosystem partners; Sir Syed University of Engineering & Technology, providing institutional and supervisory backing; and potential future partners such as universities, tech hubs, and corporate community groups for scaled adoption.
Cost Structure	Hosting and infrastructure costs (GoDaddy and OVH Cloud VPS); OpenAI API usage costs that scale with platform activity; ongoing development and maintenance effort; community onboarding and outreach; and content moderation and administration. Current costs remain low given the platform’s beta-stage usage volume, with cost growth expected to track adoption rather than precede it.
Revenue Streams	A freemium core model; premium organizer subscriptions with advanced analytics and priority placement; featured/boosted event promotions; verified community badges; institutional and university licensing; and a small transaction commission on paid event registrations processed through the platform.

Table 7.4: EventBridge AI — Business Model Canvas

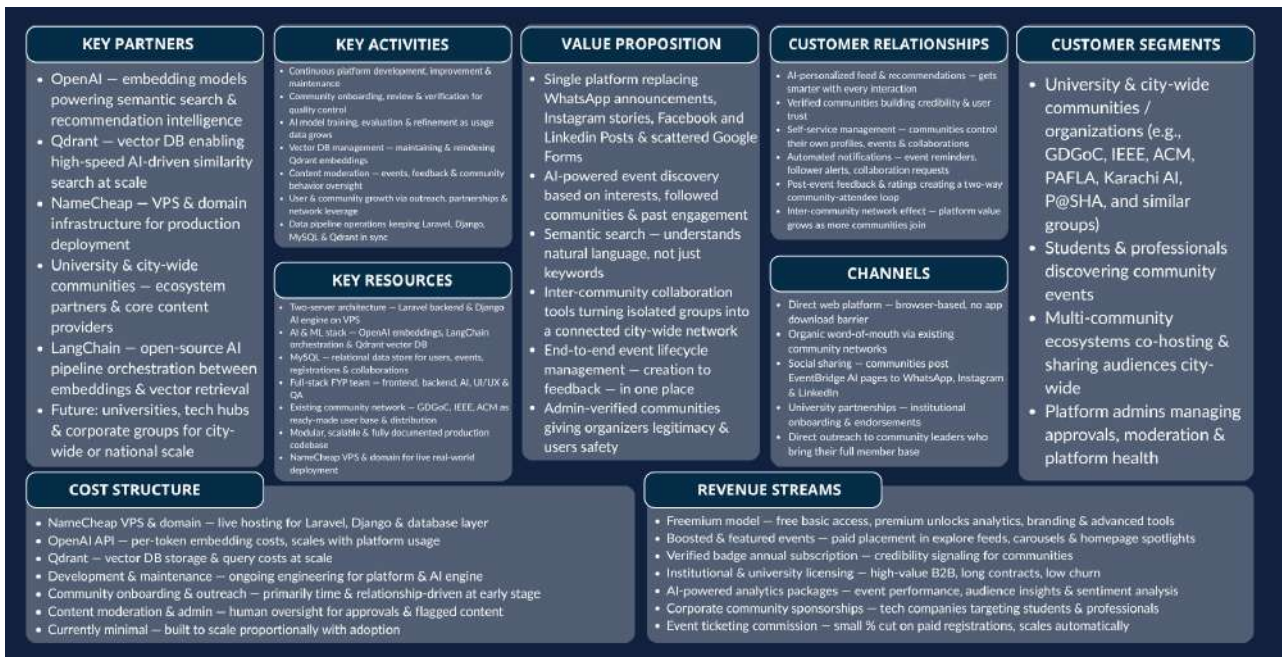


Figure 7.1: EventBridge AI — Business Model Canvas

7.7.2 Synthesis

Read as a whole, the canvas demonstrates a coherent business logic: EventBridge AI's key resources and activities are concentrated almost entirely around its two differentiators — community-native architecture and AI-driven discovery — and its customer relationships are designed to compound in value as more communities and individuals join, creating a natural network effect. Its cost structure remains deliberately lean at the current beta stage, while its revenue streams are structured to activate only once that network effect has produced a user base large enough to justify monetization, consistent with the sequencing rationale established in Section 7.4. This alignment between what the platform is built to do, who it serves, and how it plans to sustain itself forms the proposed business plan carried into the remainder of this chapter, where it is connected directly back to the Final Year Project itself.

7.8 Integration with Final Year Project

7.8.1 Project Alignment

EventBridge AI's business model is not a theoretical exercise layered onto an academic project after the fact; it is a direct extension of decisions already embedded in the system's architecture and scope. Every element of the business plan traces back to a corresponding technical or design choice made during development:

- The community-native architecture described as a Key Resource in Section 7.7 is the same architecture defined at the database and application level in Chapter 4, where communities exist as persistent entities with profiles, team structures, and event histories, rather than being modeled as incidental metadata on individual event records.
- The AI-driven discovery that anchors the Unique Value Proposition (Section 7.2) is the semantic search and recommendation pipeline built using OpenAI embeddings and

Qdrant, evaluated for performance in Chapter 6.

- The freemium and tiered monetization logic (Section 7.4) is designed around functionality that already exists in the deployed system — event publishing, discovery, and basic recommendations are fully implemented and free by default, meaning no monetization pathway described in this chapter depends on features that do not yet exist.
- The market segments and personas identified in Section 7.3 are drawn directly from the real actor roles — Communities, Individuals, and Administrators — formally defined in the system’s requirements and reflected in its role-based access control.

This alignment demonstrates that the Final Year Project was not designed purely to satisfy academic evaluation criteria, but with a working awareness of how the resulting system would need to function as a real product from its first deployment.

7.8.2 Implementation Details

Translating the business model’s requirements into a working system surfaced several technical challenges that had direct commercial implications, requiring solutions that served both engineering and business goals simultaneously.

- Independent scalability of the AI layer. Because the business model treats AI-driven discovery as the platform’s core differentiator, the AI engine could not be allowed to become a bottleneck as adoption grows. This was addressed by separating the Django AI engine from the Laravel application entirely, hosting it independently on an OVH Cloud VPS with Qdrant running in Docker, so that search and recommendation capacity can be scaled without touching the core platform, directly supporting the scalability plan discussed in Section 7.5.
- Low-cost operation at beta scale. The Cost Structure block identifies infrastructure and API costs as scaling with usage rather than being fixed upfront. This was made possible by architectural choices such as using text-embedding-3-large selectively and caching-friendly query patterns, keeping OpenAI API costs proportional to actual search and indexing volume rather than constant background load.
- Trust and credibility without a dedicated moderation team. The Customer Relationships block relies on community verification to establish trust, which was implemented through an admin approval workflow for new community registrations, giving the platform a lightweight but functioning credibility mechanism without requiring paid moderation staff at this stage.
- Cross-community collaboration as a technical and business feature simultaneously. The inter-community collaboration tooling — a request/accept workflow allowing two communities to co-manage a shared event — was one of the more complex implementation challenges in Chapter 4, precisely because it is also one of the platform’s strongest business differentiators; no competitor evaluated in Section 7.1 offers equivalent functionality.

Each of these solutions reflects the same underlying principle: technical decisions during development were made with an awareness of their downstream business consequences, not purely their engineering elegance.

7.8.3 Market Experience:

EventBridge AI's business model is also informed by real-world testing rather than assumption alone. Following deployment, a structured beta outreach effort was conducted directly with communities from the platform's core target segments — GDGoC, IEEE, and ACM chapters — using a two-track approach: community leads were asked to register their organizations, build out community profiles and team structures, and upload both past and upcoming events; general members were separately asked to test the platform's core AI features, specifically semantic search and personalized recommendations, using natural-language example queries to guide their interaction.

Feedback collection was deliberately built into the platform itself rather than relying on an external form, through an in-platform feedback mechanism accessible to users during normal use, with responses surfaced directly in the admin panel. This decision reflected a market-facing insight gained during beta planning: external feedback forms depress response rates relative to feedback prompts embedded in the product experience itself, a consideration that will continue to inform how user research is conducted as the platform scales beyond its current beta group.

This early market engagement validated two assumptions central to the business model: that communities are willing to migrate event management activity away from informal channels when given a structured alternative, and that individuals engage meaningfully with AI-driven search and recommendations when the queries feel natural rather than keyword-constrained. These findings directly reinforce the target audience definitions in Section 7.3 and the value proposition claims in Section 7.2, grounding the business plan in observed behavior rather than assumption alone.

7.9 Future Road map

7.9.1 Future Development

Building on the foundation already deployed and validated during beta testing, EventBridge AI's future development roadmap is organized into three horizons: near-term feature additions, technical platform improvements, and longer-term scale and reach.

Feature Additions

- A digital event check-in system with auto-generated attendance certificates and passes, replacing the manual attendance marking that organizers currently rely on.
- AI-powered sentiment analysis on post-event feedback, giving communities actionable insight beyond raw numerical ratings.
- Integrated payment gateway support (e.g., JazzCash, EasyPaiza, Stripe) for paid events, moving away from manual payment-proof uploads toward verified, automated transactions — a prerequisite for the ticketing commission revenue stream identified in Section 7.4.
- An AI chatbot assistant for event discovery and platform navigation, extending the platform's conversational AI capability beyond search into general user support.

Technical Improvements

- Responsive web design and, eventually, dedicated mobile applications, addressing the current beta limitation of desktop-and-laptop-only access.
- An advanced community analytics dashboard providing follower growth trends, event performance heatmaps, and AI-generated engagement insights, directly supporting the premium organizer subscription tier proposed in the monetization strategy.
- Continuous refinement of the semantic search and recommendation models as real usage data accumulates, improving relevance and personalization accuracy over time without requiring architectural changes.

Scale and Reach

- Multi-city expansion within Pakistan, extending the community-first onboarding model validated with GDGoC, IEEE, and ACM chapters to other cities' community ecosystems.
- Longer-term international expansion, opening the platform to community ecosystems facing the same event-fragmentation problem outside Pakistan, consistent with the internationalization outlook discussed in Section 7.5.

These priorities were deliberately sequenced so that feature additions and technical improvements — which strengthen the core product and unlock revenue streams — precede geographic scaling, ensuring the platform is deepened before it is widened.

7.9.2 Adaptability

EventBridge AI's business model is designed to adapt to changing market conditions along three dimensions rather than remaining fixed to its current beta-stage assumptions.

- **Modular monetization.** Because revenue streams are activated progressively rather than all at once, the platform can adjust which mechanisms it emphasizes based on actual adoption patterns. If institutional licensing proves more viable than individual premium subscriptions, for instance, the business model can shift emphasis without requiring changes to the underlying platform.
- **Architectural flexibility.** The independent scalability of the Laravel and Django/Quadrant layers, discussed in Section 7.5, means the platform can absorb shifts in where demand concentrates — whether that is a surge in AI-driven search usage, growth in community management activity, or new feature areas entirely — without a system-wide redesign.
- **Community-responsive roadmap.** Because the platform's core differentiators (community-native architecture, semantic discovery) were shaped directly by feedback from real community leads and members during beta testing (Section 7.8.3), the roadmap is structured to continue incorporating market feedback as a primary input, rather than being fixed in advance. This keeps the business model responsive to what communities and individuals actually need as the platform grows, rather than committing early to assumptions that may not hold at scale.

Taken together, this adaptability ensures that EventBridge AI's business model can evolve in step with its user base and market conditions, rather than requiring the platform to conform to a roadmap defined before real-world usage data existed.

7.10 Financial Projections

The financial projections presented in this section are illustrative and assumption-based, consistent with the platform’s current beta stage where real transactional revenue has not yet been generated. They are constructed using the platform’s actual known cost baseline (as established during product costing) combined with reasonable adoption assumptions drawn from the community ecosystem sizes discussed in Section 7.1, and are intended to demonstrate the financial viability of the business model rather than to represent audited or contractually committed figures.

7.10.1 Cost Baseline

Two cost categories anchor these projections: the sunk development investment already incurred, and the ongoing operational costs required to run the platform.

Category	Basis	Estimated Cost (USD)
Development effort	630 hours (7 months $\times$ 30 working days $\times$ 3 hrs/day) across five role categories at $\sim$ \$30/hr	$\sim$ \$18,980
IP filing (copyright + trademark)	Two copyright filings + one trademark filing, IPO-Pakistan	$\sim$ \$150 (estimated)
Total sunk investment		$\sim$\$19,130

Table 7.5: Development investment (sunk cost)

Category	Estimated Monthly Cost (USD)
GoDaddy hosting (Laravel)	$\sim$ \$15
OVH Cloud VPS (Django AI engine + Qdrant)	$\sim$ \$40
OpenAI API usage (embedding generation, gpt-4o-mini re-ranking)	$\sim$ \$25
Total monthly operating cost	$\sim$\$80

Table 7.6: Monthly operating costs at current beta scale

At current beta usage volume (roughly 500 indexed events, 50 active user profiles), operating costs remain low, consistent with the "currently low, built to scale" positioning established in the Cost Structure block of Section 7.7. These costs are expected to scale primarily with search and embedding volume rather than with the number of registered communities, since embedding generation is the dominant variable cost driver.

7.10.2 Revenue Projections

Revenue projections assume a phased activation of the monetization strategy defined in Section 7.4, beginning after the platform exits beta and establishes a stable, engaged community base. The following three-year outlook illustrates a conservative adoption curve.

Metric	Year 1 (Beta → Early Adoption)	Year 2 (Growth)	Year 3 (Scale)
Active communities	15–20	50	120
Active individual users	500	3,000	10,000
Premium organizer subscriptions (@ ~\$10/community/month)	Not yet activated	10 communities → ~\$1,200/yr	35 communities → ~\$4,200/yr
Verified badge fees (@ ~\$15/community/yr, one-time annual)	Not yet activated	20 communities → ~\$300/yr	60 communities → ~\$900/yr
Featured event promotions (@ ~\$5/event, ad hoc)	Not yet activated	~\$600/yr	~\$2,400/yr
Institutional licensing	Not applicable	Not yet activated	1 pilot institution → ~\$1,500/yr
Estimated annual revenue	\$0	~\$2,100	~\$9,000

Table 7.7: Three-year revenue projection (illustrative)

7.10.3 Break-Even Analysis and ROI

Given the sunk development investment of approximately \$19,130 and monthly operating costs of approximately \$80 (scaling gradually as adoption grows), the platform is not projected to reach cumulative break-even against its full development cost within the three-year window modeled above — a realistic outcome for a community platform whose revenue model is intentionally activated only after organic adoption is established, rather than optimized for early monetization.

However, evaluated on operating break-even rather than full investment recovery, the picture is materially healthier: at the Year 2 revenue level (~\$2,100/yr, or ~\$175/month) against operating costs of roughly \$80–120/month at that scale, the platform is projected to reach positive operating cash flow during Year 2, meaning ongoing operations become self-sustaining well before the initial development investment is recovered. This is the more meaningful milestone for a beta-stage platform of this kind, since the development cost is a one-time academic and engineering investment rather than a recurring capital expenditure.

Return on investment, calculated against the full sunk cost, remains modest within the three-year horizon (cumulative revenue of \$11,100 against \$19,130 invested, plus ongoing operating costs), which is consistent with typical early-stage platform economics where network effects and institutional partnerships — not individual subscriptions — become the dominant revenue drivers only once meaningful scale is reached. The financial case for EventBridge AI therefore rests less on near-term profitability and more on the low, usage-proportional cost structure established in Section 7.10.1, which allows the platform to sustain itself operationally at low adoption levels while remaining positioned to scale revenue meaningfully once the institutional licensing and multi-city expansion pathways discussed in Sections 7.5 and 7.9 materialize.

7.11 Legal Considerations

7.11.1 Intellectual Property

EventBridge AI’s intellectual property protection strategy reflects the specific legal realities of software protection in Pakistan, and has been approached deliberately rather than left unaddressed.

- Patents are not applicable. Under Pakistan’s Patents Ordinance, 2000, software, algorithms, and business methods are explicitly excluded from patentability. This rules out patent protection for the platform’s semantic search pipeline or recommendation algorithm as a legal option, regardless of their technical originality, and shapes the rest of the IP strategy toward the protections that are actually available under Pakistani law.
- Copyright registration is being pursued as the primary form of legal protection, covering two distinct filings: the platform’s codebase, and its original written content (including documentation and platform-authored material). Both filings are to be completed through the Copyright Office at IPO-Pakistan, Gulshan-e-Iqbal, Karachi, and remain pending finalization of the required personal filing details (CNIC numbers and addresses) for the registering parties.
- Trademark registration is being pursued for the “EventBridge AI” name and logo, also through IPO-Pakistan’s Trade Marks Registry, to protect the platform’s brand identity established in Section 7.6 from unauthorized use as the product gains visibility.
- Trade secret protection is applied to the components of the system that copyright and trademark registration do not cover as effectively — specifically, the recommendation algorithm’s design and the embedding pipeline logic have been deliberately kept out of public-facing documentation, and both GitHub repositories (Laravel and Django) are maintained as private repositories, limiting exposure of the platform’s core technical differentiators.
- Internal ownership agreement. An internal agreement between the project’s core contributors is already in place, establishing clarity on ownership and rights over the platform ahead of any formal registration, which reduces the risk of future ownership disputes as the project moves from academic deliverable toward a operating product.

Together, these measures reflect a layered approach: relying on registered rights (copyright, trademark) where the law provides them, and operational secrecy (private repositories, undocumented internals) where it does not, given that patent protection is foreclosed by statute.

7.11.2 Legal Considerations for International Markets

Should EventBridge AI pursue the multi-city or international expansion discussed in Sections 7.5 and 7.9, several additional legal considerations would apply beyond its current Pakistan-only scope:

- Jurisdiction-specific IP registration. Trademark and copyright protections secured through IPO-Pakistan are generally territorial; expansion into new countries would require separate trademark filings in each target jurisdiction, or pursuit of international mechanisms such as the Madrid System for trademark protection, to maintain brand protection outside Pakistan.

- Data protection and privacy compliance. International expansion would introduce exposure to region-specific data protection regimes (such as the GDPR in the European Union), which impose requirements around user consent, data storage location, and the right to erasure that are more stringent than Pakistan’s current regulatory environment. This would need to be assessed before onboarding users or communities from any such jurisdiction.
- Platform terms of service and liability. As the user base grows beyond a single beta-testing community, formalizing terms of service, a privacy policy, and clear liability boundaries (particularly around user-generated event content and inter-community collaboration) becomes necessary to limit the platform’s legal exposure, a step not yet required at the current beta scale but anticipated as part of the platform’s maturation.

These considerations are noted here as forward-looking requirements rather than immediate obligations, consistent with the platform’s current geographic scope, and would be revisited concretely at the point any expansion beyond Karachi is actively pursued.

7.12 Automation Testing

While the full testing methodology and detailed test coverage of EventBridge AI is addressed in Chapter 5, this section considers automated testing specifically through a business and quality-assurance lens — as a factor directly affecting the platform’s commercial viability, credibility with institutional partners, and reliability of the revenue-generating features proposed earlier in this chapter.

7.12.1 Current Testing Infrastructure

The Laravel application is built with PHPUnit as its test runner, supported by Mockery for mock objects and FakerPHP for generating realistic test data, all included as development dependencies in the project’s environment. Development follows a main/dev/feature-branch Git workflow, in which feature branches are merged into dev for integration before being promoted to main for deployment, providing a natural checkpoint at which functional issues can be caught before reaching the live platform. At present, test execution occurs manually at these checkpoints rather than through a fully automated continuous integration (CI) pipeline triggered on every commit.

7.12.2 Business Rationale for Automation

From a business standpoint, automated testing is not merely an engineering best practice but a direct enabler of several commitments made elsewhere in this chapter:

- Revenue reliability. The premium subscription and institutional licensing streams proposed in Section 7.4 depend on the platform maintaining consistent uptime and correct behavior for paying organizational clients. Manual, checkpoint-based testing carries a higher risk of regressions reaching production than an automated pipeline that runs the full test suite on every change, a risk that becomes less acceptable once paying clients are involved.

- Institutional trust. Universities or organizations considering the institutional licensing pathway (Section 7.4, Section 7.7) are more likely to trust a platform that can demonstrate a disciplined, automated quality-assurance process, particularly given that such clients will be evaluating the platform’s operational maturity, not only its feature set.
- Reduced operating cost at scale. As identified in the Cost Structure block (Section 7.7) and the financial projections (Section 7.10), development and maintenance effort is a recurring cost. Automated testing reduces the time engineers spend manually verifying existing functionality after each change, lowering the ongoing cost of maintaining the platform as its codebase and feature set grow.
- Safer scaling of the AI pipeline. Because the recommendation engine and semantic search pipeline are continuously refined as real usage data accumulates (Section 7.9.1), automated regression testing is particularly important for this layer specifically, to ensure that model or pipeline updates do not silently degrade search relevance or recommendation quality for existing users.

7.12.3 Planned Automation Roadmap

Formal continuous integration and continuous deployment (CI/CD) automation — such as running the existing PHPUnit suite automatically via GitHub Actions on every push or pull request, gating merges to main on passing tests — is identified as a near-term technical improvement, extending the future development roadmap outlined in Section 7.9.1. This would be introduced as a natural next step once test coverage across both the Laravel application and the Django AI engine is sufficiently comprehensive, ensuring that automation reinforces confidence in the pipeline rather than automating an incomplete one. Establishing this pipeline before scaling institutional partnerships or premium offerings is treated as a prerequisite to commercial expansion, not an optional enhancement, given how directly it supports the reliability commitments underlying the platform’s monetization strategy.

Chapter 8

Future Directions and Conclusion

8.1 Introduction

This chapter closes the technical report by drawing together what EventBridge AI achieved as a system, what remains unresolved or unexplored, and where the platform could reasonably go from here. Consistent with the scope of a concluding chapter, the discussion here is deliberately concise: it does not restate the detailed evaluation already presented in Chapter 6, nor the design rationale covered in Chapters 3 and 4, but instead reflects on the project as a whole — what was learned in building it, what questions remain open, and what future work would meaningfully extend it.

8.2 Summary of Key Accomplishments and Learnings

EventBridge AI set out to solve a concrete, lived problem: community-led events in Karachi’s tech and academic ecosystem were fragmented across informal channels, leaving communities under-visible and individuals unaware of opportunities relevant to them. Over the course of this project, that problem was addressed with a fully deployed, two-server platform — a Laravel web application and a Django-based AI engine — that brings community management, event discovery, and AI-powered personalization together under a single system.

The most significant technical accomplishment was building and integrating a working semantic search and recommendation pipeline using OpenAI embeddings and a Qdrant vector database, evaluated in Chapter 6 and shown to perform within acceptable bounds for the platform’s current scale. Equally significant was the architectural decision to separate business logic from AI computation across two independently scalable services, a decision that shaped nearly every implementation challenge documented in Chapter 4 and proved central to the platform’s viability as a real, growable product rather than a static academic prototype.

Beyond the system itself, several practical lessons emerged during development that extend beyond any single chapter:

- Precision in claims matters as much as precision in code. Several early assumptions from the original proposal and SRS — such as the recommendation engine being “hybrid,” or LangChain playing a central architectural role — did not hold once the system was actually built, and the report itself had to be corrected to reflect what was truly implemented

rather than what was originally planned. This reinforced a broader lesson: a system's actual behavior, not its intended design, is what an evaluation must be honest about.

- Cross-service architecture introduces coordination costs that single-service systems do not have. Keeping the Laravel and Django layers synchronized, particularly around indexing and recommendation freshness, surfaced reliability considerations (addressed through the observer-dispatched queue job sync mechanism) that would not have existed in a monolithic design, and that shaped how the team thought about resilience throughout.
- A platform's business viability and its technical architecture are inseparable. Working through the business model in Chapter 7 revealed how directly technical choices — independent scalability, low marginal cost per query, modular monetizable features — determine whether a system can realistically sustain itself beyond an academic deadline, a perspective that came into focus only once the platform was evaluated as a product rather than solely as a deliverable.
- Real user engagement surfaces gaps that internal testing cannot. Beta outreach to actual communities (GDGoC, IEEE, ACM chapters) validated core assumptions about demand and usability, but also revealed friction points — such as onboarding effort for community leads — that were not visible during isolated development and testing.

Collectively, these accomplishments and lessons reflect a project that moved beyond building a functioning system to understanding what that system needs to become reliable, sustainable, and genuinely useful to the people it was built for.

8.3 Unaddressed Problems and Limitations

Not every problem surfaced during development was resolved within the scope of this project, and several limitations were consciously deferred rather than solved. These fall into four broad categories.

8.3.1 Architectural and Operational Limitations

- No automated CI/CD pipeline currently gates deployments (discussed in Section 7.12); testing occurs at manual checkpoints in the Git workflow rather than on every commit, leaving a window for regressions to reach production undetected.
- The platform has no formal disaster recovery or backup verification strategy. Database backups exist as an administrative capability, but there is no tested restoration procedure or defined recovery time objective should the OVH VPS or GoDaddy hosting experience a failure.
- Both the Laravel application and the Django AI engine currently run as single instances with no redundancy; an outage on either server represents a single point of failure with no automatic failover.
- No formal load testing at true production scale has been conducted. Chapter 6's performance evaluation reflects representative conditions at the current beta scale, not stress-tested behavior under significantly higher concurrent usage.

8.3.2 AI and Data Limitations

- As noted in Chapter 6, the platform has no caching layer for frequently repeated search queries and no fallback recommendation strategy for entirely new users with no interaction history, both of which affect responsiveness and personalization quality at the edges of typical usage.
- The system has an equivalent but previously undocumented cold-start problem on the community side: a newly onboarded community with no historical events has limited signal for the recommendation engine to work with, and no mitigation strategy for this case currently exists.
- The platform’s semantic search and recommendation quality are entirely dependent on OpenAI’s embedding models, creating a vendor dependency with no fallback embedding provider or offline degradation path if the API becomes unavailable or its pricing model changes materially.
- There is no versioning or migration strategy for embedding model upgrades; adopting a newer or different embedding model in the future would require re-embedding the entire event and community corpus, a cost and process not yet planned for.
- Recommendation results are not explainable to end users; individuals are shown personalized results but not the reasoning behind them, which limits both user trust and the team’s own ability to debug perceived recommendation quality issues.

8.3.3 Security, Privacy, and Compliance

- No formal, independent security audit or penetration test has been conducted on either the Laravel application or the Django AI engine, leaving unknown vulnerabilities that structured internal development and testing may not surface.
- Data protection compliance has not been formally assessed against any specific regulatory framework. This is a reasonable gap given the platform’s current Karachi-only scope, but it was flagged in Section 7.11 as a requirement that would need direct attention before any international expansion.
- Content moderation currently relies entirely on manual admin review with no automated abuse, spam, or fake-community detection, which may not scale proportionally as the platform grows beyond its current beta community base.

8.3.4 Business and Adoption Limitations

- The financial projections in Section 7.10 are illustrative rather than based on observed transactional data, since no monetization mechanism has been activated to date; actual revenue behavior once premium features go live remains unvalidated.
- No formal tracking or attribution exists to measure which marketing channels (Section 7.6) actually drive community or individual adoption, meaning channel effectiveness assumptions have not been tested against real acquisition data.

These limitations do not undermine the platform’s core viability, demonstrated through both its technical evaluation in Chapter 6 and its real-world beta engagement in Section 7.8.3, but they represent the honest boundary between what has been built and validated, and what remains to be addressed as the platform matures.

8.4 Open Questions

Beyond the concrete limitations identified in Section 8.3, several open questions remain that the current project scope did not attempt to answer, either because they require longer observation periods, larger user populations, or decisions that fall outside a Final Year Project’s boundaries.

- Does recommendation quality genuinely improve as usage data accumulates, or does it plateau? The content-based nearest-neighbor approach used in this system is expected to improve with more interaction data, but this has not been observed over a long enough period to confirm whether relevance gains continue meaningfully or level off after an initial improvement phase.
- At what community and event volume does the current two-server architecture stop scaling gracefully? The architecture was designed with independent scalability in mind (Section 7.5), but the actual threshold at which Qdrant query latency, MySQL relational load, or Django worker capacity becomes a bottleneck has not been empirically determined, since current usage remains well within the platform’s beta-scale test conditions.
- Would a hybrid recommendation approach (combining collaborative and content-based filtering) meaningfully outperform the current content-based-only method, given this platform’s specific data characteristics? The original proposal envisioned something closer to a hybrid system, but the report deliberately corrected this claim to reflect what was actually built. Whether the added complexity of a true hybrid approach would be justified by a measurable accuracy gain, rather than assumed, remains unanswered.
- How would users and communities respond to the monetization mechanisms proposed in Chapter 7 once actually activated? Willingness to pay for premium organizer features, verified badges, or institutional licensing was never tested with real transactions, and community sentiment toward a previously free platform introducing paid tiers is inherently uncertain until observed directly.
- Is Karachi’s community ecosystem representative enough to generalize the platform’s design to other cities or countries without significant adaptation? The internationalization outlook in Section 7.5 assumes the underlying event-fragmentation problem is not unique to Karachi, but this assumption has not been tested against community behavior in a genuinely different market.
- What is the actual retention behavior of individuals and communities over a multi-month period? Beta engagement (Section 7.8.3) validated initial adoption and usability, but sustained, longer-term retention — whether communities keep the platform updated and whether individuals keep returning after the novelty of AI-powered discovery wears off — has not yet been observed.

These questions are left open deliberately rather than answered speculatively, consistent with the report’s broader commitment to distinguishing what has been demonstrated from what remains assumed.

8.5 Directions for Future Research and Development

Building on the limitations and open questions identified above, several concrete techniques, algorithms, and tools present promising directions for extending EventBridge AI beyond its current implementation.

8.5.1 Recommendation and Search Improvements

- **Hybrid recommendation modeling.** Introducing a collaborative filtering component alongside the current content-based nearest-neighbor approach — for example, matrix factorization or a lightweight neural collaborative filtering model — could address the open question in Section 8.4 about whether recommendation quality can meaningfully improve beyond content-based similarity alone, particularly as more user interaction data accumulates.
- **Re-ranking with explainability.** Extending the existing gpt-4o-mini re-ranking step to generate brief, user-facing justifications for recommendations (e.g., "recommended because you followed similar communities") would directly address the explainability gap noted in Section 8.3, improving user trust without requiring a new model architecture.
- **Embedding caching and incremental re-indexing.** A caching layer for frequent or near-duplicate queries, combined with a more granular incremental re-indexing strategy (rather than full weekly re-indexing), would reduce both API cost and latency, addressing the caching limitation identified in Chapter 6.
- **Cold-start mitigation.** Rule-based or popularity-weighted fallback strategies for new users and newly onboarded communities with no interaction history would close the cold-start gaps identified in Section 8.3, without requiring a fundamentally different recommendation architecture.

8.5.2 Platform Reliability and Operations

- **CI/CD automation.** Implementing GitHub Actions (or an equivalent) to run the existing PHPUnit suite automatically on every commit, as outlined in Section 7.12, would be a natural and relatively low-effort next step toward reducing regression risk ahead of any commercial scaling.
- **Load testing frameworks.** Tools such as Apache JMeter or k6 could be used to empirically determine the scaling thresholds referenced in Section 8.4, replacing current assumptions about architectural scalability with measured data.
- **Observability tooling.** Structured logging and monitoring (e.g., Prometheus/Grafana, or a lighter-weight equivalent suited to the current VPS-based deployment) would give the team visibility into real bottlenecks before they affect users, rather than relying on manual detection.

8.5.3 Security and Data Governance

- **Independent security review.** A structured penetration test or vulnerability assessment, even a lightweight one, would begin closing the audit gap identified in Section 8.3 before the platform takes on paying institutional clients.

- Automated content moderation. Lightweight spam or abuse detection (rule-based initially, potentially ML-based later) would reduce reliance on manual admin review as community volume grows.

8.5.4 Business and Adoption Research

- Controlled monetization pilots. Rather than activating all revenue streams from Section 7.4 simultaneously, a phased pilot with a small group of communities would generate real willingness-to-pay data, directly answering the open question in Section 8.4 about monetization response.
- Longitudinal retention tracking. Instrumenting the platform to track community and individual engagement over a multi-month horizon, beyond the current beta window, would provide the retention data needed to evaluate long-term product-market fit rather than initial adoption alone.

Taken together, these directions do not represent a redesign of EventBridge AI, but a set of targeted, achievable extensions to what has already been built and validated — each one addressing a specific, named gap rather than a speculative reinvention of the platform.

8.6 Concluding Remarks

EventBridge AI began as a response to a problem that was easy to observe and hard to ignore: valuable community events across Karachi’s tech and academic ecosystem were going unseen, not for lack of effort from organizers, but for lack of a system built to bring them together. This report has documented the full journey from that observation to a deployed, functioning platform — its design and architecture, its development challenges, its measured performance, its business viability, and now, in this final chapter, an honest account of what remains unresolved.

What this project demonstrates is not a finished, unchanging product, but a working foundation: a two-server architecture proven capable of independent scaling, an AI-driven discovery pipeline validated through real evaluation, and a business model grounded in actual usage rather than assumption. Equally important is what this report has chosen not to overstate. Where early claims outpaced what was actually built, they were corrected. Where performance data was representative rather than measured, that was disclosed rather than obscured. Where limitations exist, they were named specifically rather than gestured at generally. That discipline, maintained consistently from Chapter 1 through this closing chapter, is itself one of the project’s outcomes.

The problems, questions, and future directions identified in this chapter are not signs of an incomplete project, but the expected and necessary boundary of any system built to be extended rather than declared finished. EventBridge AI’s foundation is built, tested, and understood well enough to know exactly where it should go next — and that clarity, as much as the platform itself, is the result of this Final Year Project.

Appendix A - User Manual

8.7 Introduction and Purpose

This appendix provides a practical guide to using EventBridge AI, intended to help new Individuals, Communities, and Administrators navigate the platform's core features. It complements the technical documentation presented in the preceding chapters by focusing on the end-user experience rather than internal implementation details. The instructions in this appendix reflect the actual functionality available on the deployed beta platform at eventbridgeai.com.

8.8 Accessing the Platform

EventBridge AI is a web-based platform accessible through any modern web browser (Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari) at eventbridgeai.com. No installation or additional software is required. The platform supports three distinct account types, each with a dedicated authentication flow:

- Individual — for people who wish to discover, follow, and register for events.
- Community — for organizations such as GDGoC, IEEE, or ACM chapters that wish to host and manage events.
- Administrator — a platform-governance role responsible for approving Communities and monitoring overall activity.

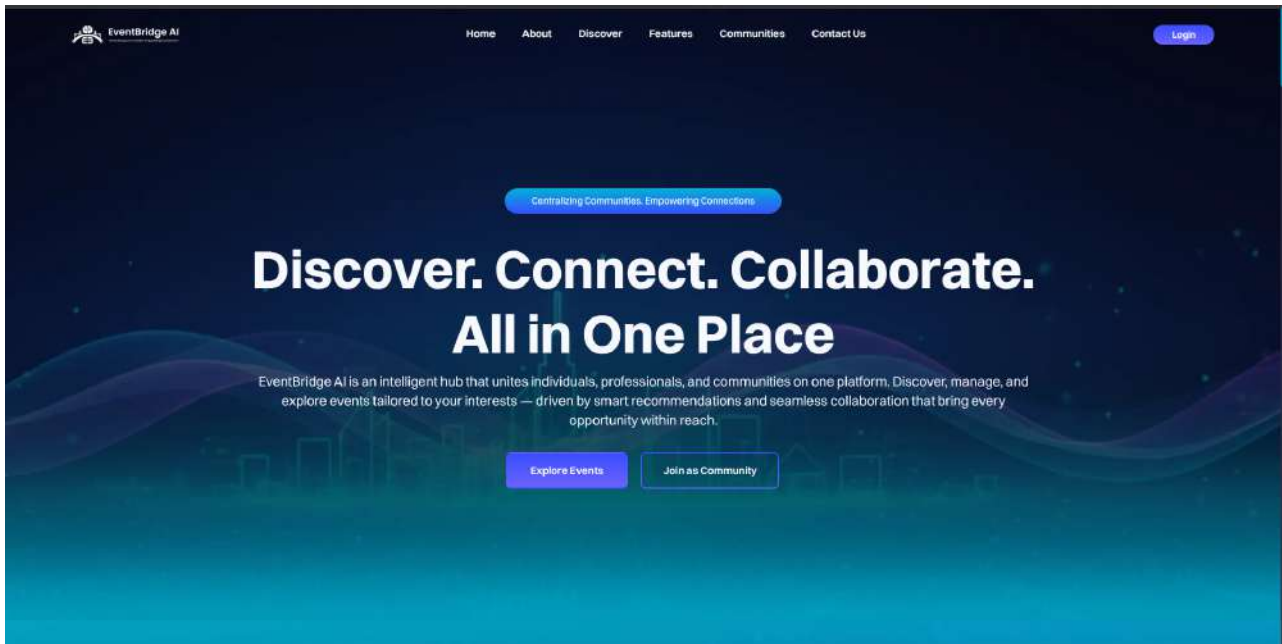


Figure 8.1: EventBridge AI Landing Page

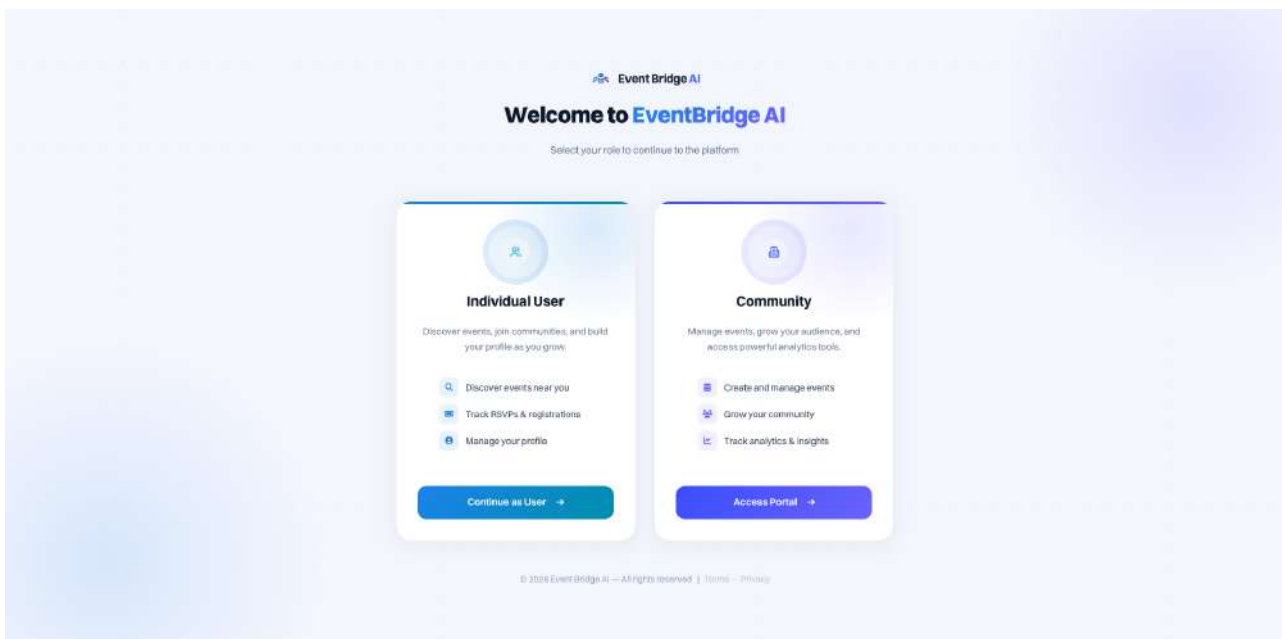


Figure 8.2: Pathway Selection Screen

8.9 Registration and Login

8.9.1 Individual Registration and Login

- From the landing page, select Sign Up and choose the Individual account type.
- Enter the required details (name, email address, and password) and submit the form.

- Upon first login, the Individual is guided through a short onboarding wizard to capture domain interests, preferred event types, and skills. This information is later used by the platform's recommendation engine to personalize event suggestions.
- Once onboarding is complete, the Individual gains full access to the Explore, Registration, and Profile features described in Section A.4.

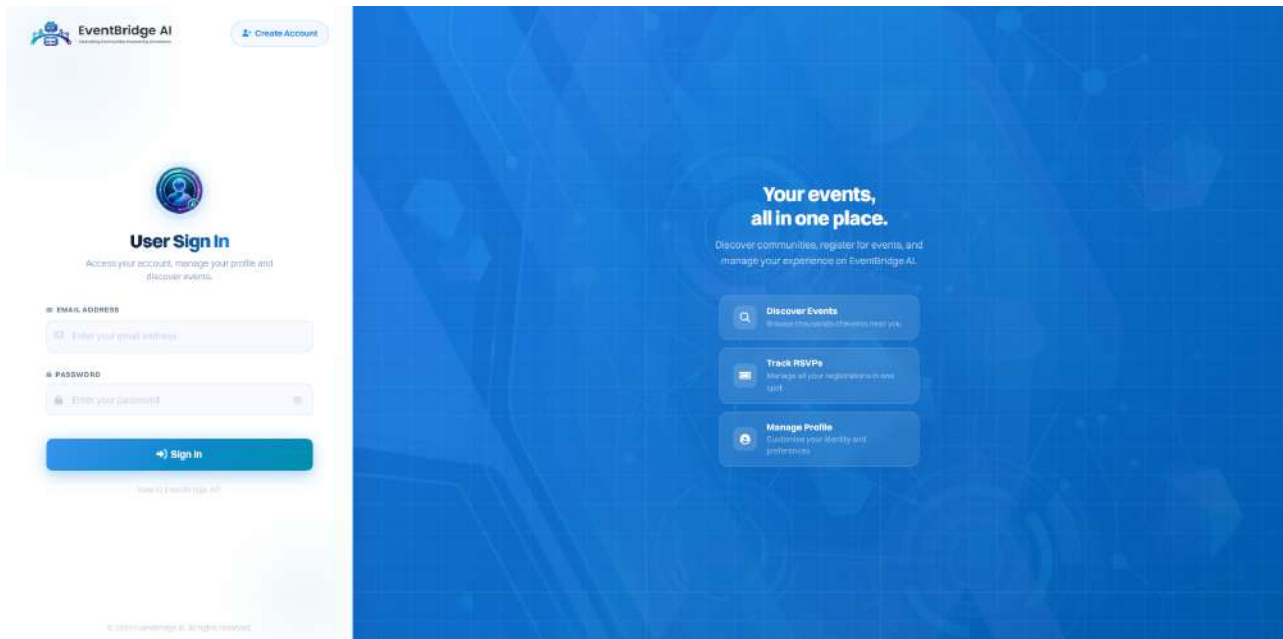


Figure 8.3: Individual User Login Screen

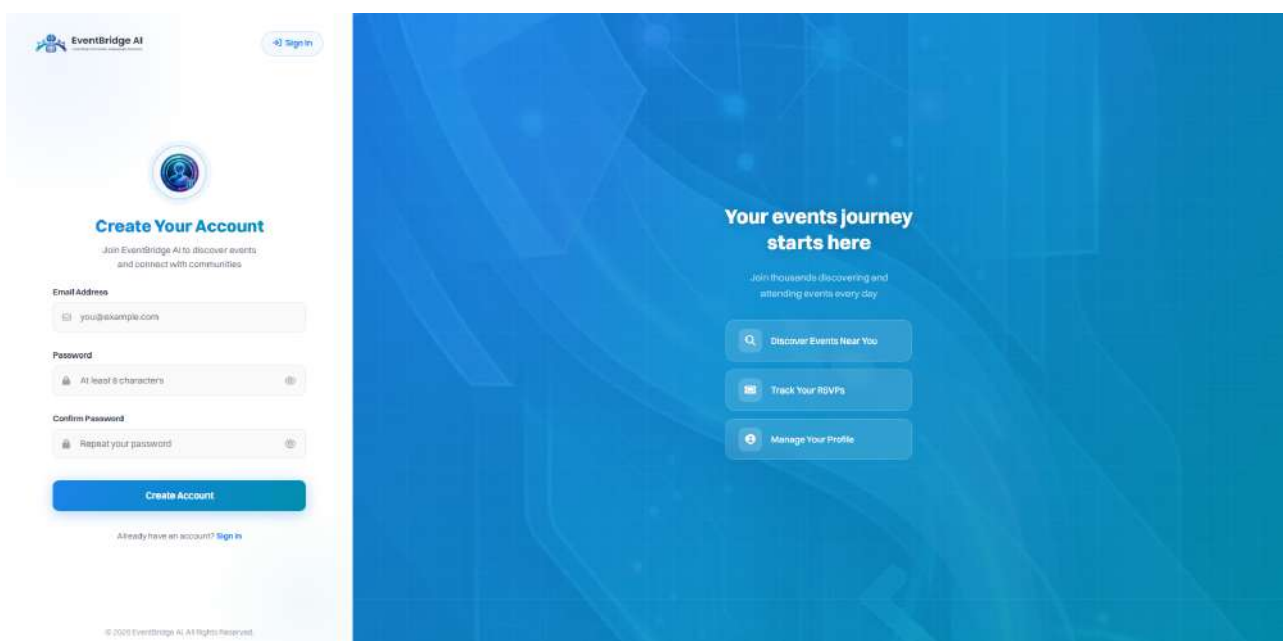


Figure 8.4: Individual User Account Registration Screen

8.9.2 Community Registration and Admin Approval

- From the landing page, select Sign Up and choose the Community account type.
- Complete the Community registration form, providing the Community name, category, and contact details.
- Upon submission, the account is placed into a Pending Verification state. A confirmation message informs the Community that the account is awaiting Administrator review.
- If the Community attempts to log in before approval, the system redirects to a status page displaying the current approval state (Pending, Approved, Rejected, or On Hold).
- Once an Administrator approves the account (Section A.6.1), the Community receives a notification and gains access to its Dashboard.

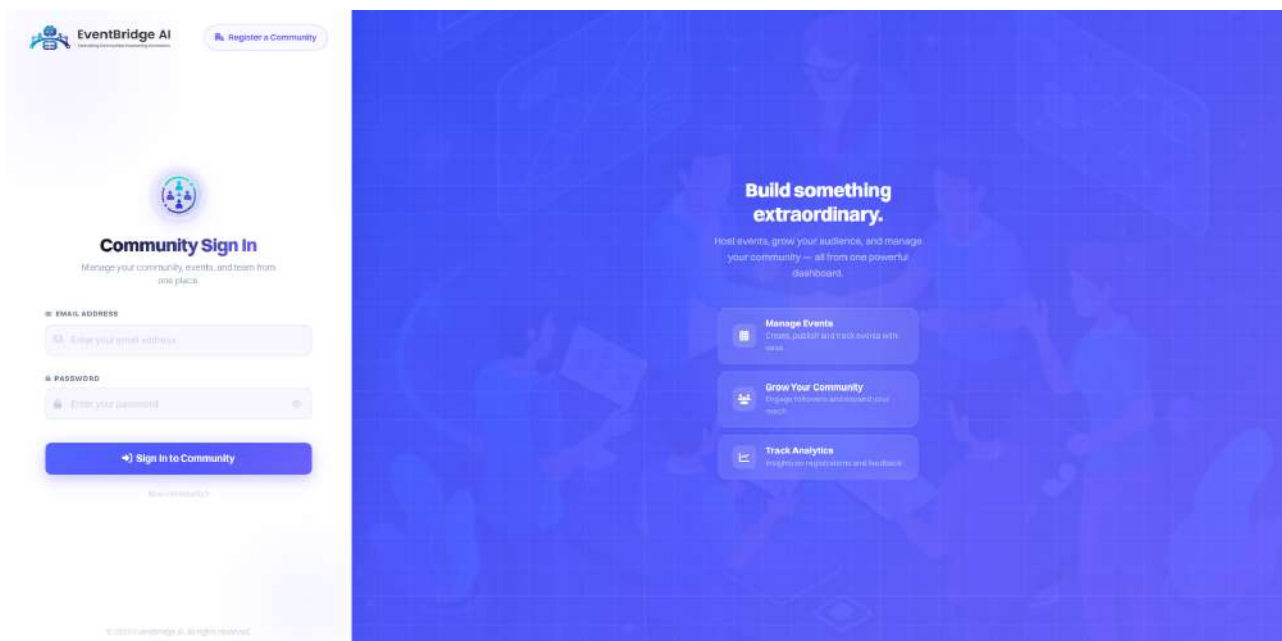
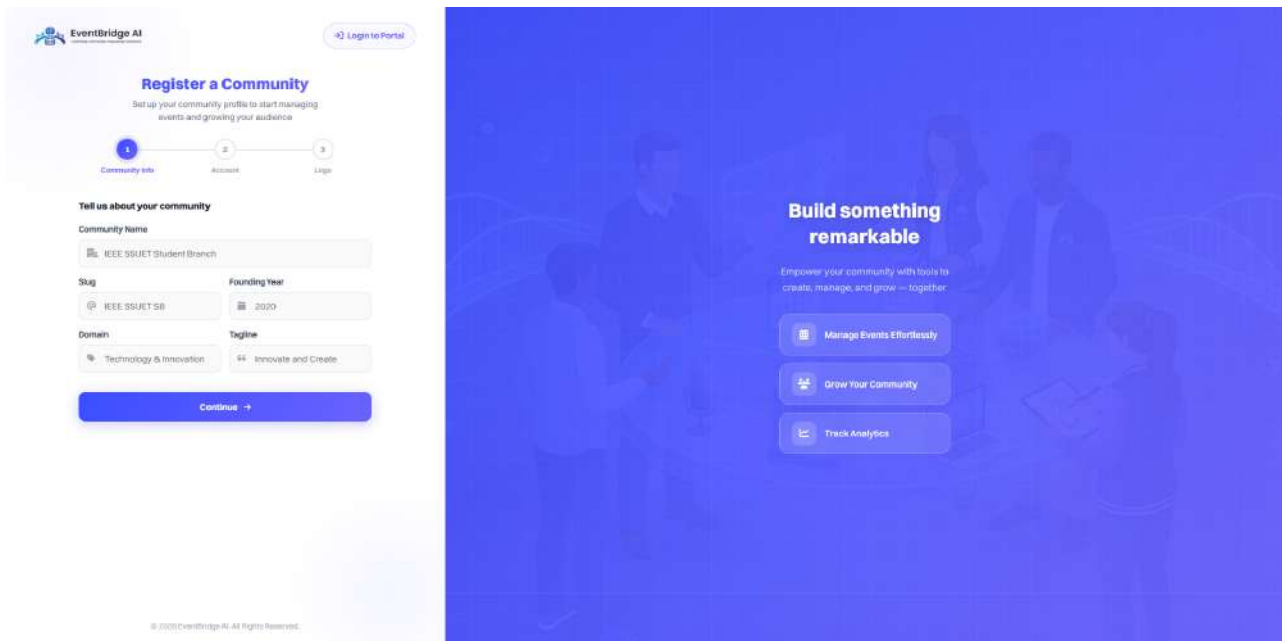


Figure 8.5: Community Login Screen

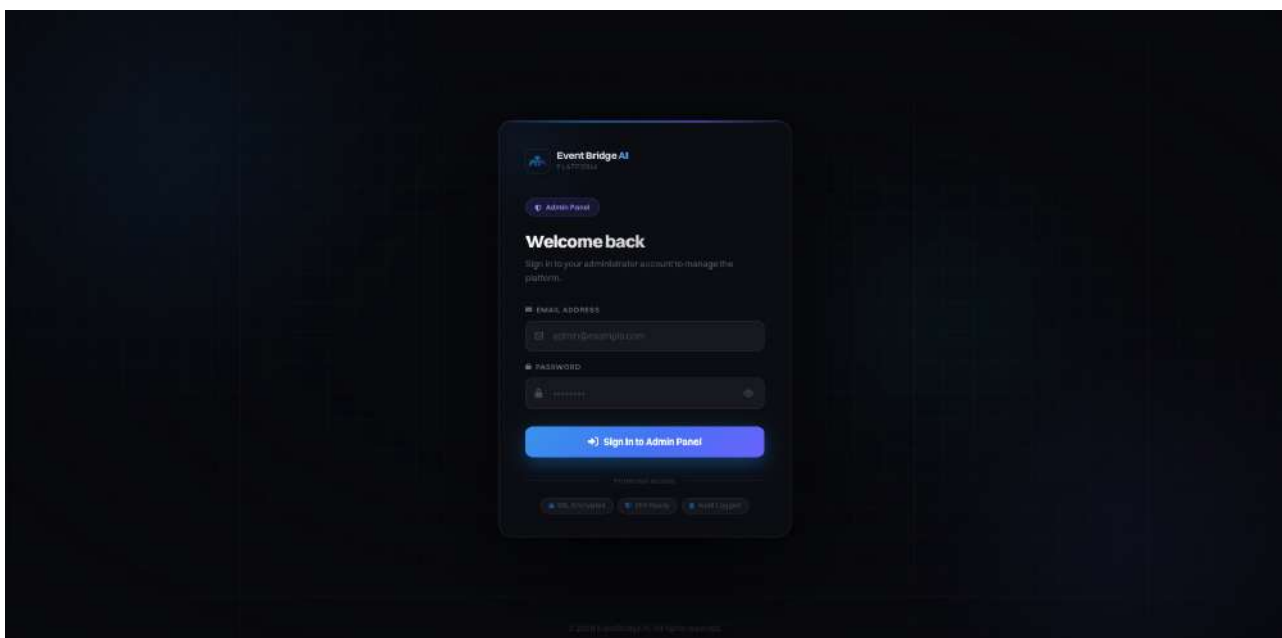


The image shows the 'Register a Community' screen of the EventBridge AI platform. The left panel is a registration form with a progress indicator (1. Community info, 2. Account, 3. Logo). The form fields include: Community Name (IEEE SSUET Student Branch), Slug (IEEE SSUET SR), Founding Year (2020), Domain (Technology & Innovation), and Tagline (Innovate and Create). A 'Continue' button is at the bottom. The right panel has a blue background with the text 'Build something remarkable' and three buttons: 'Manage Events Effortlessly', 'Grow Your Community', and 'Track Analytics'.

Figure 8.6: Community Registration Screen

8.9.3 Administrator Login

Administrator accounts are provisioned internally and are not available through public self-registration. Administrators log in through a dedicated portal using credentials issued during platform setup.



The image shows the 'Admin Login' screen of the EventBridge AI platform. The screen is dark-themed with a central login form. The form includes the 'EventBridge AI PLATFORM' logo, an 'Admin Panel' link, a 'Welcome back' message, and fields for 'EMAIL ADDRESS' (admin@eventbridgeai.com) and 'PASSWORD'. A 'Sign In to Admin Panel' button is at the bottom. There are also links for 'Forgot Password', 'Forgot Email', and 'Forgot Username'.

Figure 8.7: Admin Login Screen

8.10 Individual User Guide

8.10.1 Exploring Communities and Events

- After logging in, the Individual is directed to a personalized Home page displaying AI-recommended events, upcoming events from followed Communities, and events matching declared interests.
- Select Explore from the navigation menu to browse the full event catalog, filterable by category, date, mode (online/in-person), and price.
- Use the search bar to enter natural-language queries (e.g., "AI workshops this weekend"). The platform applies semantic search to match the query against event descriptions, falling back to keyword-based search if the AI service is unavailable.
- Select any event card to view its full details, including description, schedule, venue or online link, host Community, and registration information.

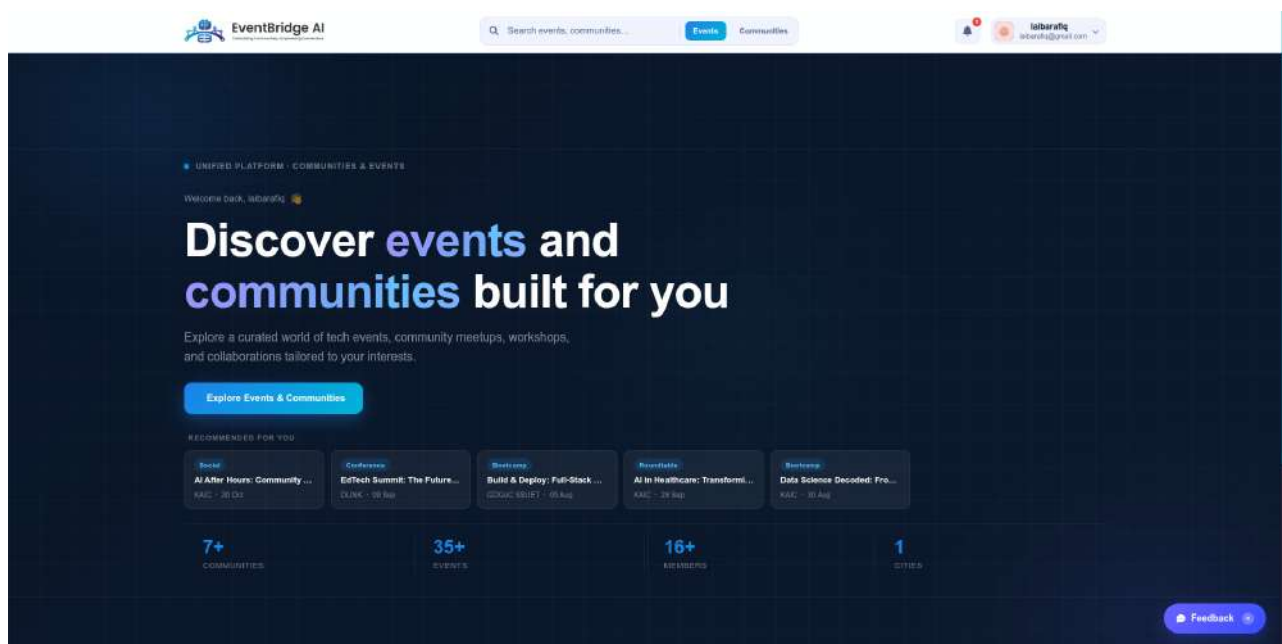


Figure 8.8: User Home Page

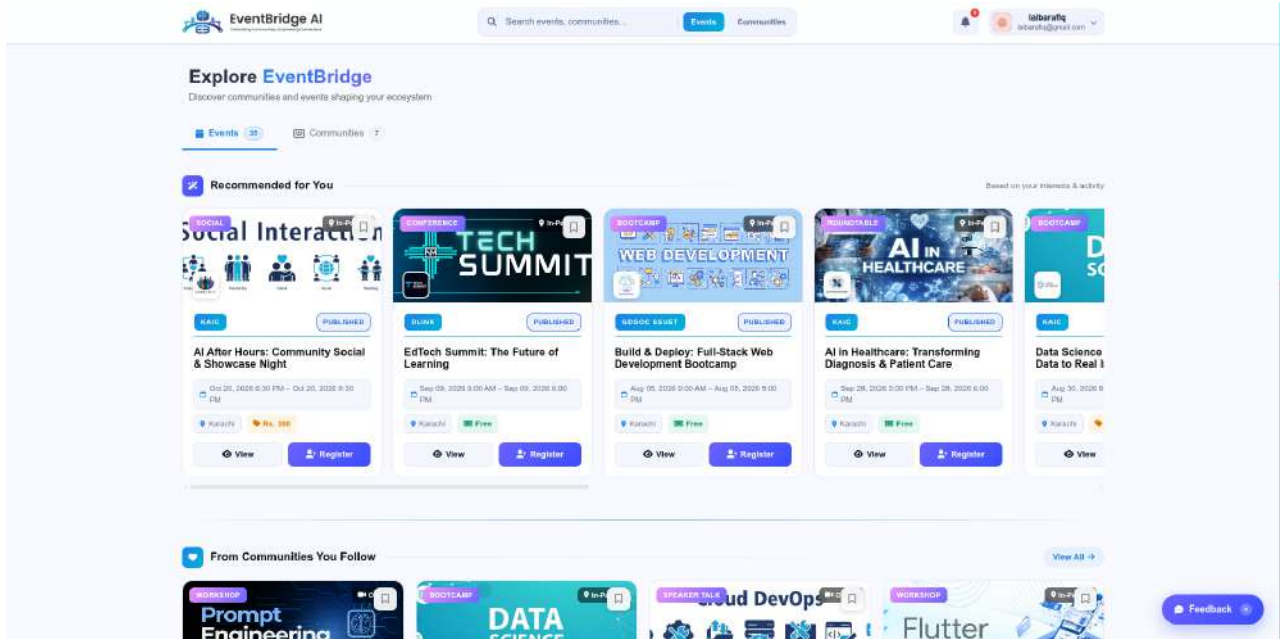


Figure 8.9: Explore Events

8.10.2 Following Communities

- Navigate to a Community's public profile (via its event pages or the Community directory).
- Select Follow to receive updates on new events published by that Community.
- Followed Communities' events are prioritized in the Individual's Home feed and factored into the recommendation engine.
- Select Unfollow at any time to reverse this.

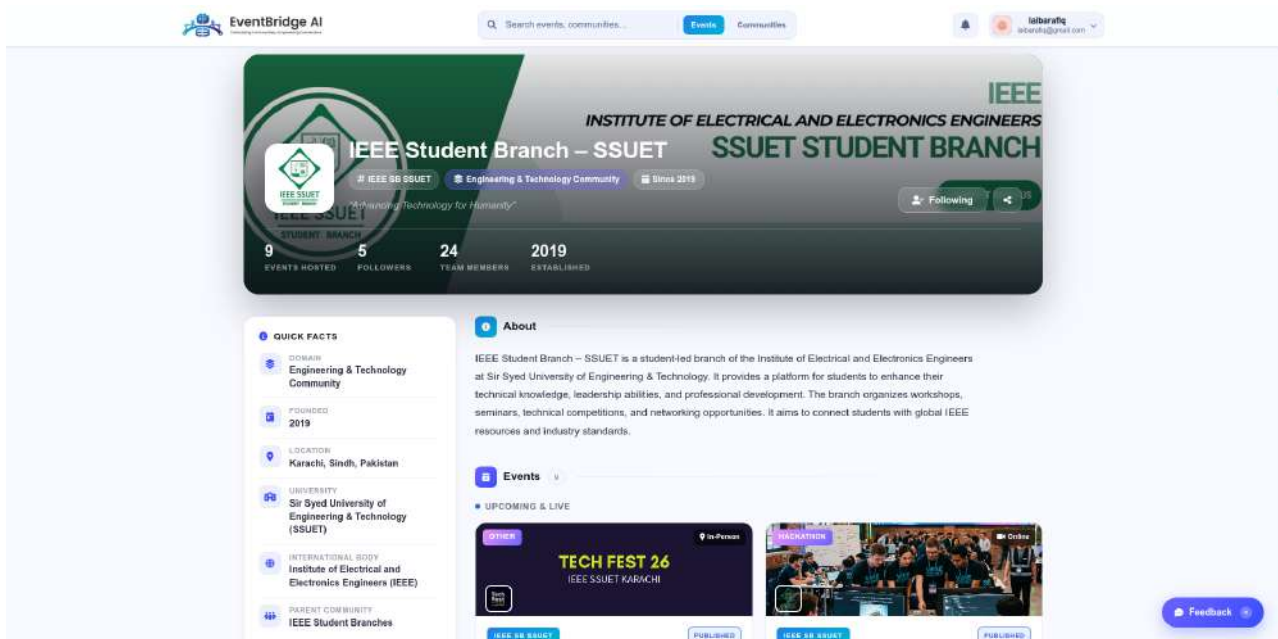


Figure 8.10: Community Public Profile Page

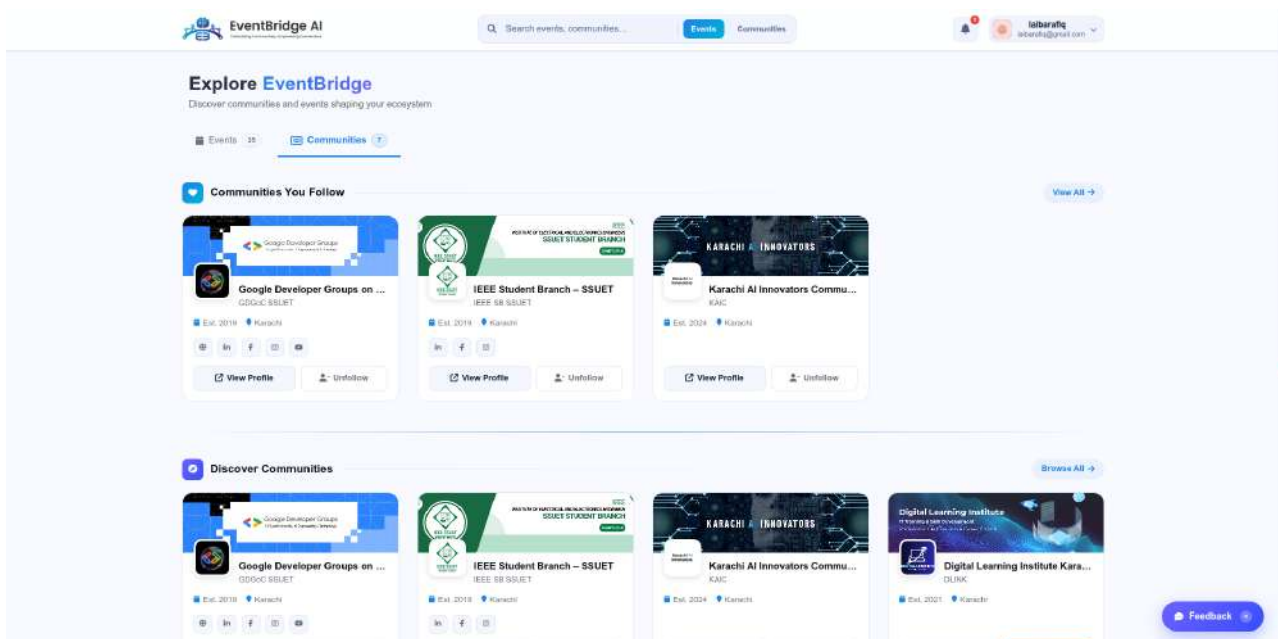


Figure 8.11: Explore Communities

8.10.3 Saving Events

- On any event page, select the Save icon to bookmark the event for later reference.
- Saved events are accessible from the Saved Events section of the profile menu.
- Select Unsave to remove an event from this list.

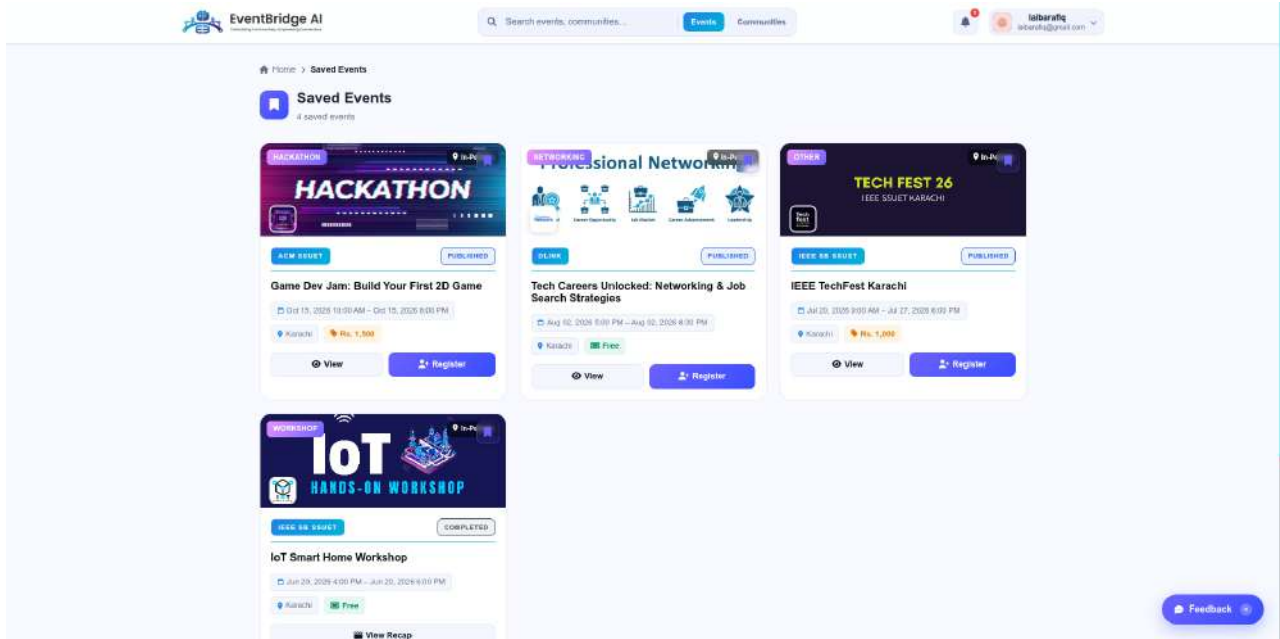


Figure 8.12: Saved Events

8.10.4 Viewing an Event's Public Page

- Selecting any event from the Explore feed, search results, or a Community's profile opens that event's public page.
- This page is accessible to any visitor, including Individuals who are not logged in, and displays the event's title, description, schedule, venue or online link, host Community, speakers/guests, media gallery, and current registration status (open, waitlisted, or closed).
- From this page, the Individual can view the hosting Community's profile, save the event, or proceed to registration (Section A.4.5).
- If not logged in, selecting Register or Save prompts the Individual to log in or create an account first.

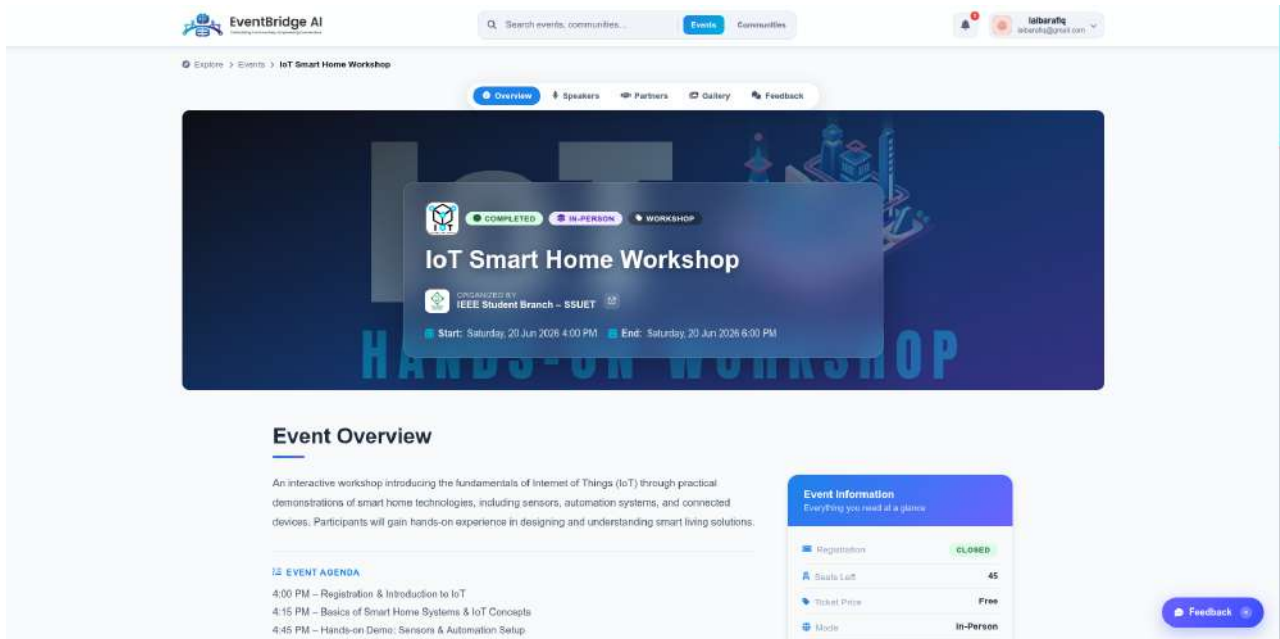


Figure 8.13: Event View Page

8.10.5 Registering for Events

- On an event's detail page, select Register.
- Depending on how the hosting Community configured the event, registration proceeds through either the platform's built-in form or an external Google Form link.
- For internal registrations, complete any additional questions set by the Community and, where required, upload proof of payment for paid events.
- Upon submission, the registration is recorded with a status of Confirmed, Pending, or Waitlisted, depending on event capacity.
- A confirmation notification is sent, and the event appears under My Registrations in the profile.

The screenshot shows the 'Register' page for the 'Vibe Coding Hackathon' on the EventBridge AI platform. The page is divided into several sections:

- Event Details:** Includes the event title 'Vibe Coding Hackathon', start and end dates (2025-07-31 10:00:00 to 2025-08-01 10:00:00), platform 'Teams', and a 'Registration Fee' of 'Free' with a note 'No payment required'.
- Your Details:** A section for 'Registering As' with a dropdown menu showing 'Laliba' and 'lalibaf@gmail.com'. It includes a link to 'Edit profile'.
- A Few Questions:** A section for 'Help the organizer understand your background' with three optional questions:
 - 'Why do you want to attend?' with a text area for the answer.
 - 'How did you hear about this event?' with a text area for the answer.
 - 'What do you expect to learn or gain?' with a text area for the answer.
- Feedback:** A button labeled 'Feedback' is located at the bottom right.

Figure 8.14: Event Registration Page

The screenshot shows the 'My Registrations' page on the EventBridge AI platform. It displays a summary of the user's registration status and a list of registered events.

Registration Status	Count
Total Registered	3
Confirmed	3
Pending Approval	0
Events Attended	1

Event Title	Date	Location	Fee	Status	Registered On	Action
Cybersecurity Essentials: Defending the Digital Frontier	Aug 23, 2025	Karachi	Free	Confirmed	Jun 26, 2025	View Event
IEEE TechFest Karachi	Jul 25, 2025	Karachi	PKR 1,000	Confirmed	Jun 19, 2025	View Event
IoT Smart Home Workshop	Jun 20, 2025	Karachi	Free	Confirmed	Jun 19, 2025	View Event

Figure 8.15: User Registrations

8.10.6 Submitting Feedback

- Once an event has concluded, the Individual may submit feedback from the event's detail page or the My Registrations section.
- Feedback consists of a satisfaction rating and an optional written comment.
- Submitted feedback is visible to the hosting Community, who may respond publicly.

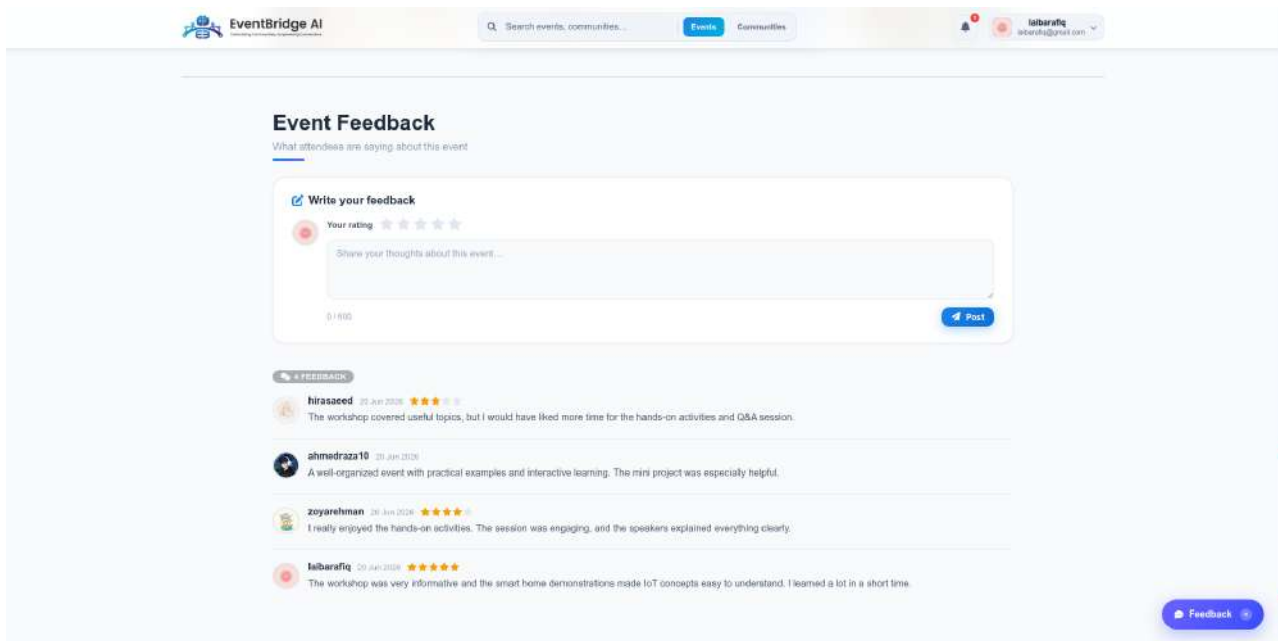


Figure 8.16: Event Feedback Section

8.10.7 Managing Profile and Interests

- Select Profile from the navigation menu to update personal details, profile photo, and declared interests (domain interests, preferred event types, and skills). Updating interests refines future AI recommendations.

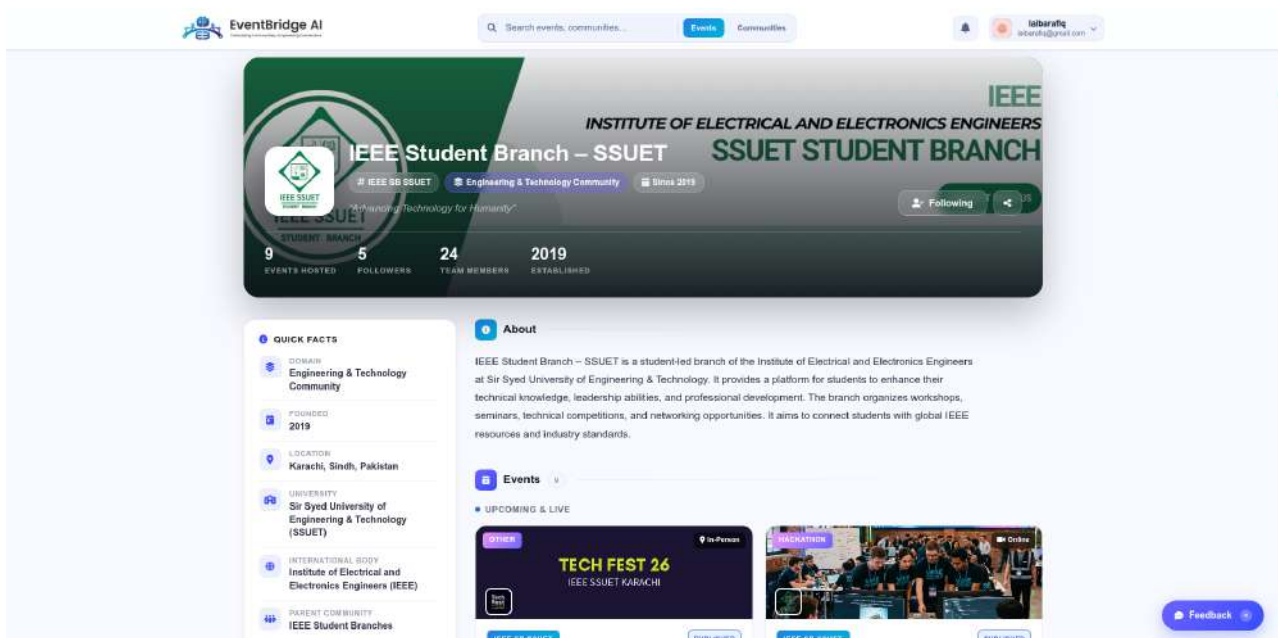


Figure 8.17: User Profile Page

8.11 Community User Guide

8.11.1 Community Dashboard

Upon login, an approved Community lands on its Dashboard, which displays summary statistics (total events, followers, registrations, and feedback count), a list of upcoming and live events, and any pending collaboration requests awaiting action.

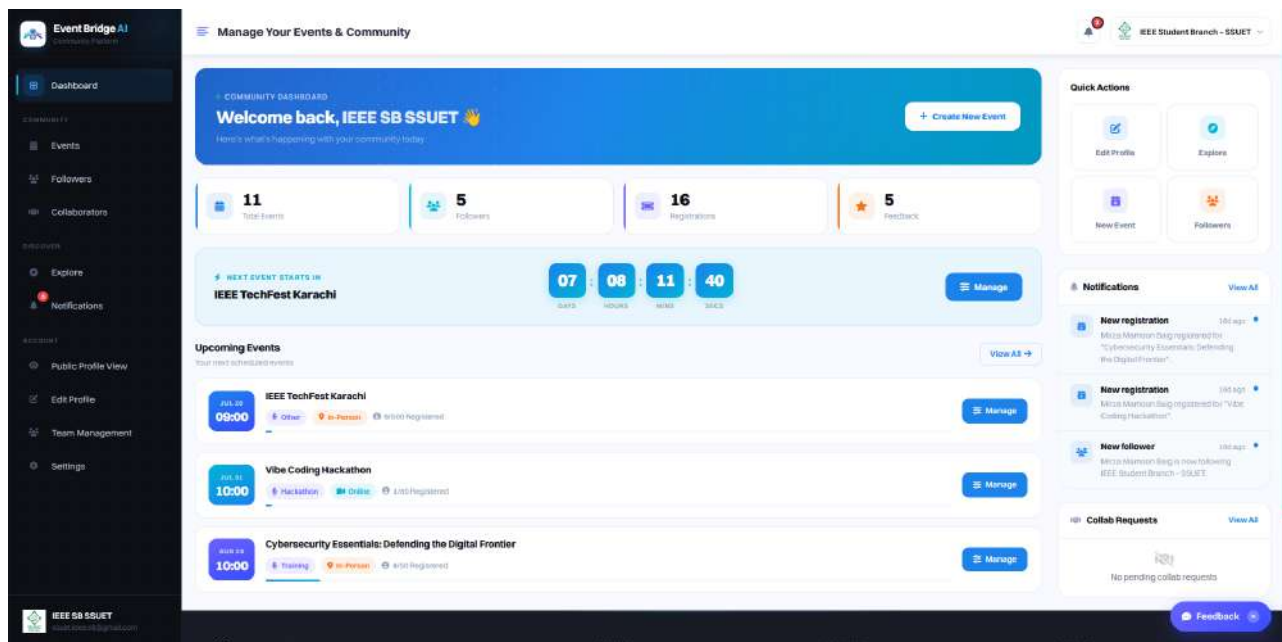


Figure 8.18: Community Dashboard

8.11.2 Profile Management

- Navigate to Profile from the Dashboard sidebar.
- Select Edit Profile to update the Community's logo, banner, bio, mission/vision/core values, location, and social links.
- Save changes to update the Community's public-facing profile page immediately.

Event Bridge AI
Community Platform

Manage Your Events & Community

Edit Community Profile
Update your community's information

Community Name
IEEE Student Branch - SSUET

Short Name
IEEE SB SSUET

Contact Email
ssuet.ieee.sb@gmail.com

Domain
Engineering & Technology Community

Established Year
2019

University Affiliation
Sir Syed University of Engineering & Technology (SSUET)

International Affiliation
Institute of Electrical and Electronics Engineers (IEEE)

Parent Community
IEEE Student Branches

Motto
Advancing Technology for Humanity

About
IEEE Student Branch - SSUET is a student-led branch of the Institute of Electrical and Electronics Engineers at Sir Syed University of Engineering & Technology. It provides a platform for students to enhance their technical knowledge, leadership abilities, and professional development. The branch organizes workshops, seminars, technical competitions, and networking opportunities. It aims to connect students with global IEEE resources and industry standards.

Mission Statement
To empower students with technical excellence, leadership skills, and professional growth opportunities. To promote innovation, research, and hands-on learning in engineering and technology. To bridge the gap between academic knowledge and real-world industry practices.

Vision Statement
To develop competent and globally aware engineers who contribute to technological advancement. To create a strong technical community that drives innovation and excellence.

Core Values
Enter values

Feedback
Add

Figure 8.19: Community Profile Management

8.11.3 Team Management

- Navigate to Team to define the Community's organizational structure (e.g., President, Vice President, department leads).
- Add team members individually, assigning each a role, group, and photo.
- Team members can be edited or removed at any time. A Community maintains one active team structure at a time.

Event Bridge AI
Community Platform

Manage Your Events & Community

Tenure Team Management
Manage your Team Members

Office Bearers

CHAIRPERSON
Tooba Shaikh
A passionate Software Engineering student with a keen interest in AI, Data Science, and Web Development — she's all set to guide the branch toward creativity, collaboration, and impactful change.
tooba.shaikh@gmail.com

VICE CHAIRPERSON
Laiba Javed
Laiba from her roles in WOH, Cyber Savvy, and Dastan Theatre Society to spearheading flagship events, she continues to inspire through action, creativity, and dedication.
laiba.javed@gmail.com

GENERAL SECRETARY
Mirza Mamoon Baig
Mamoon takes charge as the General Secretary for IEEE SSUET 2020! With expertise in Data Science and AI, he brings strong leadership, precision, and

Feedback
Add

Figure 8.20: Community Team Management

8.11.4 Creating and Managing Events

- From the Dashboard, select Create Event.
- Enter the event's basic details (title, description, category), schedule, and venue or online link.
- Choose a registration type: the platform's built-in registration form, or an external Google Form link.
- Optionally set a participant limit.
- Save the event as a Draft to continue editing later, or select Publish to make it visible to Individuals immediately.
- Published events can be edited, unpublished, or marked as Completed once concluded. Events automatically progress through the lifecycle: Draft → Published → Ongoing → Completed (or Cancelled).

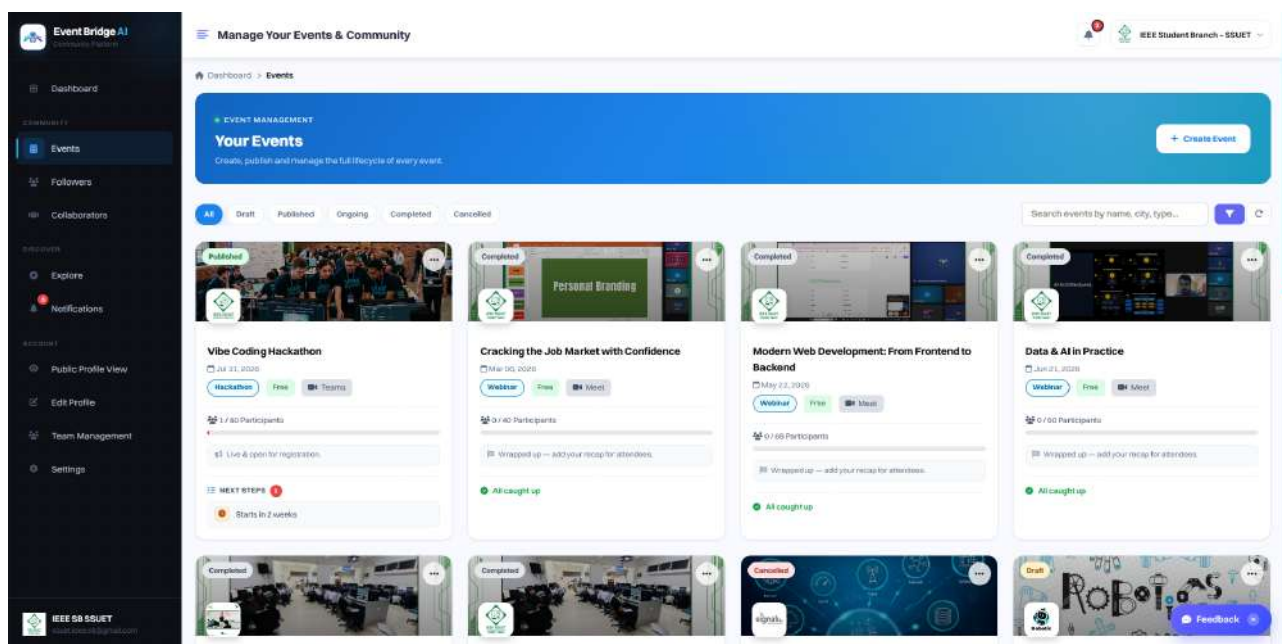


Figure 8.21: Events Management Dashboard

8.11.5 Speaker Management

- From within an event's management screen, select Speakers.
- Add speakers, guests, hosts, moderators, or judges, specifying their name, role, and bio.
- Speakers can be edited or removed prior to the event.

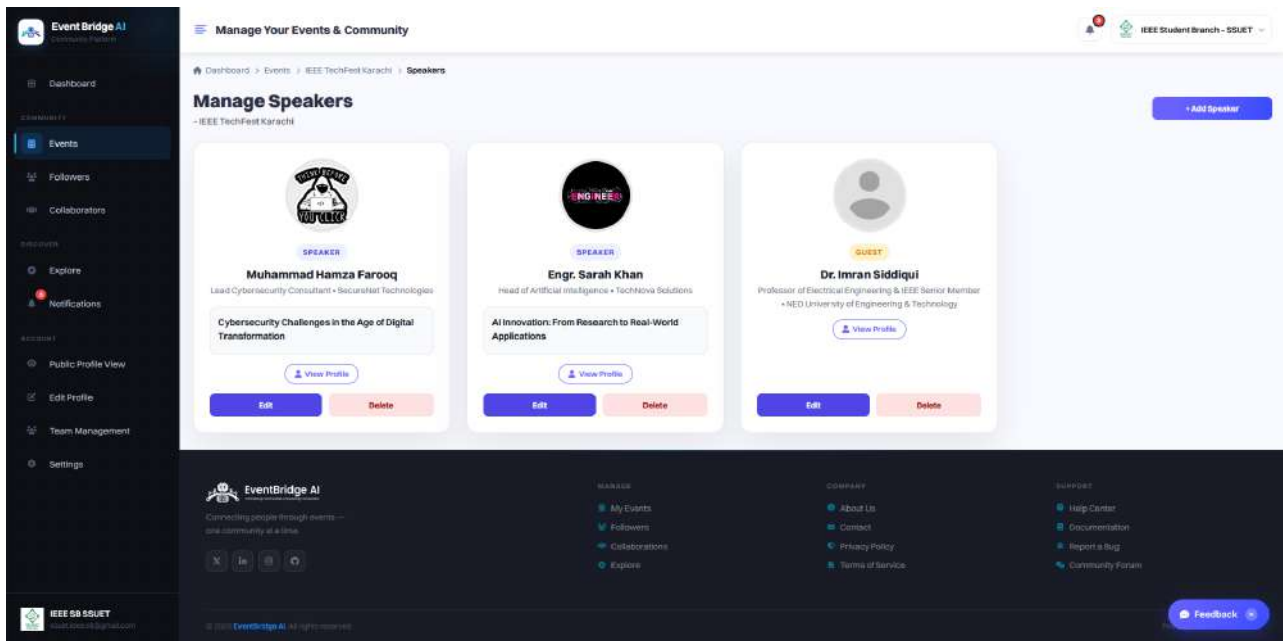


Figure 8.22: Event Speakers Management

8.11.6 Media Uploads

- From within an event's management screen, select Media.
- Upload one or more images or videos related to the event.
- Designate one uploaded image as the Featured Image, which is used as the event's cover image across the platform.

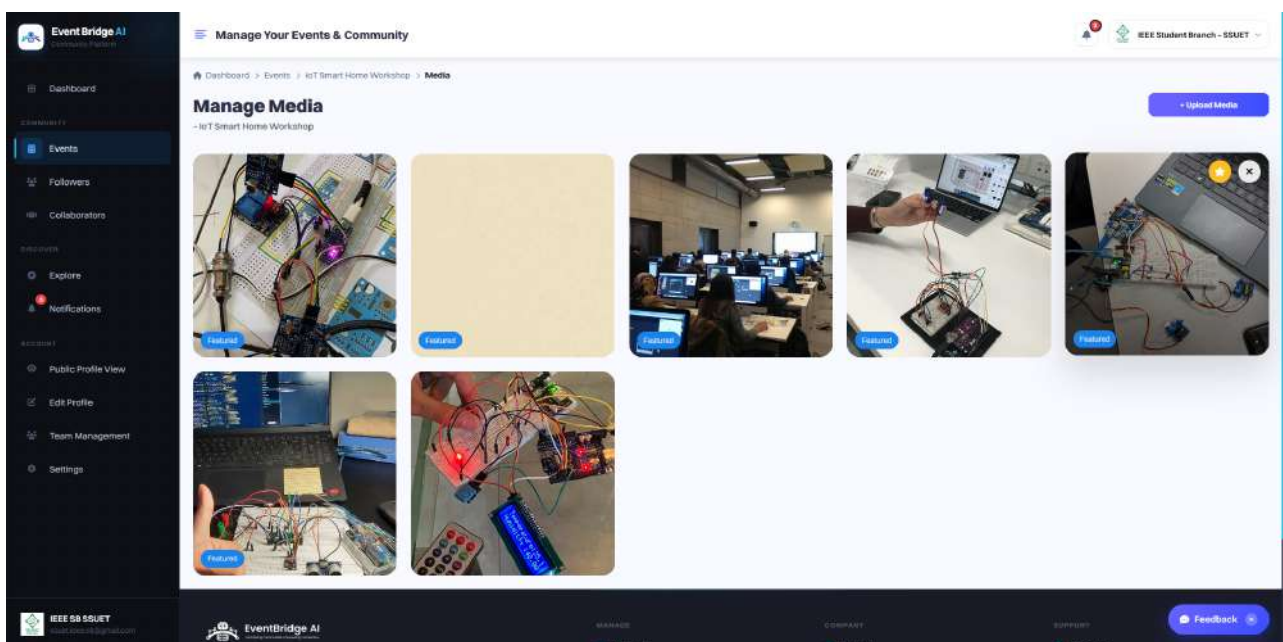


Figure 8.23: Event Media Management

8.11.7 Managing Registrations and Feedback

- Select Registrations within an event’s management screen to view registered, waitlisted, and pending attendees.
- After the event concludes, select Feedback to view submitted ratings and comments from attendees.
- Toggle feedback visibility to control whether specific feedback appears on the event’s public page.

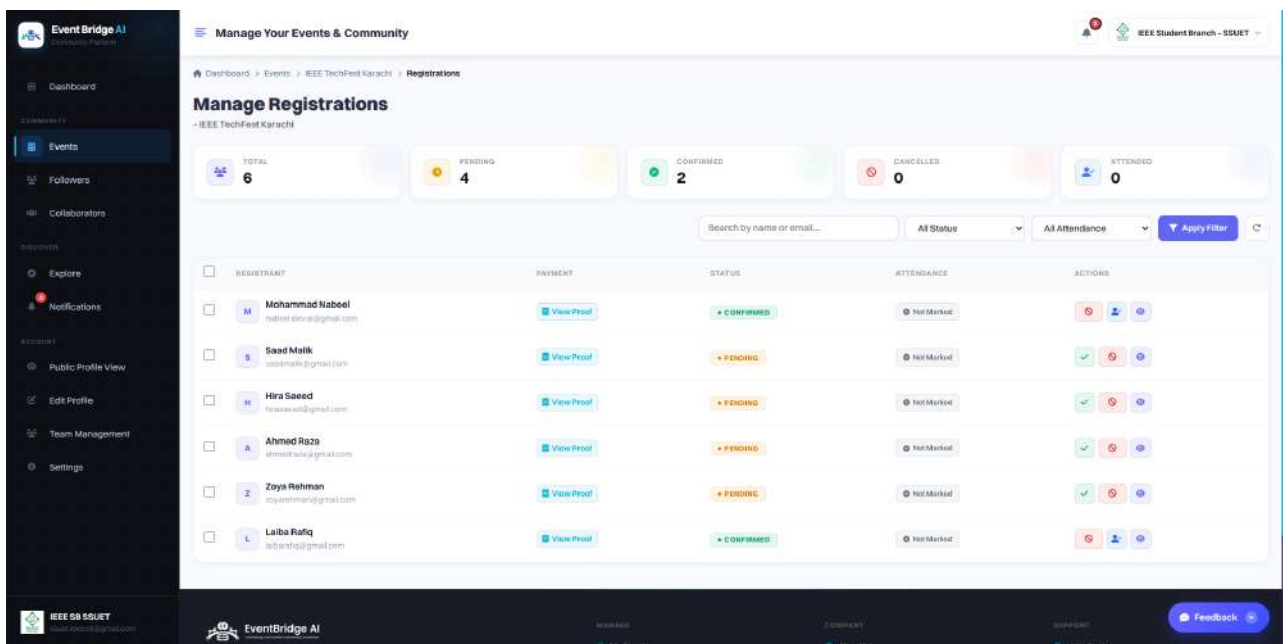


Figure 8.24: Manage Event Registrations and Attendance

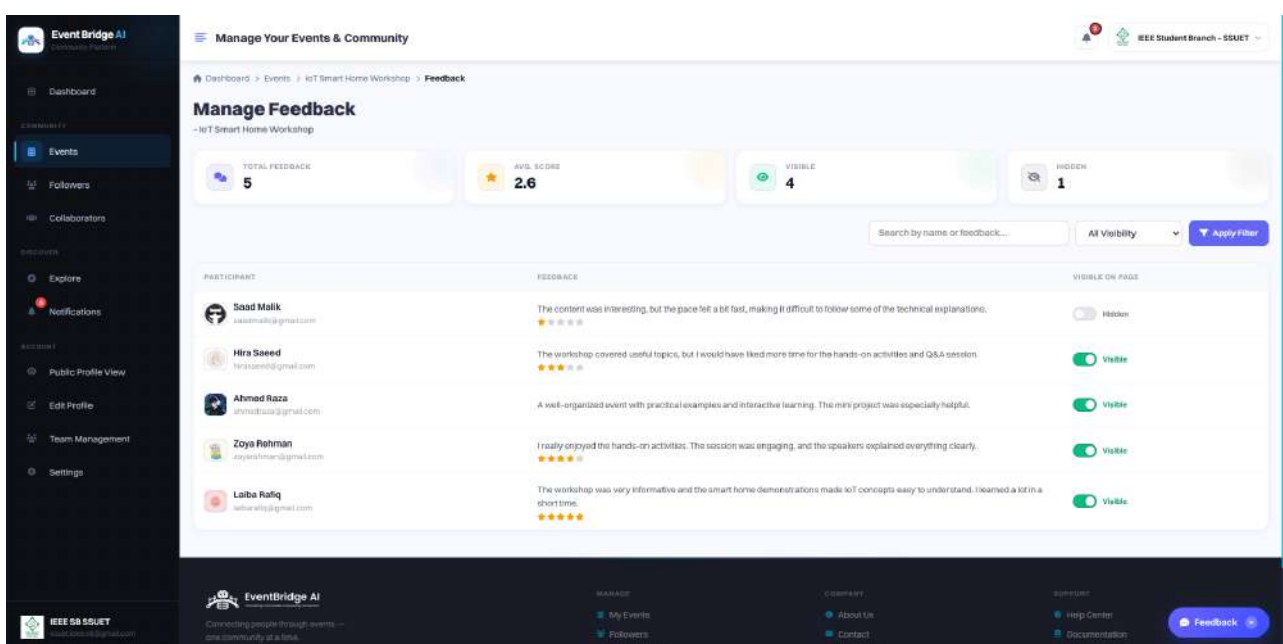


Figure 8.25: Manage Event Feedback

8.11.8 Community Collaboration

- Select Collaborations from the Dashboard sidebar.
- To invite another Community to co-host an event, select Send Collaboration Request, choose the target Community and event, and submit.
- Incoming requests appear with Accept, Reject, or Cancel options. The requesting Community is notified of the outcome.
- Accepted collaborations grant the partner Community shared visibility into the linked event.

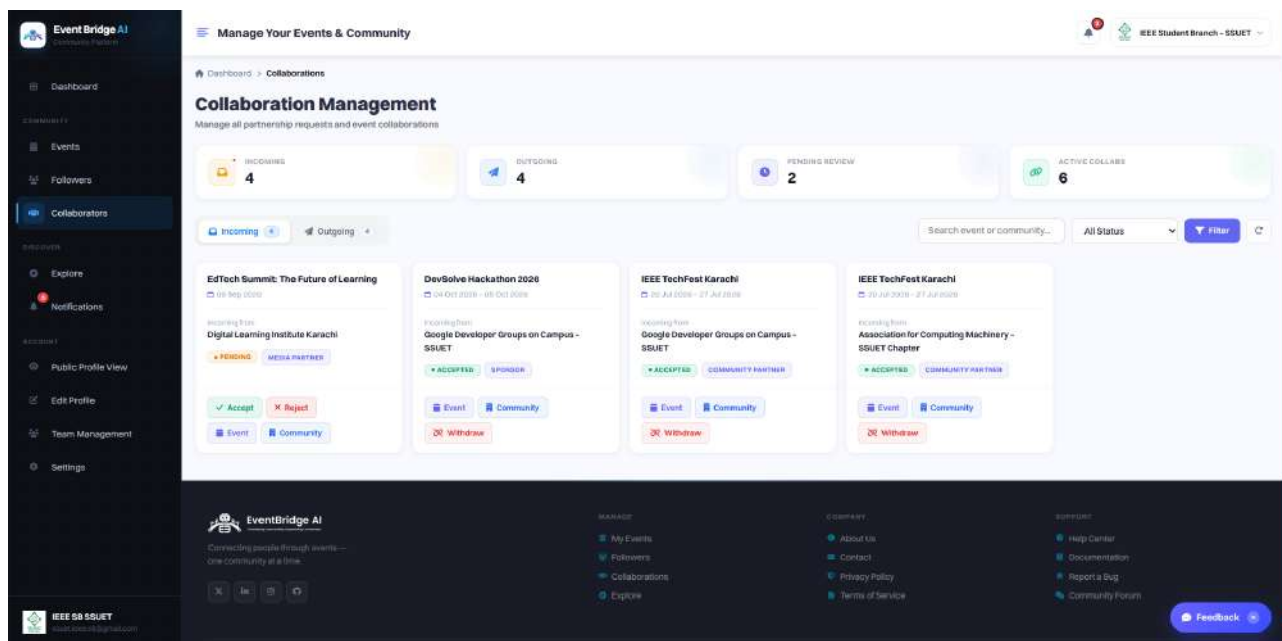


Figure 8.26: Collaboration Management Dashboard

8.11.9 Managing Followers

Select Followers from the Dashboard sidebar to view the list of Individuals and Communities following the Community's profile.

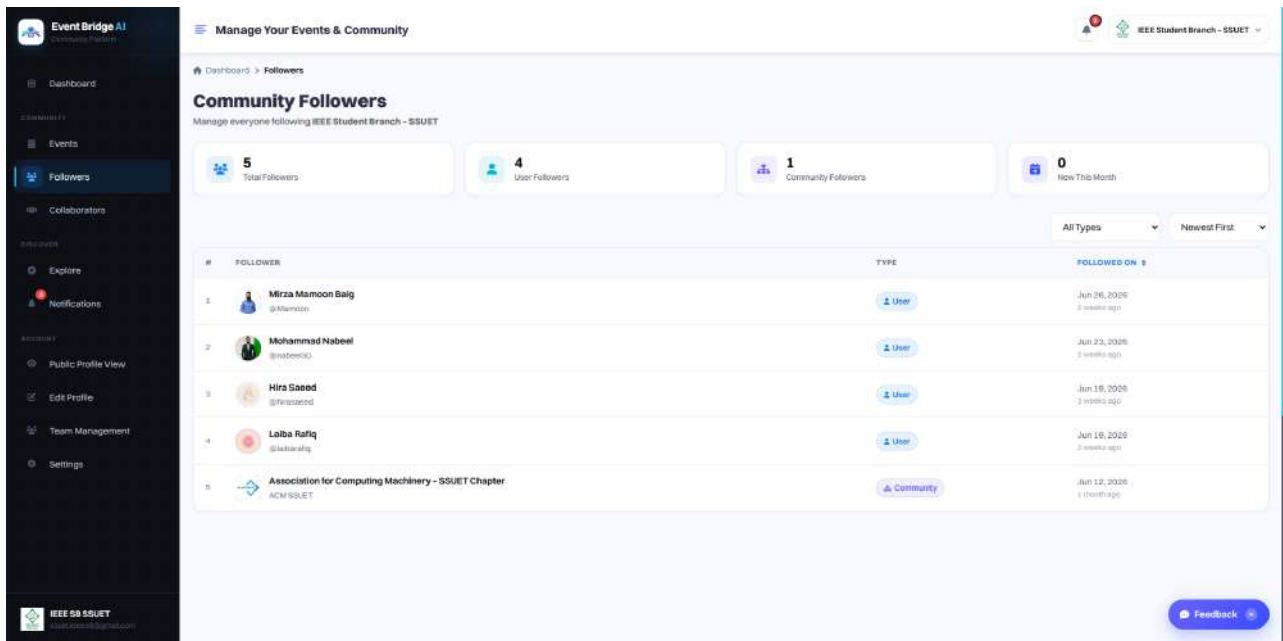


Figure 8.27: Manage Community Followers

8.11.10 Notifications

Select the Notifications icon to view alerts for new followers, registrations, feedback, and collaboration requests.

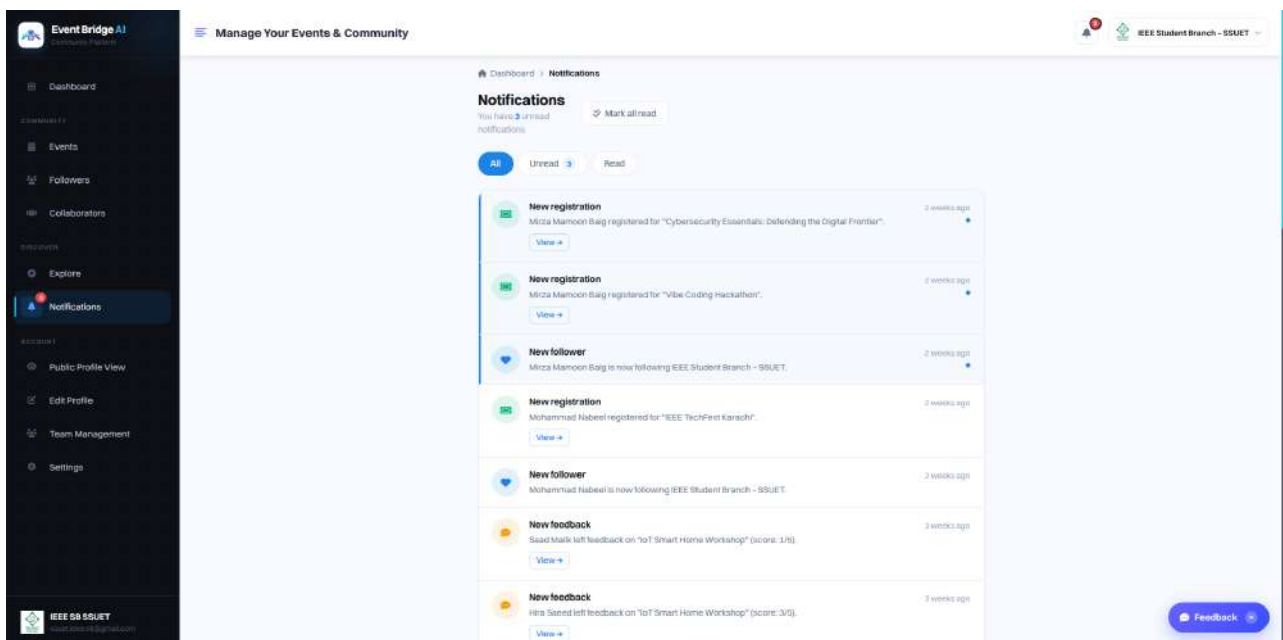


Figure 8.28: Community Dashboard - Notifications Screen

8.12 Administrator Guide

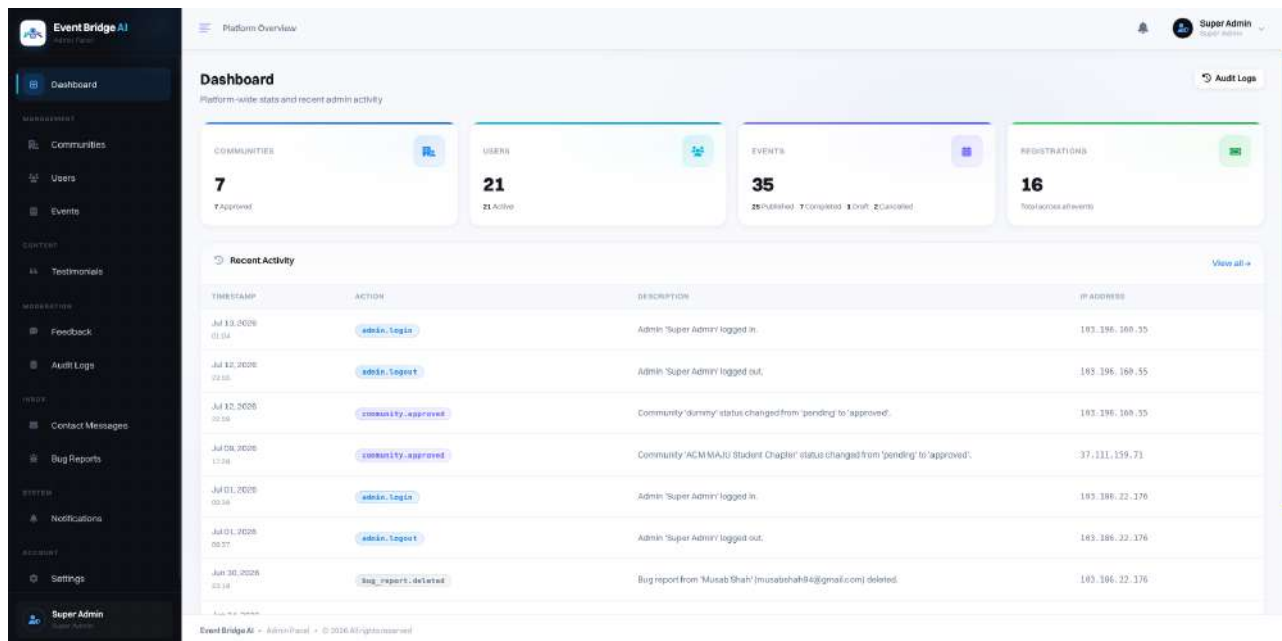


Figure 8.29: Admin Dashboard

8.12.1 Approving or Rejecting Communities

- From the Admin panel, navigate to Community Management to view all registered Communities, filterable by status (Pending, Approved, Rejected, On Hold, Suspended).
- Select a pending Community to review its submitted registration details.
- Select Approve to activate the account, or Rejected/On Hold with an accompanying reason. The Community is notified of the decision, and the action is recorded in the audit log.

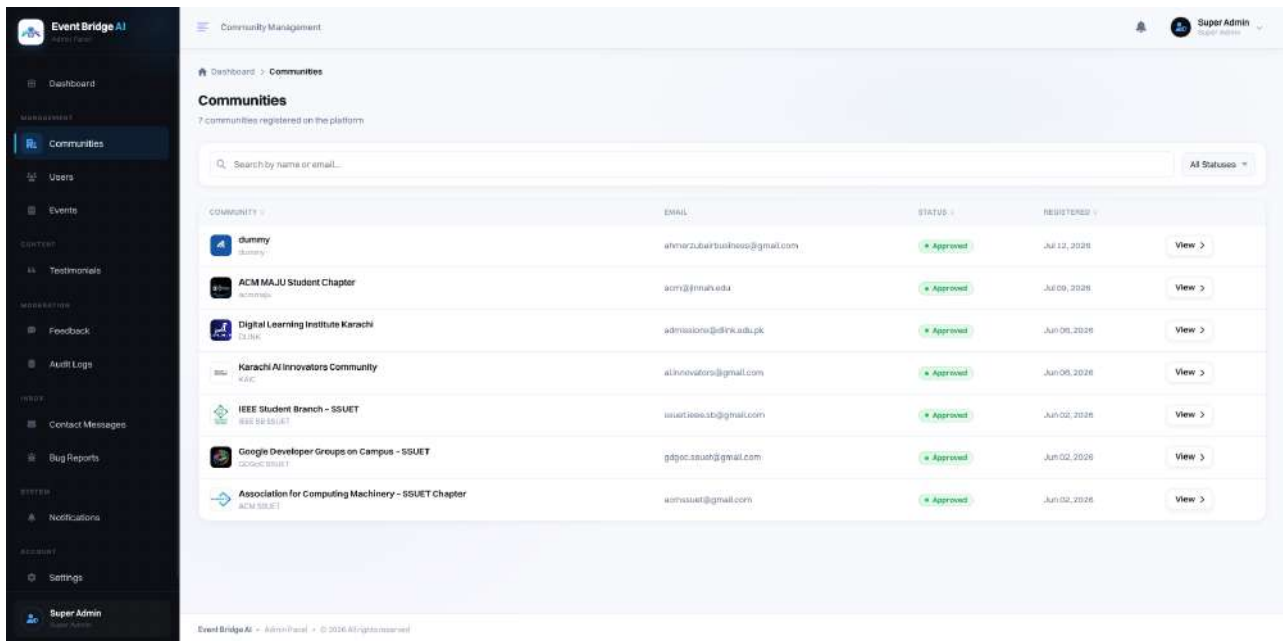


Figure 8.30: Manage Communities

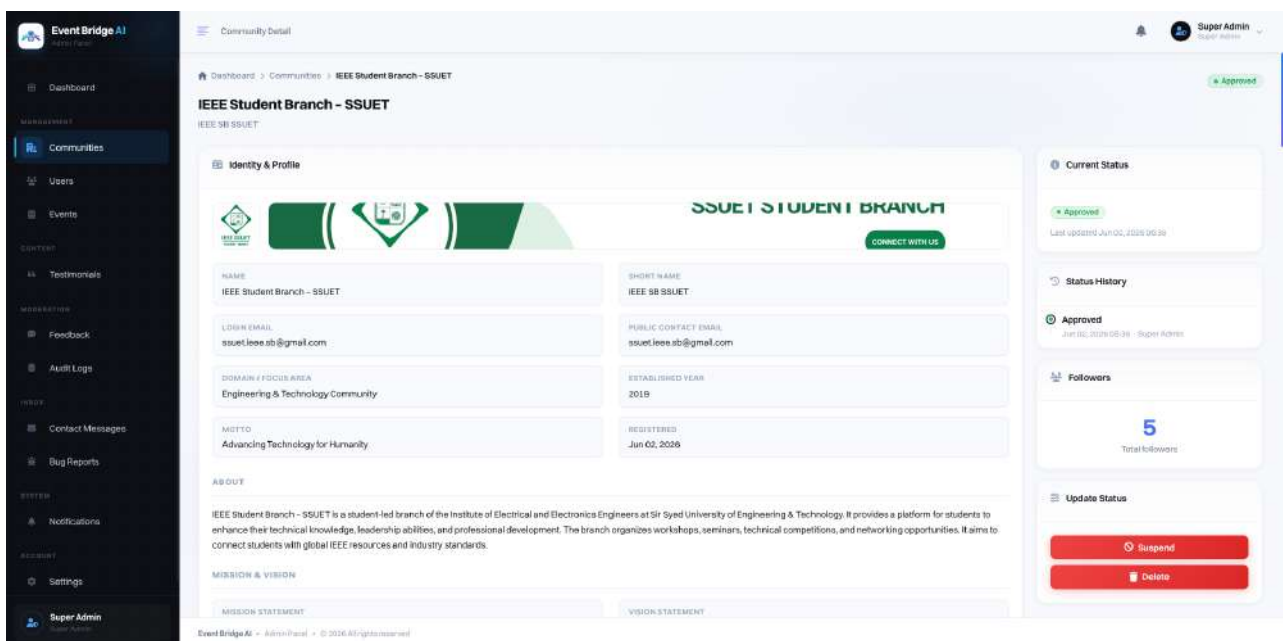


Figure 8.31: View Community Details and Manage Status

8.12.2 Managing Users and Communities

- Navigate to User Management to view all registered Individuals, searchable and filterable by name, email, registration date, or status.
- Select a user to view their profile and registration history.
- Change an account's status (active/suspended/deactivated) or reset a user's password if required. Both actions are logged in the audit trail.

- Community accounts can similarly be suspended or deactivated from Community Management for policy violations.

USER	EMAIL	USERNAME	STATUS	REGISTERED	REGISTRATIONS	
Ahmer Zubair	ahmerzubair@ssuet@gmail.com	AhmerZubair	Active	Jul 12, 2026	0	View >
HIFZA SHAHZAD	hifzashahzad11@gmail.com	Hifza_Sh	Active	Jul 09, 2026	0	View >
muhammad ismail	muhammadismail1101@gmail.com	ismail05	Active	Jul 02, 2026	0	View >
(no name)	majidragut4@gmail.com	—	Active	Jun 29, 2026	0	View >
(no name)	dariskamran2003@gmail.com	—	Active	Jun 26, 2026	0	View >
Zakir Raza	zakiraza110@gmail.com	Raza	Active	Jun 26, 2026	0	View >
Kashan Ghori	kashan.june@gmail.com	KGS	Active	Jun 26, 2026	0	View >
Mazd Kamran	mazd13835@gmail.com	Mazd	Active	Jun 26, 2026	0	View >
Mirza Mamoon Baig	mmamoonbaigdata@gmail.com	Mamoon	Active	Jun 25, 2026	2	View >
adeSe4Eruxvth5N adeSe4Eruxvth5N	ae@ss.com	65487	Active	Jun 25, 2026	0	View >

Figure 8.32: Users Management

8.12.3 Event Oversight

Navigate to Event Management to view all events across the platform, filterable by status (Draft, Published, Ongoing, Completed, Cancelled) or hosting Community. This view is read-only — Administrators can monitor event activity but do not edit or manage individual events, which remains the responsibility of the hosting Community.

EVENT	COMMUNITY	TYPE	STATUS	START DATE	CONFIRMED	
Vibe Coding Hackathon	IEEE Student Branch - SSUET	Hackathon	Published	Jul 31, 2026	3	View >
Cracking the Job Market with ...	IEEE Student Branch - SSUET	Webinar	Completed	Mar 08, 2026	0	View >
Modern Web Development: Fr...	IEEE Student Branch - SSUET	Webinar	Completed	May 22, 2026	0	View >
Data & AI in Practice	IEEE Student Branch - SSUET	Webinar	Completed	Jun 21, 2026	0	View >
Prompt Engineering Worksha...	IEEE Student Branch - SSUET	Workshop	Completed	Jun 08, 2026	0	View >
Prompt Engineering Worksha...	IEEE Student Branch - SSUET	Workshop	Completed	Jun 09, 2026	0	View >
Molestias impedit N	Association for Computing Machinery - SSUET Chapter	Competition	Cancelled	Jul 10, 2026	0	View >
AI Innovation Summit 2026: Ba...	Association for Computing Machinery - SSUET Chapter	Conference	Completed	May 01, 2026	0	View >
AI After Hours: Community So...	Karachi AI Innovators Community	Social	Published	Oct 20, 2026	0	View >
EdTech Summit: The Future of ...	Digital Learning Institute Karachi	Conference	Published	Sep 08, 2026	0	View >

Figure 8.33: Events Oversight

8.12.4 Reports and Moderation

- Navigate to Reports & Moderation to review flagged event feedback, platform feedback submissions, contact messages, and bug reports.
- Select an item to view its full content and take appropriate action, such as removing inappropriate feedback.

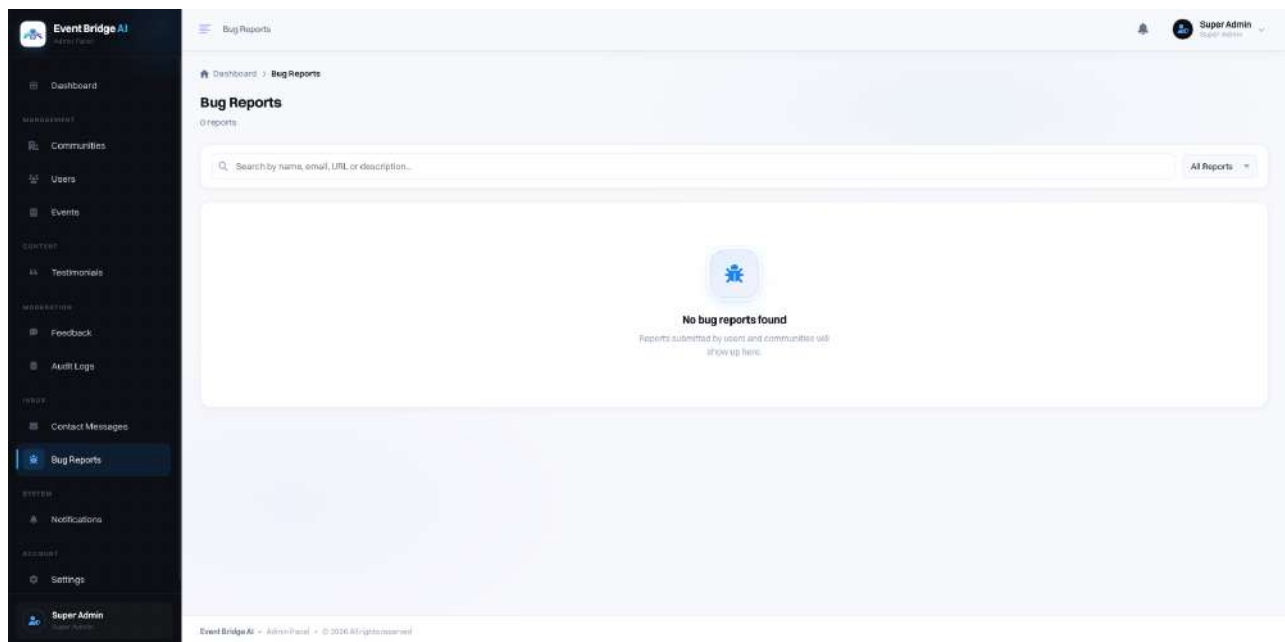


Figure 8.34: Manage Bugs Reporting

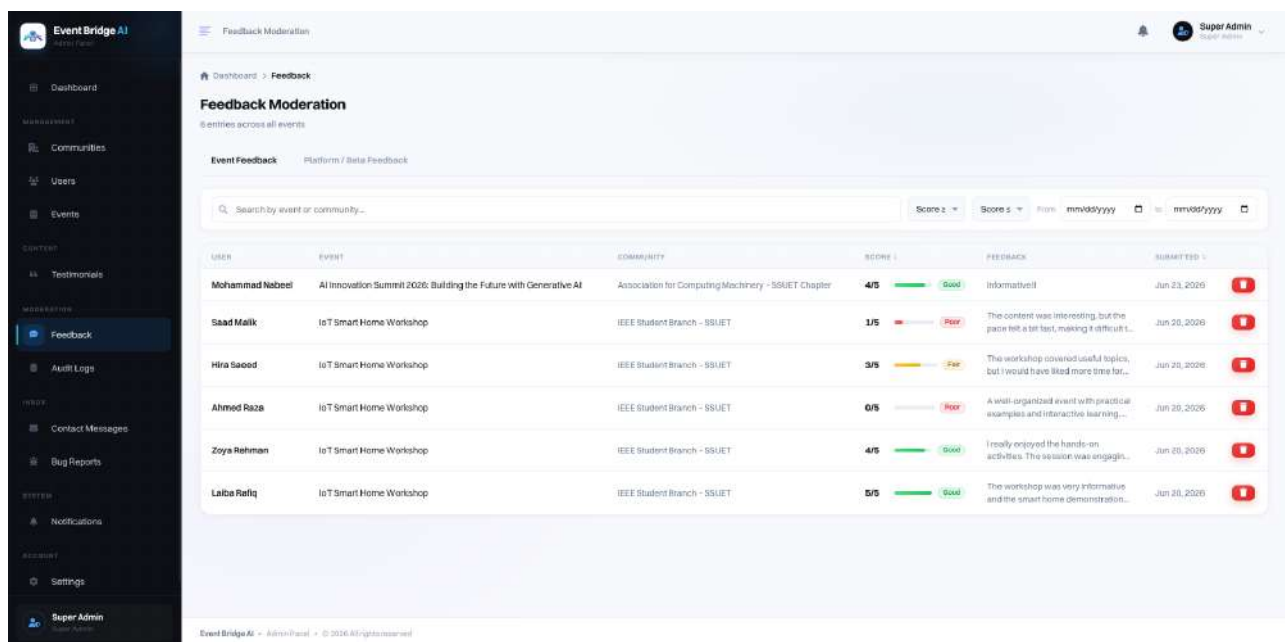


Figure 8.35: Manage Events and Platform Feedback

8.12.5 Broadcasting Announcements

Navigate to Notifications to send a platform-wide or targeted announcement to all Individuals and/or Communities.

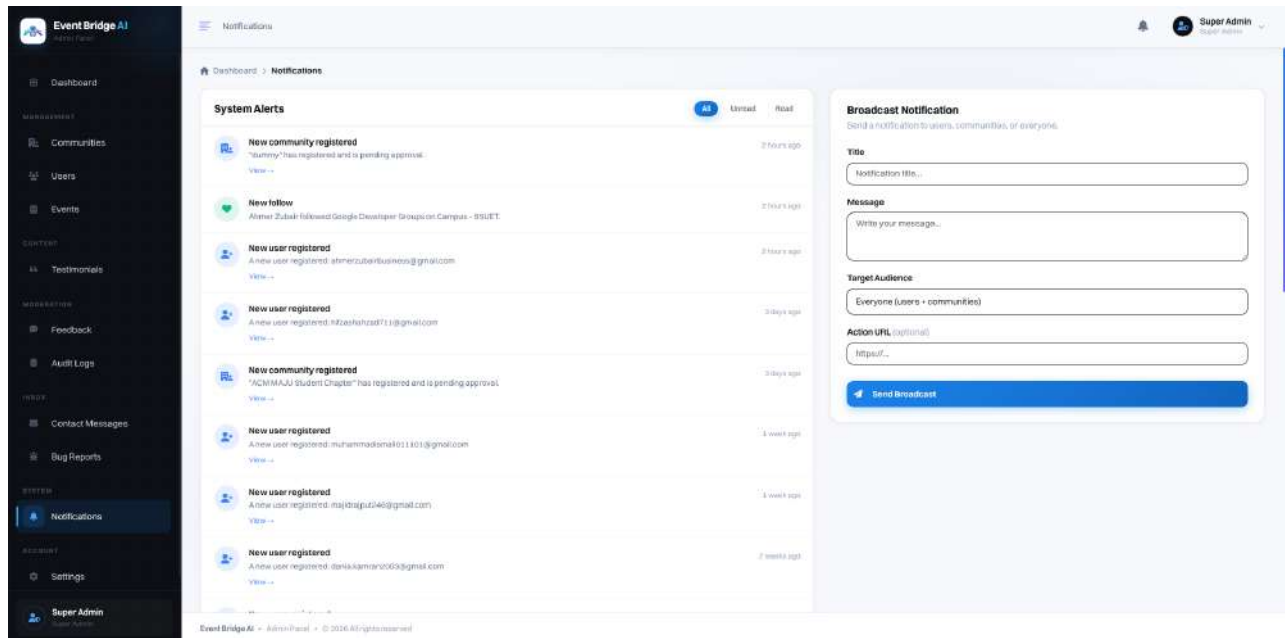


Figure 8.36: Notifications and Broadcasting Panel

8.12.6 Audit Logs

Navigate to Audit Logs to view a complete, filterable history of administrative actions — community status changes, user status changes, and password resets — each recorded with the acting Administrator, timestamp, and reason where applicable.

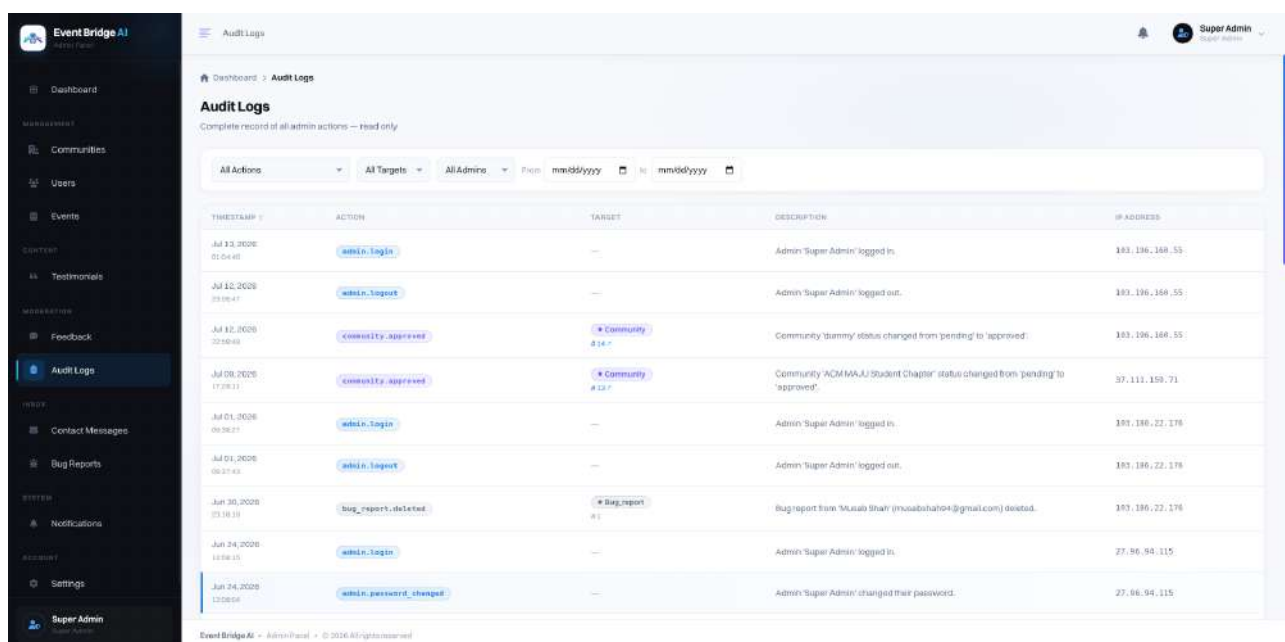


Figure 8.37: Admin Audit Logs

8.13 Notifications Overview

EventBridge AI maintains a dedicated notification system for each account type — Individual, Community, and Administrator — accessible via a bell icon in the navigation bar. Notifications are generated automatically for events such as registration confirmations, new followers, collaboration requests, community approval decisions, and upcoming event reminders.

- Select the Notifications icon to view a feed of recent alerts, with unread notifications indicated by a count badge.
- Select an individual notification to view its full detail and, where applicable, jump directly to the related event, collaboration, or account.
- Use Mark All as Read to clear the unread badge.

Notifications are checked periodically while the platform is open (polling-based) and are automatically pruned after 90 days.

8.14 Troubleshooting and Support

If an issue arises while using the platform, the following channels are available:

- Help Page — Accessible via the Help link in the footer, providing general platform guidance.
- Contact Form — Accessible via the Contact page; submit a message directly to the EventBridge AI team for account or platform-related queries.
- Report a Bug — Accessible via the Report Bug link in the footer; describe any technical issue encountered, which is logged for review by the development team.
- Platform Feedback Widget — A floating feedback button, available to all account types, allows quick submission of a 1–5 satisfaction rating with an optional comment.

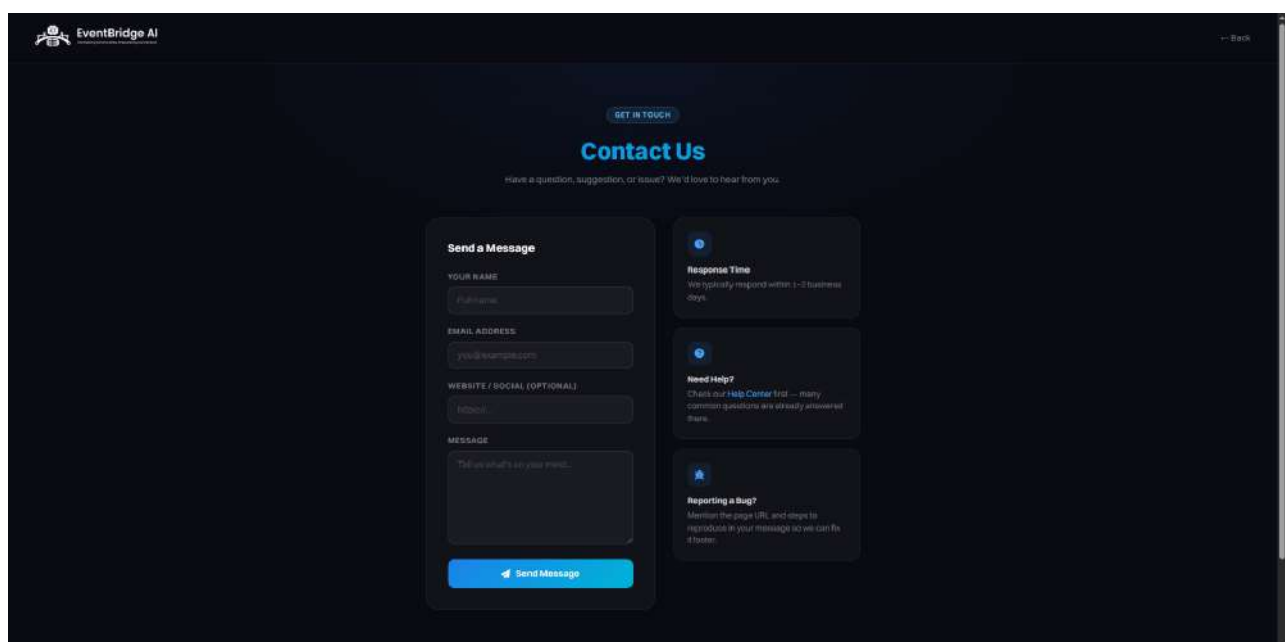


Figure 8.38: Contact Us Page

Figure 8.39: Report Bug Page

8.15 Documentation and Help Center

In addition to the guidance provided in this appendix, EventBridge AI maintains dedicated in-platform Documentation and Help Center pages, accessible via the footer navigation. These pages provide supplementary guidance, feature walkthroughs, and answers to common queries for Individuals, Communities, and Administrators, and are maintained independently of this manual as the platform evolves.

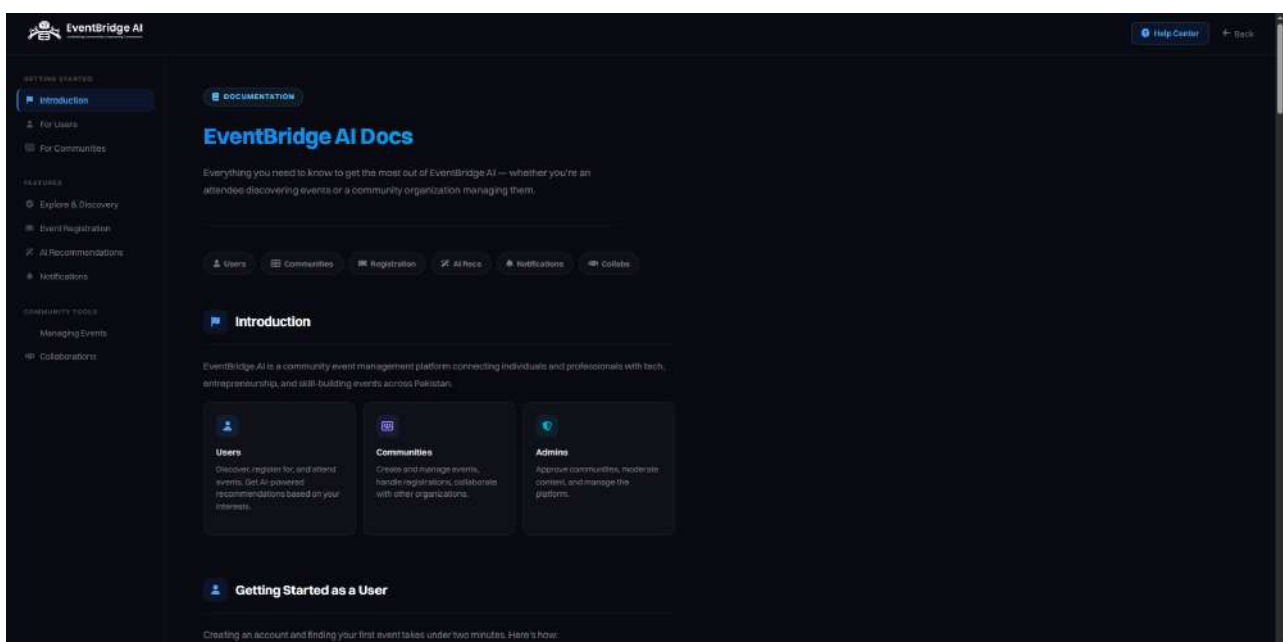


Figure 8.40: Platform Documentation

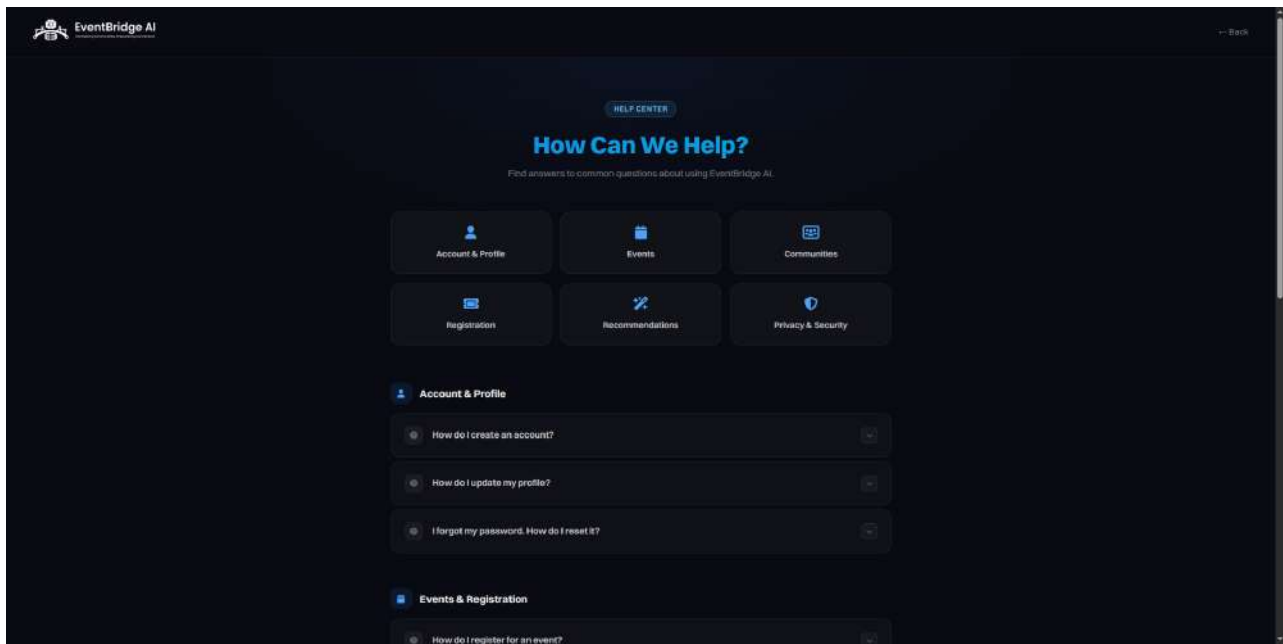


Figure 8.41: Help Center and FAQ Page

8.16 Conclusion

This appendix has outlined the core workflows available to Individuals, Communities, and Administrators on EventBridge AI, covering account registration, event discovery and participation, community and event management, platform oversight, and available support channels. Together with the in-platform Documentation and Help Center, this manual is intended to enable any new user to onboard onto the platform and make full use of its features with minimal friction.

Appendix B- Software Requirement Specification (SRS)

8.17 Introduction

8.17.1 Purpose

EventBridge AI is designed to address a specific and recurring challenge faced by individuals and communities in cities like Karachi: fragmented and disorganized event discovery and management. These communities, like IEEE, GDSC, ACM, and university clubs, frequently organize events for learning, networking, skill development, and collaboration. However, due to the lack of a centralized digital platform, such events are scattered across informal channels such as WhatsApp groups, Instagram Stories, and isolated websites. This fragmentation leads to three key problems: poor discoverability, low turnout, and missed opportunities for Individuals and Communities alike.

The proposed platform, EventBridge AI, is a centralized, AI-powered ecosystem that helps individuals discover, join, and interact with events in meaningful ways. It is not just a listing site; it intelligently understands user interests and improves participation through timely, relevant suggestions. Communities benefit from structured tools for event publishing, community building, and feedback analysis.

The broader purpose of this project includes:

- Eliminating the disconnection between events and their relevant audiences by creating a smart central hub for all Individuals and communities.
- Using AI-powered personalization and filtering to tailor event recommendations to individual preferences and community interests.
- Offering semantic search capabilities to allow users to find events not just by keywords but by context (e.g., "free tech workshops this weekend").
- Supporting future features like event check-in, attendance tracking, and feedback dashboards to help communities grow strategically.

Overall, the platform enhances engagement, reach, and operational efficiency for both event participants and organizers.

8.17.2 Scope

EventBridge AI is envisioned as a web-based application built with a scalable backend (Laravel PHP), a frontend interface (HTML/CSS/JS), and an intelligent AI engine (Python + Django).

The system is designed to support seamless interaction between individuals and communities, focusing on discoverability and personalization.

The scope of the project includes:

- **User Registration and Profiles:** Individuals can register and manage their profiles by defining personal interests. The system enables users to follow communities.
- **Community Registration:** Organizations or clubs must register by providing official entity details through a specialized onboarding form.
- **Administrative Approval:** To ensure platform quality, Community accounts remain inactive until the System Admin reviews and approves the registration.
- **Secure Authentication:** Upon approval, the community can log in using their credentials to access their community dashboard.
- **Event Management System:** End-to-end management of event listings, including title, poster, type (online/offline), fee (paid/free), and tags.
- **Explore Feed:** Users can discover communities to follow and view event details, including options to register or provide feedback. The system displays events from followed communities, personalized recommendations, and overall new live events available to all users.
- **Semantic Search:** A NLP-driven search engine that allows users to search using natural language queries for more relevant results.
- **Recommendation Engine:** The system suggests events based on user activity, preferences, and previous attendance. These recommendations are generated using Collaborative Filtering or Content-Based Filtering, or a Hybrid Recommendation approach.
- **Notification System:** The system provides a centralized notification module for both Individuals and Communities, alerting them of key activities and relevant actions.
- **Administrative Controls:** The platform administrator oversees system operations and manages community registrations. Their primary responsibility is to review, approve, or reject community registration.
- **Cross-Community Features:** Support for inter-community collaborations, shared events, and city-wide visibility of educational and professional initiatives.

Future expansion includes attendance tracking, sentiment analysis on feedback, and dashboards with visual insights for organizers to assess engagement levels.

This platform aligns with the following Sustainable Development Goals (SDGs):

- **Quality Education (SDG 4):** Enhancing learning opportunities through accessible events.
- **Industry, Innovation, and Infrastructure (SDG 9):** Providing an intelligent, scalable infrastructure for digital communities.
- **Partnerships for the Goals (SDG 17):** Promoting inter-community collaboration for shared learning and growth.

8.17.3 Definitions

This section defines key technical and domain-specific concepts used within the Event Bridge AI system to ensure clarity and consistency across stakeholders.

- **Artificial Intelligence (AI):** The simulation of human intelligence in computer systems, enabling tasks such as learning, reasoning, pattern recognition, and decision-making. In this system, AI is primarily used for semantic search and personalized event recommendations.
- **Large Language Model (LLM):** A deep learning model trained on large-scale textual data, capable of understanding and generating human-like language. It may be utilized for advanced query understanding in the platform.
- **Natural Language Processing (NLP):** A branch of AI that enables machines to understand, interpret, and process human language. It is used to interpret user search queries in semantic search.
- **Semantic Search:** An AI-driven search technique that interprets user intent and contextual meaning rather than relying solely on keyword matching, enabling more accurate event discovery.
- **Recommendation Engine:** A system that analyzes user behavior, interests, and interaction history to suggest relevant events using similarity scoring and ranking algorithms.
- **Vector Embedding:** A numerical representation of textual or behavioral data in a high-dimensional space, enabling similarity comparison between users and events.
- **Vector Database:** A specialized database optimized for storing and querying vector embeddings, used for efficient similarity search in AI-powered features.
- **Event Lifecycle:** The end-to-end process of managing an event, including creation, editing, publishing, registration, attendance tracking, and feedback collection.
- **Community Collaboration:** A system feature that allows multiple communities to co-manage and organize events with shared ownership and responsibilities.
- **Personalized Events:** A dynamically generated list of recommended events tailored to an individual user based on preferences, behavior, and AI-driven insights.
-

8.17.4 Acronyms and Abbreviations

This section lists all abbreviations and acronyms used throughout this document for ease of reference.

- AI – Artificial Intelligence
- API – Application Programming Interface
- DB – Database
- FYP – Final Year Project

- FR – Functional Requirements
- LLM – Large Language Model
- NLP – Natural Language Processing
- NFR – Non-Functional Requirements
- SDG – Sustainable Development Goals
- UI – User Interface
- UX – User Experience
- Vector DB – Vector Database

8.18 Project Planning and Management

Project planning and management are foundational pillars in the development lifecycle of EventBridge AI. Given the academic constraints and technical complexity of the system, it is essential to adopt structured, well-documented, and phased planning strategies to ensure timely progress and quality deliverables. This section outlines the strategic thinking, scheduling, work segmentation, and risk-mitigation efforts applied to the development of EventBridge AI.

8.18.1 SWOT Analysis

For the EventBridge AI project, the SWOT analysis informs:

- The technology stack selection (leveraging strengths in AI expertise).
- Design decisions that avoid complexity pitfalls.
- Features that align with user needs and market gaps.
- Safeguards against competitive and regulatory threats.

The following presents a comprehensive SWOT analysis specifically tailored to the strategic, technical, and operational dimensions of the EventBridge AI platform.

Strengths

- **AI-Driven Personalization:** EventBridge AI utilizes vector-based semantic search and user interest to tailor event recommendations to each user, increasing relevance and engagement.
- **Centralized Community Ecosystem:** Unlike fragmented event platforms, it brings together all city-wide communities into a single, structured digital hub.
- **Scalable Modular Architecture:** The platform is built with a layered, service-oriented architecture that supports expansion across cities or institutions.

Weaknesses

- **Initial Data Scarcity:** The recommendation engine requires historical user interaction data, which may not be available during early usage.
- **Technical Complexity:** Combining PHP, Python, LLMs, and vector databases adds layers of integration complexity, increasing the need for testing and coordination.
- **Non-technical User Adjustment:** First-time users may need onboarding to understand filters, semantic search, and AI-based personalization.

Opportunities

Post-COVID Digital Shift: The rise of online/hybrid events creates a sustained demand for centralized digital engagement tools. **Academic Adoption:** Universities and student bodies can adopt the system as their official platform for event management and inter-community networking. **Market Differentiation:** Eventbridge AI distinguishes itself by providing an ecosystem for academic and professional Communities to centralize their activities. Unlike generic event tools, it focuses on structured community management and intelligent discovery to foster long-term engagement and seamless networking within a unified digital space. **Future Expansion:** Potential to commercialize the platform for educational boards, youth conferences, or city-level event organizers.

Threats

- **Regulatory Constraints:** Data privacy laws must be strictly followed due to the involvement of user interaction history and behavioral profiling.
- **Technology Evolution:** Advancements in vector search engines, LLMs, or better-performing AI APIs could necessitate regular refactoring.
- **Indirect Competitors:** Generic social platforms might start offering similar features, diminishing the platform's uniqueness if not updated regularly.
- **Cybersecurity Risks:** As the system collects user profiles and behavioral data, it becomes a target for potential breaches if not well-secured.

By clearly identifying and addressing these elements, the SWOT analysis empowers the project team to take proactive, informed decisions and adapt to changing environments throughout development and deployment.

8.18.2 Gantt Chart



Figure 8.42: EventBridge AI - Gantt Chart

Project Planning (Start Date: 10/1/2025, End Date: 10/31/2025, Duration: 30 Days)

- **Description:** The Project Planning phase is the foundational step for setting the direction and scope of the EventBridge AI project. This phase focuses on gathering requirements, defining the project scope, and establishing a high-level architecture.
- **Key Activities:**
 - Requirement gathering: Collecting initial system requirements, both functional and non-functional, and aligning them with stakeholders' needs.

- Scope definition: Setting clear objectives for what the project will achieve and what it won't.
- Initial architectural planning: High-level discussions and decisions on the tech stack, tools, and methodologies to be used.
- Analysis: This phase has a defined start and end date and occupies 30 days. It is essential because it sets the stage for all future tasks by ensuring the team has a clear understanding of the project requirements, timelines, and available resources.

Requirement Gathering (Start Date: 10/15/2025, End Date: 11/14/2025, Duration: 30 Days)

- Description: The Requirement Gathering phase builds on the initial project planning by refining and finalizing the system's requirements, including detailed user stories, technical specifications, and acceptance criteria.
- Key Activities:
 - Stakeholder Interviews: Engaging with end-users, system administrators, and business stakeholders to refine system requirements.
 - Use Case Development: Formalizing how different actors (Users, Communities and admin) will interact with the platform.
 - Validation: Cross-referencing the gathered requirements with feasibility studies to ensure that the requirements are implementable within the set constraints.
- Analysis: This phase is 30 days long, running concurrently with Project Planning for 15 days. This ensures an overlap, allowing for feedback and refinement. It is crucial to gather comprehensive data to inform the design and implementation stages.

Design Phase (UI/UX) (Start Date: 10/25/2025, End Date: 12/5/2025, Duration: 41 Days)

- Description: The Design Phase is focused on developing the system's UI/UX design. The goal is to create wireframes, user flows, and design prototypes that clearly represent the user experience.
- Key Activities:
 - Wireframing: Sketching the basic layout and functionality of the platform's user interface.
 - Prototype Design: Designing interactive UI prototypes that simulate user interaction with the platform.
 - User Feedback: Conducting usability testing with stakeholders to validate design choices before development begins.
- Analysis: Lasting 41 days, this phase overlaps slightly with the Requirement Gathering phase, which helps streamline the transition from planning to actual system design. The longer duration reflects the criticality of getting the design right before development begins, as it will directly affect the user experience and system performance.

Web Application Development (Start Date: 11/1/2025, End Date: 2/28/2026, Duration: 117 Days)

- **Description:** The Web App Development phase is the core development phase for building the frontend and backend of the platform. This task involves implementing the system's design, developing features, and integrating components.
- **Key Activities:**
 - **Frontend Development:** Coding the user interface (UI) based on the wireframes and prototypes.
 - **Backend Development:** Developing server-side logic, APIs, and database structures using frameworks such as Laravel (PHP) and MySQL.
 - **Integration:** Ensuring the frontend and backend communicate effectively through APIs and data exchange mechanisms.
- **Analysis:** This phase spans 117 days and is the longest single task. The duration is justified given the complexity of developing a fully functional web application with multiple integrations. It is a foundational task that will require ongoing testing and refinement.

AI Server Development (Start Date: 3/15/2026, End Date: 5/15/2026, Duration: 92 Days)

- **Description:** This phase focuses on the development of the AI recommendation engine and semantic search functionality. This system will power the personalized event recommendations and search features for the platform.
- **Key Activities:**
 - **AI Model Development:** The system utilizes Generative AI and vector embeddings to power its recommendation engine. It employs content based or collaborative, or hybrid filtering approaches to provide users with relevant event suggestions.
 - **Semantic Search:** Building the ability for users to search for events using natural language queries.
- **Analysis:** The 92-day duration reflects the complexity of Web App Development, as both systems must integrate seamlessly. The AI engine must be functional and well-integrated with the platform before proceeding to the next phase.

Integration & Testing (Start Date: 4/1/2026, End Date: 6/28/2026, Duration: 90 Days)

- **Description:** The Integration & Testing phase ensures that all components, including the frontend, backend, and AI server, work together as expected.
- **Key Activities:**
 - **System Integration:** Integrating all platform components (web app, AI model, databases) and ensuring smooth communication between them.
 - **Unit Testing:** Conducting individual unit tests for each module.

- User Acceptance Testing (UAT): Verifying that the system meets user expectations and functional requirements.
- Analysis: With a duration of 90 days, this phase is designed to identify and address bugs, integration issues, and performance bottlenecks before deployment. The relatively short duration ensures that the platform will be ready for deployment without unnecessary delays.

Deployment (Start Date: 2/15/2026, End Date: 3/1/2026, Duration: 14 Days)

- Description: The Deployment phase involves moving the platform from the development environment to a live production environment.
- Key Activities:
 - Cloud Hosting Setup: Deploying the platform to a cloud provider, ensuring it is accessible to users.
 - Security Measures: Implementing encryption, access controls, and security auditing.
 - Final System Validation: Ensuring everything works in the live environment as expected.
- Analysis: This phase is relatively short, lasting 14 days, as the infrastructure is already in place. The goal is to ensure that the system is properly configured, secure, and fully functional in a production environment.

Documentation (Start Date: 10/1/2025, End Date: 7/1/2026, Duration: 30 Days)

- Description: The Documentation phase includes writing the technical and user-facing documentation necessary for the platform.
- Key Activities:
 - SRS (Software Requirements Specification): A formal document that describes the system's purpose, functional and non-functional requirements, and any constraints the software must follow. It serves as a foundation for the entire project development.
 - SDD (Software Design Document): A technical document that provides a detailed description of the system's architecture, data structures, and interface design, acting as a primary reference for the implementation phase.
 - Technical Report: Academic review document covering progress and learnings.
- Analysis: 273 days are allocated for this phase to ensure comprehensive documentation is available for future developers, users, and administrators.

Final Submission (Start Date: 7/15/2026, End Date: 7/16/2026, Duration: 1 Days)

- Description: The final submission marks the completion of the project. This phase involves wrapping up the project, ensuring everything is documented, and submitting the final deliverables.

- Key Activities:
 - Project Wrap-Up: Final review of all deliverables, ensuring they meet requirements.
 - Submission: The platform is submitted for academic evaluation.
- Analysis: This phase lasts 1 day, as it is a final administrative task that marks the official completion of the project.

Buffer Time and Dependencies:

- Buffer Time: There is no explicit buffer time shown in this chart, but the overlapping tasks (such as AI Server and Web App Development) provide inherent flexibility. Any delays in one phase could potentially affect the subsequent phases, but the overall project timeline allows for concurrent workstreams to mitigate such risks.
- Dependencies:
 - Web App Development is dependent on the Requirement Gathering and Design Phase.
 - AI Server is dependent on the Design Phase and overlaps with Web App Development.
 - Integration & Testing is dependent on the completion of AI Server and Web App Development, ensuring all components work together.

8.19 Work Breakdown Structure (WBS)

In EventBridge AI, the WBS is categorized into six high-level phases, each of which is broken down into clearly defined subtasks:

- Requirement Engineering: This initial phase includes identifying the core problem, analyzing existing solutions, and documenting functional and non-functional requirements. It forms the foundation upon which all other development is based.
 - Problem Identification: Recognizing and validating the issue of disconnected event ecosystems.
 - Literature Review Existing Platforms: Studying existing solutions like Eventbrite or Meetup to identify feature gaps.
 - Functional Non-Functional Requirements: Creating a structured list of system capabilities and performance expectations.
- Design Phase: This includes the visual and technical blueprint of the system.
 - Database Schema Design: Mapping out tables, relationships, and data flow.
 - System Architecture: Designing the interaction between Laravel backend, Django AI server, and databases.
 - Figma Wireframes: Crafting mockups to define frontend user interfaces and user experience flows.
- Development Phase: The coding and implementation stage divided by stack layers.

- Frontend (HTML, CSS, JS): Building user-facing components for dashboards, event listings, and profiles.
- Backend APIs (Laravel + MySQL): Creating logic for CRUD operations, authentication, and notifications.
- AI Engine (Django + NLP): Developing the semantic search, event recommendation engine, and embedding-based vector filtering.
- Testing Phase: Ensures system quality and reliability through multi-stage evaluations.
 - Unit Testing: Verifying individual modules for correctness.
 - Integration Testing: Validating how modules interact, especially across frameworks.
- Deployment: Preparing the system for live academic use.
 - Hosting Configuration: Deploying backend and AI components on cloud platforms.
 - Security Measures: Implementing encryption, data validation, and access control.
 - Final Presentation: Preparing for academic evaluation, showcasing core deliverables.
- Documentation: All reporting and user-assistance material.
 - SRS (Software Requirements Specification): A formal document that describes the system’s purpose, functional and non-functional requirements, and any constraints the software must follow. It serves as a foundation for the entire project development.
 - SDD (Software Design Document): A technical document that provides a detailed description of the system’s architecture, data structures, and interface design, acting as a primary reference for the implementation phase.
 - Technical Report: Academic review document covering progress and learnings.

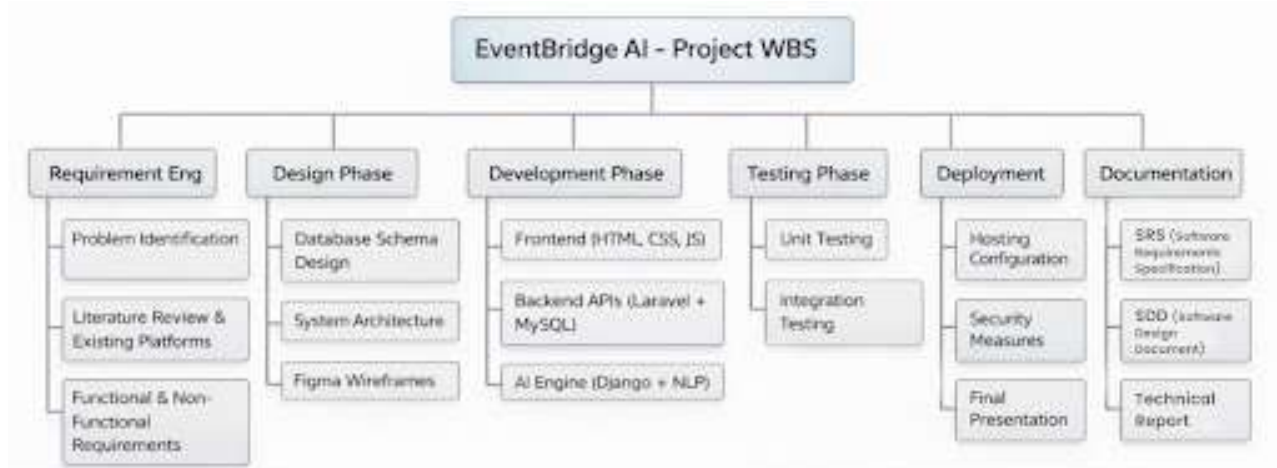


Figure 8.43: Work Breakdown Structure Diagram

Each box in the WBS diagram represents a work package, a task or collection of closely related subtasks that result in a deliverable. Arrows and branches show the hierarchy, beginning with the overall project at the top and splitting into progressively granular components. This structured visualization aids in:

- Defining the full scope of work clearly.
- Avoiding task duplication or omission.
- Assigning responsibilities at the subtask level.
- Monitoring deliverables for each development phase.

8.20 Overall Description

This section presents a comprehensive overview of the EventBridge AI platform, detailing its placement within the software ecosystem, its expected functionality, and the contextual conditions under which it will operate. This provides clarity for both technical evaluators and non-technical stakeholders by illustrating how the system is structured, what it is designed to do, and how users and system components interact.

8.20.1 Product Perspective

EventBridge AI is positioned as a standalone yet extensible web-based application designed to address the inefficiencies in event discovery and engagement across academic and community ecosystems. Rather than functioning in isolation, EventBridge AI serves as a comprehensive middleware that bridges the gap between fragmented community posts and active event participants.

At its core, EventBridge AI is a multi-layered, modular system composed of the following integrated layers:

- **Frontend Presentation Layer:** A user-facing interface developed using HTML, CSS, and JavaScript, providing dashboards, search modules, and community views.
- **Backend Application Layer:** Built on Laravel (PHP), this layer manages server-side logic, including authentication, event CRUD operations, user profiles, community workflows, and notification routing.
- **AI Services Layer:** Implemented using Django (Python), this component performs semantic search, vector filtering, recommendations, and relevance scoring using embedding models and LLMs.
- **Data Management Layer:** Utilizes MySQL for structured data such as events and user records, and Qdrant (or an equivalent vector database) for managing high-dimensional AI embeddings.

The system follows a decoupled architecture to ensure ease of maintenance, parallel development, and flexibility in updates. JSON-based Rest APIs mediate communication between the Laravel backend and the Django AI server, enabling independent scalability and modular feature enhancements.

8.20.2 Product Functions

Event Bridge AI provides a comprehensive set of functionalities designed to facilitate event management, discovery, and interaction between users and communities. The system integrates traditional event management features with AI-powered capabilities to enhance user experience and engagement.

The major functional components of the system are described below:

User Management

The system allows individual users to register, authenticate, and manage their profiles.

- **Account Creation & Authentication:** Users can create accounts and securely log in to the platform.
- **Onboarding & Preferences:** Users provide interests and preferences during onboarding.
- **Personalization Data Usage:** User data is utilized for personalization and recommendation features.

8.20.3 Community Management

Communities represent organizations that host and manage events.

- **Community Registration:** Communities can register by submitting organizational details.
- **Admin Approval:** Community accounts require administrative approval before activation.
- **Profile Management:** Approved communities can manage their public profiles and information.
- **Team Management:** Communities can maintain and update team member information.

8.20.4 Event Management

Communities are responsible for managing the complete lifecycle of events.

- **Event Creation Publishing:** Communities can create, edit, and publish events.
- **Event Details Management:** Communities can manage event schedules, venues, and descriptions.
- **Registration Handling:** Communities can manage user registrations and attendance.
- **Feedback Management:** Communities can collect and manage post-event feedback.

8.20.5 Event Discovery and Interaction

The platform provides a centralized exploration hub for both users and communities.

- Event Browsing & Search: Users can browse and search for events.
- Event Participation: Users can register for and attend events.
- Community Exploration: Communities can explore events for collaboration opportunities.
- Follow System: Users can follow communities to receive updates.

8.20.6 Community Collaboration

The system enables collaboration between multiple communities.

- Collaboration Requests: Communities can send and receive collaboration requests.
- Joint Event Management: Multiple communities can co-manage events.
- Shared Ownership: Events can have shared ownership between collaborating communities.

8.20.7 AI-Powered Features

The platform incorporates Artificial Intelligence to enhance discovery and personalization.

- Semantic Search: Enables natural language-based event discovery.
- Recommendation Engine: Provides personalized event suggestions to users.
- Behavioral Analysis: AI models utilize user behavior, preferences, and event data.

8.20.8 Notification System

The system provides real-time notifications to keep users and communities informed.

- Event-Triggered Notifications: Notifications are generated based on system activities and user actions.
- Alert Types: Includes registrations, approvals, collaborations, and updates.
- In-App Delivery: Notifications are delivered within the platform interface.

8.20.9 Feedback and Interaction System

The platform allows users to provide feedback on events.

- User Feedback Submission: Users can submit textual feedback on events.
- Visibility Control: Communities can set feedback visibility as public or private.
- Manual Moderation: Feedback is managed manually by the hosting community.

8.20.10 Administrative Control

The Admin oversees platform operations and governance.

- Community Approval: Admin can approve or reject community registrations.
- System Monitoring: Admin monitors overall platform activity.
- Platform Governance: Admin ensures proper functioning of the system.

8.21 Operating Environment

EventBridge AI is engineered to operate in a cloud-first, browser accessible environment while adhering to modern security and privacy standards. Its infrastructure is optimized for academic institutions but is also capable of scaling to accommodate larger municipal or commercial deployments.

8.21.1 Operating Environment Details

- Client-Side Access: All features are available through major browsers (Chrome, Firefox, Safari, Edge).
- Server Environment:
 - Laravel (PHP) hosted on Linux-based virtual servers (NameCheap VPS or equivalent).
 - Django (Python) services are deployed using NGINX + Gunicorn.
 - MySQL for relational data and Qdrant (or similar) for high-dimensional vector embeddings.
- External Services:
 - OpenAI or HuggingFace APIs for embeddings and LLM-based tasks.

8.21.2 Constraints and Considerations

- Academic Timelines: The fixed development window demands time-boxed sprints and buffer-inclusive planning.
- System Dependencies: Relies on stable internet connections and external APIs which may evolve or change.
- Resource Optimization: Embedding computations and semantic ranking require GPU or high-performance server resources, especially during peak usage.
- Maintenance Separation: The use of two separate backend technologies requires independent monitoring and deployment cycles.

These environmental definitions and constraints directly shape the development scope, risk planning, and deployment strategy.

8.22 Design and Implementation Constraints

This section describes the limitations and constraints that affect the design and implementation of the Event Bridge AI system.

- **Technology Stack Constraint:** The system is developed using Laravel (PHP) for the main backend and a Python-based framework, Django, for AI services. This requires maintaining interoperability between different technologies.
- **External API Dependency:** The system relies on third-party services such as OpenAI for embedding generation and natural language processing. Changes in API availability, pricing, or performance may impact system functionality.
- **Infrastructure Limitations:** Deployment is expected on virtual private servers (VPS) or cloud-based environments with limited computational resources. AI-related operations such as embedding generation and similarity search may require high-performance resources.
- **Academic Timeline Constraint:** As a Final Year Project (FYP), development must be completed within a fixed academic schedule, limiting the scope for extensive scalability or optimization.
- **Deployment Complexity:** The use of multiple backend services (Laravel + Python AI services) requires separate deployment, monitoring, and maintenance processes.
- **Security and Privacy Requirements:** The system must adhere to standard web security practices, including secure authentication, data protection, and safe API communication.

8.23 Assumptions and Dependencies

This section outlines the assumptions made during system design and the external dependencies required for proper system operation.

8.23.1 Assumptions

- **Internet Availability:** Users are expected to have access to a stable internet connection to interact with the platform.
- **Valid User Data:** Users and communities provide accurate and valid information during registration and usage.
- **Community Compliance:** Communities follow platform guidelines while creating and managing events.
- **Moderate Initial Scale:** The system is assumed to operate at a moderate scale during initial deployment (academic or limited public usage).

8.23.2 Dependencies

- Web Browsers: The system depends on modern web browsers for client-side access.
- Backend Frameworks: Laravel (PHP) and Python-based frameworks for core functionality.
- Database Systems: MySQL for structured data and Qdrant (or similar) for vector data storage.
- External AI Services: APIs such as OpenAI for embeddings and NLP processing.
- Hosting Environment: Linux-based servers or cloud infrastructure for deployment.

8.24 Requirement Identifying Technique

Identifying, refining, and validating system requirements is one of the most critical phases of software development. For EventBridge AI, which is rooted in solving real-world problems through intelligent automation and semantic technologies, the process of requirement gathering was intentionally thorough and iterative.

This section explains the methodologies employed to collect actionable requirements and illustrates how user expectations, system capabilities, and technical feasibility were aligned into one coherent framework. It also introduces system use cases derived from these activities that support the modeling of interaction diagrams.

8.24.1 Use Case Diagram

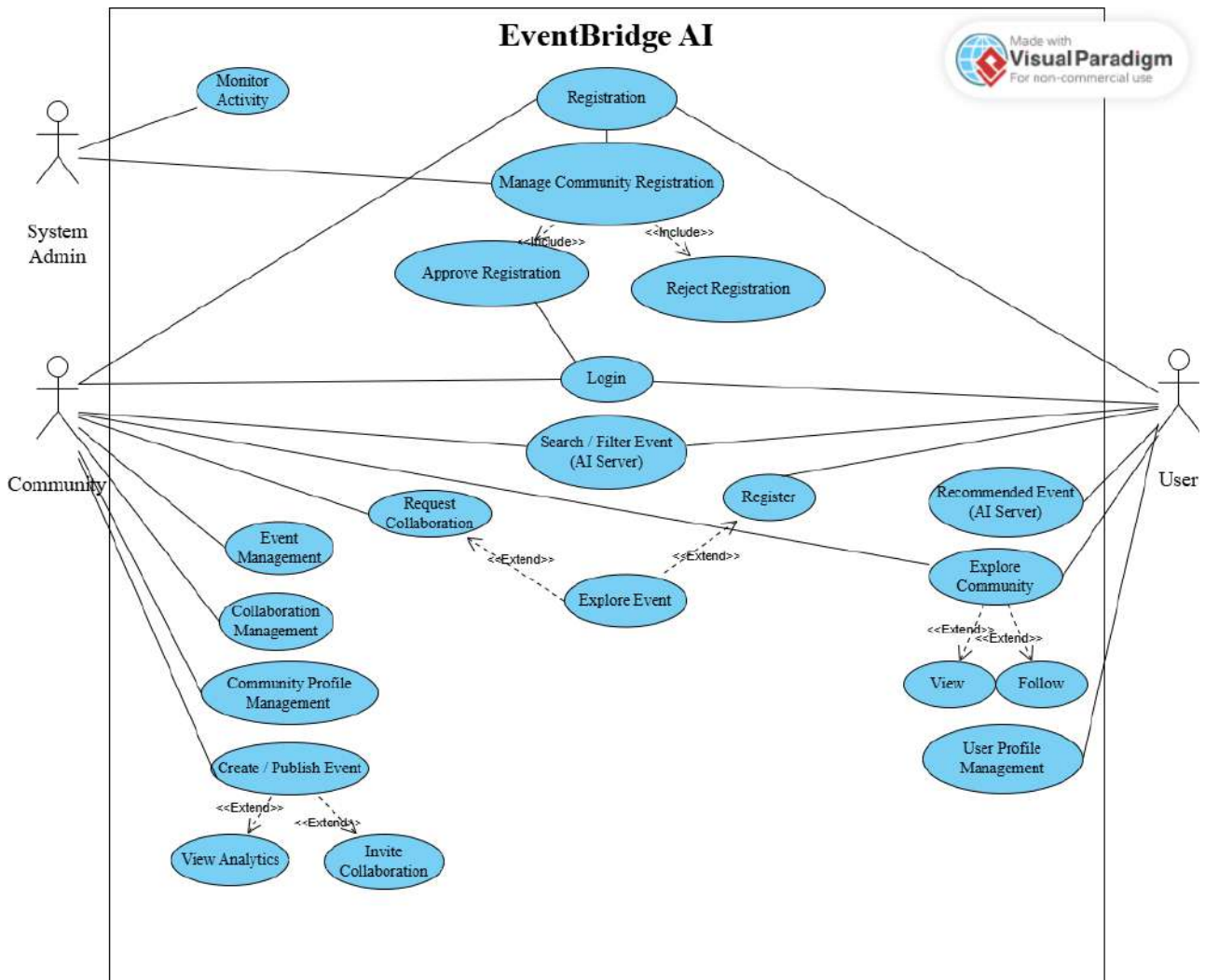


Figure 8.44: Use Case Diagram

8.24.2 Use Case Description

Use Case Diagram for EventBridge AI illustrates the key actors involved in the system (Individuals, Communities, and Admin) and their respective interactions with the system's functionality. Each use case represents a functional requirement or interaction, showing how users engage with the platform and how their activities are processed by the system.

Actors in the System:

- **Individual User:** The User is an Individual or attendee who engages with the platform to discover events and receive recommendations.
- **Community:** The Community is responsible for creating and managing events within EventBridge AI.
- **Admin (System Admin):** The Admin is a system-level actor responsible for accepting and rejecting the request for community account creation.

Key Use Cases and Interactions:

- Individual User
 - Register (Individual User):
 - Description: This use case allows an individual to create a new account on the EventBridge AI platform to access user-level features.
 - Interaction: The user submits their details (email, password, etc.), which the system stores to create their profile and manage access.
- Login (Individual User):
 - Description: This functionality enables registered users to authenticate and securely access their personalized dashboard.
 - Interaction: The user enters their credentials, and the system verifies them against the database to grant access.
- User Profile Management (Individual User):
 - Description: This allows users to update and manage their personal details, preferences, and account settings.
 - Interaction: The user edits their profile information, and the system saves and updates these changes in the database.
- Search / Filter Event (AI Server) (Individual User):
 - Description: This allows users to search for events using keywords, filters, or natural language queries. The platform uses AI-driven search to provide relevant results.
 - Interaction: The user performs a search, and the AI server queries the event database to return tailored results based on the search criteria.
- Recommended Event (AI Server) (Individual User):
 - Description: The system provides personalized event suggestions to the user based on their preferences and past behavior.
 - Interaction: The AI Recommendation Engine analyzes user data and displays customized event suggestions exclusively on the user's interface.
- Explore Community (Individual User):
 - Description: This allows users to browse through different registered communities on the platform to find groups that match their interests.
 - Interaction: The user browses community profiles. They can choose to "View" specific community details or "Follow" a community to receive future updates.
- Explore Event (Individual User):
 - Description: Users can browse through various active event listings on the platform.

- Interaction: The user explores available events. While viewing a specific event, they can choose to "Register" to officially attend it.

Community:

- Registration (Community):
 - Description: This use case allows an organization to submit a request to create a community account on the platform.
 - Interaction: The community submits their organizational details, which are sent to the System Admin for approval before the account is fully activated.
- Login (Community):
 - Description: This enables approved communities to authenticate and access their organizational tools and features.
 - Interaction: The community enters their credentials, and the system verifies them to grant access to the community dashboard.
- Community Profile Management (Community):
 - Description: This allows community administrators to update their public profile, description, and organizational settings.
 - Interaction: The community edits their public-facing details, and the system updates their profile across the platform.
- Event Management (Community):
 - Description: This provides a centralized hub for communities to oversee their overall portfolio of past, present, and drafted events.
 - Interaction: The community navigates their dashboard to track, edit, or manage the lifecycle of all their associated events.
- Create / Publish Event (Community):
 - Description: Communities can generate new events by providing details such as event name, category, location, time, and description.
 - Interaction: The community fills in the details and publishes the event. From here, they can also "View Analytics" for that specific event or "Invite Collaboration" from other communities.
- Explore Event (Community):
 - Description: Communities can browse the platform to see events hosted by other organizations for networking or market research.
 - Interaction: The community browses event listings. If they find an aligned event, they can "Request Collaboration" to co-host or sponsor.
- Search / Filter Event (AI Server) (Community):
 - Description: This functionality allows communities to search for events based on keywords, filters, or natural language queries to find partnership opportunities.

- Interaction: The community inputs a search query, and the AI server processes it to return relevant event results.
- Collaboration Management (Community):
 - Description: This enables communities to manage incoming and outgoing collaboration requests to share resources and co-host programs.
 - Interaction: The community uses the system to accept, reject, or track partnership requests with other organizations.

Admin:

- Monitor Activity (System Admin):
 - Description: Admins can oversee general platform operations, user behavior, and overall system health to ensure a safe environment.
 - Interaction: The admin views system dashboards and activity logs to track platform usage and flag any issues.
- Manage Community Registration (System Admin):
 - Description: Admins are responsible for reviewing incoming account creation requests from new communities.
 - Interaction: The admin reviews the community’s submitted details and will either “Approve Registration” to grant access or “Reject Registration” if the guidelines are not met.

8.25 Non-Functional Requirements

This section defines the quality attributes, constraints, and system characteristics required for the effective operation of the Event Bridge AI platform. These requirements ensure system performance, security, scalability, and overall usability.

8.25.1 Performance Requirements

- Response Time: The system shall respond to user actions (e.g., page loads, API requests) within 2–3 seconds under normal load conditions.
- Search Performance: Semantic search queries shall return results within 3–5 seconds, depending on dataset size and AI processing.
- Concurrent Users: The system shall support multiple concurrent users without significant degradation in performance.
- Scalability Handling: The system shall maintain acceptable performance levels as the number of users and events increases.
- AI Processing Efficiency: Embedding generation and recommendation processing shall be optimized to minimize latency and resource usage.

8.25.2 Safety Requirements

- **Data Integrity:** The system shall ensure that user and event data is not lost or corrupted during operations.
- **Failure Handling:** The system shall handle unexpected failures gracefully without causing system crashes.
- **Backup Mechanism:** Regular backups shall be maintained to prevent data loss.
- **User Action Safety:** Critical actions (e.g., event deletion) shall include confirmation mechanisms to prevent accidental operations.

8.25.3 Security Requirements

- **Authentication:** The system shall implement secure authentication mechanisms for users, communities, and admin.
- **Authorization:** Role-based access control shall be enforced to restrict access to sensitive functionalities.
- **Data Protection:** Sensitive user data shall be securely stored and protected against unauthorized access.
- **Secure Communication:** All communication between client and server shall use secure protocols (e.g., HTTPS).
- **API Security:** External API integrations shall be secured using API keys and proper access control mechanisms.

8.25.4 Software Quality Attributes

- **Usability:** The system shall provide an intuitive and user-friendly interface for both technical and non-technical users.
- **Reliability:** The system shall operate consistently with minimal downtime.
- **Availability:** The system shall be available to users at all times except during scheduled maintenance.
- **Efficiency:** System resources shall be utilized efficiently to ensure optimal performance.
- **Accuracy:** AI-based recommendations and search results shall maintain a high level of relevance and correctness.

8.25.5 Business Rules

- **Community Approval Rule:** A community must be approved by the admin before accessing platform features.
- **Event Ownership Rule:** Events must be associated with at least one community.
- **Collaboration Rule:** Collaboration between communities requires mutual agreement.

- User Registration Rule: Users must register before participating in events.
- Feedback Rule: Only registered participants can provide feedback on events.

8.25.6 Interoperability

- API Integration: The system shall support integration with external AI services such as OpenAI.
- Cross-System Communication: The system shall enable communication between the Laravel backend and Python-based AI services.
- Data Exchange Format: Standard data formats (e.g., JSON) shall be used for communication between system components.

8.25.7 Extensibility

- Modular Architecture: The system shall be designed using a modular structure to allow the addition of new features.
- AI Expansion: The AI module shall support integration of additional models and algorithms.
- Feature Enhancement: New functionalities (e.g., analytics dashboards, advanced recommendations) can be added without major restructuring.

8.25.8 Maintainability

- Code Structure: The system shall follow clean coding practices and a standardized structure.
- Documentation: Proper technical documentation shall be maintained for all modules.
- Error Handling: Clear error messages and logging mechanisms shall be implemented.
- Update Capability: System updates and bug fixes shall be applied without major downtime.

8.25.9 Portability

- Platform Independence: The system shall be accessible across different operating systems via web browsers.
- Deployment Flexibility: The system shall be deployable on various cloud or VPS environments.

8.25.10 Reusability

- Reusable Components: Common modules (e.g., authentication, notification system) shall be reusable across the system.
- Service Reusability: AI services and APIs shall be designed for reuse in other applications or modules.

8.25.11 Installation

- **Server Setup:** The system shall support installation on Linux-based servers.
- **Dependency Management:** Required frameworks, libraries, and services shall be clearly defined and installable.
- **Configuration:** Environment configuration (e.g., database, API keys) shall be manageable through environment files.
- **Deployment Process:** The system shall support a structured deployment process for smooth setup and updates.

8.26 Other Requirements

This section specifies additional requirements that are not covered in previous sections but are essential for the complete functioning, usability, and compliance of the Event Bridge AI system.

8.26.1 On-line User Documentation and Help System Requirements

- **User Help Interface:** The system shall provide an accessible help section within the platform for user guidance.
- **Guided Onboarding:** The system shall include onboarding assistance to help new users and communities understand platform features.
- **FAQ Section:** Frequently Asked Questions shall be available to address common user queries.
- **Error Guidance:** User-friendly error messages and suggestions shall be provided to assist users in resolving issues.
- **Feature Instructions:** Key functionalities (e.g., event creation, registration, collaboration) shall include brief usage instructions.

8.26.2 Purchased Requirements

- **Third-Party AI Services:** The system may utilize external AI services such as OpenAI for embeddings and NLP tasks.
- **Hosting Services:** Deployment may rely on third-party hosting providers (e.g., VPS or cloud platforms).
- **Domain and SSL:** The system may require domain registration and SSL certificates for secure access.
- **Database Tools:** External tools or services may be used for database hosting and management.

8.26.3 Licensing Requirements

- **Software Licenses:** All frameworks and libraries used (e.g., Laravel, Python frameworks) shall comply with their respective open-source licenses.
- **API Usage Licensing:** Usage of third-party APIs shall adhere to their licensing terms and usage policies.
- **Content Ownership:** Users and communities retain ownership of content they create on the platform.
- **Compliance:** The system shall comply with applicable software licensing regulations.

8.26.4 Legal, Copyright, and Other Notices

- **Data Privacy:** The system shall respect user privacy and handle data responsibly.
- **Copyright Protection:** Users shall not upload or share content that violates copyright laws.
- **Terms of Use:** The platform shall define clear terms and conditions for all users.
- **Liability Disclaimer:** The platform shall not be held responsible for user-generated content or external interactions.

8.26.5 Applicable Standards

- **Web Standards:** The system shall follow standard web development practices (HTML, CSS, JavaScript compatibility).
- **Security Standards:** The system shall follow standard web security practices (e.g., HTTPS, secure authentication).
- **Software Engineering Practices:** The system shall follow structured development methodologies and coding standards.
- **Data Handling Standards:** Proper data management and storage practices shall be followed.

8.26.6 Reports

- **User Reports:** The system shall provide data related to user registrations and participation.
- **Event Reports:** Communities shall have access to event-related data such as registrations and attendance.
- **Feedback Reports:** The system shall store and display user feedback for events.
- **Usage Analytics:** Basic usage insights (e.g., number of users, events, interactions) shall be available.
- **Administrative Reports:** Admin shall have access to system-level summaries for monitoring platform activity.

Appendix C- Software Design Description (SDD)

8.27 Introduction

8.27.1 Purpose

The purpose of this Software Design Document (SDD) is to define the architectural blueprint for EventBridge AI. This platform is designed to be a centralized AI-powered web solution that connects communities with individuals across the city. It details the system's modules, including the core web application and the AI engine, ensuring a clear roadmap for development and deployment.

8.27.2 Scope

The scope of EventBridge AI covers the development of a web-based ecosystem that serves two primary user bases: Communities (organizers) and Individuals (attendees/learners).

- For Communities: Tools for event posting, management, and inter-community collaboration.
- For Individuals: A unified dashboard for event discovery, profile management, and AI-driven recommendations based on interests.
- Core Technology: Integration of a PHP/Laravel frontend with a Python/Django AI backend.

8.27.3 Project Background

Currently, Individuals and Communities in Karachi are actively involved in organizing and attending events. However, information is fragmented across multiple disparate platforms like WhatsApp, Instagram, Facebook, and LinkedIn. This fragmentation results in low visibility for events and missed opportunities for learning and networking.

8.27.4 Motivation

The motivation behind this project is to solve the problem of "scattered information". By creating a centralized hub, the platform aims to enhance visibility for Communities and simplify the search process for Individuals, ensuring they don't miss out on valuable opportunities due to a lack of awareness.

8.27.5 Project Objective

- To provide a centralized platform for Communities to manage and publish events.
- To implement an AI-powered recommendation system to personalize event feeds for Individuals.
- To facilitate inter-community collaboration and unified notification systems.

8.28 Design Methodology and Software Process Model

8.28.1 Design Methodology

The system follows an Object-Oriented Design (OOD) methodology. This approach allows for the creation of modular classes (e.g., User, Event, Community) that encapsulate data and behavior, ensuring the system is scalable and easier to maintain given the integration between Laravel and Python services.

8.28.2 Design Pattern

The project utilizes the MVC pattern, which is the standard architectural pattern for the Laravel framework.

- Model: Manages the data logic and interactions with the MySQL database (e.g., Event, User models).
- View: Handles the presentation layer (UI) displayed to Individuals and Communities.
- Controller: Processes user inputs and serves as the intermediary between the Model and View.

8.28.3 Software Process Models

An Agile approach is selected to accommodate the iterative nature of developing AI features alongside core web modules.

- Iterative Development: The project is divided into sprints (e.g., Frontend, Backend, AI Integration) as outlined in the project timeline.
- Adaptability: Allows for continuous testing and refinement of the AI recommendation engine based on user feedback.

8.29 System Overview

8.29.1 Architectural Design

The system employs a Service-Oriented Architecture (Client-Server model) with a distinct separation between the core application and the AI service.

- Client Side: Web browsers accessing the platform.

- Server Side:
 - Web Server: Runs the Laravel application for core logic and UI.
 - AI Server: A Python/Django-based API that handles Vector DB interactions and LLM processing.
- Database: A dual-database approach using MySQL for relational data and a Vector Database for semantic search embeddings.

8.29.2 Process Flow (Functional Requirement)

Based on the System Flowchart Diagram:

- User Entry: The user lands on the application and selects their type: Community or Individual (User).
- Authentication:
 - Communities: Register and await approval from the admin before accessing the dashboard.
 - Individuals: Register/login to access their personal dashboard.
- Dashboard Functions:
 - Communities: Can create events, manage collaborations, and view analytics.
 - Individuals: Can explore communities, search for events, and view "Recommended Events" generated by the AI Server.
- Event Interaction: When a Community publishes an event, it is indexed by the AI Server. Individuals can then view, follow, or register for these events.

8.30 Project Architecture

8.30.1 Web Module

Functions (UI Designing & UX Methodology)

- Tools: Figma is utilized for designing high-fidelity prototypes and wireframes to ensure a professional user experience.
- UX Methodology: The design focuses on accessibility and clarity, ensuring individuals can easily filter events by category (Tech, Gaming), format (Physical/Online), and mode (Free/Paid).

Architecture (Box and Line Diagram)

- Frontend: HTML/CSS/JS + Laravel Blade Templates.
- Backend API: Laravel handling Authentication, Event Management, and Notifications.
- AI Engine: Python API handling Recommendations and Smart Search (NLP).

8.30.2 Database Module

- Type: Relational Database (MySQL) + Vector Database.
- Justification: MySQL is essential for maintaining structured relationships between Individuals, Communities, and Events. The Vector Database is required to store embeddings for the content-based filtering algorithms.
- DDL (Data Definition Overview):
 - Users Table: Stores Individual profile data.
 - Communities Table: Stores Community profiles and approval status.
 - Events Table: Stores event metadata (Title, Description, Date).

8.30.3 Administration Module

- Approval System: Validates Community registration requests.
- Content Management: Monitors event postings for compliance.

8.30.4 Data Dictionary

- Individual: An end-user who registers to attend events.
- Community: An entity (university society or city group) that organizes events.
- Embedding: A vector representation of event data used for AI similarity matching.
- Collaborator: A secondary community invited to co-host an event.

8.31 System Analysis & Design Overview

8.31.1 Class Diagram

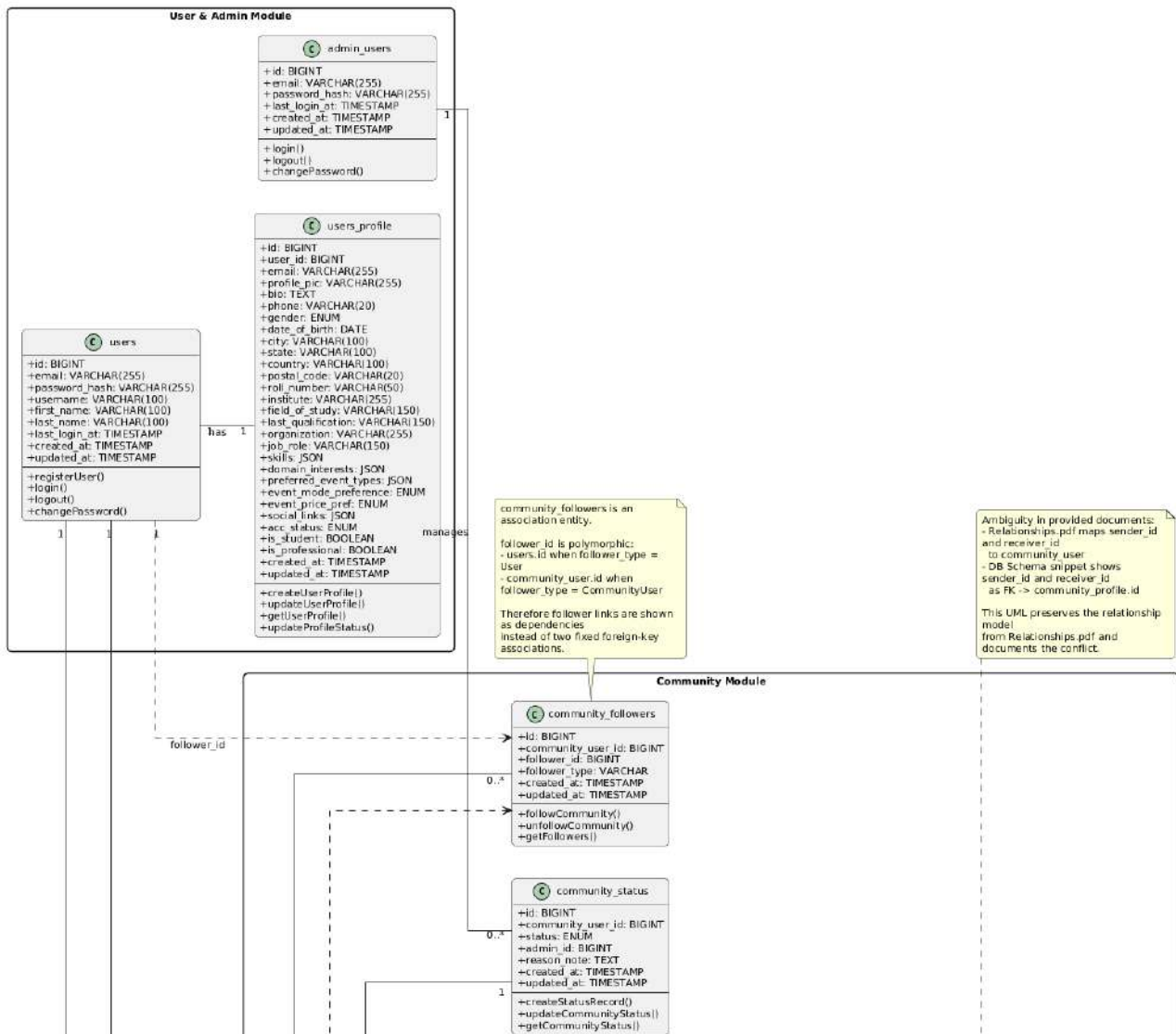


Figure 8.45: Class Diagram Part 01

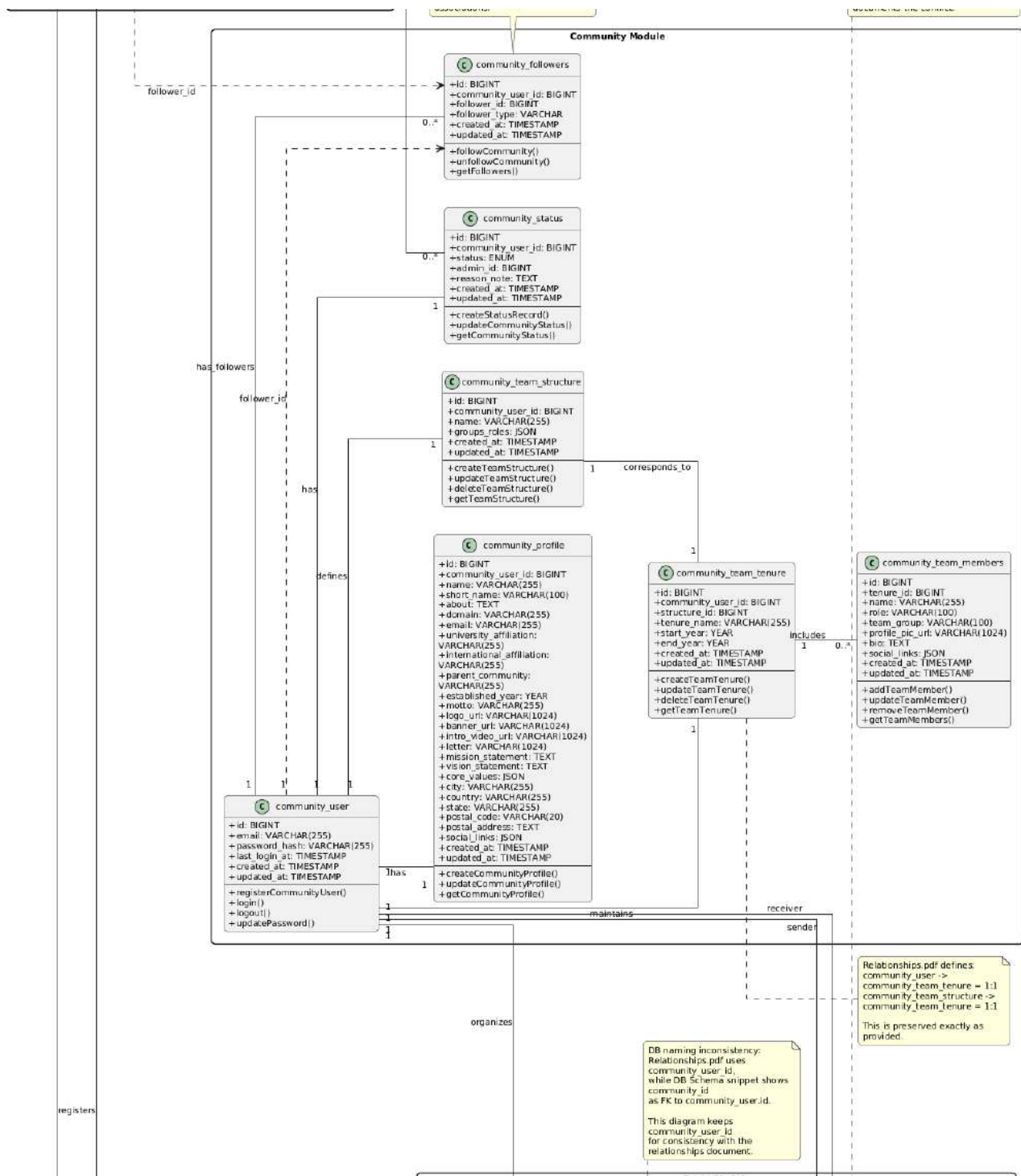


Figure 8.46: Class Diagram Part 01

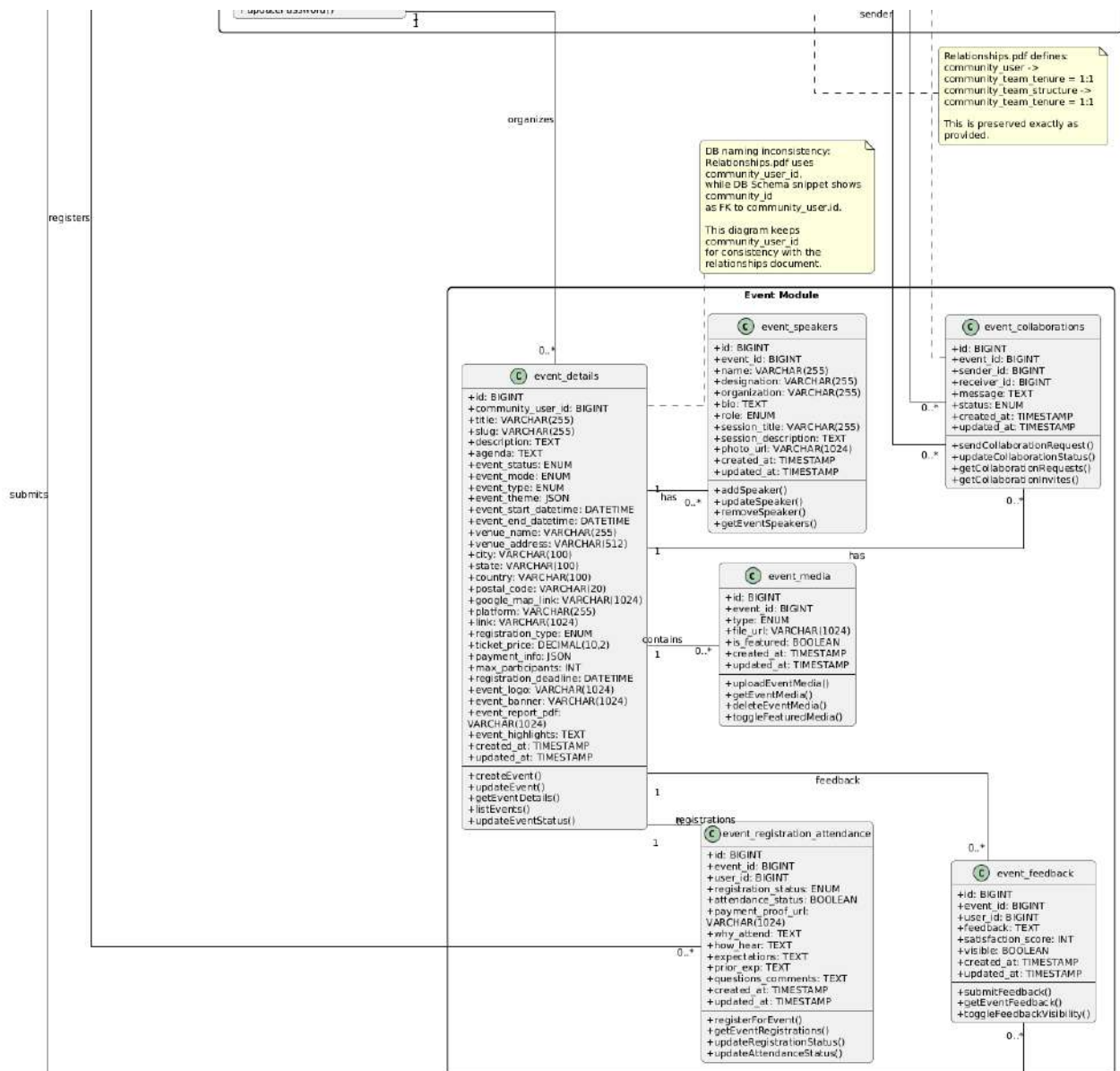


Figure 8.47: Class Diagram Part 01

8.31.2 System Sequence Diagram

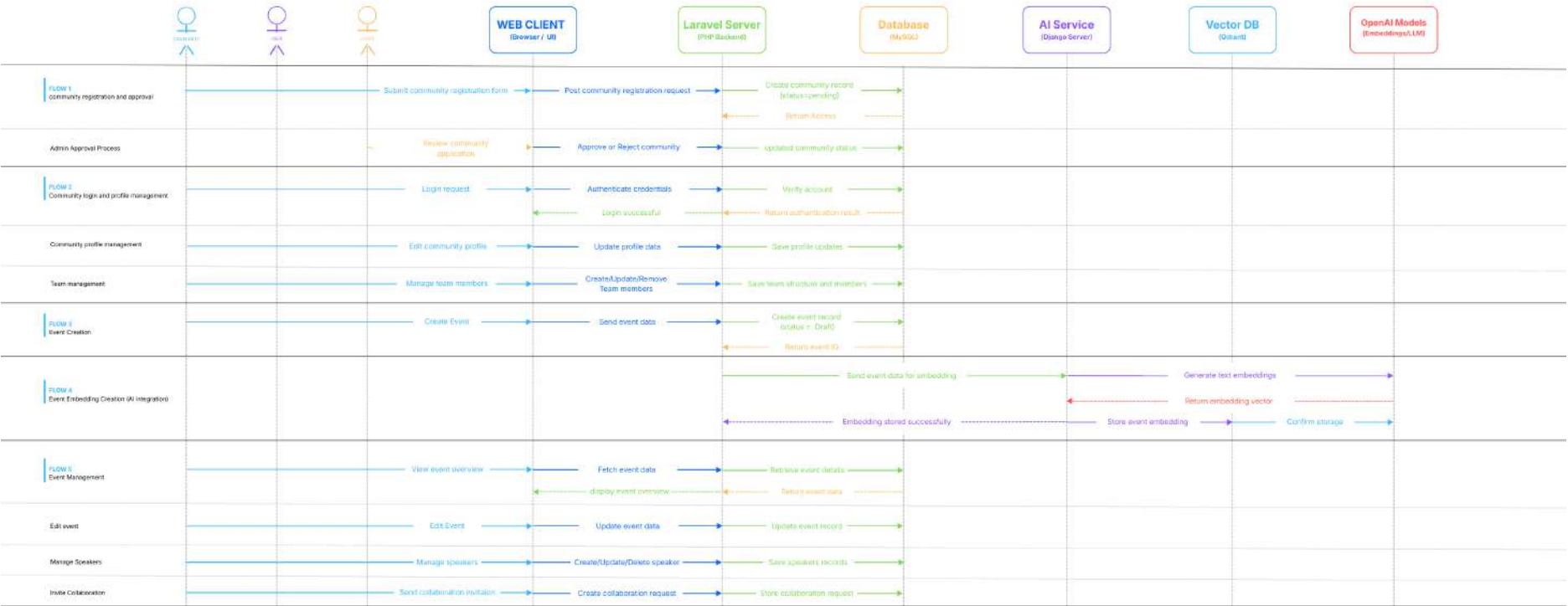


Figure 8.48: Sequence Diagram (Part 1 of 3)



Figure 8.49: Sequence Diagram (Part 2 of 3)

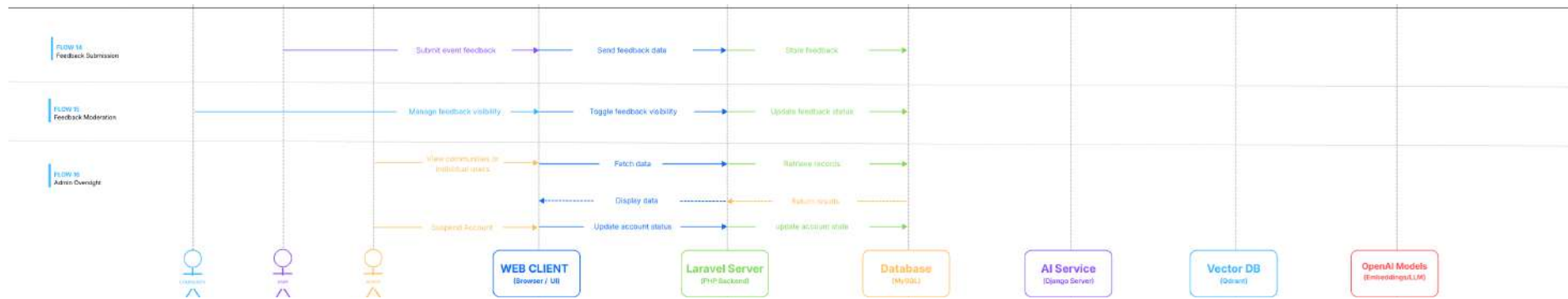


Figure 8.50: Sequence Diagram (Part 3 of 3)

8.31.3 State Transition Diagram



Figure 8.51: UML State Transition Diagram

Appendix D- Dissemination Activity

This appendix documents the dissemination activities undertaken in connection with the EventBridge AI project, comprising participation in a public technology exhibition and the preparation of an academic research paper based on the project's system design and implementation.

8.32 Research Paper

In addition to the exhibition, the findings and technical contributions of the EventBridge AI project were consolidated into a formal IEEE-format research paper, prepared as academic evidence of the project's research depth and contribution. The paper is titled:

"EventBridge AI: A Content-Based, Embedding-Driven Approach to Community Event Discovery and Recommendation"

Authored by Laiba Rafiq and Muhammad Nabeel, under the supervision of Engr. Tauseef Mubeen, Department of Software Engineering, Sir Syed University of Engineering and Technology, Karachi. The paper covers the system's architecture, the content-based nearest-neighbor recommendation methodology, implementation details, and an illustrative evaluation of the platform's semantic search and recommendation performance. As of this report, the paper has been written and is pending submission. The complete manuscript is reproduced below.

**EventBridge AI: A Content-Based, Embedding-Driven Approach to
Community Event Discovery and Recommendation**

Authors: Laiba Rafiq, Muhammad Nabeel

Department of Software Engineering, Sir Syed University of Engineering and Technology, Karachi, Pakistan
Supervised by Engr. Tauseef Mubeen, Assistant Professor, Department of Software Engineering, SSUET

Figure 8.52: EventBridge AI IEEE Research Paper P1

Abstract—Community-led technical and academic ecosystems in Karachi — including chapters of GDGoC, IEEE, and ACM — organize a continuous stream of events that remain fragmented across disconnected channels such as WhatsApp, Instagram, Facebook, and LinkedIn. This fragmentation reduces event visibility and limits opportunities for learning, networking, and cross-community collaboration. This paper presents EventBridge AI, a community-centric event discovery and management platform that addresses this problem through a dual-stack architecture: a Laravel-based web application for core platform logic and a Django-based AI microservice for semantic search and recommendation. Events and user preference profiles are represented as dense vector embeddings using OpenAI's text-embedding-3-large model and stored in a Qdrant vector database, enabling cosine-similarity-based nearest-neighbor retrieval. Semantic search results are further refined using a large language model re-ranking step. The recommendation engine is deliberately designed as a pure content-based filtering system — concatenating user interest, behavioral, and interaction signals into a single embedding query rather than relying on collaborative or hybrid signals — a design choice suited to a growing platform with limited historical interaction data. We describe the system architecture, the recommendation methodology in technical detail, and the platform's implementation, and we discuss the rationale for the content-based design relative to collaborative and hybrid alternatives. Performance characteristics are presented as illustrative of the architecture's expected behavior rather than as controlled empirical benchmarks.

Index Terms—semantic search, event recommendation, content-based filtering, vector embeddings, community platforms, Qdrant

I. INTRODUCTION

In community ecosystems such as those in Karachi, individuals actively organize and attend events led by technical and academic communities, including GDGoC, IEEE, and ACM chapters. However, these events are scattered across multiple, disconnected platforms, including WhatsApp groups, Instagram pages, Facebook groups, and LinkedIn posts. This fragmentation leads to low visibility and missed opportunities for learning, networking, and inter-community collaboration.

General-purpose platforms such as Eventbrite and Facebook Events cater to broad audiences and are not tailored to the needs of city-based technical and academic communities. These platforms lack community profiles, cross-community collaboration tools, and personalization mechanisms suited to a fragmented but tightly-knit local ecosystem. No existing solution specifically addresses this niche with an integrated, AI-driven discovery and management approach.

This paper presents EventBridge AI, a centralized, AI-powered web platform that brings together events from across Karachi's technical and academic communities into a single discovery hub. The platform's core contribution is not merely centralization but the integration of a semantic search and content-based recommendation layer that helps

individuals discover events aligned with their interests without relying on manually curated categories or sparse user-rating histories. The remainder of this paper is organized as follows. Section II reviews related work in event recommendation, content-based filtering, and campus/community event platforms. Section III describes the system architecture. Section IV details the recommendation methodology. Section V discusses implementation aspects. Section VI presents illustrative performance characteristics. Section VII discusses design trade-offs and limitations, and Section VIII concludes with directions for future work.

II. RELATED WORK

A. Event Recommendation in Event-Based Social Networks

Event recommendation has been studied extensively in the context of event-based social networks (EBSNs) such as Meetup and Douban Event. Huang *et al.* [1] survey context-aware recommendation schemes in EBSNs and observe that events typically have short lifetimes and sparse attendance history, making conventional collaborative filtering poorly suited to this domain — a finding that directly motivates content-based approaches for platforms with similarly short-lived, high-turnover event listings. Xu and Liu [2] propose a semantic-enhanced, context-aware hybrid collaborative filtering method for EBSNs, combining content semantics with contextual influence, while Liao *et al.* [3] address group-level event recommendation using attention-based neural models. These works establish event recommendation as a well-studied problem that is largely approached through collaborative or hybrid filtering, in contrast to the purely content-based design adopted in this work.

B. Content-Based Filtering and Embedding-Based Recommendation

Zhao *et al.* [4] provide a comprehensive survey of embedding techniques in recommender systems, distinguishing embeddings derived from collaborative signals (user-item interactions) from those derived directly from content representations. Ezaldeen *et al.* [5] demonstrate a content-based recommendation framework for e-learning that combines semantic term-graph construction with deep learning to match learner profiles against resource descriptions — a domain-adjacent precedent for using rich textual content, rather than interaction history, as the primary recommendation signal. Jha *et al.* [6] apply cosine-similarity-based recommendation directly within a social events networking platform, offering a close methodological parallel to the embedding-and-cosine-similarity pipeline used in EventBridge AI, though without large language model-based embeddings.

C. Centralized Event and Campus Platforms

Several recent systems address the fragmentation problem in campus and community event discovery. Manikandan *et al.* [7] present "Evento," an event-based social networking platform with an integrated recommendation engine driven

Figure 8.53: EventBridge AI IEEE Research Paper P2

by user interactions such as likes and replies, reporting a purity index of 0.80 across a small user base. Lakhani *et al.* [8] describe "Evently," an AI-powered college event management platform centralizing event creation, registration, and promotion with AI-assisted discovery features, reflecting the broader industry trend toward augmenting centralized event hubs with personalization. Both systems, however, rely on interaction-based or clustering signals rather than semantic content embeddings, and neither targets a multi-community, city-wide ecosystem spanning distinct technical and academic organizations, as EventBridge AI does for Karachi.

D. Positioning of This Work

Existing literature reveals two dominant patterns: EBSN-focused recommenders that emphasize collaborative or hybrid filtering over social interaction data [1], [2], [3], and centralized campus platforms that provide event management with limited or interaction-driven personalization [7], [8]. EventBridge AI differs from both by adopting a purely content-based, embedding-driven nearest-neighbor approach — using OpenAI's text-embedding-3-large model and cosine similarity over a Qdrant vector store — deliberately avoiding the cold-start and sparsity limitations of collaborative methods [1] while extending the content-based paradigm demonstrated in adjacent domains [5], [6] to a multi-community, city-scoped event discovery context not previously addressed in the literature.

III. SYSTEM ARCHITECTURE

EventBridge AI is composed of two independently deployed backend systems, each with a distinct responsibility. The web application, built with Laravel (PHP) and backed by a MySQL relational database, is the source of truth for all user, community, and event data and handles authentication, event lifecycle management, community collaboration, registrations, and notifications. A separate Django-based AI microservice, implemented in Python with Django REST Framework, is responsible exclusively for semantic search, embedding generation, and recommendation logic. The Django service is stateless from a relational-data perspective — it stores no event or user data of its own, relying instead on a Qdrant vector database for all persisted embeddings.

The Laravel application is the sole caller of the AI microservice; the Django service never initiates a request back to Laravel. Communication occurs over HTTP, authenticated via a shared-secret key transmitted in a custom request header and validated by a dedicated middleware component on the Django side. The integer event identifier assigned by Laravel's relational database is reused directly as the point identifier in Qdrant, forming the only shared key between the two systems and avoiding the need for a translation or mapping layer.

Qdrant maintains two logically separate collections, both storing 3072-dimensional vectors under cosine distance: one collection supports keyword-independent semantic search over event listings, and a second, richer collection supports the recommendation engine, storing additional fields such as

registration type and event status that are used for post-retrieval filtering. In production, the Django AI engine runs behind an Nginx reverse proxy under Gunicorn on an OVH Cloud VPS, while the Laravel application is hosted separately. This two-server separation keeps the platform's business logic and its intelligence layer independently scalable and maintainable.

IV. METHODOLOGY — AI RECOMMENDATION ENGINE

A. Event and User Representation

Each event is converted into a single natural-language text string by concatenating its structured attributes — title, event type, mode (online/physical), registration type, city and country, description, and agenda — into a labeled template (e.g., "[Title]: ...", "[Description]: ..."). This text is embedded synchronously, at the moment an event is published or updated, using OpenAI's text-embedding-3-large model, producing a 3072-dimensional dense vector that is upserted into the relevant Qdrant collection alongside its metadata payload.

User preference representations are constructed analogously. At recommendation time, a single textual profile is assembled by concatenating the user's declared domain interests, preferred event types, skills, and mode/price preferences with contextual signals drawn from their interaction history: the titles of previously registered, saved, viewed, and clicked events, and recent search queries. All of these signals — declared preferences and behavioral history alike — are merged into one text string prior to embedding; there is no separate scoring, weighting, or combination step across signal types. This single string is embedded using the same text-embedding-3-large model used for events, ensuring that user and event vectors share a common representational space.

B. Retrieval and Filtering

The resulting user vector is queried against the recommendation-oriented Qdrant collection using approximate nearest-neighbor cosine similarity search, retrieving the top fifty candidate events. This candidate set is then filtered in application code: events already dismissed, registered for, or saved by the user are excluded regardless of similarity score, as are events that are not in a published state or whose start date has already passed. The remaining candidates are truncated to the requested result size (a default of ten, with a maximum of twenty) and returned in descending order of cosine similarity score.

It is important to characterize this mechanism precisely: it is pure nearest-neighbor content-based filtering. There is no matrix factorization, no user-user or item-item collaborative signal, and no learned combination weighting across the different input signals — every signal, whether a declared interest or a behavioral trace, is folded into a single embedding query through text concatenation. This design is a deliberate choice rather than a limitation to be engineered around: for a multi-community platform still accumulating interaction history, a purely content-based approach avoids the cold-start and data-sparsity problems that collaborative

Figure 8.54: EventBridge AI IEEE Research Paper P3

and hybrid methods are known to face in comparable event-recommendation settings [1].

C. Semantic Search Pipeline

Independently of the recommendation engine, EventBridge AI provides free-text semantic search over the event catalog. A user's natural-language query is embedded with the same text-embedding-3-large model and used to retrieve the top candidates from the search-oriented Qdrant collection via cosine similarity. Rather than returning this ranking directly, the candidate set — together with the original query — is passed to a large language model (GPT-4o-mini) acting as a re-ranker, which receives short text excerpts of each candidate event and returns a refined ordering of event identifiers. This two-stage retrieve-then-re-rank design allows vector search to handle recall efficiently across the full event catalog while delegating fine-grained relevance judgments to a model with stronger contextual reasoning.

V. IMPLEMENTATION

The web application is implemented in Laravel 12 on PHP 8.2 with a MySQL relational schema covering users, communities, events, registrations, collaborations, and notifications. The AI microservice is implemented in Django with Django REST Framework, exposing a small set of stateless JSON endpoints for collection setup, event indexing (both for search and for recommendation), semantic search, and recommendation generation. All endpoints other than the Django admin interface are protected by shared-secret API key authentication.

A notable implementation characteristic is that all AI processing is synchronous and request-driven: embedding generation, Qdrant queries, and re-ranking calls all occur inline within the HTTP request/response cycle of the Django service, with no background task queue, scheduler, or asynchronous job runner inside the AI microservice itself. Recurring work, such as the daily computation of personalized recommendation feeds for all users, is instead orchestrated entirely from the Laravel side, whose own task scheduler invokes the AI microservice's HTTP endpoints on a schedule and persists the results into the relational database for fast retrieval on subsequent page loads.

Beyond the AI layer, the platform provides modules for community profile management, event lifecycle management (draft, published, ongoing, completed, archived), cross-community collaboration on shared events, a notification and alerting system, and an administrative oversight module for moderation, reporting, and platform-wide analytics.

VI. EVALUATION

Because EventBridge AI is an actively developed platform rather than a system with an extended production history, the performance characteristics presented here are illustrative of the architecture's expected behavior rather than controlled empirical benchmarks collected under fixed load and traffic conditions. They are reported in this spirit to give a realistic sense of how the system behaves without overstating the rigor of the underlying measurement.

Within the search and recommendation pipelines, the dominant sources of latency are the synchronous calls to external OpenAI endpoints — embedding generation and, in the search path, LLM-based re-ranking — rather than the Qdrant retrieval step itself, which is designed for low-latency approximate nearest-neighbor lookup and is expected to scale comfortably as the event corpus grows into the thousands. The recommendation pipeline's post-retrieval filtering step (excluding dismissed, already-interacted-with, unpublished, or past events) operates over a bounded candidate set of fifty items and is computationally inexpensive relative to the embedding and vector-search stages.

Relevance of retrieved results was additionally assessed using a metadata-alignment proxy — comparing retrieved event categories, types, and modes against the declared interests and historical interactions of a user — as a practical alternative to manual ground-truth relevance labeling under project time constraints.

VII. DISCUSSION

The decision to implement a purely content-based recommendation engine, rather than a collaborative or hybrid one, reflects the practical realities of a platform still building its user and interaction base. Collaborative filtering methods depend on dense historical interaction data to identify meaningful user-user or item-item similarity, and are known to struggle with cold-start users and newly published events — precisely the conditions under which a growing, multi-community event platform operates most of the time [1]. By contrast, a content-based approach can recommend a newly published event immediately, since its relevance is derived from its own descriptive content rather than from accumulated interaction signals.

This design choice is not without trade-offs. Because all signals are merged into a single embedding query through text concatenation, the system cannot currently assign different importance to, for example, a strong declared interest versus a single incidental click, nor can it produce the kind of serendipitous, cross-user recommendations that collaborative signals sometimes surface. Extending the system with a learned weighting scheme over signal types, or introducing a complementary collaborative signal once sufficient interaction data accumulates, are natural directions for future refinement rather than corrections to a flawed design.

The platform's geographic scope is intentionally bounded to Karachi's technical and academic community ecosystem — including GDGoC, IEEE, and ACM chapters — rather than generalized to a national or multi-city footprint at this stage. This bounded scope is consistent with the platform's origin as a solution to a locally observed fragmentation problem, and mirrors the scoping seen in comparable single-institution or single-city systems such as Evento [7] and Evently [8], both of which similarly rely on interaction-based rather than content-embedding-driven personalization.

VIII. CONCLUSION AND FUTURE WORK

Figure 8.55: EventBridge AI IEEE Research Paper P4

This paper presented EventBridge AI, a community-centric event discovery and management platform designed for Karachi's fragmented technical and academic event ecosystem. The platform's central technical contribution is a content-based, embedding-driven recommendation engine that represents both events and user preferences as dense vectors in a shared semantic space, retrieved via cosine similarity over a Qdrant vector database, deliberately avoiding the cold-start limitations associated with collaborative filtering in comparable event-recommendation contexts. A complementary semantic search pipeline, combining vector retrieval with large language model re-ranking, further improves free-text event discovery beyond simple keyword matching.

Future work includes extending the platform with event check-in and digital pass issuance, post-event attendance and feedback analytics dashboards for community organizers, and a refined recommendation model that introduces explicit per-signal weighting or a complementary collaborative component as the platform's interaction history grows. The project's community-centric, technology-driven approach to local event discovery also aligns with several United Nations Sustainable Development Goals, including Quality Education, Industry, Innovation and Infrastructure, and Partnerships for the Goals [21], through its emphasis on accessible knowledge-sharing and cross-organizational collaboration within Karachi's community ecosystem.

REFERENCES

- [1] X. Huang, G. Liao, N. Xiong, A. V. Vasilakos, and T. Lan, "A Survey of Context-Aware Recommendation Schemes in Event-Based Social Networks," *Electronics*, vol. 9, no. 10, p. 1583, 2020.
- [2] M. Xu and S. Liu, "Semantic-enhanced and context-aware hybrid collaborative filtering for event recommendation in event-based social networks," *IEEE Access*, vol. 7, pp. 17493–17502, 2019.
- [3] G. Liao, X. Huang, N. N. Xiong, and C. Wan, "An Intelligent Group Event Recommendation System in Social Networks," *arXiv preprint arXiv:2006.08893*, 2020.
- [4] X. Zhao, M. Wang, X. Zhao, J. Li, S. Zhou, D. Yin, Q. Li, J. Tang, and R. Guo, "Embedding in Recommender Systems: A Survey," *arXiv preprint arXiv:2310.18608*, 2023.
- [5] H. Ezaldeen, S. K. Bisoy, R. Misra, and R. Alatrash, "Semantics aware intelligent framework for content-based e-learning recommendation," *Natural Language Processing Journal*, vol. 3, p. 100008, 2023.
- [6] P. B. Jha et al., "Enhancing Event Exploration and Engagement: A Social Events Networking Platform Leveraging Cosine Similarity Recommendations and Google Maps Integration," *Zenodo*, 2024.
- [7] G. Manikandan, R. S. Sam, S. F. Gilbert, and K. Srikanth, "Event-Based Social Networking System With Recommendation Engine," *International Journal of Intelligent Information Technologies*, vol. 20, no. 1, pp. 1–16, 2024.
- [8] S. Lakhani, P. Sharma, M. Parmar, D. Patel, S. Shrivastava, and J. Parmar, "Evently: AI-Powered College Event Management System," *International Journal of Research and Innovation in Applied Science*, vol. 11, no. 2, pp. 1699–1709, 2026.
- [9] M. R. Khan and T. Zia, "Smart Event Recommendation System Based on User Interests Using NLP and AI," *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 5, pp. 456–463, 2021.
- [10] P. Ganeshkumar, S. Padmanabhan, A. Kumar B. R., and V. A., "Social Media Personal Event Notifier Using NLP and Machine Learning," *arXiv preprint arXiv:2210.05001*, 2022.
- [11] D. Aljohani, H. Davis, and R. Mitchell, "A Comparative Study of Event Management Systems in Higher Education," in *Proc. Int. Conf. on Information Society (i-Society)*, 2017, pp. 34–40.
- [12] S. Zhang, L. Yao, A. Sun, and Y. Tay, "Deep Learning Based Recommender System: A Survey and New Perspectives," *ACM Computing Surveys*, vol. 52, no. 1, pp. 1–38, 2019.
- [13] F. Narducci, "Personalized Search and Filtering in Event Management Systems Using Machine Learning," *Expert Systems with Applications*, vol. 150, 2020.
- [14] T. Singh and D. P. Singh, "Enhancing Event Discovery Using AI-Based Semantic Search," *International Journal of Computer Applications*, vol. 176, no. 39, pp. 7–12, 2020.
- [15] P. Duhan, M. Wadhwa, and S. Jain, "Semantic Search and Recommendation Algorithm," *arXiv preprint arXiv:2412.06649*, 2024.
- [16] S. Mirajkar, N. Nair, and R. Patil, "A Systematic and Comparative Analysis of Semantic Search Algorithms," *International Journal of Computer Applications*, vol. 186, no. 43, pp. 10–18, 2024.
- [17] L. Huang, Y. Chen, and J. Zhang, "Improved Content Recommendation Algorithm Integrating Semantic Information," *Journal of Big Data*, vol. 10, no. 56, pp. 1–18, 2023.
- [18] K. Rizkallah, H. Hajj, and W. El-Hajj, "New Vector-Space Embeddings for Recommender Systems," *Applied Sciences*, vol. 11, no. 14, pp. 1–16, 2021.
- [19] J. Zhao and L. Zhang, "An Interactive Dashboard for Event Analysis and Participation Metrics," *IEEE Access*, vol. 8, pp. 45678–45690, 2020.
- [20] S. A. Smith, "Feedback Systems in Educational Event Platforms Using AI Sentiment Analysis," *Journal of Educational Computing Research*, vol. 59, no. 4, 2021.
- [21] United Nations, "Transforming Our World: The 2030 Agenda for Sustainable Development." [Online]. Available: <https://sdgs.un.org/goals>

Figure 8.56: EventBridge AI IEEE Research Paper P5

Appendix E- Marketing / Promotional Material

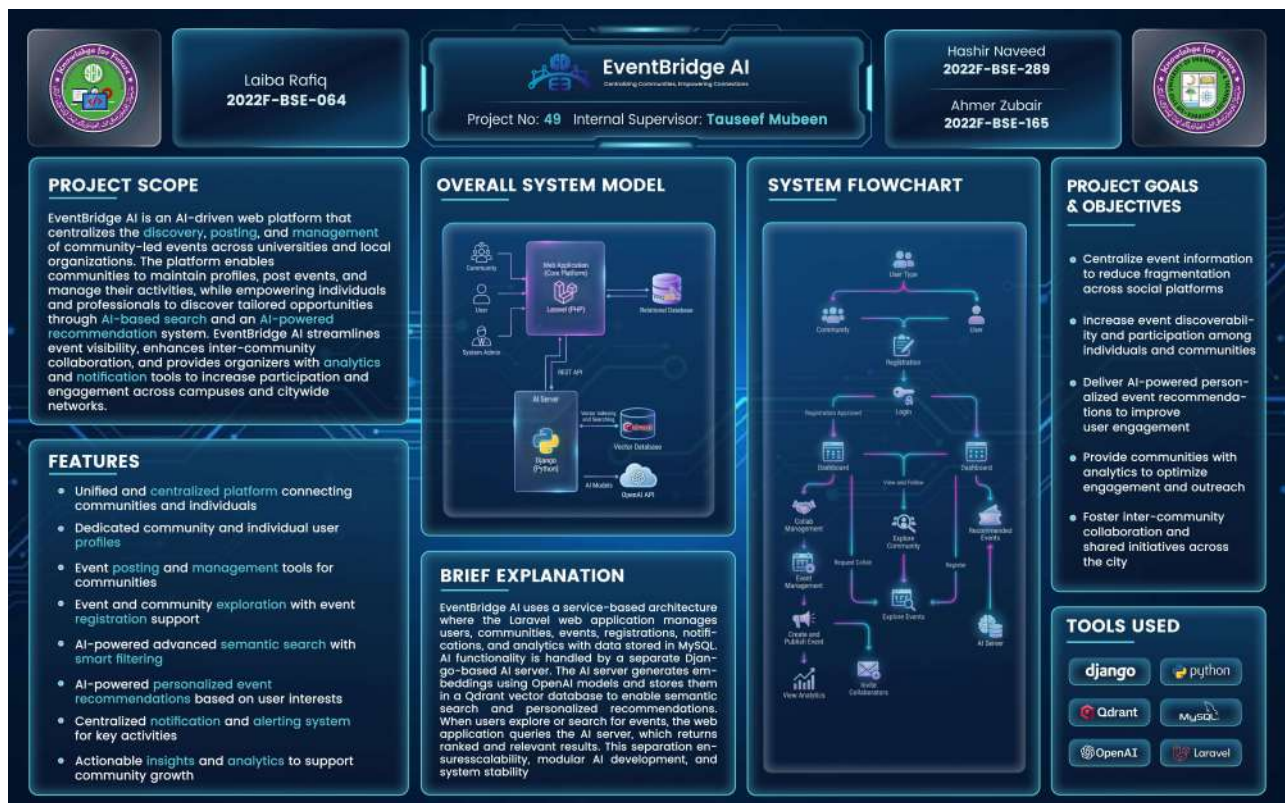


Figure 8.57: Wall Banner

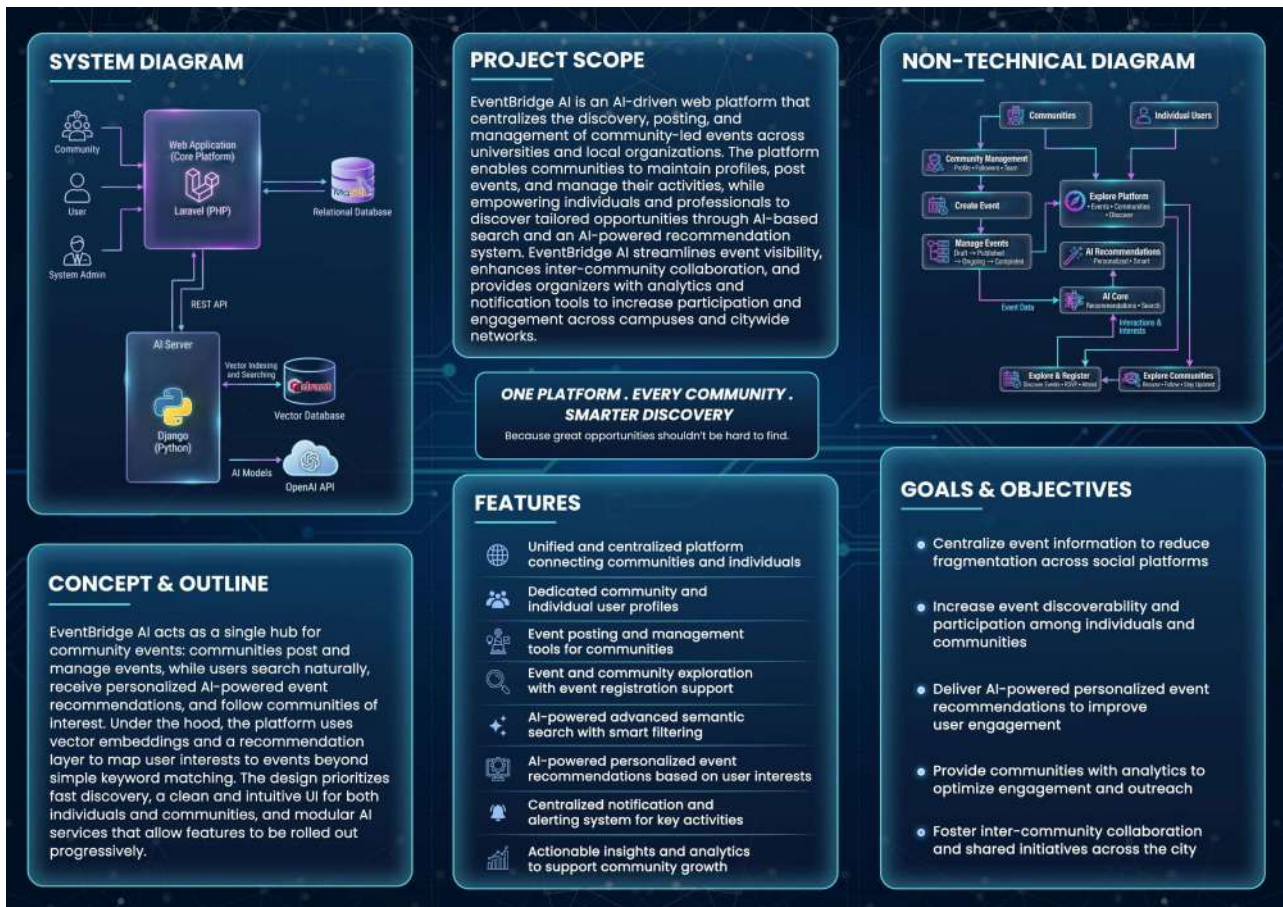


Figure 8.58: Brochure Inside



Figure 8.59: Brochure Outside



Figure 8.60: Standee

References

- [1] J. Beel, B. Gipp, S. Langer and C. Breiteringer, “Research-paper recommender systems: A literature survey,” *International Journal on Digital Libraries*, 2016.
- [2] “An anatomization of research paper recommender system: Overview, approaches and challenges,” *Expert Systems with Applications*, ScienceDirect, 2022.
- [3] C. Basu, H. Hirsh, W. W. Cohen and C. Nevill-Manning, “Technical paper recommendation: A study in combining multiple information sources,” arXiv:1106.0248.
- [4] Y. Zhang, H. Wu, V. Sorathia and V. K. Prasanna, “Event recommendation in social networks with linked data enablement,” in *Proceedings of the Indian International Conference on Artificial Intelligence*, 2006.
- [5] T. De Pessemier, S. Coppens, K. Geebelen, C. Vleugels, S. Bannier, E. Mannens, K. Vanhecke and L. Martens, “Collaborative recommendations with content-based filters for cultural activities via a scalable event distribution platform,” *Multimedia Tools and Applications*, 2011.
- [6] “A Novel Method for Event Recommendation in Meetup,” in *Proceedings of the IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, 2017.
- [7] “Deep Learning Social Filtering Model for Event Recommendation Services,” *American Journal of Artificial Intelligence*, Science Publishing Group, 2024.
- [8] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of EMNLP-IJCNLP*, 2019. Available: <https://arxiv.org/abs/1908.10084>
- [9] “Comparative analysis of word embeddings in assessing semantic similarity of complex sentences,” arXiv:2010.12637. Available: <https://arxiv.org/abs/2010.12637>
- [10] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [11] “Orchestration Framework: LangChain Deep Dive,” *Codesmith*, 2024.
- [12] “Retrieval-Augmented Generation Architectures: Integrating LangChain, LangGraph, Vector Stores, and Open-Source Large Language Models – A Survey and Case Study with GPT-OSS-20B,” *ResearchGate*, 2025.
- [13] Qdrant, “Qdrant – High-performance, massive-scale Vector Database and Vector Search Engine,” *GitHub Repository*, 2026.
- [14] “Understanding Embeddings and Vector Databases with Qdrant,” *eusoj.dev*, 2025.
- [15] “Addressing the Cold-Start Problem in Recommender Systems Based on Frequent Patterns,” *Algorithms (MDPI)*, vol. 16, no. 4, p. 182, 2023.
- [16] “Review-based Recommender Systems: A Survey of Approaches, Challenges and Future Perspectives,” arXiv:2405.05562. Available: <https://arxiv.org/abs/2405.05562>
- [17] “Building Online Communities for Professional Networks,” *ResearchGate Case Study*.
- [18] “Do Online Communities Support Research Collaboration?,” *ResearchGate*.
- [19] “Online Community Platform Market Research Report 2034,” *Dataintel*, 2026.
- [20] Invest2Innovate, “Pakistan Startup Ecosystem Report 2019,” *i2i*, 2019.
- [21] “Inside Karachi, Pakistan’s Thriving Tech Hub: Startups and Success Stories,” *Nucamp*, 2024.
- [22] “Top 5 Must-Attend Tech Meetups and Conferences in Karachi, Pakistan,” *Nucamp*, 2024.

- [23] “Karachi Tech Scene: Complete Guide for Developers & Digital Nomads,” *itsocialevent.com*.